INTRODUCTION TO ARTIFICIAL INTELLIGENCE, DATA SCIENCE & DATA MINING

UNIVERSITY OF BUEA - COLLEGE OF TECHNOLOGY

Defining the Fields

📊 Data Science

- Extracts knowledge & insights from structured and unstructured data
- Combines statistics, programming, and domain expertise
- Covers the FULL pipeline: raw data → decisions
- Output: insights, predictions, dashboards, reports

🔍 *Analogy: A detective analysing clues to solve a mystery*

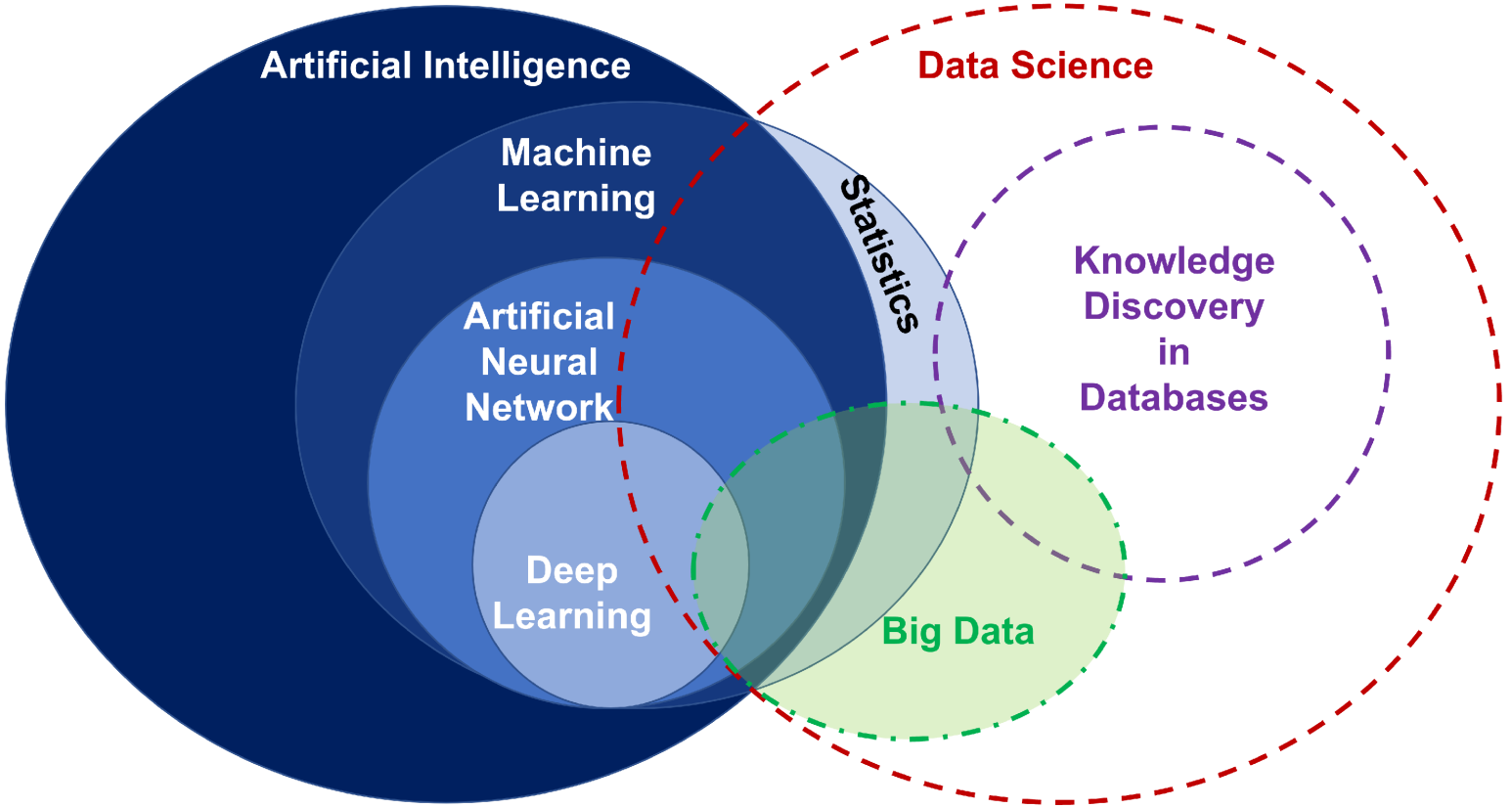
🤖 Artificial Intelligence

- Machines that simulate human intelligence and behaviour
- Includes rule-based systems, ML, Deep Learning, LLMs
- Learns patterns from data to make predictions or act
- Output: intelligent systems, autonomous agents

🤖 *Analogy: Building a robot that can think, learn, and act*

⚡ **Golden Rule: Every AI project IS a Data Science project — but not every Data Science project uses AI.**

LINGO DATA SCIENCE AND AI

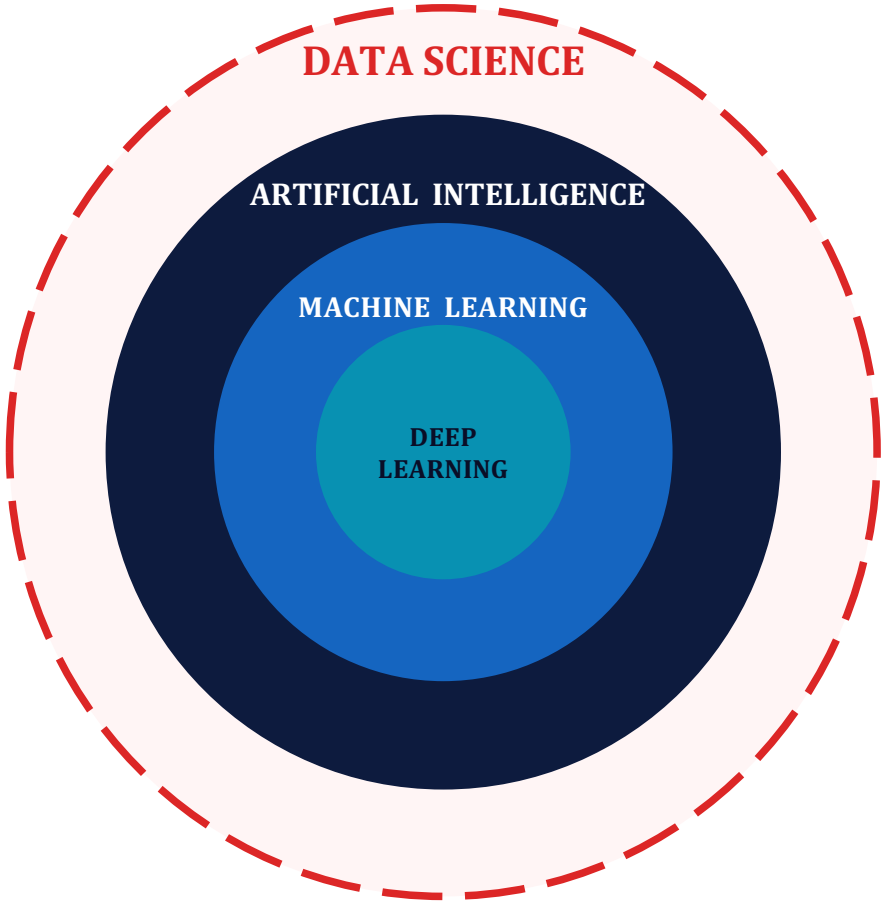


Data Science and AI

DS vs AI — Six Key Dimensions

Dimension	📊 Data Science	🤖 Artificial Intelligence
Core Question	"What does the data tell us?"	"How can machines think and act?"
Primary Output	Insights · reports · predictions	Intelligent systems · autonomous agents
Human Role	High — analyst interprets & decides	Low — system acts autonomously
Key Techniques	Statistics · EDA · Visualisation · ML	Deep Learning · NLP · CV · Transformers
Explainability	High priority — decisions must justify	Often sacrificed for performance
Typical Tools	Python · R · SQL · Pandas · Tableau	TensorFlow · PyTorch · HuggingFace · LangChain

How AI & Data Science Relate



📊 **Data Science contains:**

Statistics · KDD · Data Engineering
Visualization · Domain Knowledge

🤖 **AI adds:**

Expert Systems · Planning · Rule-based Logic

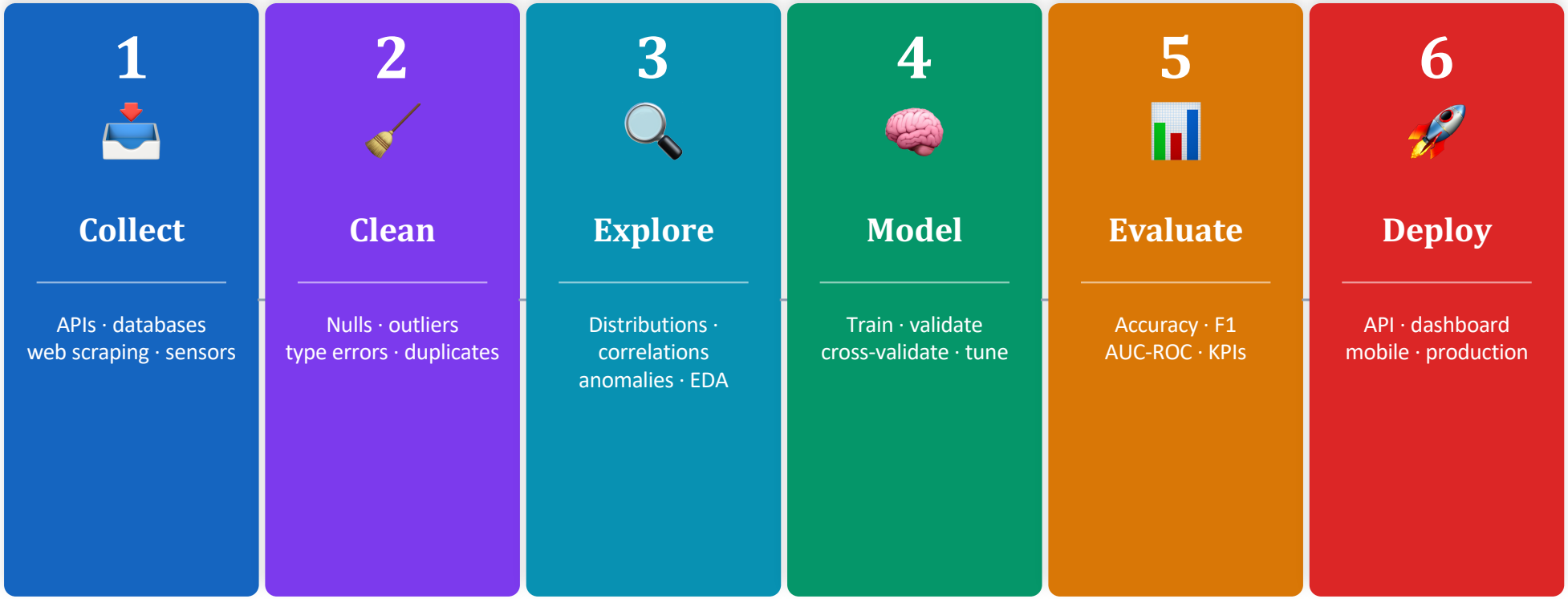
🧠 **Machine Learning adds:**

SVM · Random Forest · Regression · Clustering

🧠 **Deep Learning adds:**

CNNs · RNNs · Transformers · LLMs · Foundation Models

The Data Science Workflow



⏱ *Steps 2 & 3 (Clean + Explore) consume 70–80% of total project time*

Introduction To AI

Artificial Intelligence (AI) is a branch of computer science focused on building systems capable of performing tasks that traditionally require human intelligence. Rather than following a rigid set of rules, these systems use algorithms and data to mimic cognitive functions such as **learning**, **reasoning**, and **problem-solving**.

In humans and animals, intelligence generally involves the **ability to perceive, synthesize, and infer information, and to adaptively achieve goals in complex, uncertain environments.**
Key facets include:

- **Learning & Adaptation:** Acquiring knowledge and skills from experience or data, and adjusting to new situations.
- **Reasoning & Problem-Solving:** Using logic, rules, or mental models to reach conclusions or find solutions to novel problems.
- **Generalization:** Applying knowledge learned in one context to different, but related, contexts.
- **Perception & Understanding:** Interpreting sensory data (visual, linguistic, etc.) to form a meaningful model of the world.
- **Goal-Directed Behavior:** Planning and taking actions to achieve objectives, often while managing trade-offs and uncertainty.
- Crucially, intelligence is a **spectrum** (from a worm to a human) and is **multidimensional** (e.g., social, emotional, spatial intelligence).

A simple definition of Artificial Intelligence would be a collection of algorithms that **imitate** human reasoning and behavior

How do we define intelligence?

Human beings have innate intelligence, defined as the intelligence that governs every activity in our body. This intelligence is what causes an oak tree to grow out of a little seed, and an elephant to form from a single-celled organism.

DEFINITION

AI

as anything that makes machines act more intelligently

POLICE
INTERCEPTOR
POLICE
POLICE
07
07
FOUNDATION SERIES
SCI-FI
164 SFX - 96KHZ/24BIT - META DATA
angelitoon

Classical AI Techniques

Machine Learning

- Supervised: regression, classification (labelled data)
- Unsupervised: clustering, dimensionality reduction
- Key libs: Scikit-learn, XGBoost, LightGBM

Deep Learning

- Multi-layer neural networks learn hierarchical features
- Excels at images, audio, video, and long sequences
- Key libs: TensorFlow, Keras, PyTorch

Transformers & LLMs

- Self-attention: every token relates to all others
- Pre-trained at scale → fine-tune for any task
- GPT-4 · Claude · Gemini · LLaMA · Mistral

NLP

- Understand & generate human language
- Sentiment, translation, NER
- BERT, Hugging Face Transformers

Computer Vision

- Extract meaning from images and video
- Object detection, segmentation, OCR
- OpenCV, YOLO, Detectron2

The correlation between various concepts within the realm of data science and related disciplines is intricate and interconnected. Here's a breakdown of how each concept relates to the others:

1. Data Science:
   - Data science is a multidisciplinary field that encompasses the entire data lifecycle, from data collection and preprocessing to analysis, interpretation, and visualization. It integrates techniques and methodologies from statistics, computer science, domain knowledge, and other disciplines to extract insights and knowledge from data.

2. Artificial Intelligence (AI):
   - AI refers to the development of computer systems capable of performing tasks that typically require human intelligence. In the context of data science, AI techniques, such as machine learning and deep learning, are used to analyze data, make predictions, and automate decision-making processes.

3. Machine Learning (ML):
   - ML is a subset of AI that focuses on developing algorithms and models that can learn from data and improve over time without being explicitly programmed. ML techniques enable computers to identify patterns, make predictions, and uncover insights from data.

4. Deep Learning (DL):
- DL is a subfield of ML that utilizes artificial neural networks with multiple layers to learn from large amounts of data. DL algorithms are particularly effective for tasks such as image recognition, natural language processing, and speech recognition.

5. Big Data:
- Big data refers to large and complex datasets that exceed the processing capabilities of traditional data management tools. Data science techniques, including AI, ML, and data mining, are often used to analyze big data and extract valuable insights from it.

6. Data Analysis:
- Data analysis involves inspecting, cleaning, transforming, and modeling data to uncover meaningful patterns and insights. It encompasses a variety of techniques, including statistical analysis, exploratory data analysis, and hypothesis testing.

7. Data Analytics:
- Data analytics is the process of examining datasets to draw conclusions and make decisions based on the information they contain. It involves the application of statistical and computational techniques to analyze data and extract actionable insights.

8. Data Mining:
   - Data mining is a subset of data analysis that focuses on discovering patterns, trends, and relationships in large datasets. It involves the application of ML and statistical techniques to identify valuable information hidden within the data.

In summary, data science serves as the overarching framework that integrates AI, ML, DL, big data, data analysis, data analytics, and data mining to extract insights, make predictions, and solve complex problems using data-driven approaches. Each concept contributes to different stages of the data lifecycle and complements one another to achieve the overarching goal of deriving value from data.

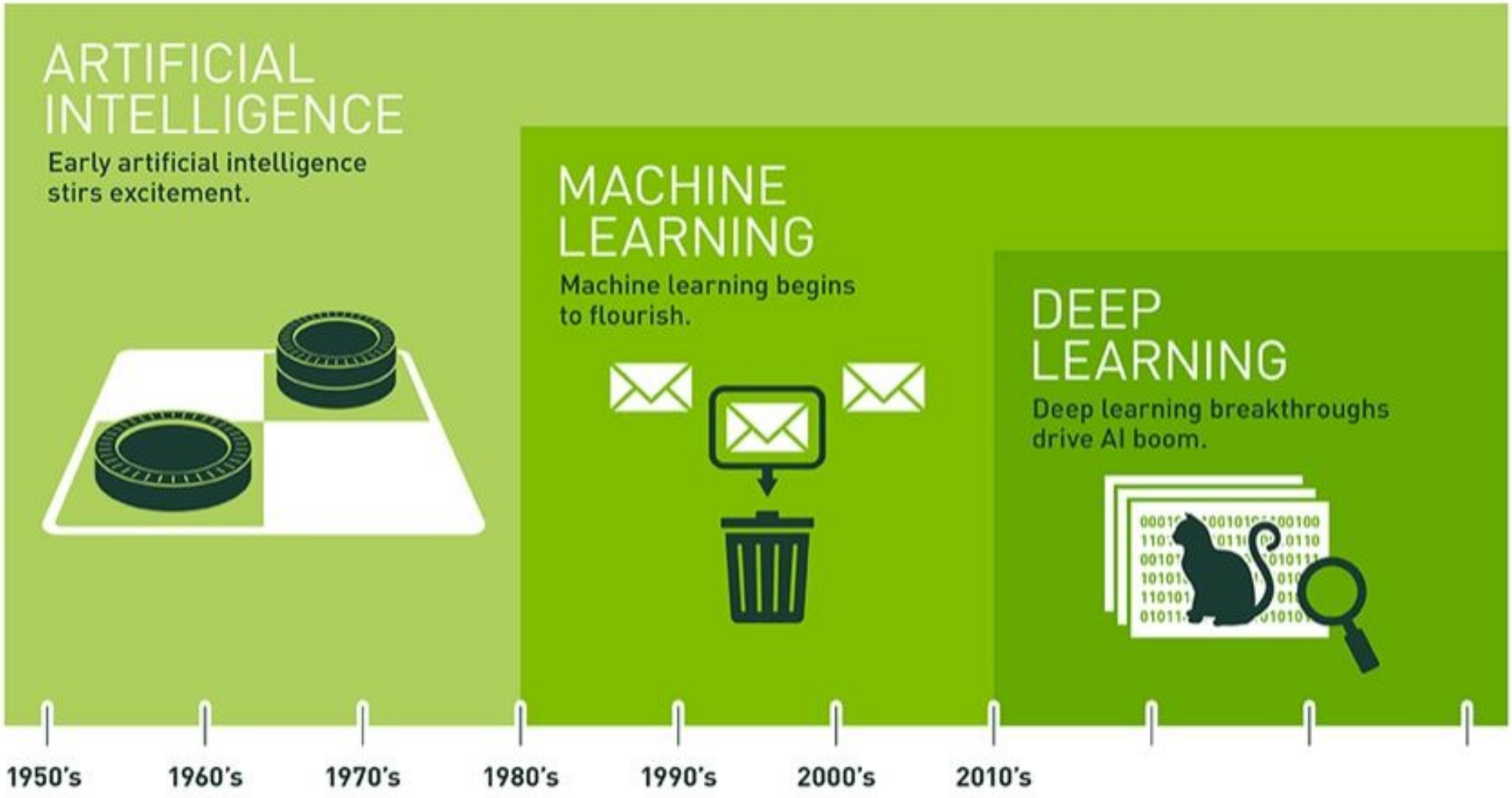
Based on strength, breadth, and application,

AI can be described in different ways

Weak or Narrow AI
is AI that is applied to a specific domain. For example, language translators, virtual assistants, self-driving cars, AI-powered web searches, recommendation engines, and intelligent spam filters. Applied AI can perform specific tasks, but not learn new ones, making decisions based on programmed algorithms, and training data.

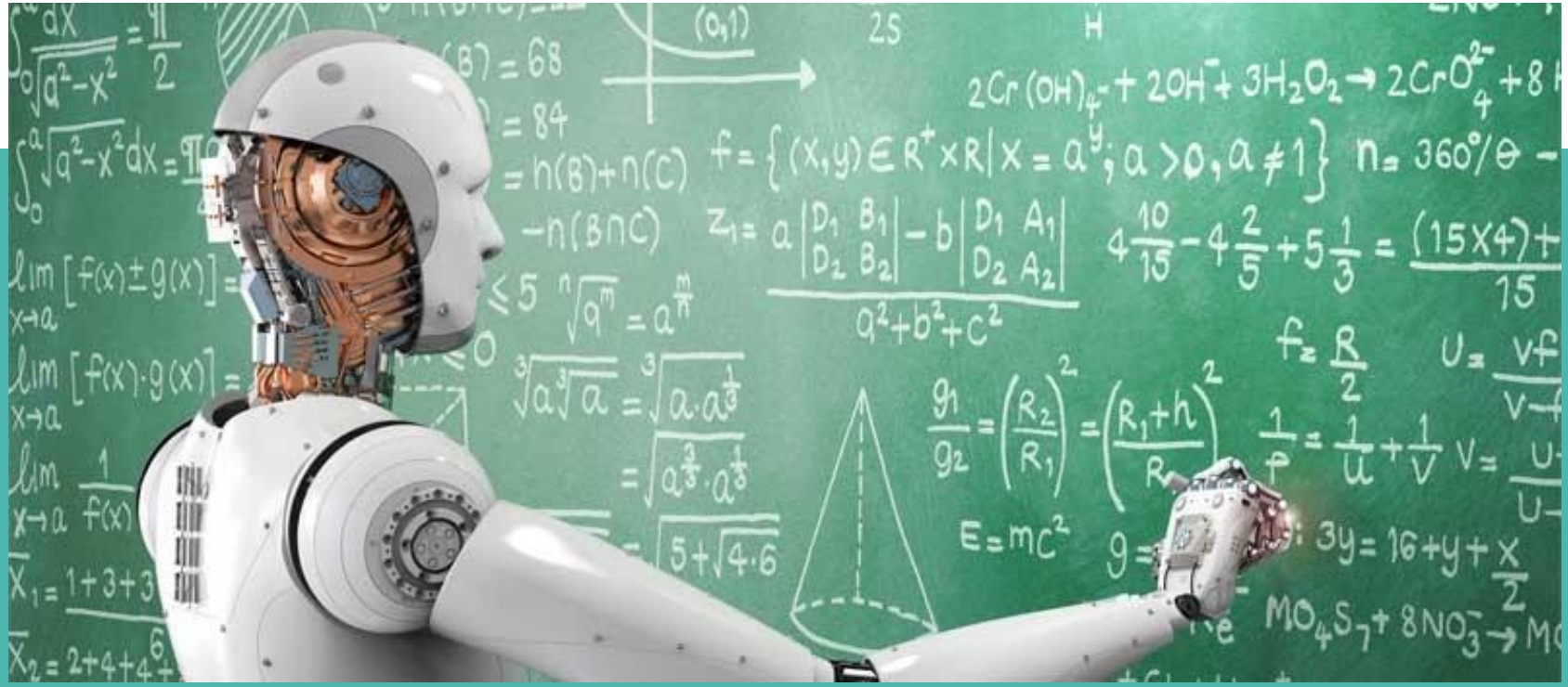
Strong AI or Generalized AI
is AI that can interact and operate a wide variety of independent and unrelated tasks. It can learn new tasks to solve new problems, and it does this by teaching itself new strategies. Generalized Intelligence is the combination of many AI strategies that learn from experience and can perform at a human level of intelligence.

Super AI or Conscious AI
is AI with human-level consciousness, which would require it to be self-aware. Because we are not yet able to adequately define what consciousness is, it is unlikely that we will be able to create a conscious AI in the near future.



Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

Artificial Intelligence ASSETS and LIMITATIONS



Here's a broad list of what AI is currently used for:

- voice/language recognition;
- spam email filtering;
- improving phone battery life;
- image recognition/ facial recognition;
- playing games (go, chess);
- self-driving cars;
- controlling IoT devices;
- managing your daily schedule;
- chatbots;
- AI-optimized hardware;
- urban traffic management;
- business decision management;
- understanding Natural Language;
- job search;
- dating websites;
- health: computer-aided interpretation of medical images;
- AI Agents (Chat GPT, Llama, Gemeni…)

- An AI robot (Bot) can be made for anything that's programmable and does not require much adaptation and creative skills.
- There are also particular cases when AI Neural Networks have been used for composing music , writing poetry, and painting. And a lot more fun stuff and applications can be found on the internet.

KEY CONCEPTS

1. **Natural Language Processing (NLP)**: NLP is a branch of AI that deals with the interaction between computers and human language. It involves tasks like language understanding, language generation, and language translation. Chatbots and virtual assistants are applications of NLP.

2. **Computer Vision**: Computer vision focuses on enabling computers to interpret and understand visual information from the world, such as images and videos. It plays a crucial role in applications like facial recognition, object detection, Image captioning, and autonomous vehicles.

3. **Expert Systems**: Expert systems are AI programs that mimic the decision-making abilities of a human expert in a particular domain. They use rules and knowledge bases to make decisions or solve problems.

Remarks on Expert Systems

Expert systems are a specific type of artificial intelligence (AI) application designed to mimic the decision-making abilities of a human expert in a particular domain or field of knowledge. They are computer programs that use knowledge representation and inference techniques to solve problems, make decisions, or provide advice within a specialized area. Expert systems are characterized by their ability to capture and apply human expertise and domain-specific knowledge effectively.

Expert systems are used in a variety of domains, including:

Medical Diagnosis: Expert systems can assist healthcare professionals in diagnosing medical conditions based on patient symptoms and medical knowledge.

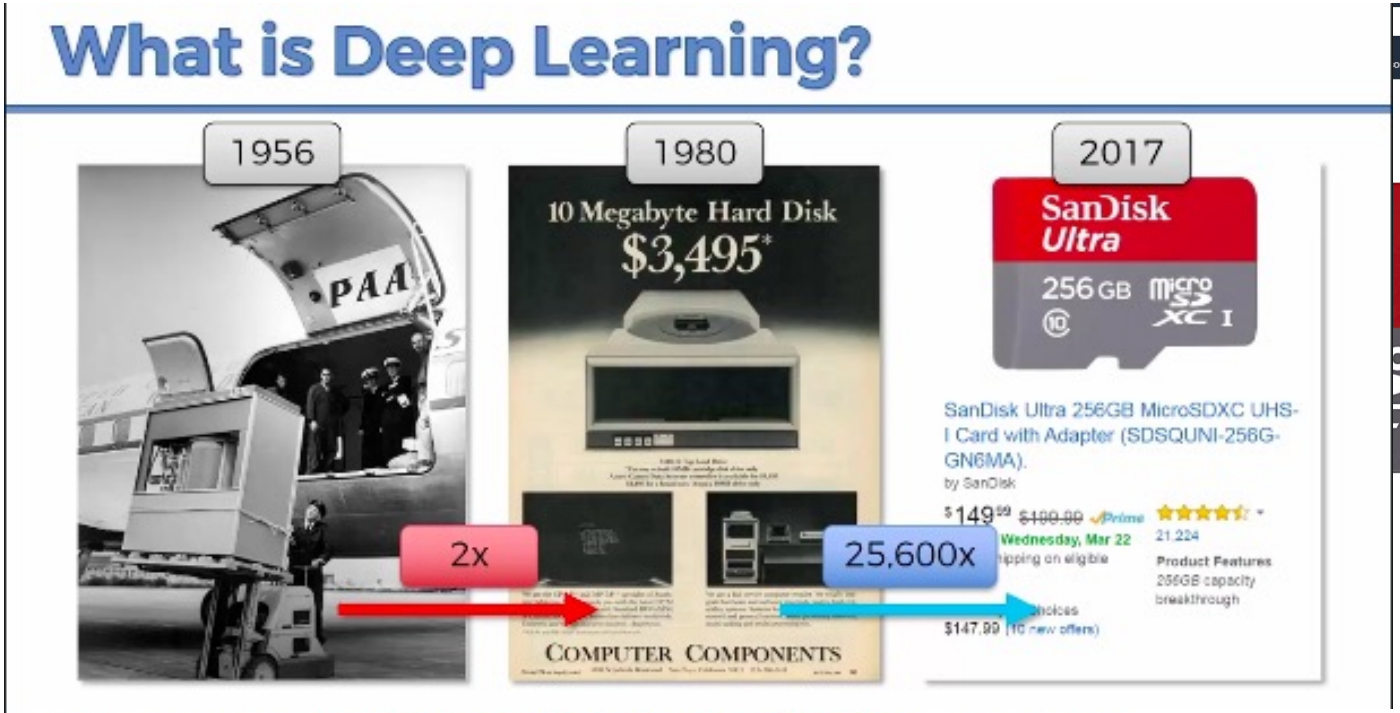
Financial Analysis: They can provide financial advisors with recommendations for investment strategies or risk assessment based on market data and financial expertise.

Troubleshooting: Expert systems can help technicians and engineers diagnose and troubleshoot technical issues in complex machinery or systems.

Customer Support: Some customer service chatbots and virtual assistants use expert systems to provide answers to customer queries based on predefined knowledge.

Natural Language Processing: Expert systems can be used in natural language processing tasks like text summarization, translation, and sentiment analysis.

What is Deep Learning?



Customers who viewed Samsung 256GB 95MB/s MicroSDXC... also viewed:

Samsung 256GB EVO Plus Class 10 UHS-I microSDXC U3 with Adapter (MB-M...
Buy new: $48.79
26 Used & new from $43.86
(653) prime

SanDisk Ultra 256GB MicroSDXC UHS-I Card with Adapter (SDSQUNI-256G-G...
Buy new: $46.99
8 Used & new from $44.95
(32,283) prime

Samsung EVO+ 256GB UHS-I microSDXC U3 Memory Card with Adapter (MB-MC...
Buy new: $54.98
10 Used & new from $50.00
(293)

Electronics › Computers & Accessories › Computer Accessories & Peripherals › Memory Cards › Micro SD Cards

Samsung 256GB 95MB/s MicroSDXC EVO Select Memory Card with Adapter (MB-ME256DA/AM)

by Samsung

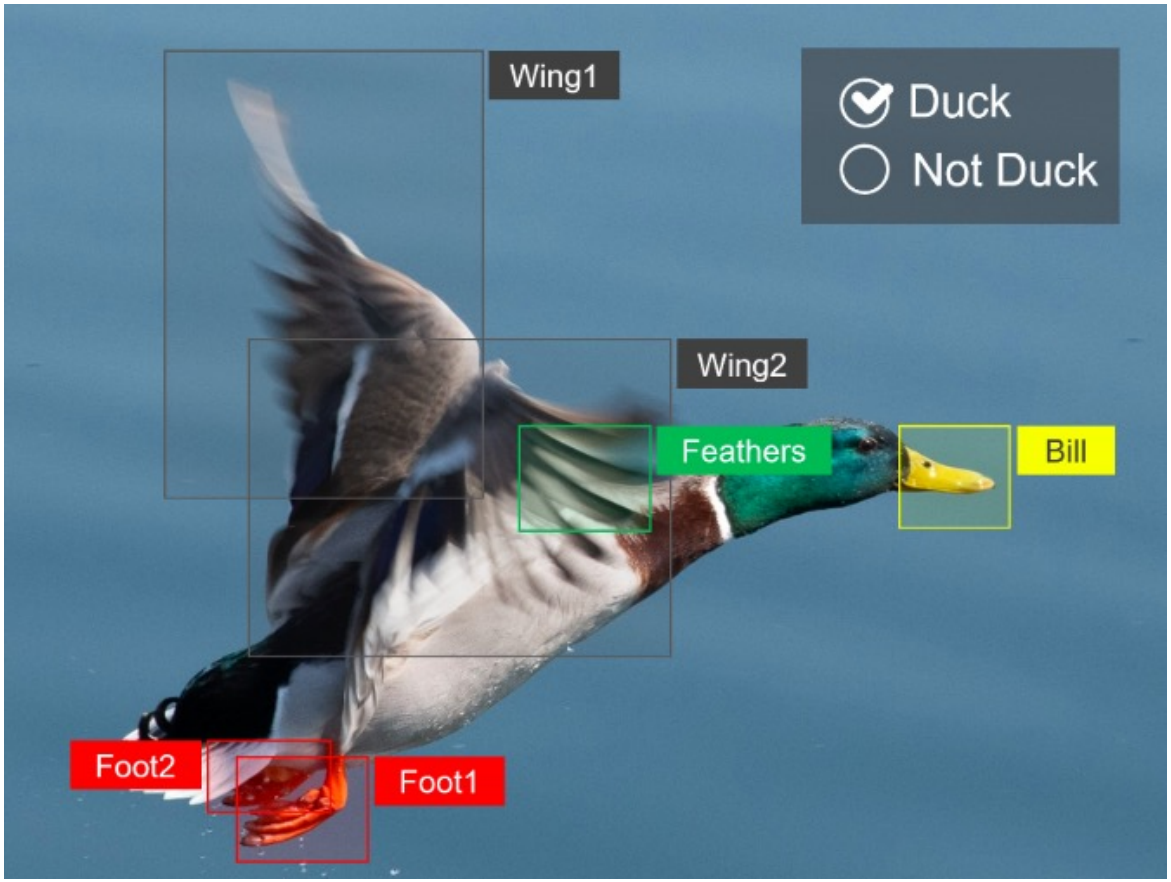
122 customer reviews | 77 answered questions

Price: $120.00 + $55.06 shipping

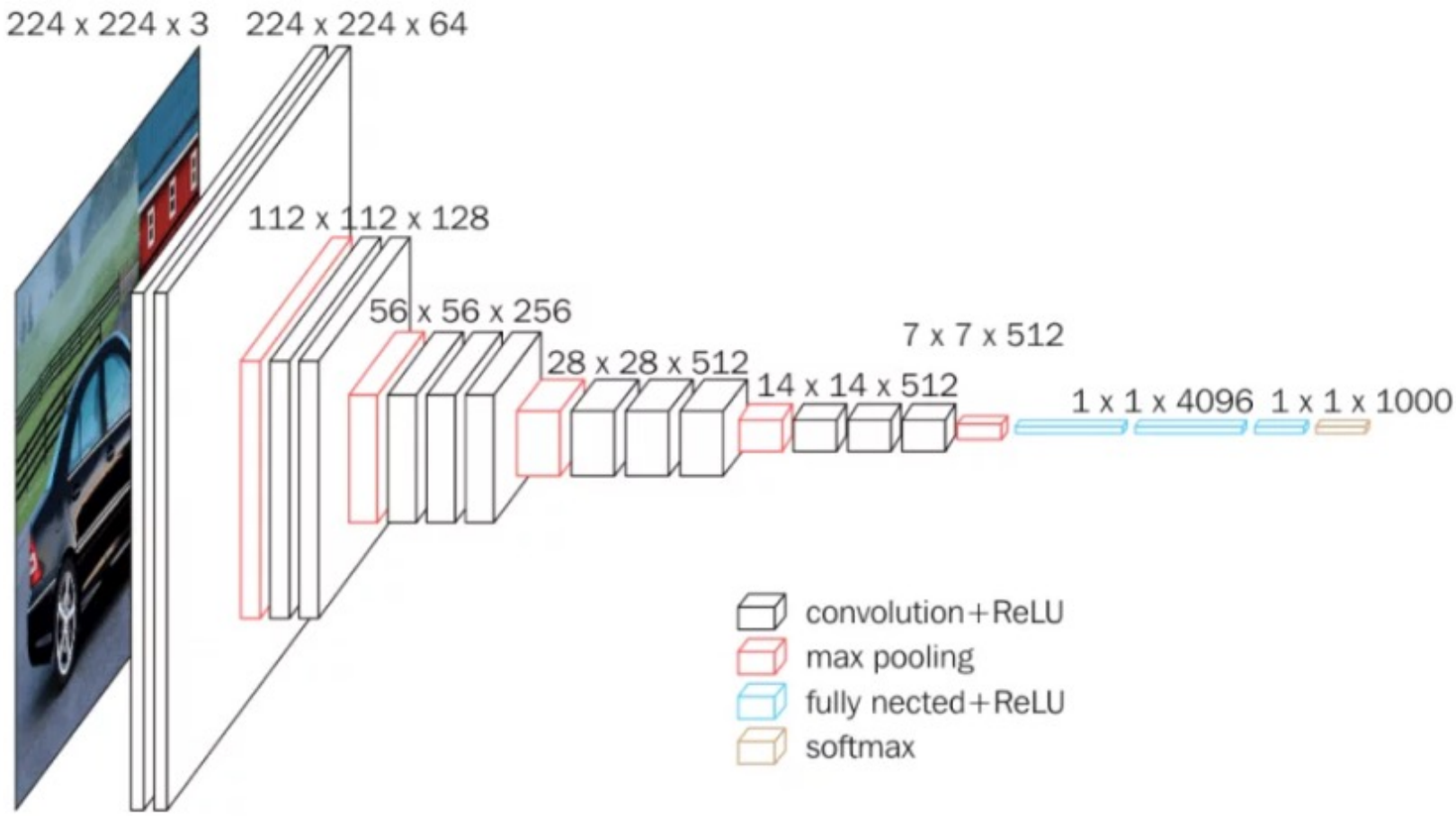
$120.00
+ $55.06 shipping

This item ships to **Cameroon. Get it by Friday, June 14 - Wednesday, June 19** Choose this

Artificial Intelligence (AI) has developed very quickly because it found its uses in almost every aspect of our lives. Basically, because **AI tools are very accessible**



VGG16 – Convolutional Network for Classification and Detection



Future of AI and Ethical Considerations

The future of Artificial Intelligence (AI) holds immense potential for transformative advancements across various domains. However, it also raises significant ethical concerns and challenges. Here's a look at both aspects:

Future of AI:

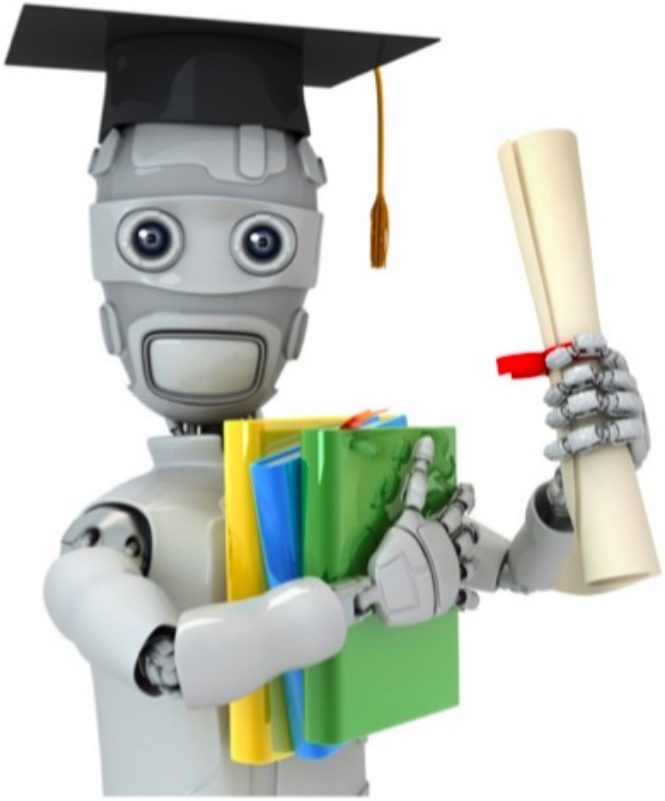
1. **Automation and Efficiency**: AI will continue to revolutionize industries by automating routine tasks and processes, leading to increased efficiency, reduced costs, and improved productivity.
2. **Healthcare**: AI will play a pivotal role in healthcare, aiding in early disease detection, personalized medicine, and drug discovery. AI-driven medical imaging, diagnostics, and robotic surgery will become more common.
3. **Autonomous Systems**: AI-powered autonomous vehicles, drones, and robots will continue to advance, revolutionizing transportation, delivery, and manufacturing industries.
4. **Natural Language Processing (NLP)**: NLP models will become even more sophisticated, enabling better chatbots, language translation, content generation, and more accurate sentiment analysis.
5. **AI in Education**: AI will reshape education through personalized learning experiences, intelligent tutoring systems, and adaptive educational content.
6. **AI for Sustainability**: AI can help address environmental challenges, such as climate change and resource conservation, by optimizing energy consumption, monitoring ecosystems, and predicting natural disasters.
7. **AI Ethics and Governance**: There will be increased focus on AI ethics, fairness, and accountability, with the development of AI governance frameworks and regulatory measures.

Future of AI and Ethical Considerations

Ethical Considerations:

1. **Bias and Fairness**: AI algorithms can inherit biases present in training data, leading to unfair and discriminatory outcomes. Addressing bias and ensuring fairness in AI systems is a critical ethical concern.
2. **Privacy**: AI systems that process personal data raise privacy concerns. Protecting individuals' data and ensuring transparency in data usage are essential ethical considerations.
3. **Job Displacement**: The automation of tasks by AI can lead to job displacement. Ensuring a just transition for affected workers and creating new opportunities is an ethical challenge.
4. **Transparency and Accountability**: AI systems often operate as "black boxes," making it challenging to understand their decisions. Ensuring transparency and accountability in AI decision-making is crucial.
5. **Autonomous Weapons**: The development of AI-powered autonomous weapons raises concerns about the ethical implications of using AI in warfare and the potential for loss of human control.
6. **AI in Surveillance**: The use of AI in surveillance, facial recognition, and monitoring can infringe on civil liberties and privacy rights, sparking ethical debates.
7. **AI in Healthcare Ethics**: Ethical dilemmas in healthcare AI include issues related to patient data privacy, medical decision-making, and the role of AI in end-of-life care.
8. **Existential Risk**: Long-term ethical concerns surround the potential for AI systems to surpass human intelligence and pose existential risks. Ensuring safe AI development is paramount.

Addressing these ethical challenges and shaping the future of AI in a responsible and ethical manner will require collaboration between technologists, policymakers, ethicists, and the broader society. Ethical considerations must be integrated into AI research, development, and deployment processes to ensure that AI benefits humanity while minimizing potential harms.



Machine Learning

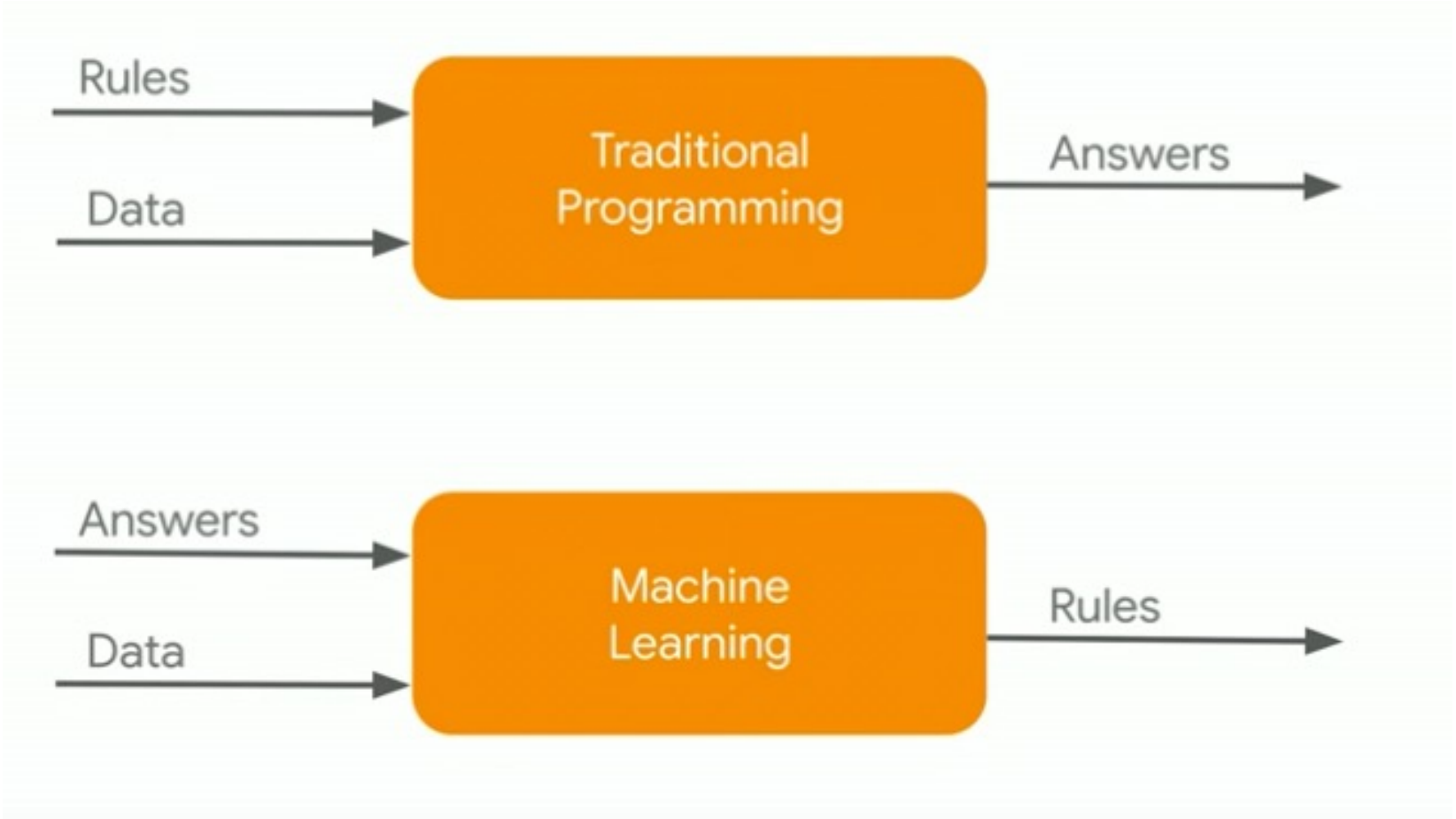
Introduction

What is machine learning

Machine Learning definition

- Arthur Samuel (1959). Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed.
- Tom Mitchell (1998) Well-posed Learning Problem: A computer program is said to *learn* from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

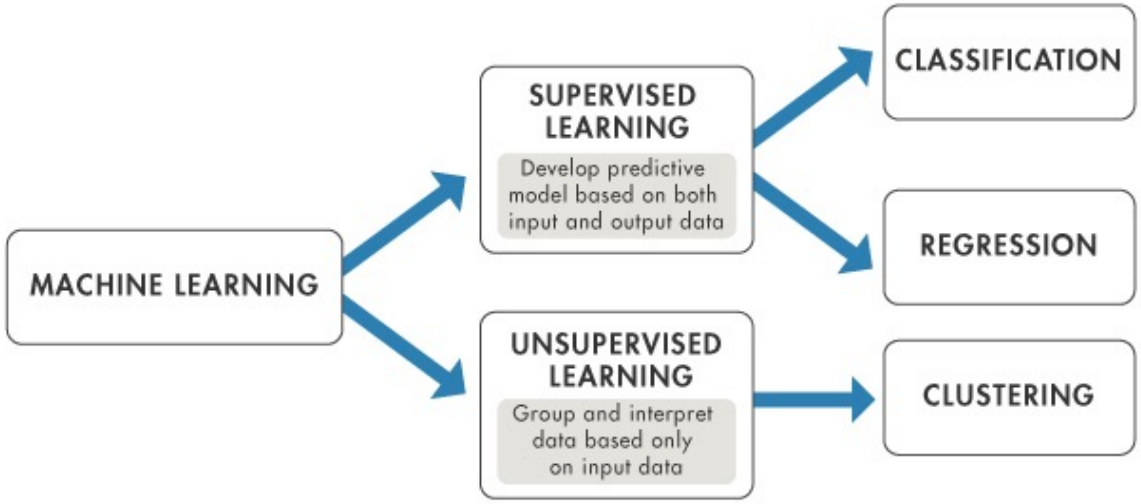
Programming Paradigms

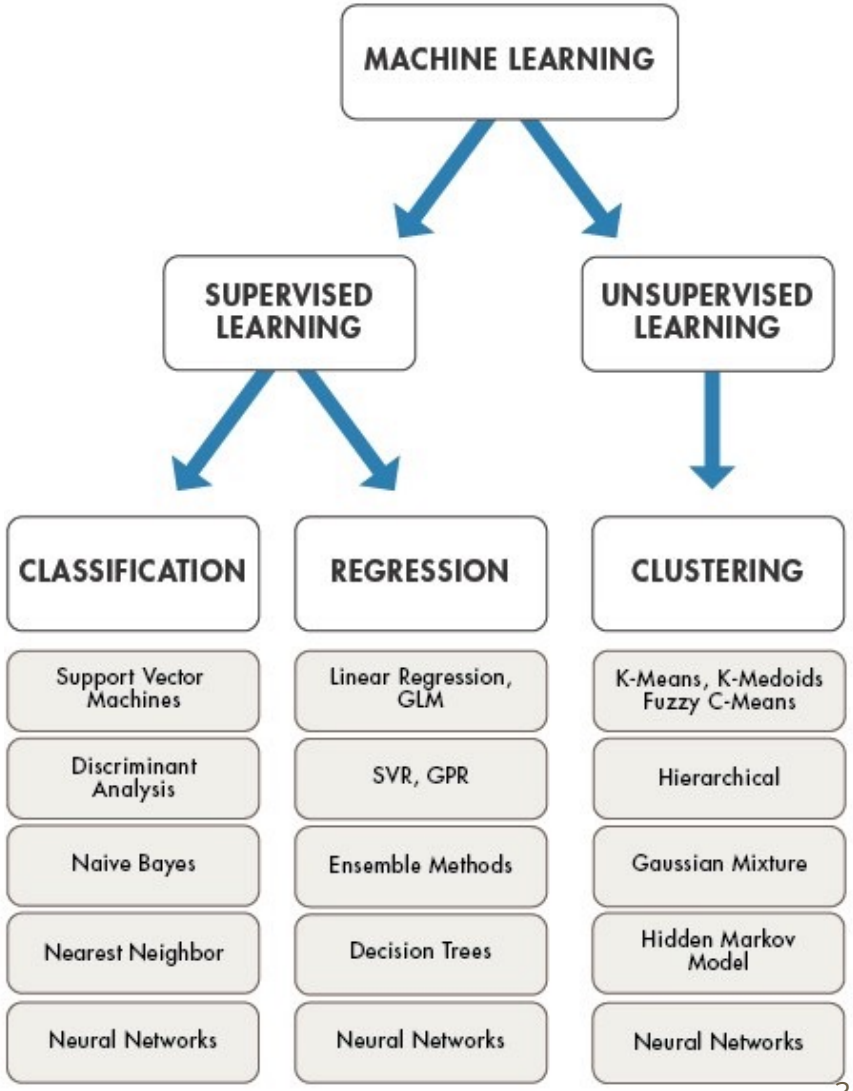


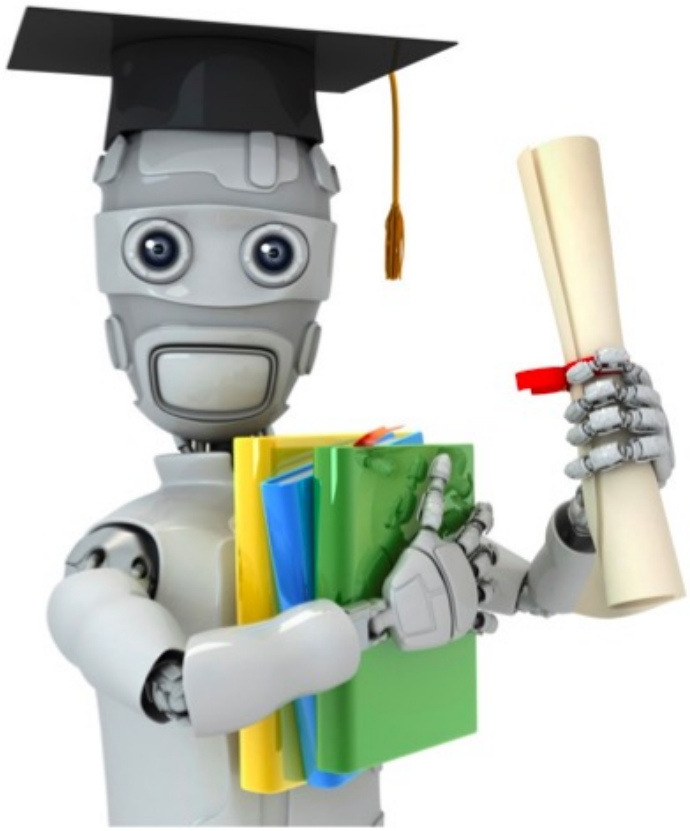
AI/ML Algorithms

If your problem is...	Start with these algorithms...
Simple linear regression / classification	Linear/Logistic Regression
Tabular data, need interpretability	Decision Trees, Random Forest
Tabular data, need maximum accuracy	Gradient Boosting (XGBoost, LightGBM)
Image/Video data	Convolutional Neural Networks (CNNs)
Text/Sequence data	Transformers (e.g., BERT, GPT families), LSTMs (legacy)
Finding groups in unlabeled data	K-Means, DBSCAN
Visualizing high-dim data	PCA (linear), t-SNE/UMAP (non-linear)
Sequential decision-making (e.g., games, robots)	Reinforcement Learning (PPO, DQN)

BASIC LEARNING PARADIGMS





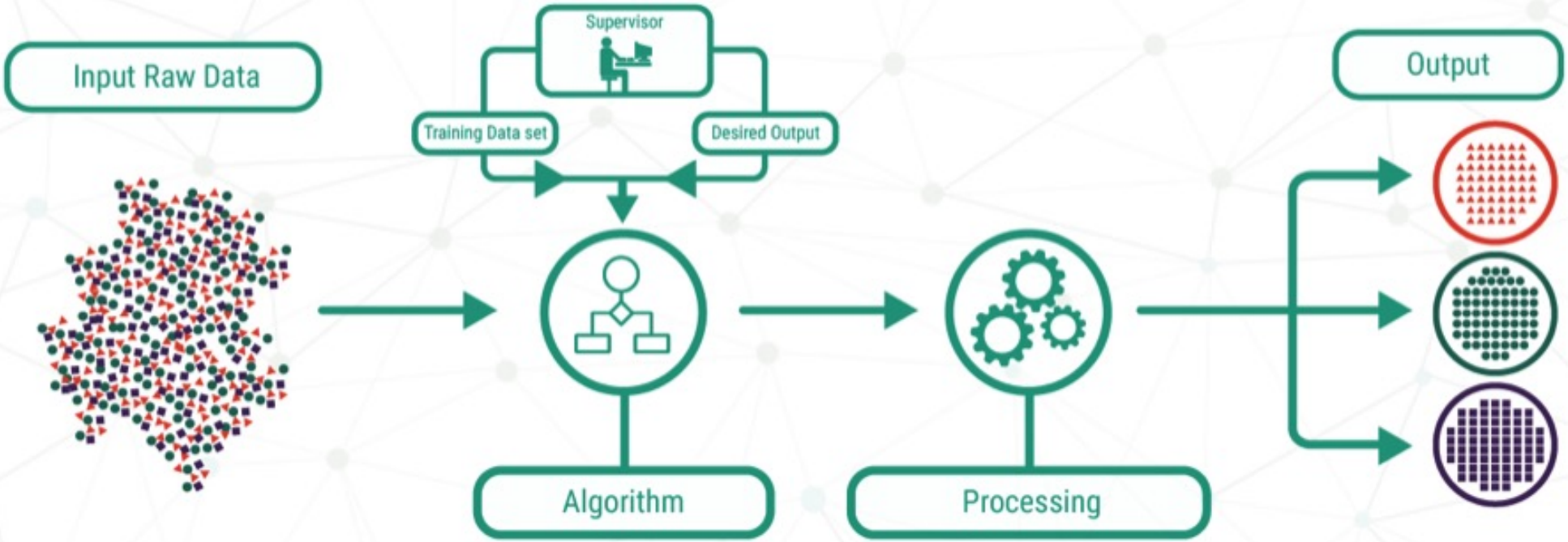


Machine Learning

Introduction

Supervised Learning

SUPERVISED LEARNING



Core Idea: Learn a mapping function from input (X) to output (Y) using a dataset of labeled examples.

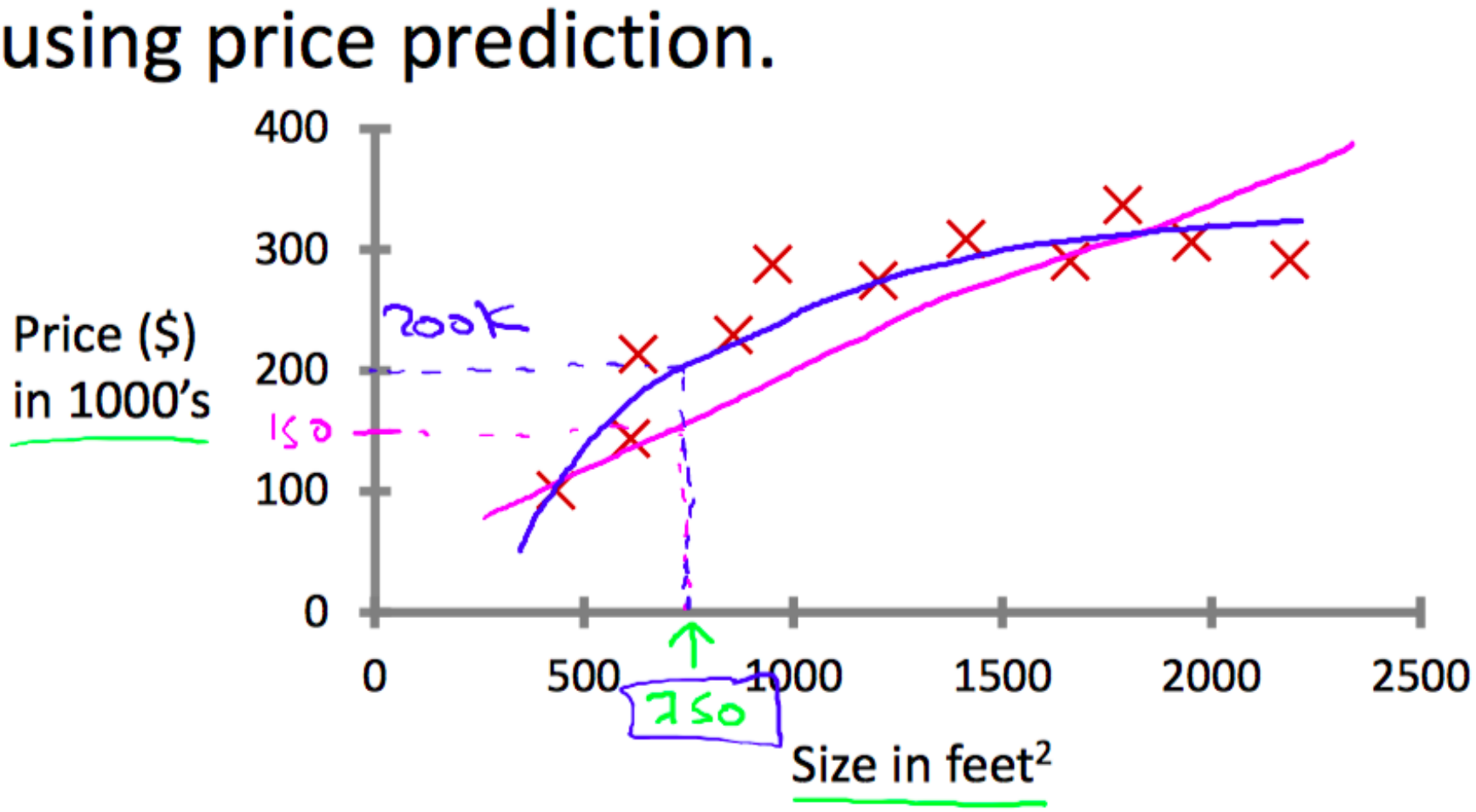
The "supervisor" is the labeled data that provides the correct answer during training.

Analogy: A student learning with a textbook that has the answer key.

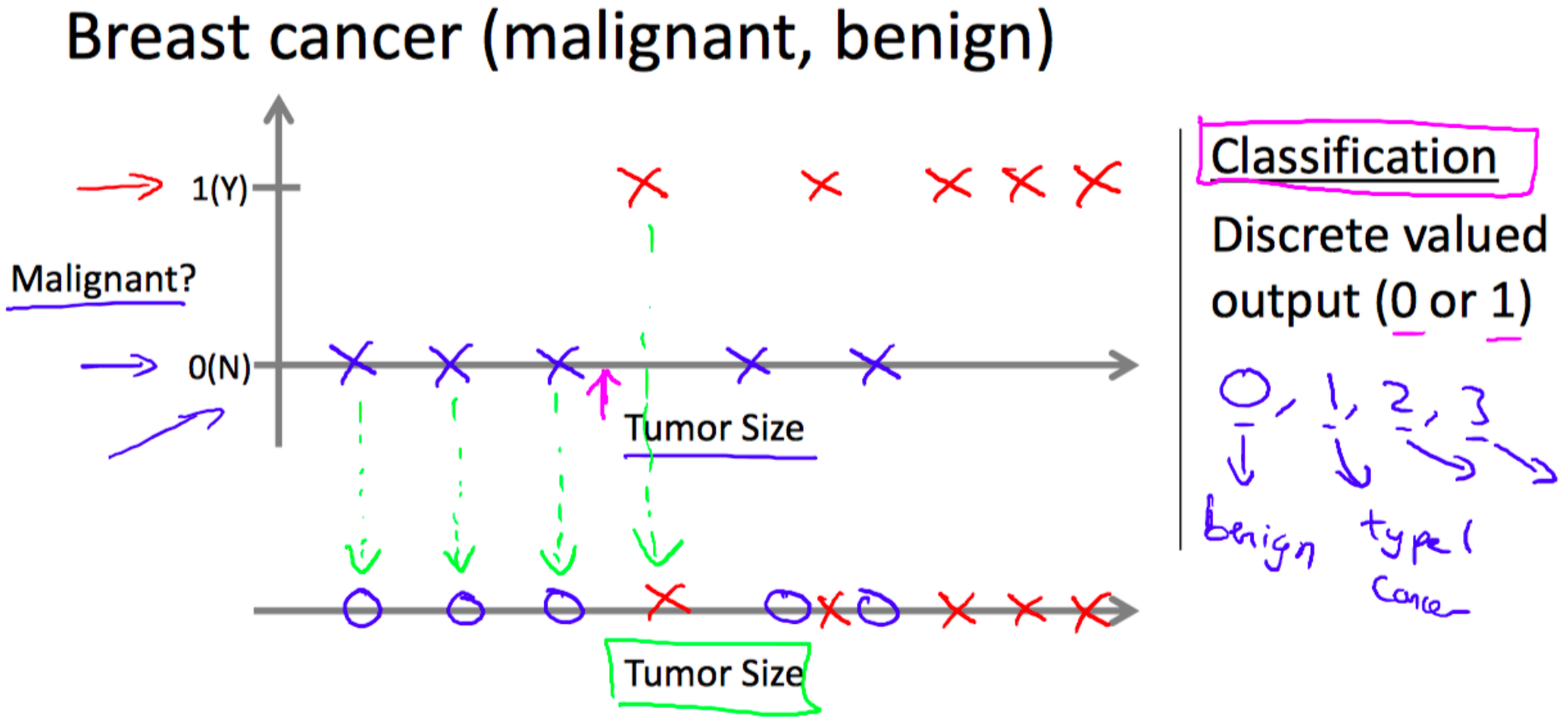
- **How it Works:**
  1. You provide a dataset: `{Input, Correct Output}` pairs.
     - *Example: {Email text, "spam" or "not spam"}, {House features, price}, {Image of digit, "7"}.*
  2. The algorithm makes a prediction, compares it to the true label, calculates the error (loss),
  3. and adjusts its internal parameters to minimize that error.
  3. This repeats until performance is satisfactory.

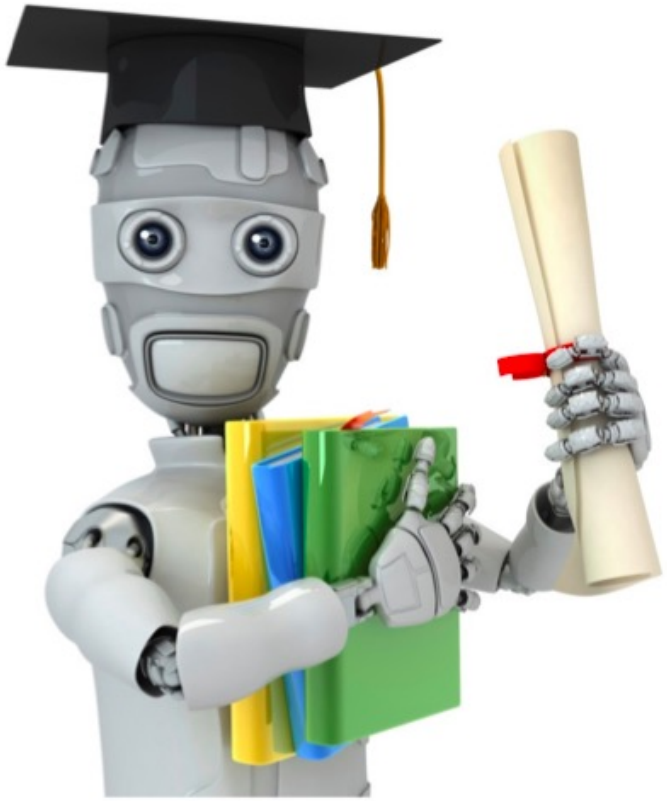
- **Common Tasks:**
  - **Classification:** Predicting a discrete category (spam/not spam, disease type, object name).
  - **Regression:** Predicting a continuous numerical value (house price, stock price, temperature).
- **Key Algorithms:**
  - Linear/Logistic Regression
  - Support Vector Machines (SVMs)
  - Decision Trees, Random Forests
  - Most Deep Neural Networks (when used with labeled data)
- **Strengths:** Highly effective for well-defined tasks with good labeled data. Performance is straightforward to measure.
- **Weaknesses: Requires large, high-quality labeled datasets**, which are expensive and time-consuming to create. Can only learn what it has seen labels for.

Regression


Housing price prediction.
Price ($) in 1000's
400
300
200
100
0
200K
150
0
500
1000
1500
2000
2500
750
Size in feet²

Classification



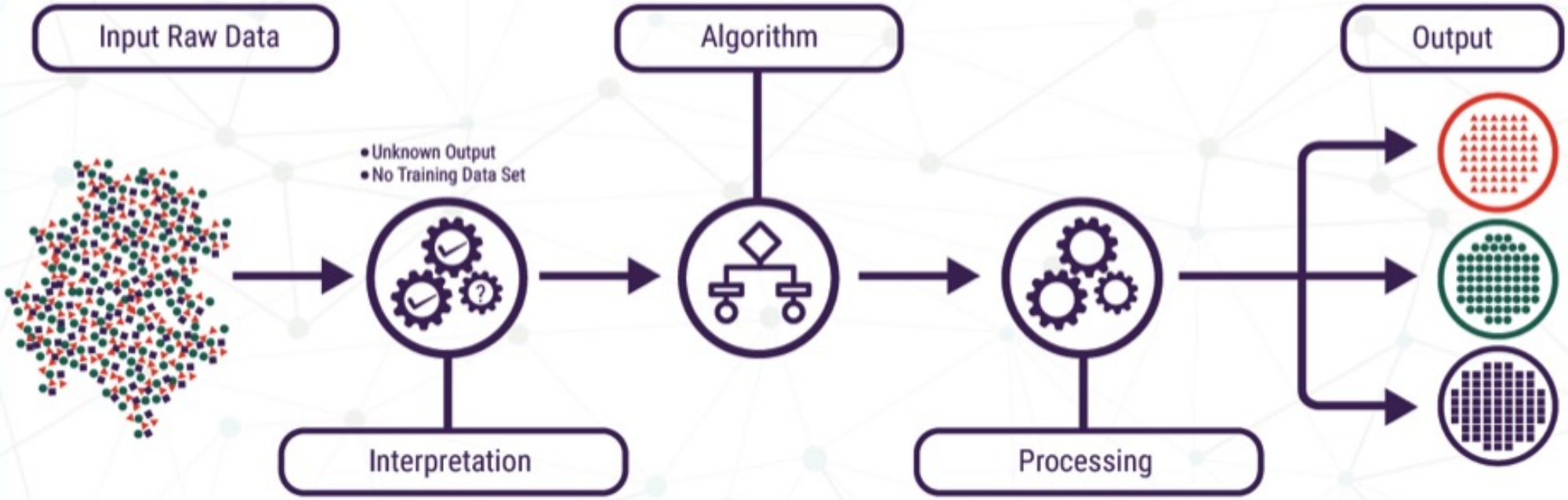


Machine Learning

Introduction

Unsupervised Learning

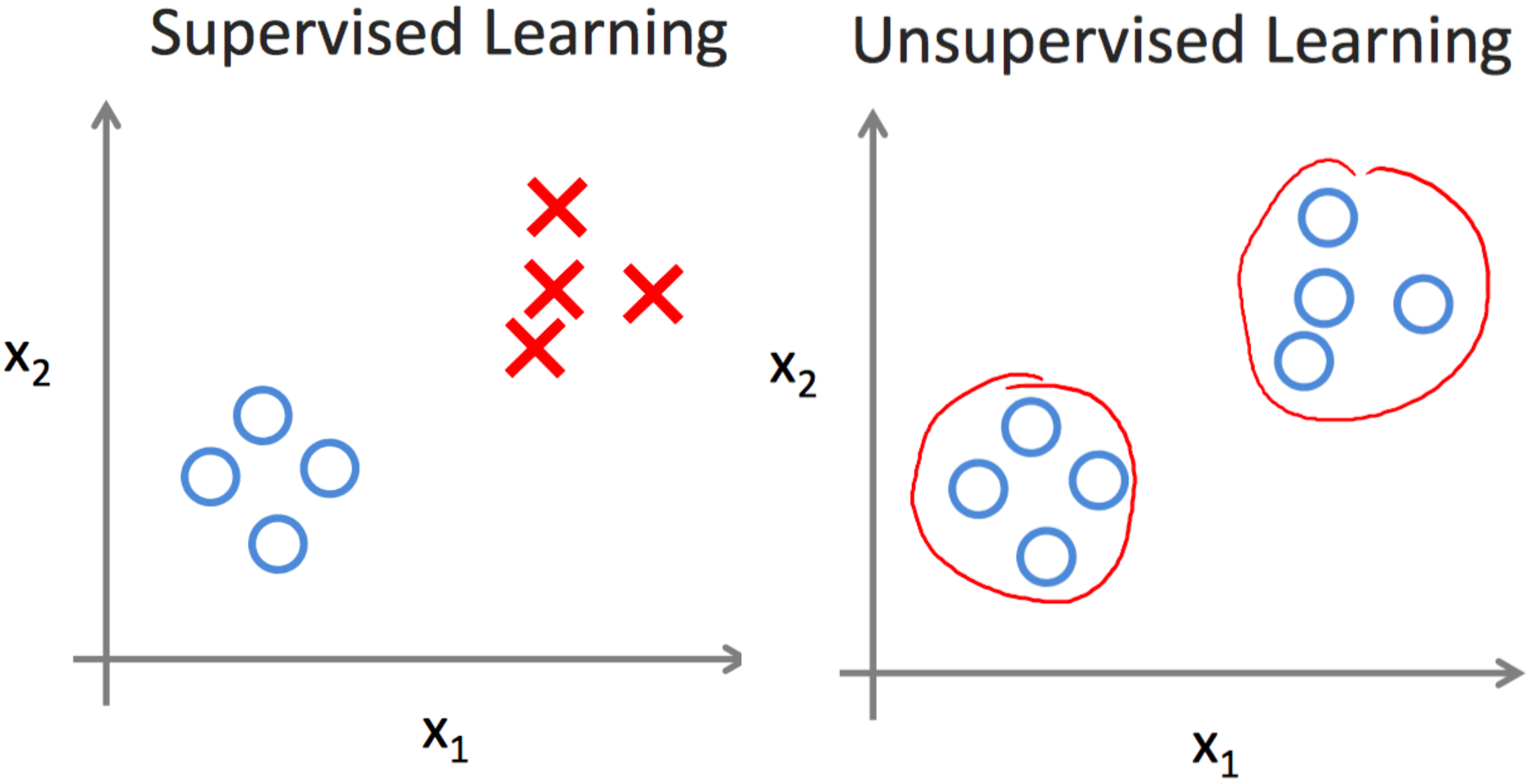
UNSUPERVISED LEARNING

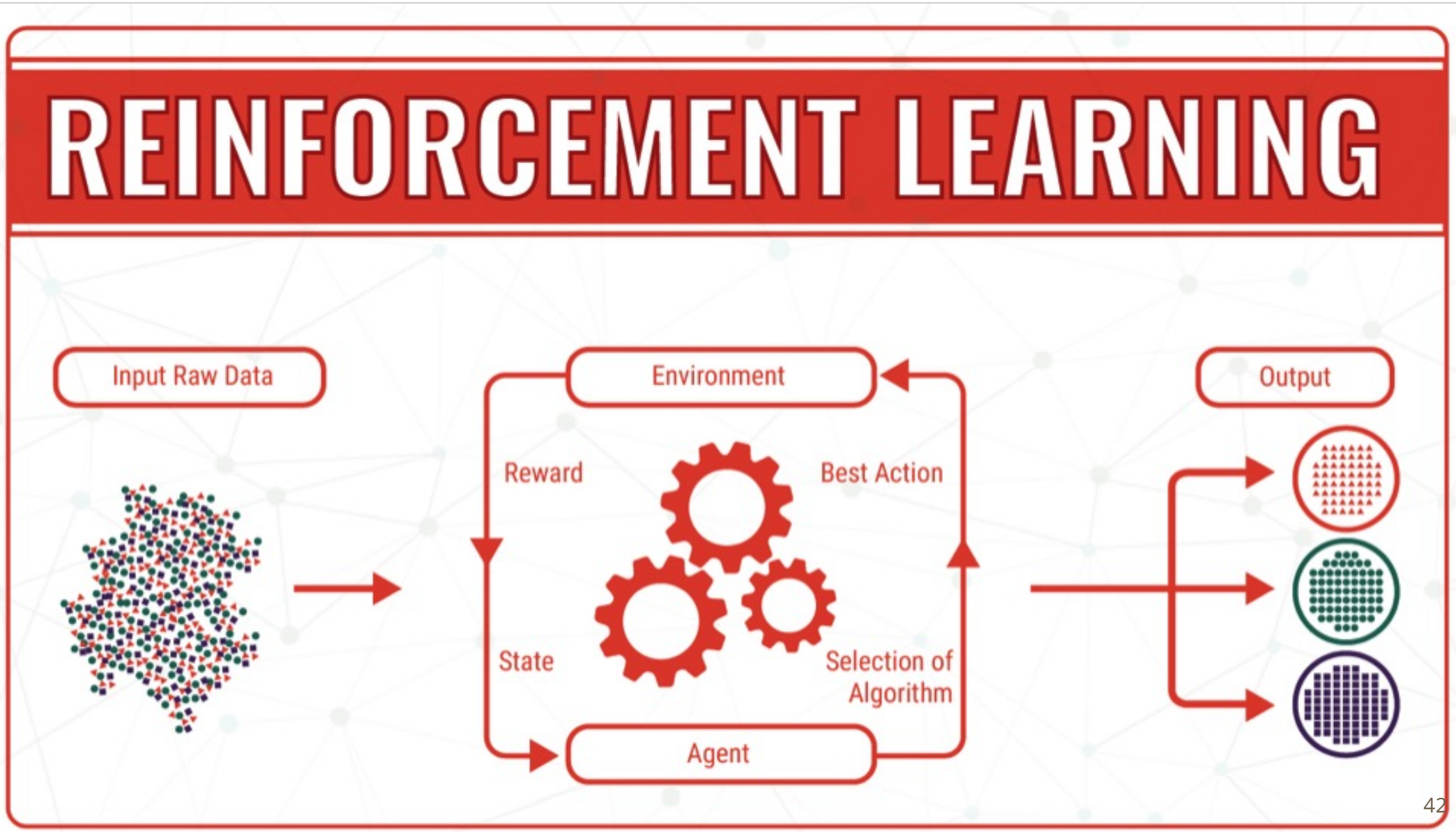


Core Idea: Discover inherent patterns, groupings, or structures in input data (X) without any labeled outputs

- The algorithm explores the data on its own.
  Analogy: A student given a stack of unorganized documents and asked to find natural categories or themes.
- **How it Works:** The algorithm analyzes the data's features to find similarities, differences, or compact representations without guidance on what to look for.
- **Common Tasks:**
  - **Clustering:** Grouping similar data points (customer segmentation, organizing document topics).
  - **Dimensionality Reduction:** Simplifying data while preserving its essence (visualizing high-dimensional data, feature compression). e.g., PCA, t-SNE.
  - **Anomaly Detection:** Identifying rare or unusual items (fraud detection, system failure).
  - **Association Rule Learning:** Discovering rules that describe relationships in data (market basket analysis: "customers who buy X also buy Y").

- **Key Algorithms:**
  - K-Means, Hierarchical Clustering
  - Principal Component Analysis (PCA)
  - Autoencoders (neural networks for compression)
  - Generative Models (like GANs - in part)
- **Strengths:** Works on raw, unlabeled data—which is abundant. Can reveal surprising, novel insights.
- **Weaknesses:** Results are often **harder to evaluate objectively** (no ground truth). The "meaning" of discovered patterns must be interpreted by humans.


Supervised Learning
x_2
x_1
Unsupervised Learning
x_2
x_1


REINFORCEMENT LEARNING
Input Raw Data
Environment
Reward
Best Action
State
Selection of Algorithm
Agent
Output

Core Idea: An agent learns to make decisions by interacting in an environment to maximize a cumulative reward signal. It learns from trial, error, and delayed feedback.

Analogy: Training a dog with treats (rewards) and reprimands (penalties), or learning to play a game by playing it repeatedly.

- **How it Works:**
  - The agent observes the **state** of the environment.
  - It takes an **action**.
  - It receives a **reward** (or penalty) and a new state.
  - The agent's goal is to learn a **policy** (a strategy) that maximizes long-term total reward.
- **Common Tasks:**
  - Game Playing (AlphaGo, Dota 2 bots)
  - Robotics (teaching a robot to walk or grasp)
  - Autonomous Systems (self-driving car control, resource management)
  - Real-time Strategy & Optimization

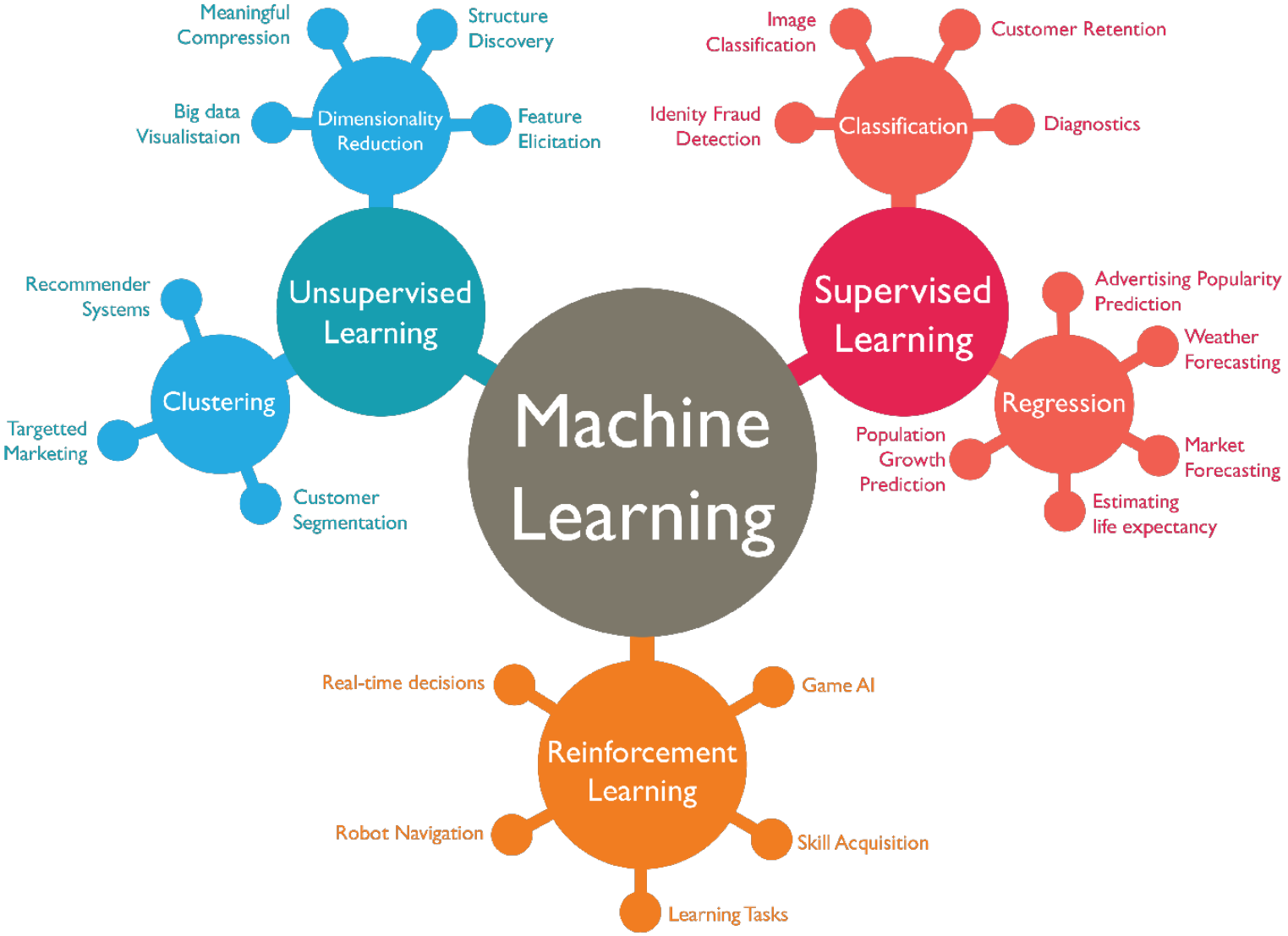
Reinforcement Learning(RL) is a type of machine learning technique that enables an agent to learn in an interactive environment by trial and error using feedback from its own actions and experiences.

Reinforcement learning

- **Key Algorithms:**
  - Q-Learning, Deep Q-Networks (DQN)
  - Policy Gradient Methods (REINFORCE, PPO)
  - Actor-Critic Methods

- **Strengths:** Ideal for sequential decision-making problems. Can master complex strategies beyond human capability.

- **Weaknesses:** Can be **extremely data-inefficient** (requires millions of simulations). Reward function design is critical and difficult. Prone to instability during training.


Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Population Growth Prediction
Market Forecasting
Estimating life expectancy
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

Fundamental **learning paradigms** in Machine Learning (ML)

Learning Type	Data Requirement	Feedback	Goal
Supervised	Labeled (X, Y)	Direct & Immediate	Learn mapping from input to known output
Unsupervised	Unlabeled (X)	None	Discover hidden structure in data
Reinforcement	Environment & Reward Signal	Delayed & Scalar (Reward/Penalty)	Learn optimal policy for sequential decisions
Semi-Supervised	Mostly Unlabeled, Some Labeled	Partial	Leverage abundant unlabeled data
Self-Supervised	Unlabeled (X)	Automatically Generated	Learn rich representations for downstream tasks

Other Hybrid learning paradigms in Machine Learning (ML),
a) Semi-Supervised Learning

- **Idea:** Uses a **small amount of labeled data** with a **large amount of unlabeled data**. This is a pragmatic middle ground, as labeled data is scarce but unlabeled data is cheap.
- **How:** The model uses the labeled data to learn initial patterns and then uses the unlabeled data to refine its understanding of the data's overall structure.
- **Example:** Google Photos identifying people: you label a few photos of "John," and the system uses unlabeled photos to find other images of John.

b) Self-Supervised Learning

- **Idea:** A **subset of unsupervised learning** where the data automatically generates its own labels. The system creates a **pretext task** to learn powerful representations.
- **How:** For example, in text, a model is trained to predict a missing word in a sentence (like BERT). In images, a model learns by predicting missing parts of an image or the rotation angle of an image. The "label" is derived from the data itself.
- **Significance:** This is the **secret sauce behind modern Large Language Models (LLMs like GPT)**. They are trained to predict the next word on a vast corpus of unlabeled text.

c) Transfer Learning

- **Idea:** Leverage knowledge gained while solving **one problem** and apply it to a **different but related problem**.
- **How:** A model pre-trained on a huge, general dataset (e.g., ImageNet for vision) is taken, and its final layers are fine-tuned on a smaller, specific dataset (e.g., medical X-rays).
- **Benefit:** Dramatically reduces data and computation requirements for new tasks. The foundation model provides a "world understanding."

d) Federated Learning

- **Idea:** Train an algorithm **across multiple decentralized devices (like phones)** holding local data samples, **without exchanging the data itself**. Only model updates are shared.
- **Benefit:** Enhances **privacy** and security, reduces communication costs. Used by Google for improving keyboard predictions without sending your typing data to the cloud.

Key Evolution: FOUNDATION MODELS

- The field is moving from pure **Supervised Learning** (which is bottlenecked by labels) towards **Self-Supervised** and **Foundation Models**, which learn general-purpose "world models" from vast unlabeled data, and then use **Transfer Learning** to adapt efficiently to thousands of specific tasks. This shift is what has powered the recent AI revolution.
- Foundation Models represent one of the most significant paradigm shifts in modern AI. They are not just another algorithm, but a new architectural and methodological approach that is reshaping the entire field.
- A **Foundation Model** is a large-scale AI model (typically based on a neural network architecture like a Transformer) that is pre-trained on vast, broad, and diverse datasets (often internet-scale text, images, audio, or multi-modal data) using self-supervised learning. It serves as a foundational base that can be adapted (via fine-tuning, prompting, etc.) to a wide variety of downstream tasks, often with minimal task-specific data.

1. **Scale:** They are defined by immense size in three dimensions:
    1. **Data Scale:** Trained on terabytes to petabytes of data (e.g., most of the public internet).
    2. **Parameter Scale:** Billions to trillions of learnable parameters (GPT-3: 175B, GPT-4: ~1.7T, Claude 3: undisclosed but massive).
    3. **Compute Scale:** Require massive computational resources (thousands of GPUs/TPUs for weeks/months).
2. **Generality:** They are not built for a single task. Instead, they develop a broad, **emergent understanding** of language, vision, or the physical world during pre-training. This general capability can be *adapted* to specific uses.
3. **Adaptability (The "Foundation" Property):** A single foundation model can be the starting point for hundreds of different applications. This is achieved through:
    1. **Fine-tuning:** Updating the model's weights on a smaller, task-specific dataset.
    2. **Prompting/In-Context Learning:** Providing instructions and examples within the input prompt itself, **without changing the model's weights** (zero-shot, one-shot, few-shot learning).
    3. **Retrieval-Augmented Generation (RAG):** Augmenting the model with external knowledge.

Key Enabling Technologies

1. **Transformer Architecture:** The neural network backbone that made this scale possible. Its self-attention mechanism allows it to handle long-range dependencies in data (e.g., relationships between words far apart in a text) efficiently and in parallel.
2. **Self-Supervised Learning (SSL):** The pre-training objective that doesn't require human-labeled data. The model learns by predicting hidden parts of its input data.
    1. **For Text (Language Models):** Predict the next word (autoregressive, like GPT) or a masked word (autoencoding, like BERT).
    2. **For Images:** Predict missing patches or learn by contrasting different augmented views of an image.
3. **Unprecedented Compute & Data:** The availability of large-scale cloud computing and massive, scrapable digital corpora (the internet) provided the fuel.

FOUNDATION MODELS

Modality	Description	Examples
Language (LLMs)	Trained on text. Understand and generate human language.	GPT-4, Claude 3, LLaMA 3, Gemini
Vision	Trained on images/videos. Understand and generate visual content.	DALL-E 3, Stable Diffusion, Midjourney, CLIP
Multi-Modal	Trained on multiple data types (text, images, audio, video). Can process and generate across modalities.	GPT-4V (Vision), Gemini 1.5 Pro, Sora (video)
Scientific	Trained on domain-specific data (code, biology, chemistry, physics).	AlphaFold (proteins), Codex/Github Copilot (code)

Foundations

- 1. Prerequisites
  - Python basics (functions, loops, classes, modules)
  - NumPy (arrays, vectorized operations)
  - Pandas (dataframes, preprocessing)
  - Matplotlib/Seaborn (basic visualization)
- Setup
  - Install sklearn: pip install scikit-learn
  - Explore official documentation: Scikit-learn Docs

1. Foundations

- Datasets
  - Learn to load built-in datasets (load_iris, load_digits, etc.)
  - Work with external datasets using Pandas
- Data Preprocessing
  - Handling missing values (SimpleImputer)
  - Scaling and normalization (StandardScaler, MinMaxScaler)
  - Encoding categorical variables (OneHotEncoder, LabelEncoder)
  - Train-test split (train_test_split)

2. SUPERVISED LEARNING

- Classification
  - Logistic Regression
  - Decision Trees
  - Random Forests
  - Support Vector Machines (SVM)
  - k-Nearest Neighbors (KNN)
  - Naïve Bayes
- Regression
  - Linear Regression
  - Ridge/Lasso Regression
  - Decision Tree Regression
  - Random Forest Regression

3. UNSUPERVISED LEARNING

- Clustering
  - K-Means
  - Hierarchical Clustering
  - DBSCAN
- Dimensionality Reduction
  - Principal Component Analysis (PCA)
  - t-SNE (via sklearn.manifold)

4. MODEL EVALUATION AND SELECTION

- Performance metrics
  - Classification: accuracy, precision, recall, F1-score, ROC-AUC
  - Regression: MSE, RMSE, MAE, R^2
- Cross-validation (cross_val_score)
- Hyperparameter tuning (GridSearchCV, RandomizedSearchCV)

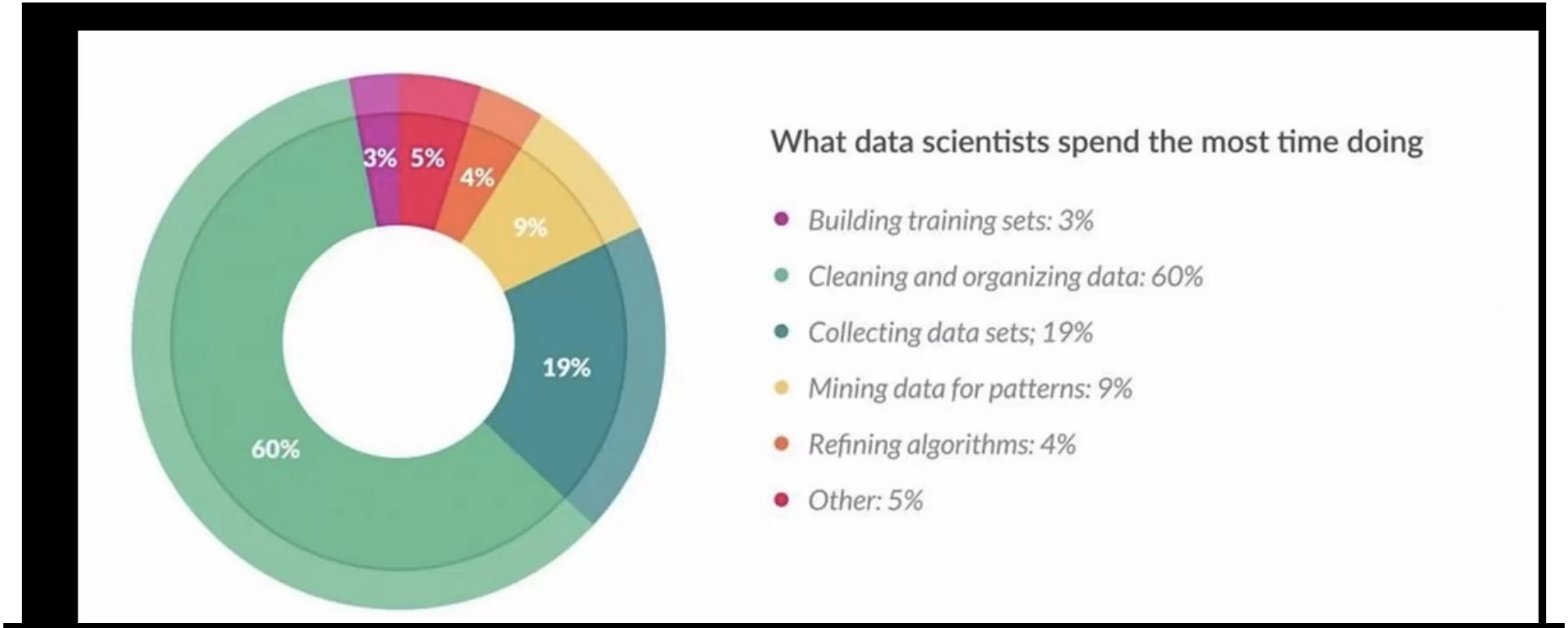
5. Advanced Topics

- Pipelines
  - Automate preprocessing + modeling (Pipeline)
- Feature Engineering
  - Polynomial features, feature selection (SelectKBest)
- Ensemble Methods
  - Bagging, Boosting (AdaBoost, Gradient Boosting, XGBoost integration)
- Model Persistence
  - Save/load models (joblib, pickle)

6. APPLICATION

- Text classification (using CountVectorizer, TfidfVectorizer)
- Image classification (digits dataset, feature extraction)
- Time series forecasting (with sklearn + external libraries)
- Deploying models (Flask, FastAPI, or Streamlit)

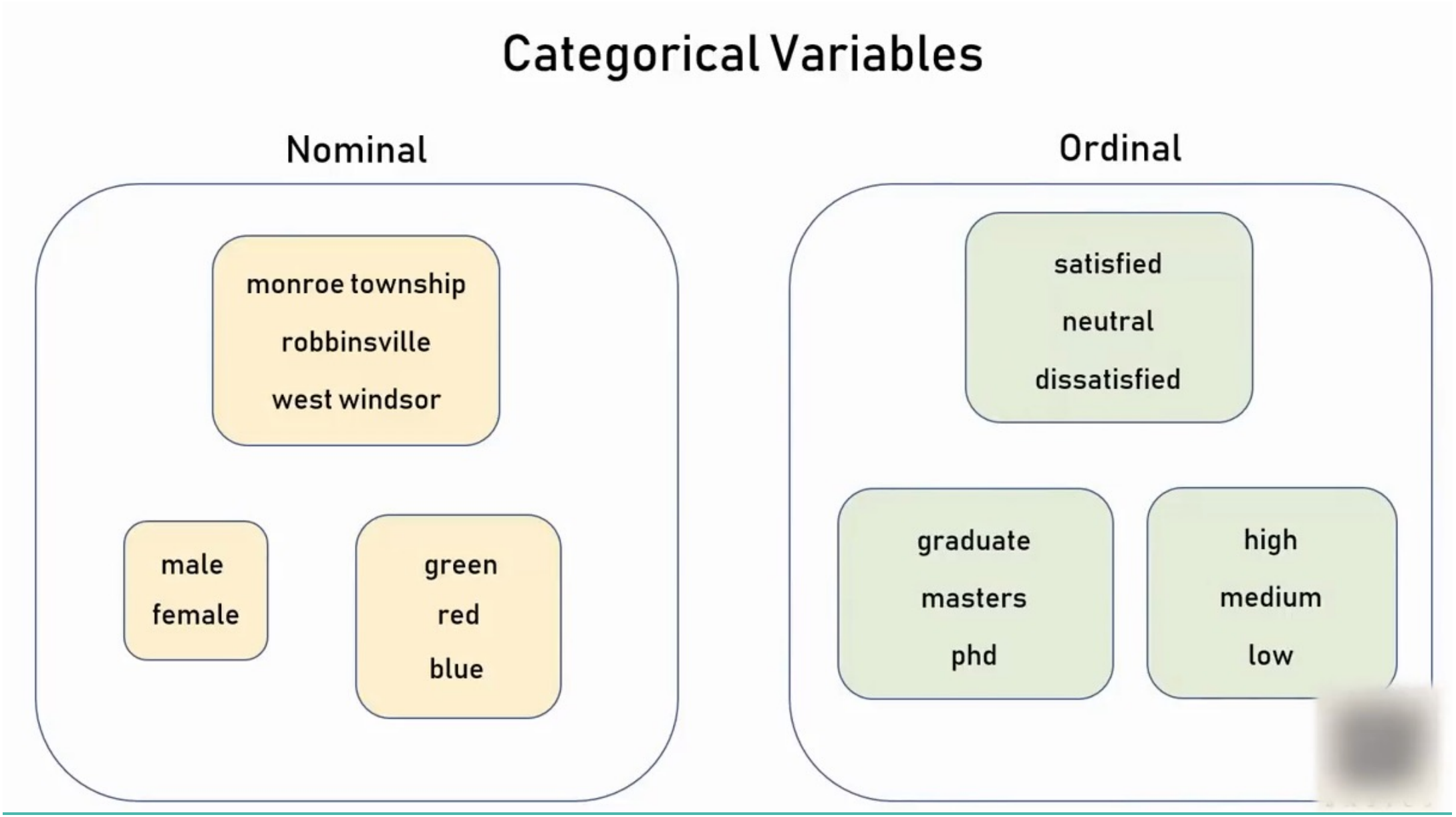
FEATURE ENGINEERING



Outlier Detection, one hot encoding, handling missing values are few basic examples of feature engineering

FEATURE ENGINEERING / DATA PREPROCESSING

- Handling Missing Values
- Label / One Hot Encoding
- Outlier Detection
- Data Splitting


Categorical Variables
Nominal
monroe township
robbinsville
west windsor
male
female
green
red
blue
Ordinal
satisfied
neutral
dissatisfied
graduate
masters
phd
high
medium
low

town	area	price
monroe township	2600	550000
monroe township	3000	565000
monroe township	3200	610000
monroe township	3600	680000
monroe township	4000	725000
west windsor	2600	585000
west windsor	2800	615000
west windsor	3300	650000
west windsor	3600	710000
robbinsville	2600	575000
robbinsville	2900	600000
robbinsville	3100	620000
robbinsville	3600	695000

Build a predictor function to predict price of a home,

1) With 3400 sqr ft area in west windsor

2) 2800 sqr ft home in robbinsville

town	area	price
monroe township	2600	550000
monroe township	3000	565000
monroe township	3200	610000
monroe township	3600	680000
monroe township	4000	725000
west windsor	2600	585000
west windsor	2800	615000
west windsor	3300	650000
west windsor	3600	710000
robbinsville	2600	575000
robbinsville	2900	600000
robbinsville	3100	620000
robbinsville	3600	695000

town	area	price
1	2600	550000
1	3000	565000
1	3200	610000
1	3600	680000
1	4000	725000
2	2600	585000
2	2800	615000
2	3300	650000
2	3600	710000
3	2600	575000
3	2900	600000
3	3100	6
3	3600	6

Monroe township = 1, West Windsor = 2, Robbinsville = 3

Monroe township < West Windsor < Robbinsville ??

Or

Monroe township + West Windsor = Robbinsville ??

```
Entrée [1]: import pandas as pd
```

```
Entrée [2]: df = pd.read_csv("homeprices.csv")
            df
```

Out[2]:

	town	area	price
0	monroe township	2600	550000
1	monroe township	3000	565000
2	monroe township	3200	610000
3	monroe township	3600	680000
4	monroe township	4000	725000
5	west windsor	2600	585000
6	west windsor	2800	615000
7	west windsor	3300	650000
8	west windsor	3600	710000
9	robinsville	2600	575000
10	robinsville	2900	600000
11	robinsville	3100	620000
12	robinsville	3600	695000

GET DUMMIES

town	area	price	monroe township	west windsor	robbinsville
monroe township	2600	550000	1	0	0
monroe township	3000	565000	1	0	0
monroe township	3200	610000	1	0	0
monroe township	3600	680000	1	0	0
monroe township	4000	725000	1	0	0
west windsor	2600	585000	0	1	0
west windsor	2800	615000	0	1	0
west windsor	3300	650000	0	1	0
west windsor	3600	710000	0	1	0
robbinsville	2600	575000	0	0	1
robbinsville	2900	600000	0	0	1
robbinsville	3100	620000	0	0	1
robbinsville	3600	695000	0	0	1

Using pandas to create dummy variables

Entrée [3]:
```
dummies = pd.get_dummies(df.town)
dummies
```

Out[3]:

	monroe township	robinsville	west windsor
0	1	0	0
1	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0
5	0	0	1

PRACTICALS - ANACONDA -> SPYDER

NumPy

NumPy is the fundamental package for scientific computing with Python. It contains among other things:

- a powerful N-dimensional array object
- sophisticated (broadcasting) functions
- tools for integrating C/C++ and Fortran code
- useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined. This allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

NumPy is licensed under the BSD license, enabling reuse with few restrictions.

Panda

pandas is an open source, BSD-licensed library, providing high-performance, easy-to-use data structures and data analysis tools for the Python programming language. It is built on top on **NumPy**,It uses its multi-dimensional arrays and fast operations internally to provide higher level methods for manipulation and analysis

Seaborn

Seaborn is a Python library for making statistical visualizations. From our experience, they complement each other really well, and are worth learning together.

Scikit-Learn

`Scikit-learn`

Machine Learning in Python

- Simple and efficient tools for data mining and data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Indexing with iloc, loc and ix in pandas python

Indexing in pandas python is done mostly with the help of iloc, loc and ix. lets see an example of each .

iloc – iloc is used for indexing or selecting based on position .i.e. by row number and column number

loc – loc is used for indexing or selecting based on name .i.e. by row name and column name

ix – indexing can be done by both position and name using ix. (NOW DEPRECATED)

Ref:

Basic selection:
http://www.datasciencemadesimple.com/indexing-iloc-loc-ix-pandas-python/
Advanced selection:
https://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/

Python Pandas Selections and Indexing

.iloc selections - position based selection

data.iloc[<row selection], <column selection>]

Integer list of rows: [0,1,2]
Slice of rows: [4:7]
Single values: 1

Integer list of columns: [0,1,2]
Slice of columns: [4:7]
Single column selections: 1

loc selections - position based selection

data.loc[<row selection], <column selection>]

Index/Label value: 'john'
List of labels: ['john', 'sarah']
Logical/Boolean index: data['age'] == 10

Named column: 'first_name'
List of column names: ['first_name', 'age']
Slice of columns: 'first_name':'address'

Dataset Creation and Importation

```
1  import pandas as pd
2  import numpy as np
3
4  #Create a DataFrame
5  d = {
6      'Name':['Alisa','Bobby','Cathrine','Alisa','Bobby','Cathri
7              'Alisa','Bobby','Cathrine','Alisa','Bobby','Cathri
8      'Exam':['Semester 1','Semester 1','Semester 1','Semester 1
9              'Semester 2','Semester 2','Semester 2','Semester 2
10
11     'Subject':['Mathematics','Mathematics','Mathematics','Scie
12                'Mathematics','Mathematics','Mathematics','Scie
13    'Score':[62,47,55,74,31,77,85,63,42,67,89,81]}
14
15 df = pd.DataFrame(d,columns=['Name','Exam','Subject','Score'])
16 df
```

IO Tools (Text, CSV, HDF5, …)

The pandas I/O API is a set of top level `reader` functions accessed like **`pandas.read_csv()`** that generally return a pandas object. The corresponding `writer` functions are object methods that are accessed like **`DataFrame.to_csv()`**. Below is a table containing available `readers` and `writers`.

Format Type	Data Description	Reader	Writer
text	CSV	read_csv	to_csv
text	JSON	read_json	to_json
text	HTML	read_html	to_html
text	Local clipboard	read_clipboard	to_clipboard
binary	MS Excel	read_excel	to_excel
binary	HDF5 Format	read_hdf	to_hdf
binary	Feather Format	read_feather	to_feather
binary	Parquet Format	read_parquet	to_parquet
binary	Msgpack	read_msgpack	to_msgpack
binary	Stata	read_stata	to_stata
binary	SAS	read_sas	
binary	Python Pickle Format	read_pickle	to_pickle
SQL	SQL	read_sql	to_sql
SQL	Google Big Query	read_gbq	to_gbq

Indexing with iloc:

.iloc [1:m, 1:n] – is used to select or index rows based on their position from 1 to m rows and 1 to n columns

Select row by using row number in pandas with .iloc

```
# select first 2 rows
df.iloc[:2]
# or
df.iloc[:2,]
```

```
# select 1st and 4thcolumn
df.iloc[:,[0,3]]
```

Indexing with ix

Indexing with ix is the combination of indexing with iloc and loc. ix is used for indexing based on the position and as well by names.

```
# select value by row label and column label using loc

df.loc[[1,2,3,4,5],['Name','Score']]
```

View a column in pandas

```
# Get all values of column 'Score'

df.ix[:,'Score']
```

Indexing with ix

Indexing with ix is the combination of indexing with iloc and loc. ix is used for indexing based on the position and as well by names.

View a column in pandas

```
# Get all values of column 'Score'

df.ix[:,'Score']
```

View a row based on row numbers

```
# Get 3rd & 4th row

df.ix[4:5,]
```

Output:

	Name	Exam	Subject	Score
4	Bobby	Semester 1	Science	31
5	Cathrine	Semester 1	Science	77

View the value based on row and column number

```
df.ix[3,2]
```

Some Data sorting with .loc

1. When selecting multiple columns or multiple rows in this manner, remember that in your selection e.g.[1:5], the rows/columns selected will run from the first number to *one minus* the second number. e.g. [1:5] will go 1,2,3,4., [x,y] goes from x to y-1.
2. Note that .iloc returns a Pandas Series when one row is selected, and a Pandas DataFrame when multiple rows are selected, or if any column in full is selected. To counter this, pass a single-valued list if you require DataFrame output.
3. You can make more complex sorting

```
In [32]: data.loc[data['first_name'] == 'Erasmo', ['company_name', 'email', 'phone1']]
```

Out[32]:

	company_name	email	phone1
last_name			
Talentino	Active Air S		
Gath	Pan Optx		
Rhea	Martin Morr		

```
# Select rows with first name Antonio AND hotmail email addresses
data.loc[data['email'].str.endswith("gmail.com") & (data['first_name'] == 'Antonio')]


# select rows with id column between 100 and 200, and just return 'postal' and 'web' co
data.loc[(data['id'] > 100) & (data['id'] <= 200), ['postal', 'web']]
```

Handle missing data in your training dataset with SciKit Imputer

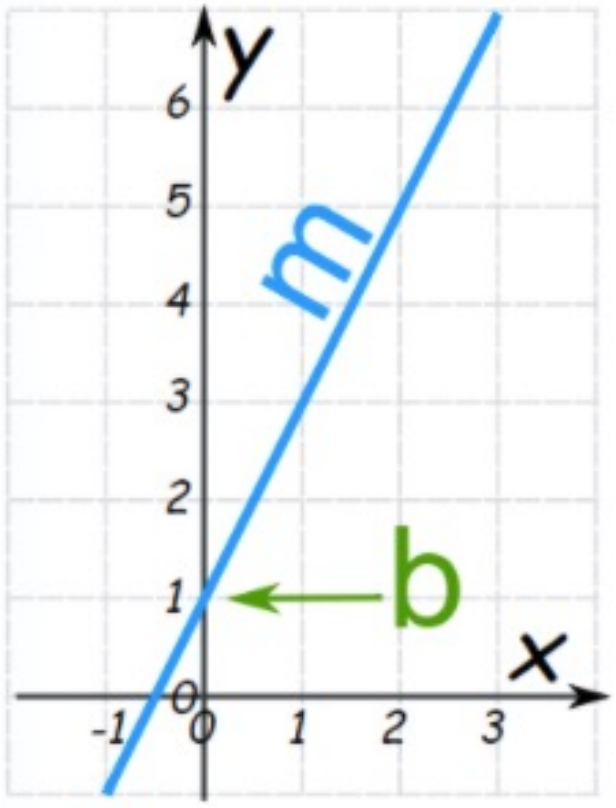
```
from sklearn.preprocessing import Imputer
imputer = Imputer(missing_values = 'NaN', strategy = 'mean', axis = 0)
imputer = imputer.fit(X[:, 1:3])
X[:, 1:3] = imputer.transform(X[:, 1:3])
```

- **missing_values** — This is the va

  this value, NaN will be the default value. So wherever we have NaN in our dataset, the Imputer object will replace it with a new value.
- **strategy** — This is the strategy we'll be using to calculate the value which has to replace the NaN occurrences in the dataset. There are three different strategies we can use:
  a. mean
  b. Median (Center of list)
  c. Most_frequent
- **axis** — This can take one of two values — 0 and 1. This will decide if the Imputer will apply the strategy along the rows or along the columns. 0 for columns, and 1 for rows.
- **verbose** — This will just decide the verbosity of the Imputer. By default, it's set to 0.
- **copy** — This will decide if a copy of the original object has to be made, or if the Imputer should change the dataset in-place. By default, it is set to True.

LINEAR REGRESSION

UNIVARIATE LINEAR REGRESSION

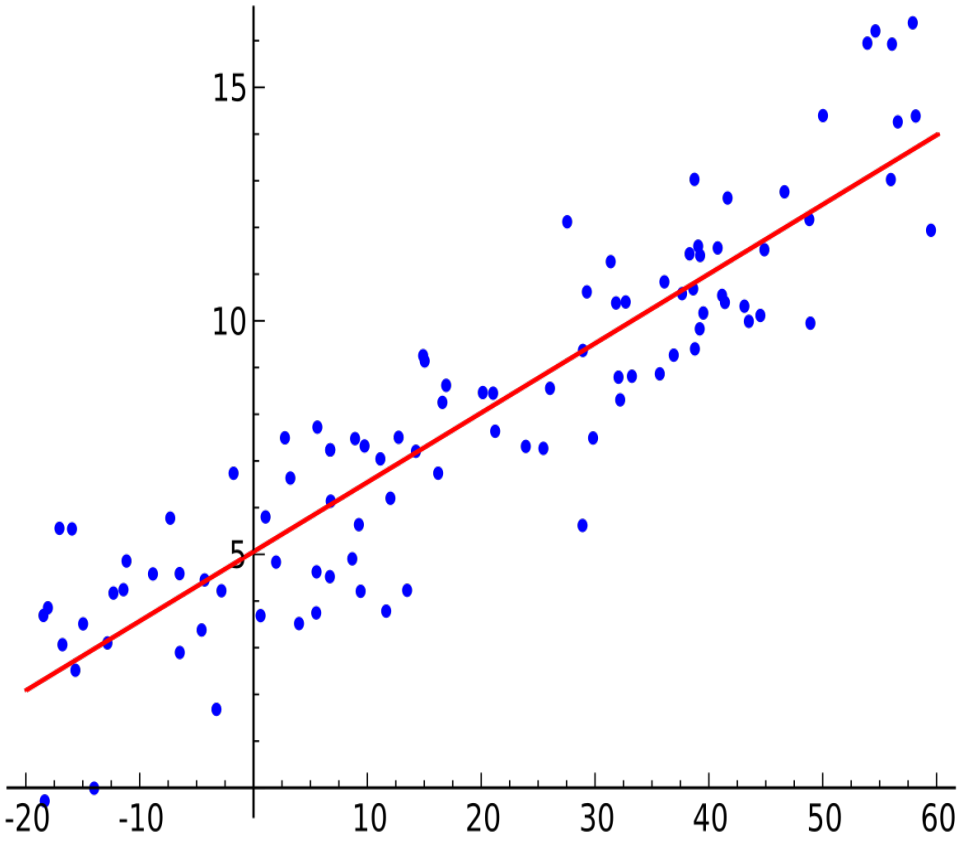


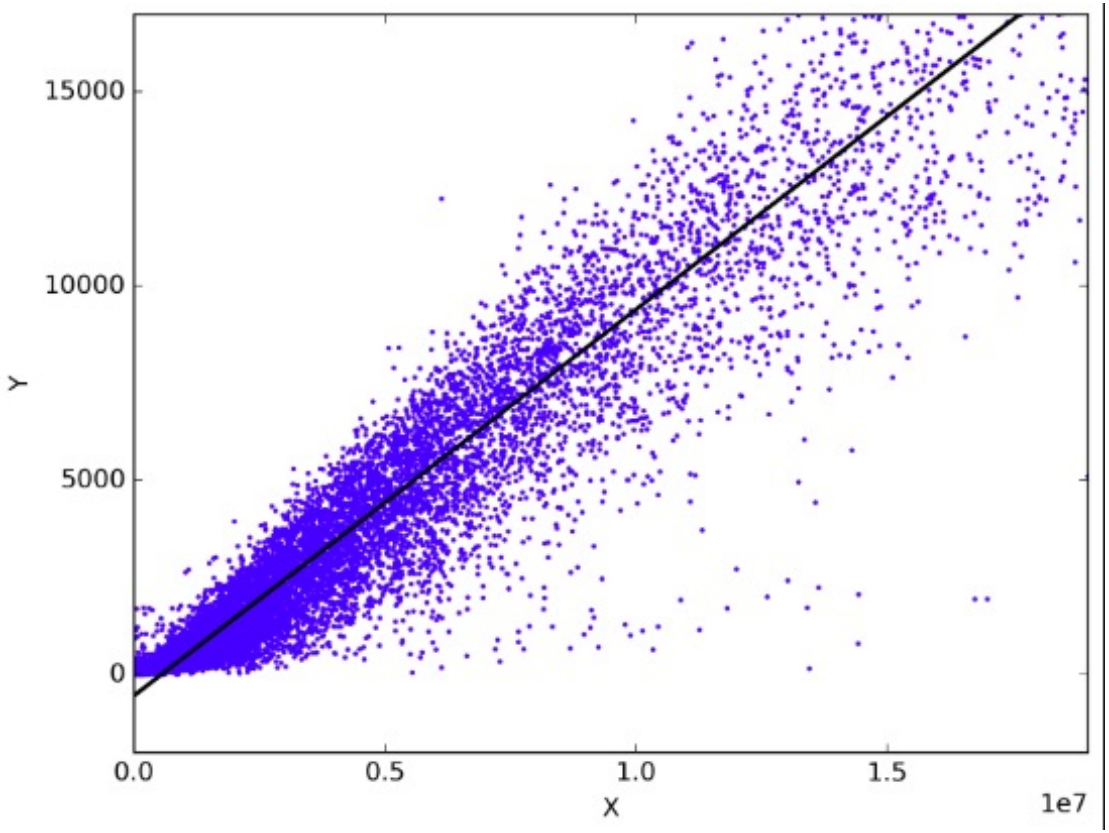

```
price = m * area + b
```

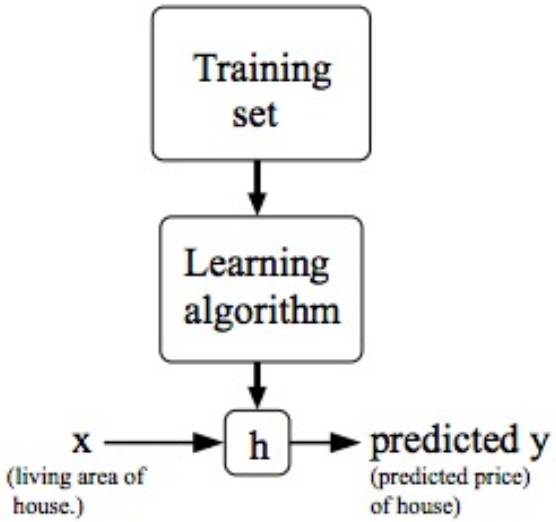
$$y = mx + b$$

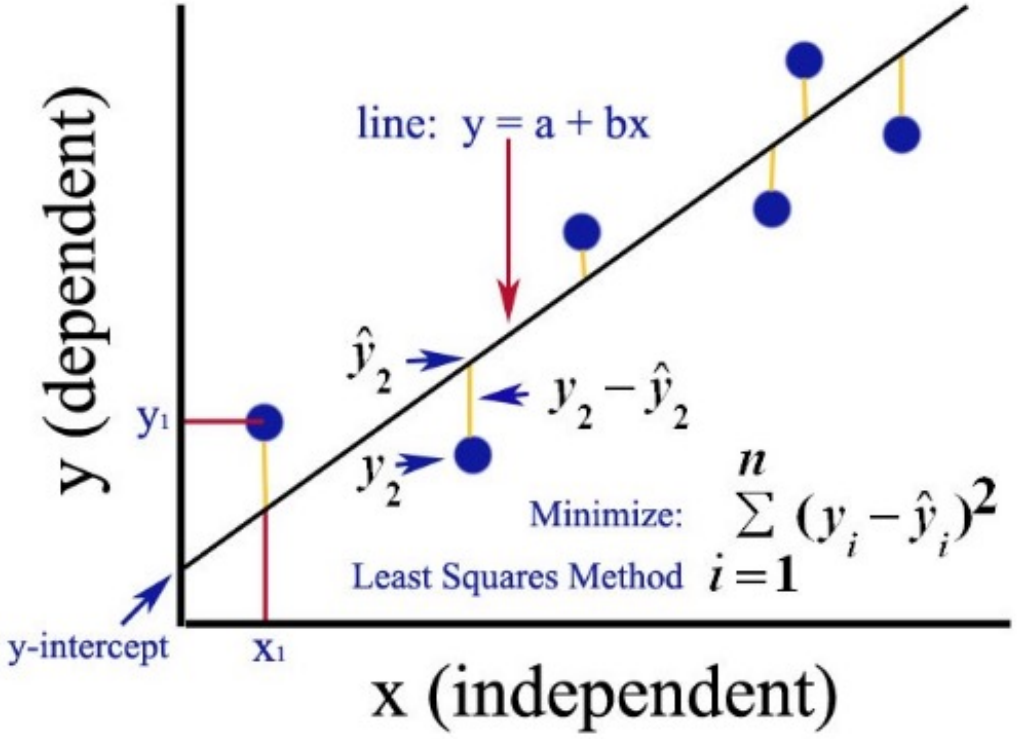
Slope (or Gradient) Y Intercept

Linear Regression



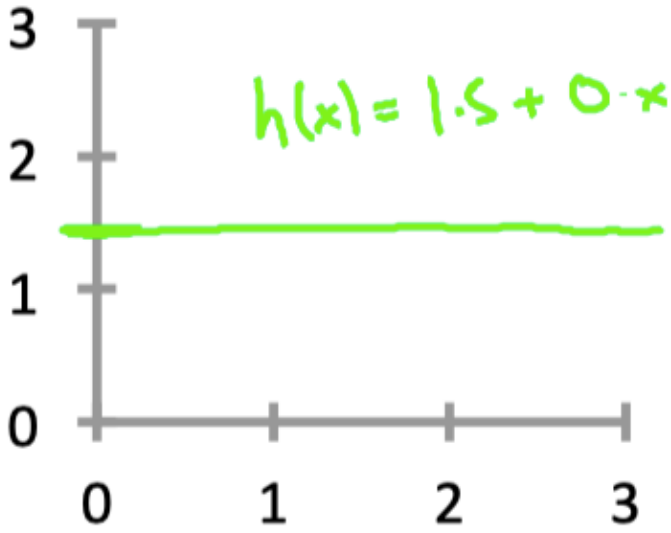






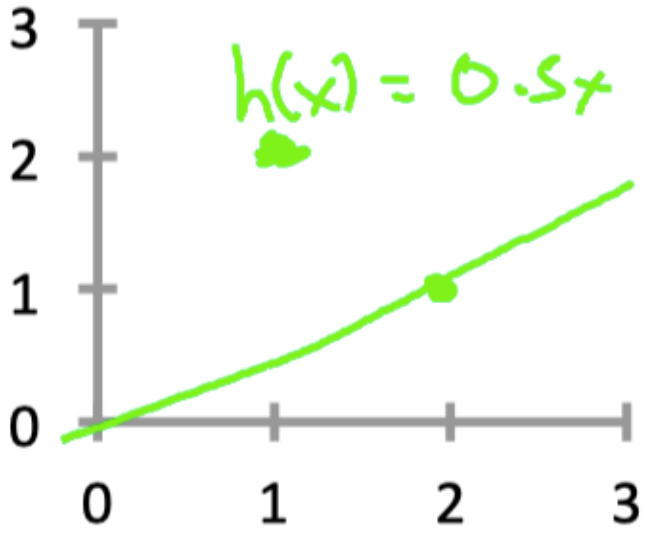
When the target variable that we're trying to predict is continuous, such as in our housing example, we call the learning problem a regression problem. When y can take on only a small number of discrete values (such as if, given the living area, we wanted to predict if a dwelling is a house or an apartment, say), we call it a classification problem.

$h_\theta(x) = \theta_0 + \theta_1 x$



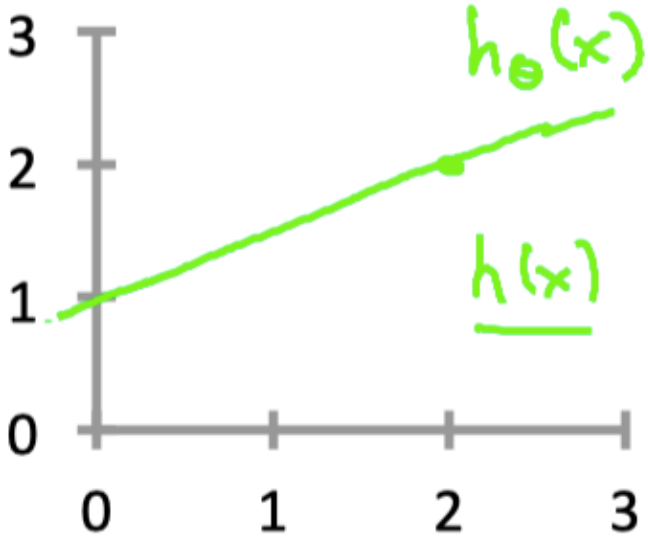
→ $\theta_0 = 1.5$

→ $\theta_1 = 0$



→ $\theta_0 = 0$

→ $\theta_1 = 0.5$



→ $\theta_0 = 1$

→ $\theta_1 = 0.5$

GRADIENT DESCENT

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

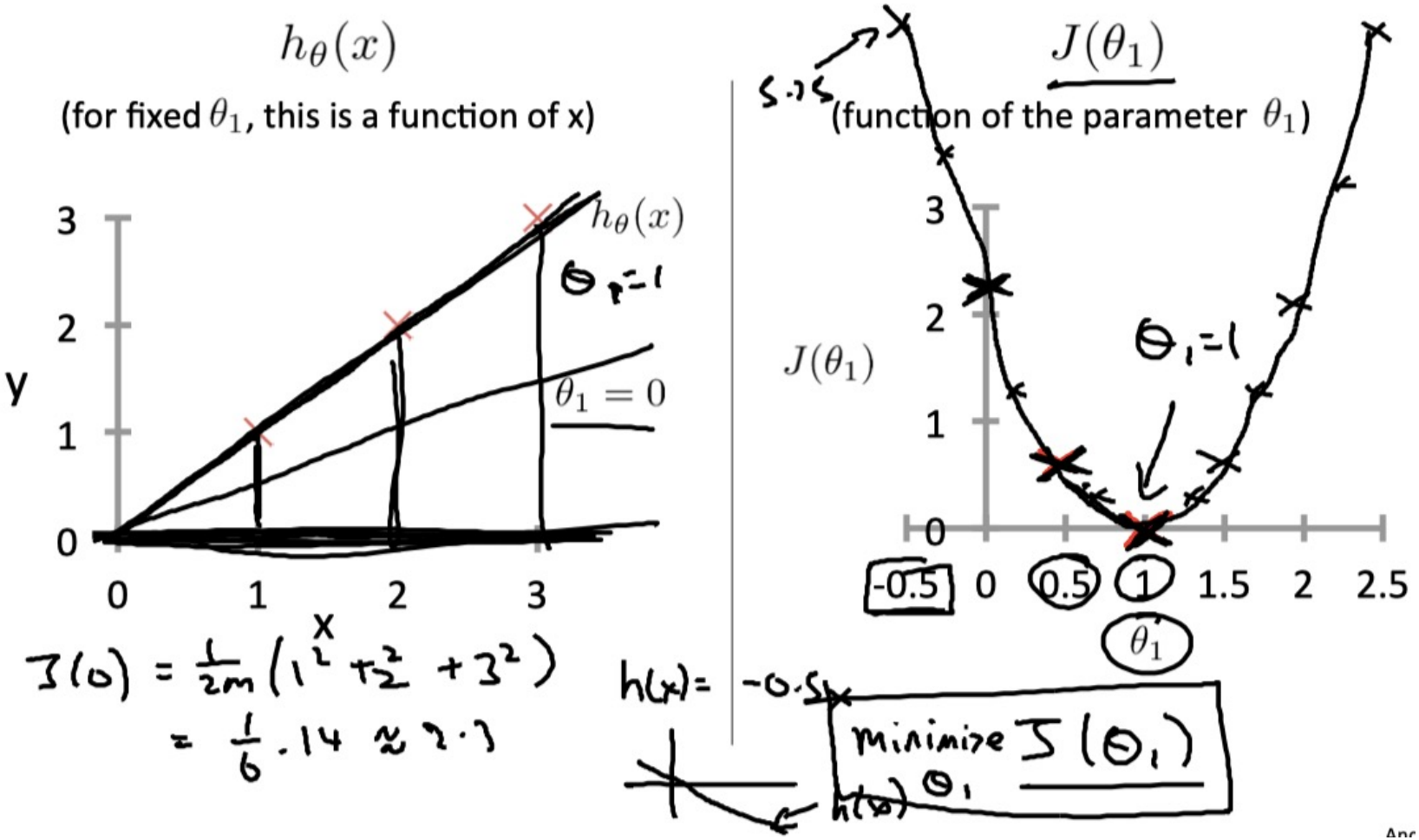
(for $j = 1$ and $j = 0$)

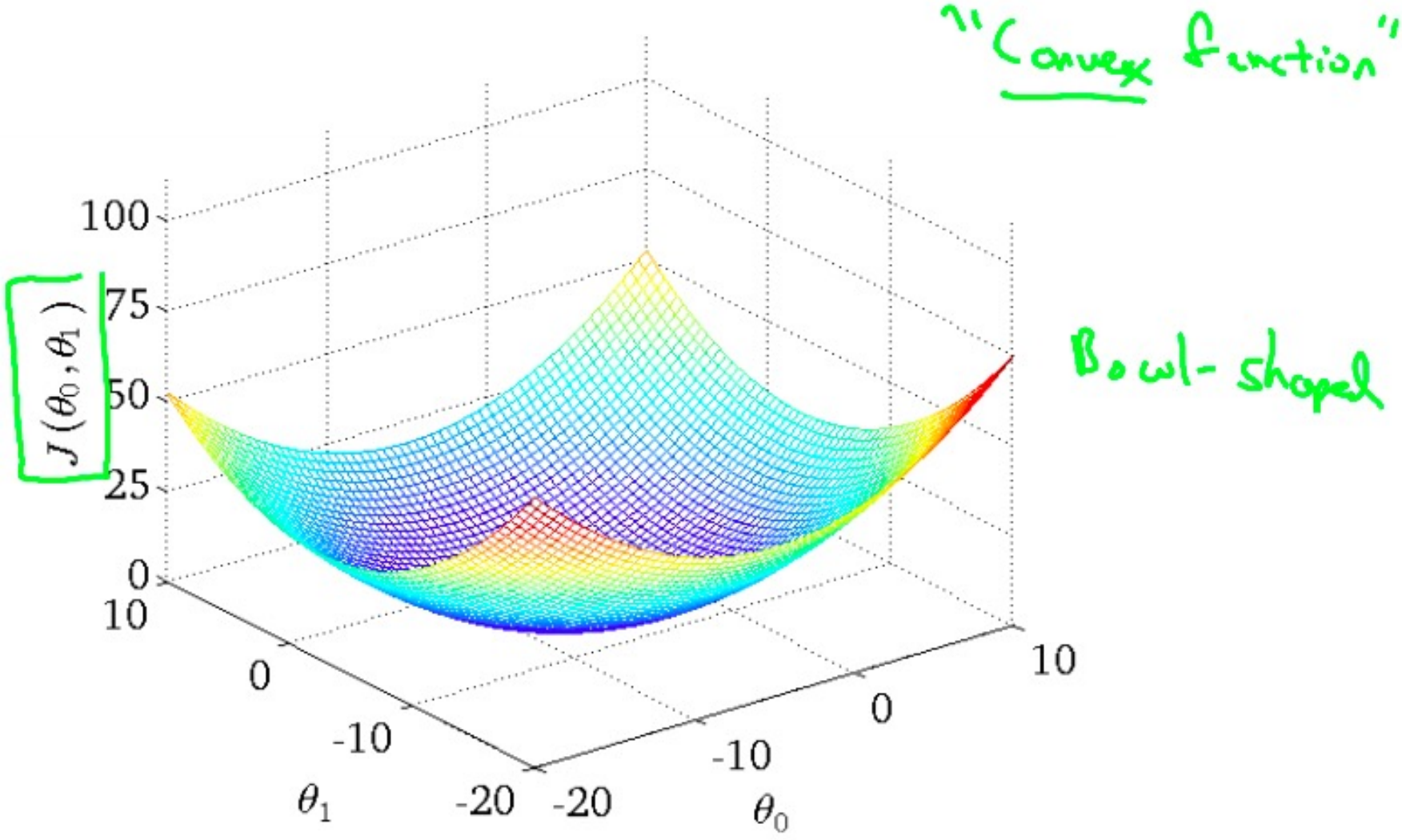
}

Linear Regression Model

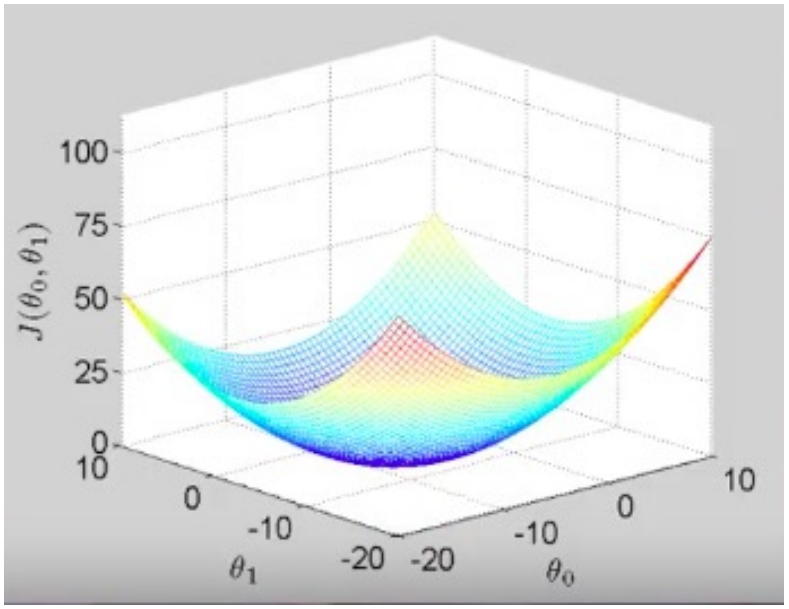
$$h_\theta(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$


$h_\theta(x)$
(for fixed θ_1, this is a function of x)
$h_\theta(x)$
$\theta_1 = 1$
$\theta_1 = 0$
y
x
0
1
2
3
$J(\theta_1) = \frac{1}{2m}(1^2 + 2^2 + 3^2)$
$= \frac{1}{6} \cdot 14 \approx 2.3$
$J(\theta_1)$
(function of the parameter θ_1)
$J(\theta_1)$
$\theta_1 = 1$
-0.5
0
0.5
1
1.5
2
2.5
θ_1
$h(x) = -0.5 \times$
minimize $J(\theta_1)$
θ_1

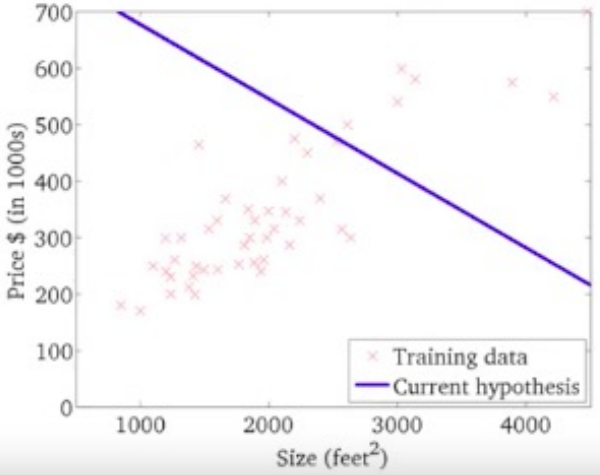

"Convex function"
$J(\theta_0, \theta_1)$
100
75
50
25
0
10
0
-10
-20
θ_1
-20
-10
0
10
θ_0
Bowl-shaped

3D VS CONTOURS



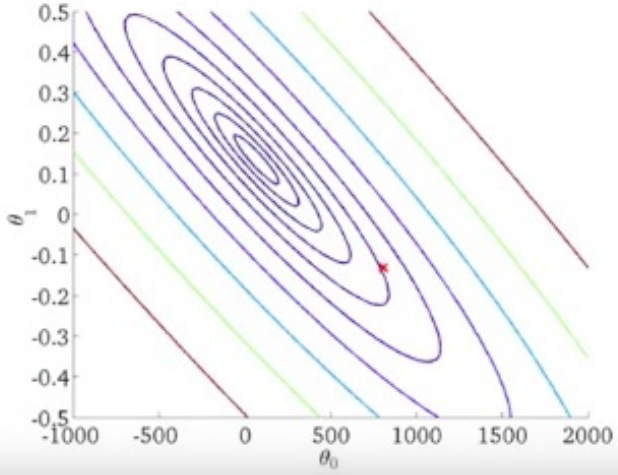
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)

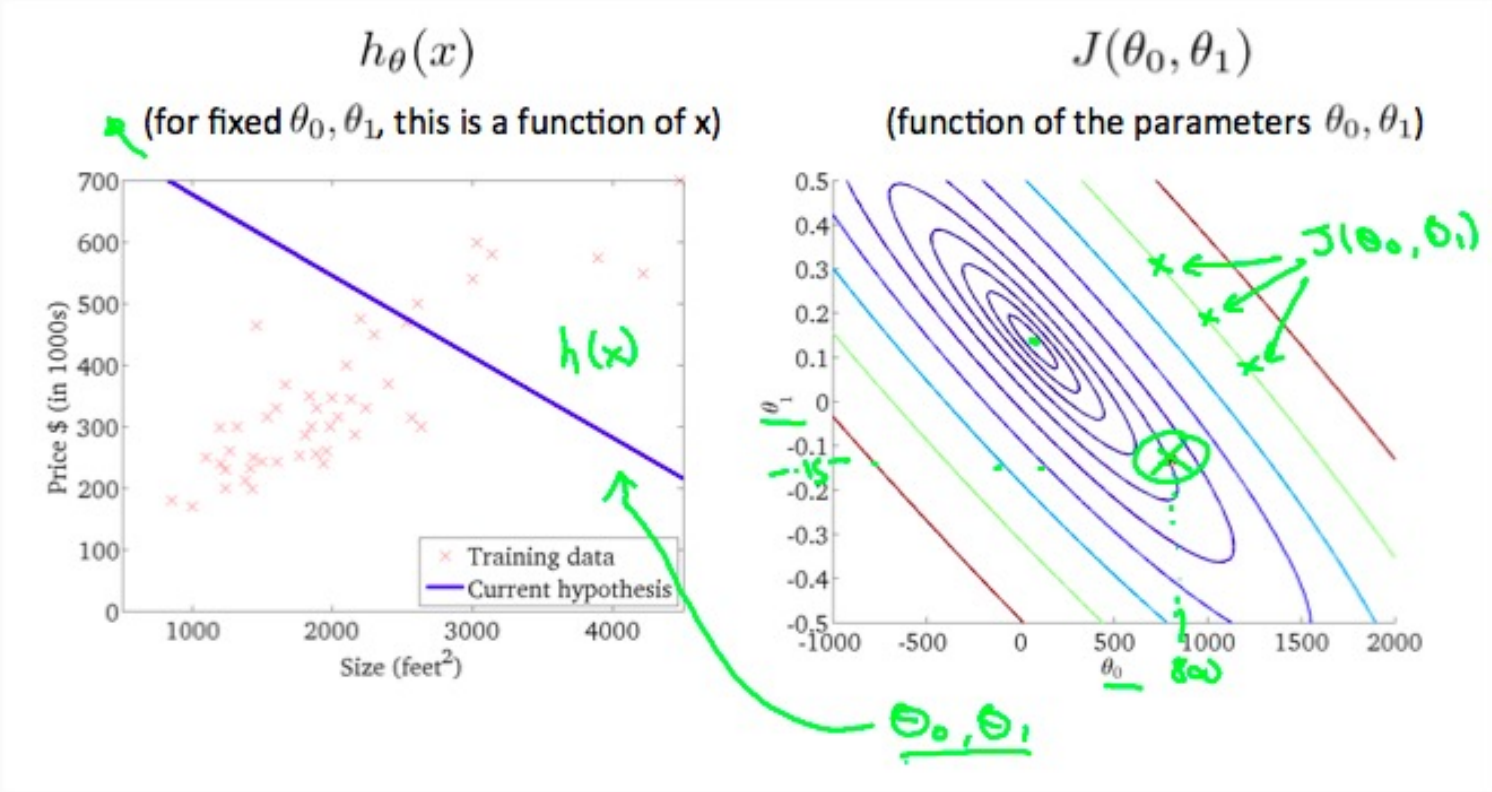


$J(\theta_0, \theta_1)$

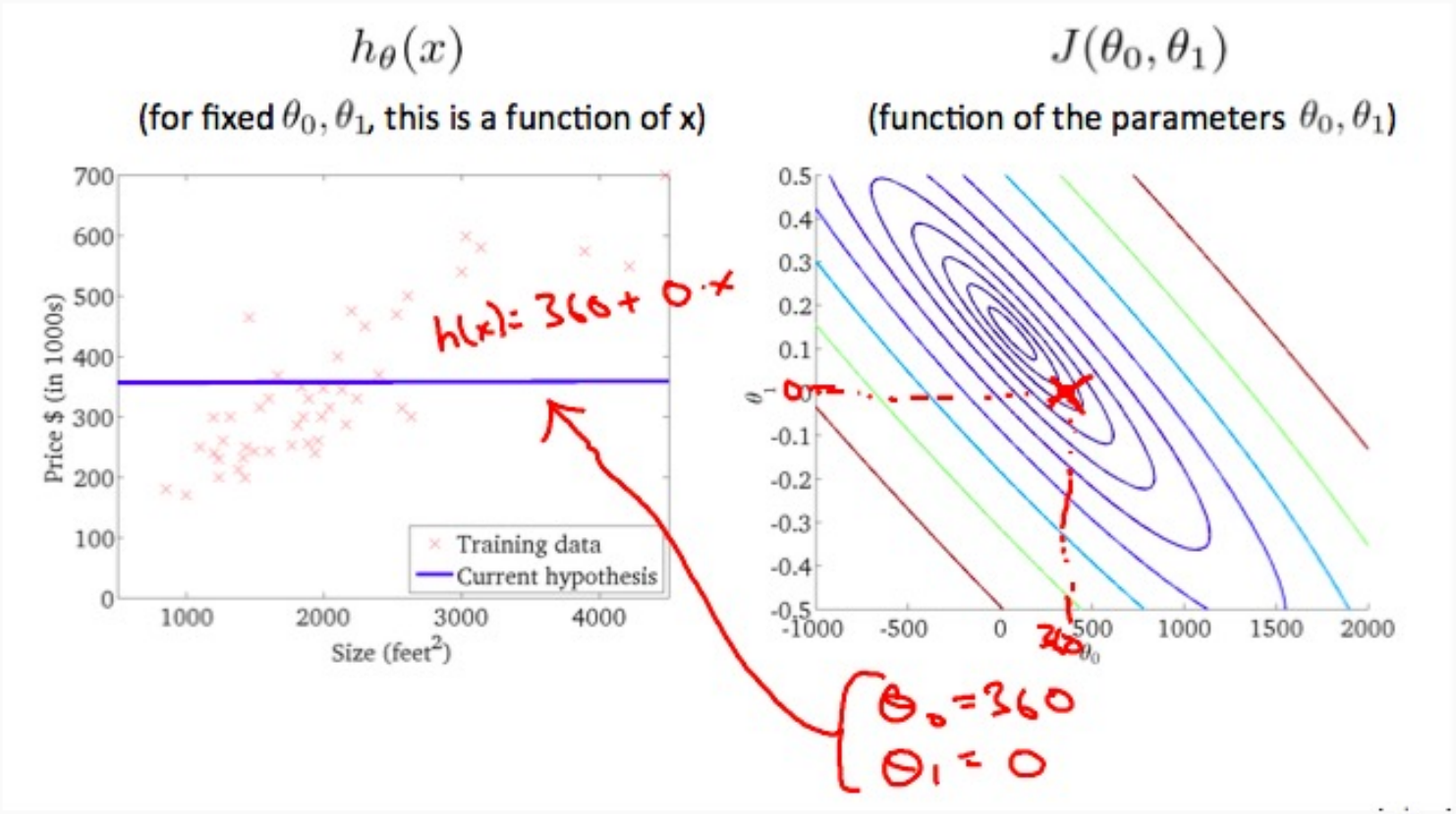
(function of the parameters θ_0, θ_1)



A contour plot is a graph that contains many contour lines. A contour line of a two variable function has a constant value at all points of the same line. An example of such a graph is the one to the right below.



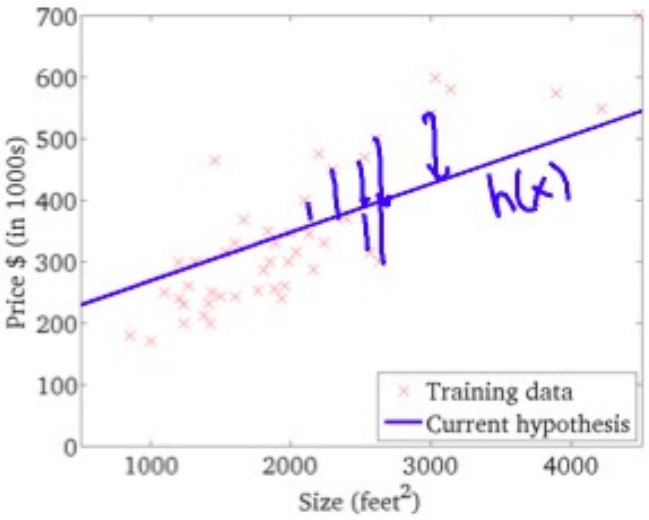
Taking any color and going along the 'circle', one would expect to get the same value of the cost function. For example, the three green points found on the green line above have the same value for $J(\theta_0, \theta_1)$ and as a result, they are found along the same line. The circled x displays the value of the cost function for the graph on the left when θ_0 = 800 and θ_1 = -0.15. Taking another h(x) and plotting its contour plot, one gets the following graphs:



When θ_0 = 360 and θ_1 = 0, the value of $J(\theta_0, \theta_1)$ in the contour plot gets closer to the center thus reducing the cost function error. Now giving our hypothesis function a slightly positive slope results in a better fit of the data.

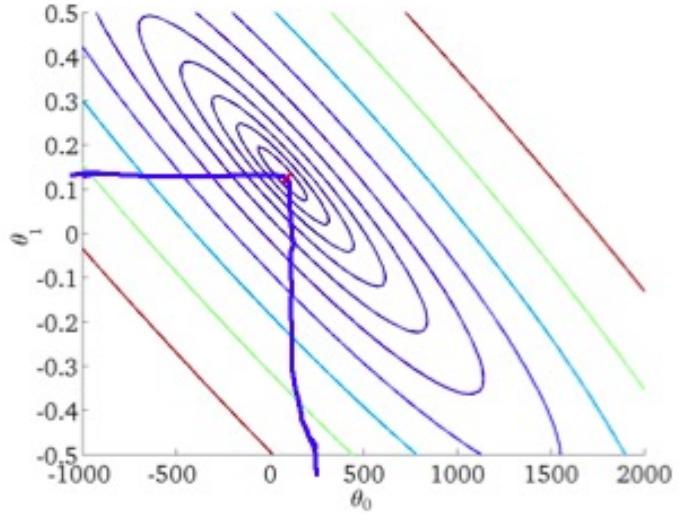
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



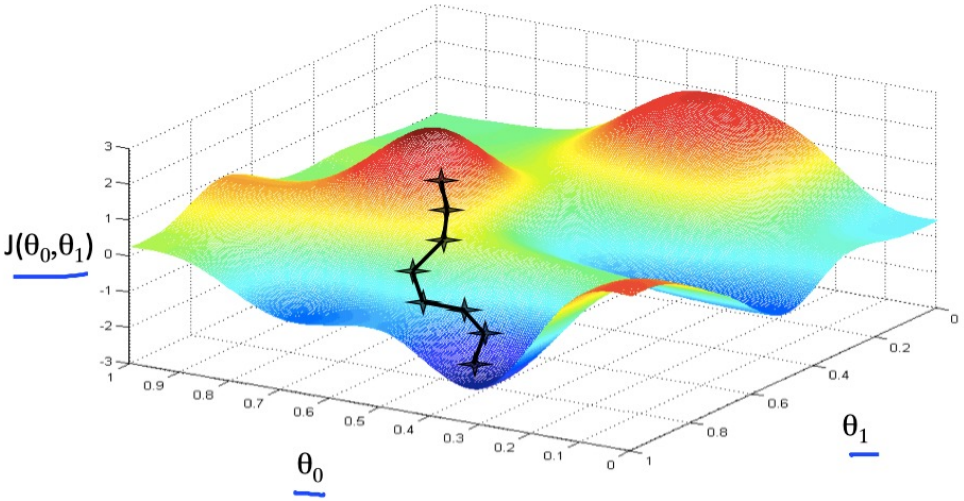
$J(\theta_0, \theta_1)$

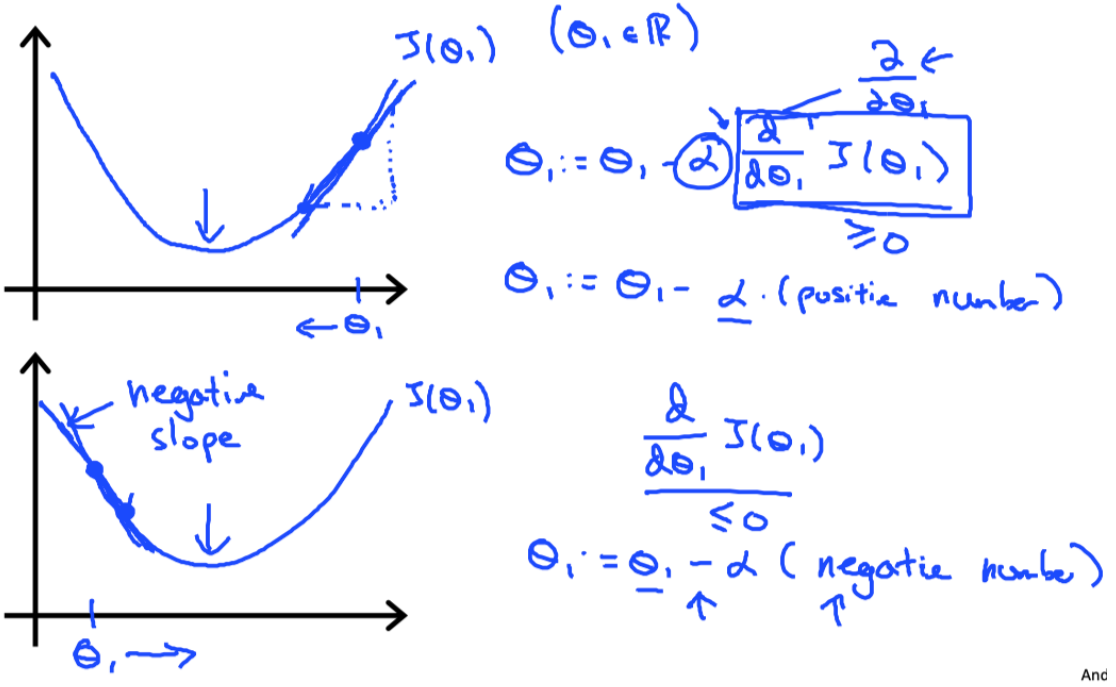
(function of the parameters θ_0, θ_1)

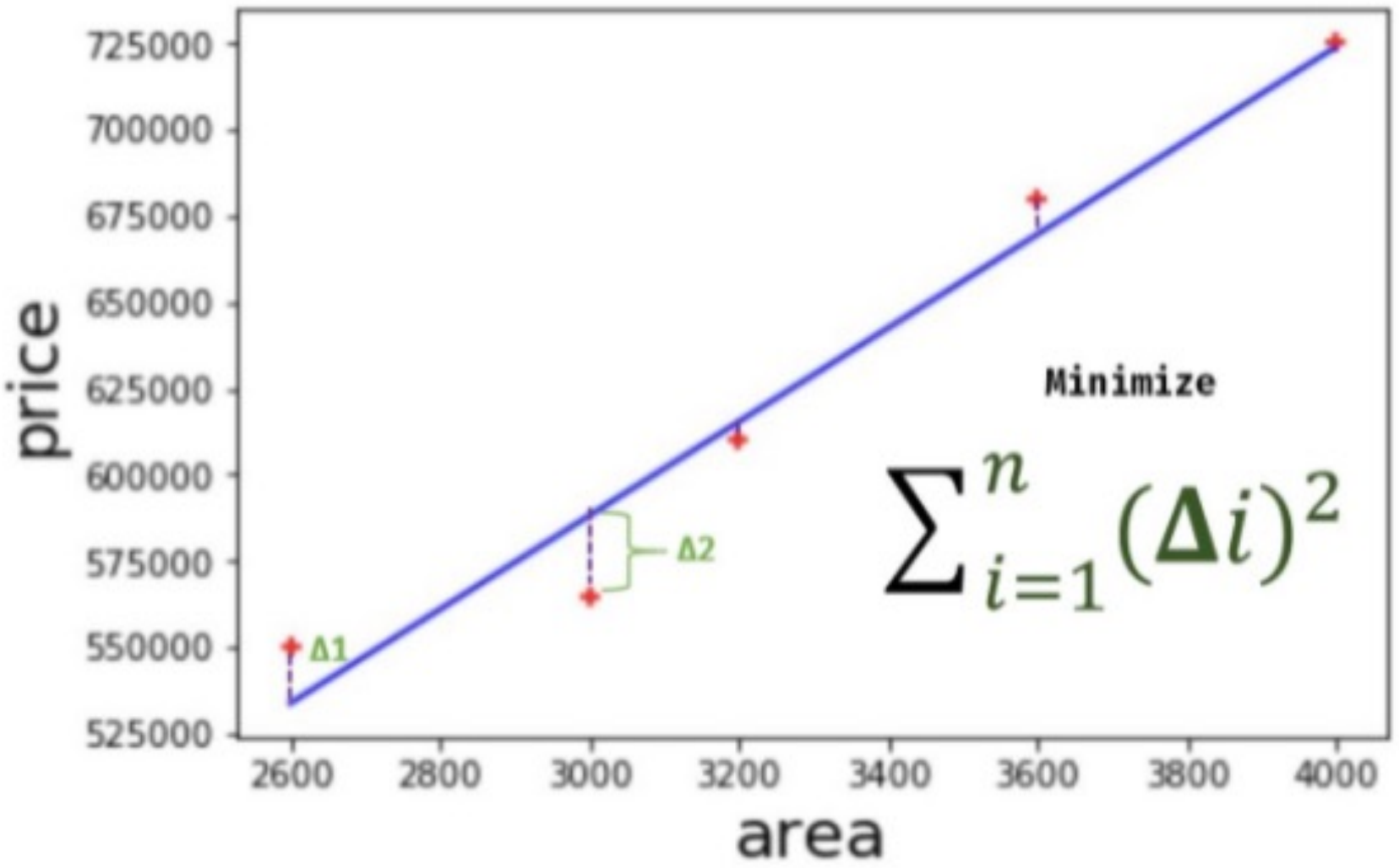


The graph above minimizes the cost function as much as possible and consequently, the result of θ_1 and θ_0 tend to be around 0.12 and 250 respectively. Plotting those values on our graph to the right seems to put our point in the center of the inner most 'circle'.

MINIMUM CONVERGENCE






725000
700000
675000
650000
625000
600000
575000
550000
525000
price
2600
2800
3000
3200
3400
3600
3800
4000
area
Δ1
Δ2
Minimize
$\sum_{i=1}^{n}(\Delta i)^2$

Implementing Gradient descent

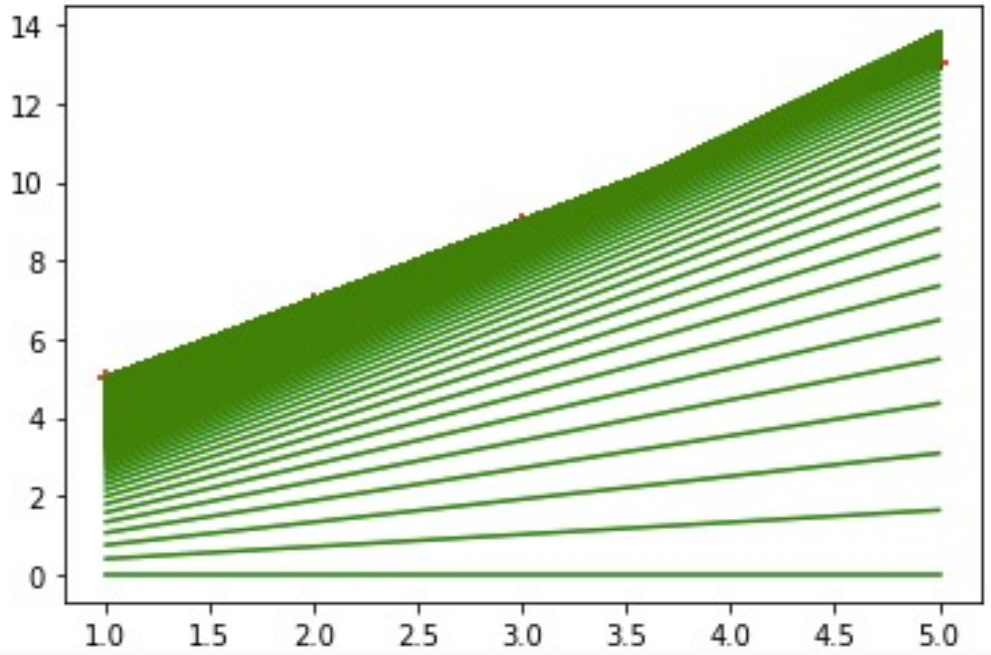
Entrée [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

Entrée [8]:
```
def gradientDescent(X,y):
    theta0 , theta1 = 8, -1
    costL = []
    iterations = 2000
    l_rate = 0.01
    m = len(X)

    #plt.scatter(X,y)
    #plt.show()
    for i in range(1,iterations + 1):
        y_predicted = theta0 + X * theta1
        print(f"y_predicted: {y_predicted}")
        cost = (1/(2*m)) * sum ([epsilons**2 for epsilons in (y_predicted - y)])
        costL.append(cost)
        plt.plot(X,y_predicted, color = 'green')
        #plt.show()
        theta0 = theta0 - l_rate *  1/m * sum(y_predicted - y)
        theta1 = theta1 - l_rate * 1/m * sum(X*(y_predicted - y))
        print(f" i={i}  H(0)x = {theta1} X1 + {theta0} , loss = {cost}")
    plt.plot(X,y_predicted, color = 'Black', linewidth = 2)
    plt.scatter(X,y,color = 'red',marker = '+', linewidth = 5)
    plt.show()
    return costL
```

Entrée [9]:
```
x = np.array([1,2,3,4,5])
y = np.array([5,7,9,11,13])
```

Entrée [10]:
```
gradient_descent(x,y)
```

```
m_curr= 2.000000028340344, b_curr= 2.999999897682365, Cost =9.559907537917085e-16 , i9997
m_curr= 2.000000028292435, b_curr= 2.9999998978553313, Cost =9.527613196559246e-16 , i9998
m_curr= 2.000000028244607, b_curr= 2.9999998980280047, Cost =9.49542801540366e-16 , i9999
```



SUPERVISED LEARNING

learn from labelled data (X, y)

Classification

Predict a category

e.g. spam / not spam · crop disease

FAMILY	COMMON MODELS
Linear	`LogisticRegression`
Neighbours	`KNeighborsClassifier`
SVM	`SVC`
Trees	`DecisionTreeClassifier`
Ensembles	`RandomForestClassifier`
Boosting	`HistGradientBoostingClassifier`
Bayes	`GaussianNB`
Neural net	`MLPClassifier`

`sklearn.linear_model, .ensemble, .svm …`

Regression

Predict a number

e.g. rent price · electricity demand

FAMILY	COMMON MODELS
Linear	`LinearRegression`
Regularised	`Ridge · Lasso · ElasticNet`
Neighbours	`KNeighborsRegressor`
SVM	`SVR`
Trees	`DecisionTreeRegressor`
Ensembles	`RandomForestRegressor`
Boosting	`HistGradientBoostingRegressor`
Neural net	`MLPRegressor`

`sklearn.linear_model, .ensemble, .svm …`

UNSUPERVISED LEARNING

find structure in unlabelled data (X)

Clustering

Group similar samples

e.g. customer segments

FAMILY	COMMON MODELS
Centroid	`KMeans · MiniBatchKMeans`
Density	`DBSCAN · HDBSCAN`
Hierarchical	`AgglomerativeClustering`
Probabilistic	`GaussianMixture`
Graph	`SpectralClustering`
Mode seeking	`MeanShift`

`sklearn.cluster, .mixture`

Dimensionality reduction

Compress features

e.g. visualise · speed up models

FAMILY	COMMON MODELS
Linear	`PCA · TruncatedSVD`
Matrix factorisation	`NMF`
Supervised	`LinearDiscriminantAnalysis`
Manifold	`TSNE · Isomap`
Feature selection	`SelectKBest · RFE`

`sklearn.decomposition, .manifold`

WORKFLOW TOOLS

prepare data, tune and evaluate models

Preprocessing

Get data model-ready

e.g. scale · encode · impute

FAMILY	COMMON MODELS
Scaling	`StandardScaler · MinMaxScaler`
Encoding	`OneHotEncoder · OrdinalEncoder`
Missing values	`SimpleImputer · KNNImputer`
Feature creation	`PolynomialFeatures`
Text	`CountVectorizer · TfidfVectorizer`
Mixed columns	`ColumnTransformer`

`sklearn.preprocessing, .impute`

Model selection & evaluation

Choose and trust a model

e.g. split · tune · score

FAMILY	COMMON MODELS
Splitting	`train_test_split`
Cross-validation	`cross_val_score · KFold`
Tuning	`GridSearchCV · RandomizedSearchCV`
Chaining	`Pipeline`
Class. metrics	`accuracy · f1 · confusion_matrix`
Reg. metrics	`MAE · RMSE · r2_score`

`sklearn.model_selection, .metrics`

sklearn.linear_model.LinearRegression

class `sklearn.linear_model.`**`LinearRegression`**(**, fit_intercept=True, copy_X=True, n_jobs=None, positive=False*)

[source]

Ordinary least squares Linear Regression.

LinearRegression fits a linear model with coefficients w = (w1, ..., wp) to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

Parameters:

fit_intercept : *bool, default=True*

Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

copy_X : *bool, default=True*

If True, X will be copied; else, it may be overwritten.

n_jobs : *int, default=None*

The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly `n_targets > 1` and secondly `X` is sparse or if `positive` is set to `True`. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

positive : *bool, default=False*

When set to `True`, forces the coefficients to be positive. This option is only supported for dense arrays.

New in version 0.24.

Methods

`fit`(X, y[, sample_weight])	Fit linear model.
`get_params`([deep])	Get parameters for this estimator.
`predict`(X)	Predict using the linear model.
`score`(X, y[, sample_weight])	Return the coefficient of determination of the prediction.
`set_params`(**params)	Set the parameters of this estimator.

fit(*X*, *y*, *sample_weight=None*)

Fit linear model.

Parameters:

X : *{array-like, sparse matrix} of shape (n_samples, n_features)*
Training data.

y : *array-like of shape (n_samples,) or (n_samples, n_targets)*
Target values. Will be cast to X's dtype if necessary.

sample_weight : *array-like of shape (n_samples,), default=None*
Individual weights for each sample.

New in version 0.17: parameter *sample_weight* support to LinearRegression.

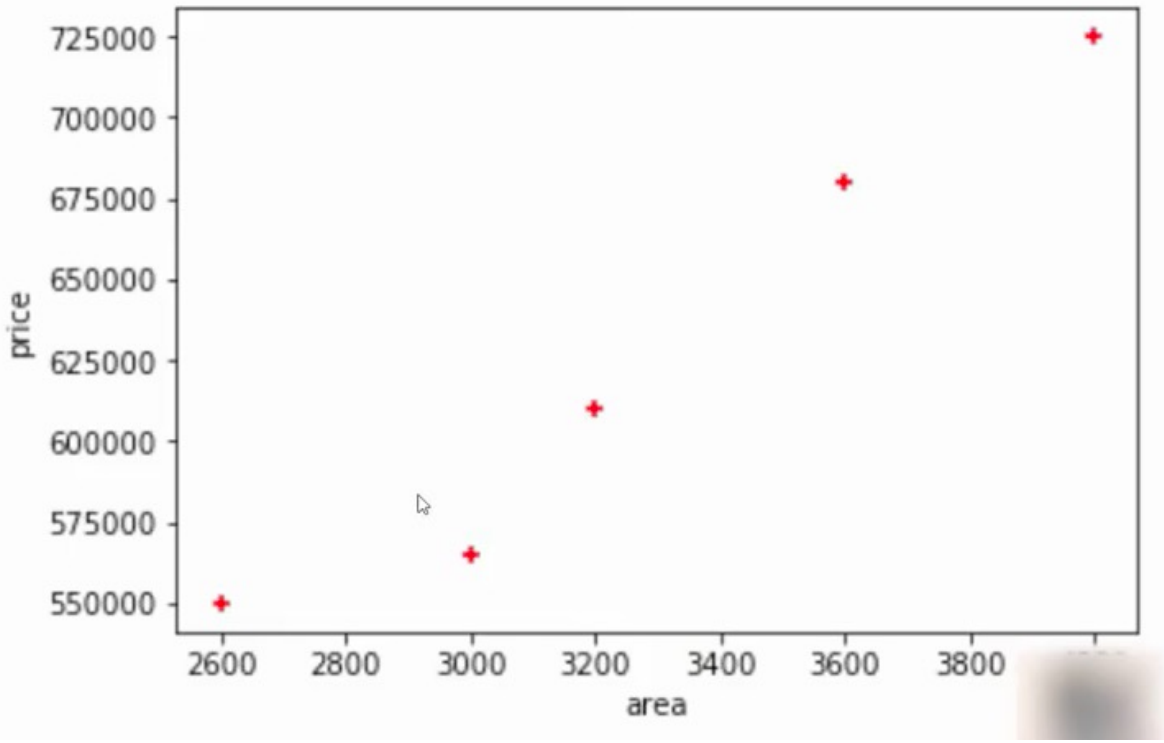
Returns:

self : *object*
Fitted Estimator.

```
>>> import numpy as np
>>> from sklearn.linear_model import LinearRegression
>>> X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
>>> # y = 1 * x_0 + 2 * x_1 + 3
>>> y = np.dot(X, np.array([1, 2])) + 3
>>> reg = LinearRegression().fit(X, y)
>>> reg.score(X, y)
1.0
>>> reg.coef_
array([1., 2.])
>>> reg.intercept_
3.0...
>>> reg.predict(np.array([[3, 5]]))
array([16.])
```

Solve this Case with linear Regression

area	price
2600	550000
3000	565000
3200	610000
3600	680000
4000	725000



Entrée [1]:
```
import pandas as pd
import numpy as np
from sklearn import linear_model
import matplotlib.pyplot as plt
```

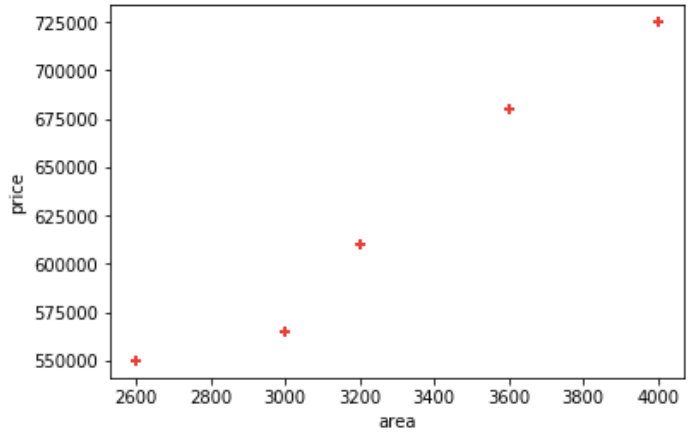
Entrée [2]:
```
df = pd.read_csv('homeprices.csv')
df
```

Out[2]:

	area	price
0	2600	550000
1	3000	565000
2	3200	610000
3	3600	680000
4	4000	725000

Entrée [3]:
```
%matplotlib inline
plt.xlabel('area')
plt.ylabel('price')
plt.scatter(df.area,df.price,color='red',marker='+')
```

Out[3]: <matplotlib.collections.PathCollection at 0x1a24d3dcf8>



```
Entrée [6]: new_df = df.drop('price',axis='columns')
            new_df.shape

Out[6]: (5, 1)
```

```
Entrée [7]: price = df.price
            price.shape

Out[7]: (5,)
```

```
Entrée [10]: new_df = df.drop('price',axis='columns')
             new_df
```

Out[10]:

	area
0	2600
1	3000
2	3200
3	3600
4	4000

```
Entrée [9]: price = df.price
            price

Out[9]: 0    550000
        1    565000
        2    610000
        3    680000
        4    725000
        Name: price, dtype: int64
```

Entrée [9]:
```
# Create linear regression object
reg = linear_model.LinearRegression()
reg.fit(new_df,price)
```

Out[9]:
```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
         normalize=False)
```

(1) Predict price of a home with area = 3300 sqr ft

Entrée [10]:
```
reg.predict([[3300]])
```

Out[10]:
```
array([628715.75342466])
```

Entrée [11]:
```
reg.coef_
```

Out[11]:
```
array([135.78767123])
```

Entrée [12]:
```
reg.intercept_
```

Out[12]:
```
180616.43835616432
```

Y = m * X + b (m is coefficient and b is intercept)

Entrée [13]:
```
3300*135.78767123 + 180616.43835616432
```

Out[13]:
```
628715.7534151643
```

(1) Predict price of a home with area = 5000 sqr ft

Entrée [14]:
```
reg.predict([[5000]])
```

Out[14]:
```
array([859554.79452055])
```

Generate CSV file with list of home price predictions

Entrée [15]:
```
area_df = pd.read_csv("areas.csv")
area_df.head(3)
```

Out[15]:

	area
0	1000
1	1500
2	2300

Entrée [16]:
```
p = reg.predict(area_df)
p
```

Out[16]:
```
array([ 316404.10958904,  384297.94520548,  492928.08219178,
        661304.79452055,  740061.64383562,  799808.21917808,
        926090.75342466,  650441.78082192,  825607.87671233,
        492928.08219178, 1402705.47945205, 1348390.4109589 ,
       1144708.90410959])
```

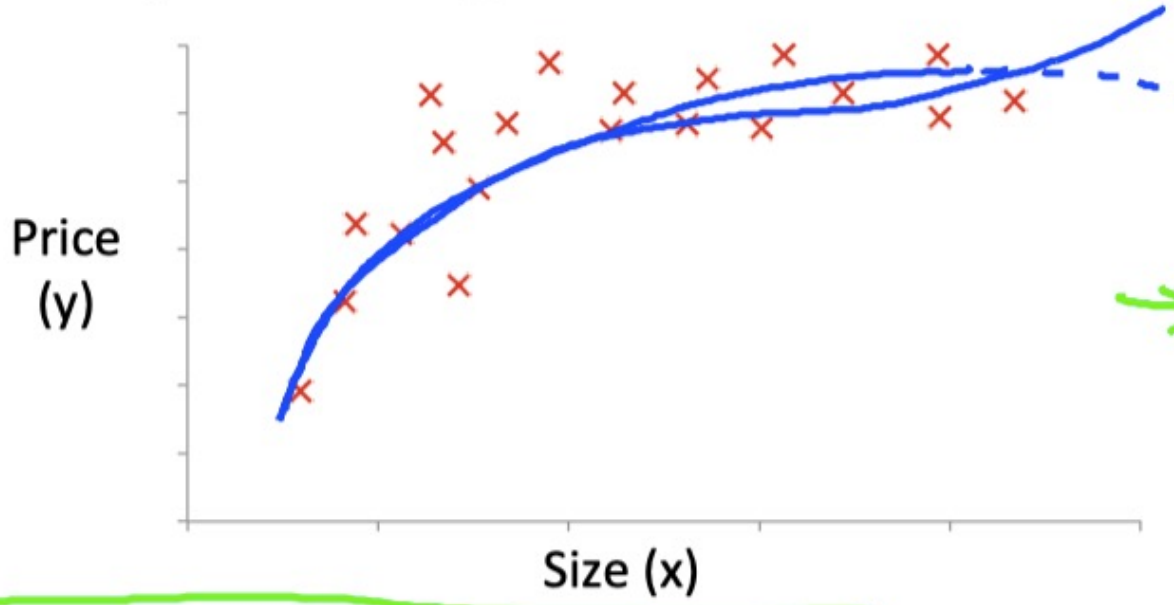
Entrée [17]:
```
area_df['prices']=p
area_df
```

Out[17]:

	area	prices
0	1000	3.164041e+05
1	1500	3.842979e+05
2	2300	4.929281e+05
3	3540	6.613048e+05
4	4120	7.400616e+05
5	4560	7.998082e+05
6	5490	9.260908e+05
7	3460	6.504418e+05
8	4750	8.256079e+05
9	2300	4.929281e+05
10	9000	1.402705e+06
11	8600	1.348390e+06
12	7100	1.144709e+06

Entrée [18]:
```
area_df.to_csv("prediction.csv")
```

Polynomial regression



$\theta_0 + \theta_1 x + \theta_2 x^2$

$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$
$$= \theta_0 + \theta_1 (size) + \theta_2 (size)^2 + \theta_3 (size)^3$$

- $x_1 = (size)$
- $x_2 = (size)^2$
- $x_3 = (size)^3$

Size: 1 - 1000

$Size^2$: 1 - 1000,000

$Size^3$: 1 - 10^9

Spyder (Python 3.7)

/Users/christian/Documents/PYTHONPROJECTS/LinearRegression2

Editor - /Users/christian/Documents/PYTHONPROJECTS/LinearRegression2/LinearRegression01.py

temp.py | LinearRegression01.py

```
 9 import numpy as np
10 import pandas as pd
11 from sklearn.linear_model import LinearRegression
12 import matplotlib.pyplot as plt
13
14 # Read in csv file
15 # File: https://github.com/mGalarnyk/Python_Tutorials/blob/master/Python_Basics/Linear_Regression/li
16 raw_data = pd.read_csv("linear.csv")
17
18 # Removes rows with NaN in them
19 filtered_data = raw_data[~np.isnan(raw_data["y"])]
20
21 x_y = np.array(filtered_data)
22 x, y = x_y[:,0], x_y[:,1]
23
24 # Reshaping
25 x, y = x.reshape(-1,1), y.reshape(-1, 1)
26
27 # Linear Regression Object
28 lin_regression = LinearRegression()
29
30 # Fitting linear model to the data
31 # h(x) = theta0 x0 + theta1 x1
32 lin_regression.fit(x,y)
33
34 # Get slope of fitted line
35 theta1 = lin_regression.coef_
36
37 # Get y-Intercept of the Line
38 theta0 = lin_regression.intercept_
39
40 # Get Predictions for original x values
41 # you can also get predictions for new data
42 predictions = lin_regression.predict(x)
43
44 # following slope intercept form
45 #print('Hello, my name is ' + name + ' and I am ' + str(age))
46 #print('My name is {name} and I am {age}'.format(name=name, age=age))
47 print ('formula: h(x) = '+str(theta0)+'x0 + '+str(theta1)+' x1')
48
49 # Plot the Original Model (Black) and Predictions (Blue)
50 plt.scatter(x, y,  color='red')
51 plt.plot(x, predictions, color='blue',linewidth=3)
52 plt.show()
```

Variable explorer

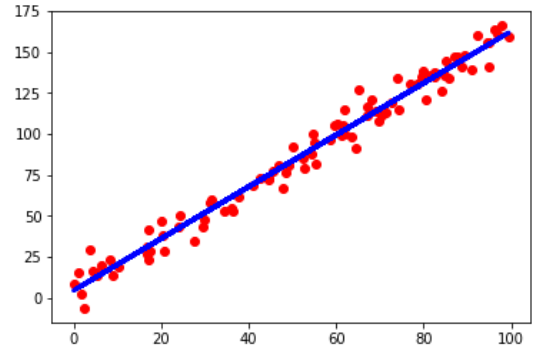
Name	Type	Size	Value
b	float64	(1,)	[4.4701969]
filtered_data	DataFrame	(92, 2)	Column names: x, y
m	float64	(1, 1)	[[1.5831968]]
predictions	float64	(92, 1)	[[135.21568589] [121.50400849]
raw_data	DataFrame	(100, 2)	Column names: x, y
theta0	float64	(1,)	[4.4701969]
theta1	float64	(1, 1)	[[1.5831968]]
valX	int64	(1, 1)	[[81]]

Variable explorer | File explorer | Help

IPython console

Console 1/A

```
In [16]: runfile('/Users/christian/Documents/PYTHONPROJECTS/LinearRegression2/
LinearRegression01.py', wdir='/Users/christian/Documents/PYTHONPROJECTS/
LinearRegression2')
formula: h(x) = [4.4701969]x0 + [[1.5831968]] x1
```




```
In [17]:
```

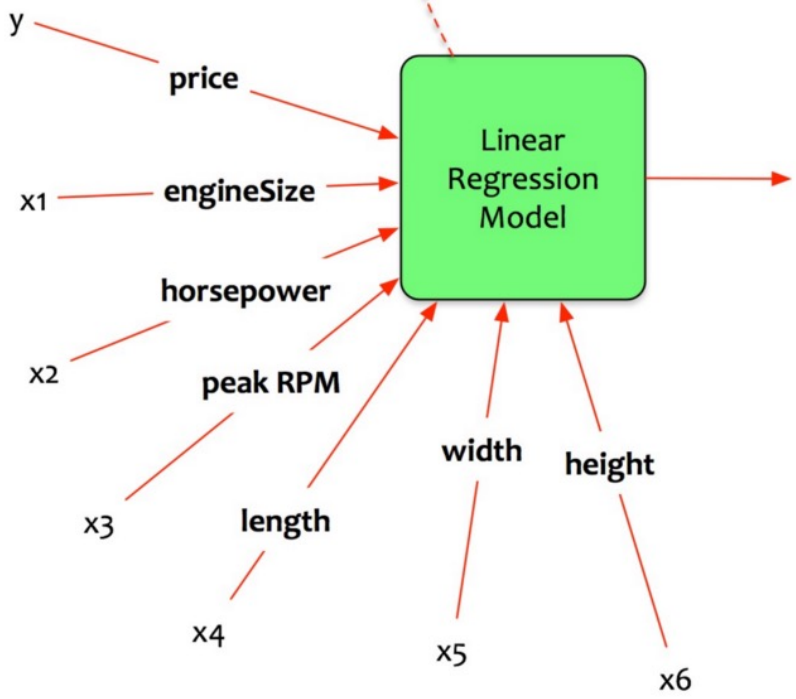
IPython console | History log

Permissions: RW End-of-lines: LF Encoding: UTF-8 Line: 23 Column: 1 Memory: 74 %

Multivariate Linear Regression

Multiple Features

- estimates β0, β1, β2, β3, β4, β5, β6
- creates model performance metrics



	Estimate
(Intercept)	-85086.39
engineSize	102.85
horsePower	43.79
peakRpm	1.52
length	-37.91
width	908.12
height	364.33

price = -85090 + 102.85* engineSize + 43.79* horse power + 1.52* peak RPM - 37.91* length + 908.12* width + 364.33* height

Size (feet)^2	Number of bedrooms	Number of floors	Age of home (years)	Price ($1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

In the training set above, what is $x_1^{(4)}$?

Gradient Descent

Previously (n=1):

Repeat $\{$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})$$

$\frac{\partial}{\partial \theta_0} J(\theta)$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update θ_0, θ_1)

$\}$

New algorithm $(n \geq 1)$:

Repeat $\{$

$\frac{\partial}{\partial \theta_j} J(\theta)$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update θ_j for $j = 0, \ldots, n$)

$\}$

$x_0^{(i)} = 1$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

$\ldots$

Sample problem of predicting home price in monroe, new jersey (USA)

Below is the table containing home prices in monroe twp, NJ. Here price depends on **area (square feet), bed rooms and age of the home (in years)**. Given these prices we have to predict prices of new homes based on area, bed rooms and age.

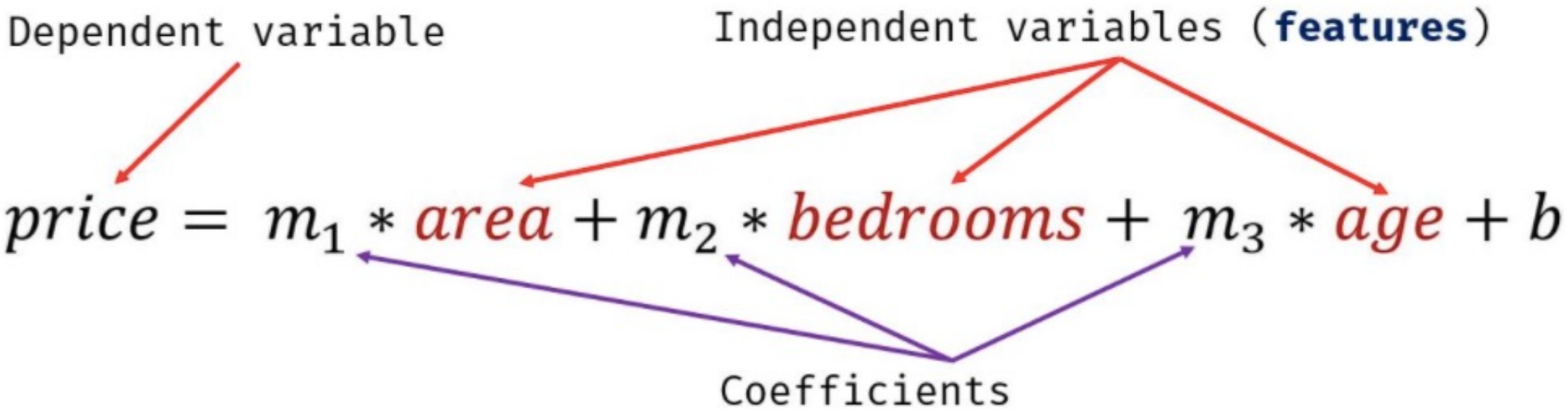
area	bedrooms	age	price
2600	3	20	550000
3000	4	15	565000
3200		18	610000
3600	3	30	595000
4000	5	8	760000
4100	6	8	810000

Given these home prices find out price of a home that has,

3000 sqr ft area, 3 bedrooms, 40 year old

2500 sqr ft area, 4 bedrooms, 5 year old

MULTIVARIATE LINEAR REGRESSION



$$y = m_1 x_1 + m_2 x_2 + m_3 x_3 + b$$

area	bedrooms	age	price
2600	3	20	550000
3000	4	15	565000
3200		18	610000
3600	3	30	595000
4000	5	8	760000
4100	6	8	810000

Entrée [19]:
```
import pandas as pd
import numpy as np
from sklearn import linear_model
```

Entrée [20]:
```
df = pd.read_csv('homeprices.csv')
df
```

Out[20]:

	area	bedrooms	age	price
0	2600	3.0	20	550000
1	3000	4.0	15	565000
2	3200	NaN	18	610000
3	3600	3.0	30	595000
4	4000	5.0	8	760000
5	4100	6.0	8	810000

Data Preprocessing: Fill NA values with median value of a column

Data Preprocessing: Fill NA values with median value of a column

Entrée [4]:
```
df.bedrooms.median()
```

Out[4]: 4.0

Entrée [5]:
```
df.bedrooms = df.bedrooms.fillna(df.bedrooms.median())
df
```

Out[5]:

	area	bedrooms	age	price
0	2600	3.0	20	550000
1	3000	4.0	15	565000
2	3200	4.0	18	610000
3	3600	3.0	30	595000
4	4000	5.0	8	760000
5	4100	6.0	8	810000

MODEL AND COEFICIENTS

```
Entrée [11]: reg = linear_model.LinearRegression()
             reg.fit(df.drop('price',axis='columns'),df.price)
             #df.drop('price',axis='columns')

  Out[11]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)

Entrée [8]: reg.coef_

   Out[8]: array([  112.06244194, 23388.88007794, -3231.71790863])

Entrée [12]: reg.intercept_

  Out[12]: 221323.00186540408
```

Find price of home with 3000 sqr ft area, 3 bedrooms, 40 year old

```
Entrée [17]: reg.predict([[3000, 3, 40]])
  Out[17]: array([498408.25158031])
```

```
Entrée [14]: 112.06244194*3000 + 23388.88007794*3 + -3231.71790863*40 + 221323.00186540384
  Out[14]: 498408.25157402386
```

Find price of home with 2500 sqr ft area, 4 bedrooms, 5 year old

```
Entrée [18]: reg.predict([[2500, 3, 40]])
  Out[18]: array([442377.03060924])
```

GET DUMMIES

town	area	price	monroe township	west windsor	robbinsville
monroe township	2600	550000	1	0	0
monroe township	3000	565000	1	0	0
monroe township	3200	610000	1	0	0
monroe township	3600	680000	1	0	0
monroe township	4000	725000	1	0	0
west windsor	2600	585000	0	1	0
west windsor	2800	615000	0	1	0
west windsor	3300	650000	0	1	0
west windsor	3600	710000	0	1	0
robbinsville	2600	575000	0	0	1
robbinsville	2900	600000	0	0	1
robbinsville	3100	620000	0	0	1
robbinsville	3600	695000	0	0	1

```
Entrée [1]: import pandas as pd
```

```
Entrée [2]: df = pd.read_csv("homeprices.csv")
            df
```

Out[2]:

	town	area	price
0	monroe township	2600	550000
1	monroe township	3000	565000
2	monroe township	3200	610000
3	monroe township	3600	680000
4	monroe township	4000	725000
5	west windsor	2600	585000
6	west windsor	2800	615000
7	west windsor	3300	650000
8	west windsor	3600	710000
9	robinsville	2600	575000
10	robinsville	2900	600000
11	robinsville	3100	620000
12	robinsville	3600	695000

Using pandas to create dummy variables

Entrée [3]:
```
dummies = pd.get_dummies(df.town)
dummies
```

Out[3]:

	monroe township	robinsville	west windsor
0	1	0	0
1	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0
5	0	0	1

Entrée [4]:
```
df_dummies= pd.concat([df,dummies],axis='columns')
#df_dummies
```

Entrée [5]:
```
df_dummies.drop('town',axis='columns',inplace=True)
#df_dummies
```

Dummy Variable Trap

When you can derive one variable from other variables, they are known to be multi-colinear. Here if you know values of california and georgia then you can easily infer value of new jersey state, i.e. california=0 and georgia=0. There for these state variables are called to be multi-colinear. In this situation linear regression won't work as expected. Hence you need to drop one column.

NOTE: sklearn library takes care of dummy variable trap hence even if you don't drop one of the state columns it is going to work, however we should make a habit of taking care of dummy variable trap ourselves just in case library that you are using is not handling this for you

Entrée [6]:
```
df_dummies.drop('west windsor',axis='columns',inplace=True)
#df_dummies
```

Entrée [7]:
```
X = df_dummies.drop('price',axis='columns')
y = df_dummies.price

from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X,y)
```

Out[7]:
```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
         normalize=False)
```

Entrée [8]:
```
X
```

Out[8]:

	area	monroe township	robinsville
0	2600	1	0
1	3000	1	0
2	3200	1	0
3	3600	1	0
4	4000	1	0
5	2600	0	0
6	2800	0	0
7	3300	0	0

```
Entrée [9]: model.predict(X)

   Out[9]: array([539709.7398409 , 590468.71640508, 615848.20468716, 666607.18125134,
                  717366.15781551, 579723.71533005, 605103.20361213, 668551.92431735,
                  706621.15674048, 565396.15136531, 603465.38378844, 628844.87207052,
                  692293.59277574])

Entrée [10]: model.score(X,y)

  Out[10]: 0.9573929037221873

Entrée [11]: model.predict([[3300,0,0],[2800,0,1]]) # 3400 sqr ft home in west windsor

  Out[11]: array([668551.92431735, 590775.63964739])

Entrée [12]: model.predict([[2800,0,1]]) # 2800 sqr ft home in robbinsville

  Out[12]: array([590775.63964739])
```

First step is to use label encoder to convert town names into numbers

Entrée [13]:
```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
```

Entrée [14]:
```
dfle = df
dfle.town = le.fit_transform(dfle.town)
dfle
```

Out[14]:

	town	area	price
0	0	2600	550000
1	0	3000	565000
2	0	3200	610000
3	0	3600	680000
4	0	4000	725000
5	2	2600	585000
6	2	2800	615000
7	2	3300	650000
8	2	3600	710000

```
Entrée [15]: X = dfle[['town','area']].values
```

```
Entrée [16]: X
```

```
Out[16]: array([[   0, 2600],
                [   0, 3000],
                [   0, 3200],
                [   0, 3600],
                [   0, 4000],
                [   2, 2600],
                [   2, 2800],
                [   2, 3300],
                [   2, 3600],
                [   1, 2600],
                [   1, 2900],
                [   1, 3100],
                [   1, 3600]])
```

```
Entrée [17]: y = dfle.price.values
             y
```

```
Out[17]: array([550000, 565000, 610000, 680000, 725000, 585000, 615000, 650000,
                710000, 575000, 600000, 620000, 695000])
```

Now use one hot encoder to create dummy variables for each of the town

Now use one hot encoder to create dummy variables for each of the town

Entrée [18]:
```
from sklearn.preprocessing import OneHotEncoder
#ohe = OneHotEncoder(categories ='auto')
ohe = OneHotEncoder(categorical_features=[0])
#ohe = OneHotEncoder(ColumnTransformer)
X
```

Out[18]:
```
array([[   0, 2600],
       [   0, 3000],
       [   0, 3200],
       [   0, 3600],
       [   0, 4000],
       [   2, 2600],
       [   2, 2800],
       [   2, 3300],
       [   2, 3600],
       [   1, 2600],
       [   1, 2900],
       [   1, 3100],
       [   1, 3600]])
```

Entrée [19]:
```
X = ohe.fit_transform(X).toarray()
X
#X = ohe.fit_transform(X[:,0].reshape(-1, 1)).toarray()
#X_encode = ohe.fit_transform(X[:,0].reshape(-1, 1)).toar
#X[:,0].reshape(-1, 1)
```

```
Entrée [22]: model.fit(X,y)

  Out[22]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                            normalize=False)

Entrée [23]: model.predict([[0,1,3400]]) # 3400 sqr ft home in west windsor

  Out[23]: array([681241.6684584])

Entrée [24]: model.predict([[1,0,2800]]) # 2800 sqr ft home in robbinsville

  Out[24]: array([590775.63964739])
```

Saving a Model with Pickle and JobLib

Entrée [1]:
```
import pandas as pd
import numpy as np
from sklearn import linear_model
```

Entrée [2]:
```
df = pd.read_csv("homeprices.csv")
df
```

Out[2]:

	area	price
0	2600	550000
1	3000	565000
2	3200	610000
3	3600	680000
4	4000	725000

Entrée [7]:
```
model = linear_model.LinearRegression()
model.fit(df[['area']],df.price)
#model.fit(df.area,df.price)
#df[['area']]
#df.area
```

Out[7]:
```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
         normalize=False)
```

```
Entrée [4]: model.coef_
   Out[4]: array([135.78767123])

Entrée [5]: model.intercept_
   Out[5]: 180616.43835616432

Entrée [6]: model.predict([[5000],[2000],[10000]])
   Out[6]: array([ 859554.79452055,  452191.78082192, 1538493.15068493])
```

Save Model To a File Using Python Pickle

```
Entrée [7]: import pickle

Entrée [8]: with open('model_pickle','wb') as file:
                pickle.dump(model,file)
```

SAVING MODEL WITH JOBLIB AND PICKLE

Save Model To a File Using Python Pickle

Entrée [11]:
```
import pickle
```

Entrée [12]:
```
with open('model_pickle','wb') as file:
    pickle.dump(model,file)
```

Load Saved Model

Entrée [13]:
```
with open('model_pickle','rb') as file:
    mp = pickle.load(file)
```

Entrée [14]:
```
mp.coef_
```

Out[14]: array([135.78767123])

Entrée [15]:
```
mp.intercept_
```

Out[15]: 180616.43835616432

Entrée [16]:
```
mp.predict([[5000]])
```

Out[16]: array([859554.79452055])

Save Trained Model Using joblib

Entrée [17]:
```
import joblib
```

Entrée [18]:
```
joblib.dump(model, 'model_joblib')
```

Out[18]: ['model_joblib']

Load Saved Model

Entrée [19]:
```
mj = joblib.load('model_joblib')
```

Entrée [20]:
```
mj.coef_
```

Out[20]: array([135.78767123])

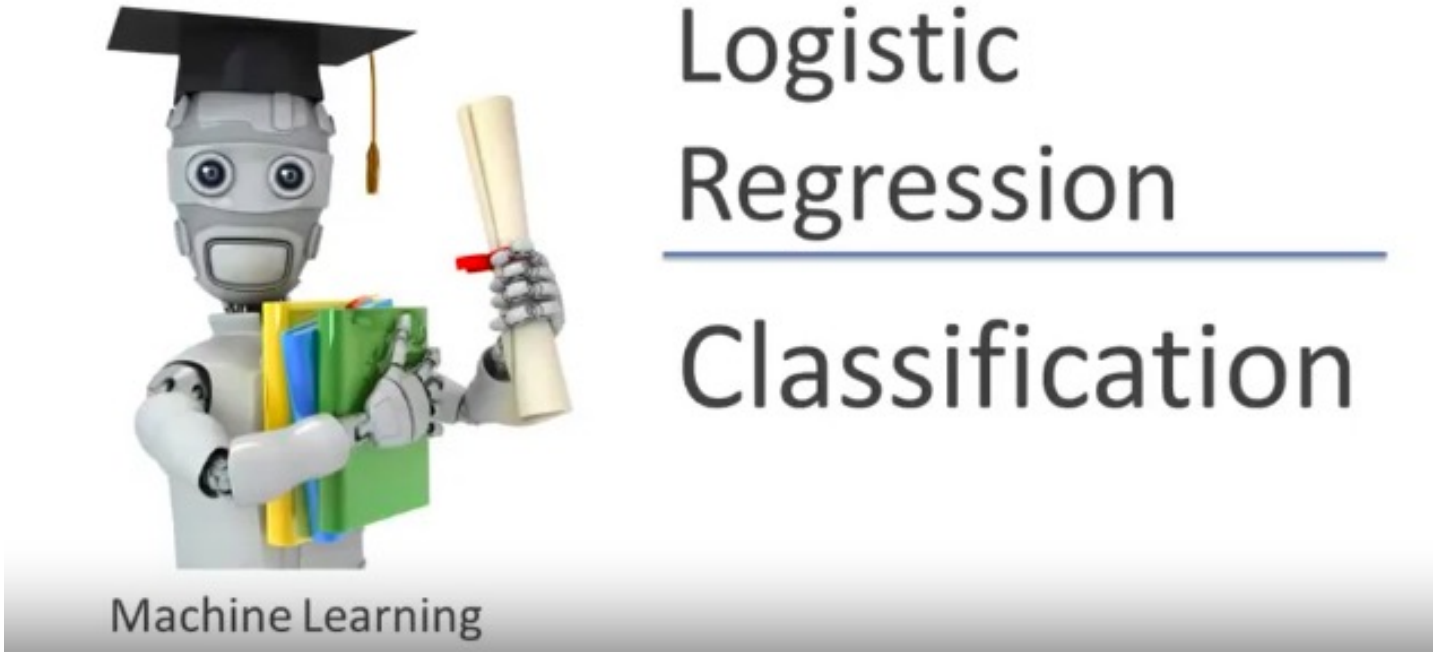
Entrée [21]:
```
mj.intercept_
```

Out[21]: 180616.43835616432

Entrée [18]:
```
mj.predict([[5000]])
```

Out[18]: array([859554.79452055])

Logistic Regression



Classification problems

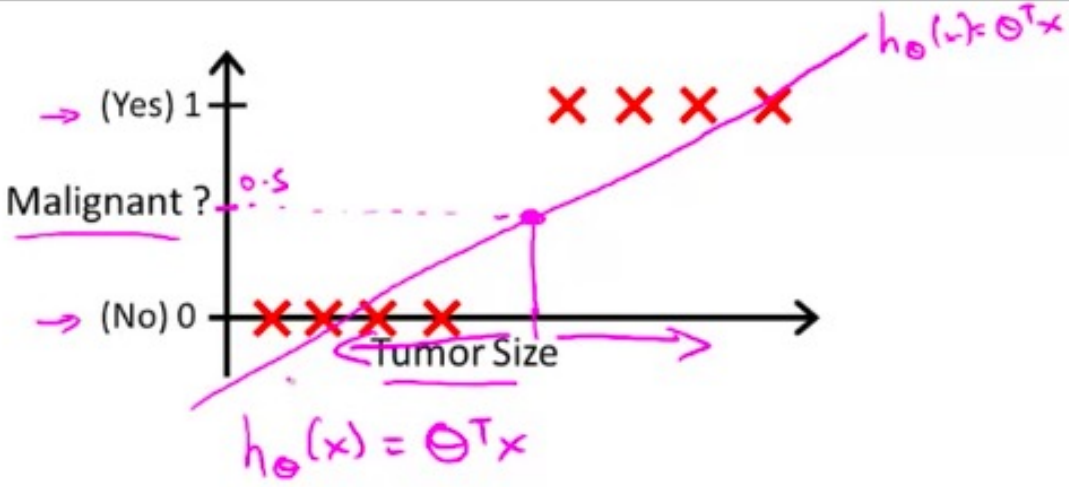
- Email -> spam/not spam?
- Online transactions -> fraudulent?
- Tumor -> Malignant/benign
- House Price: Cheap/Expensive
- Person ->Male/Female

Variable in these problems is Y

a. Y is either 0 or 1 ($Y \in \{0,1\}$)
   i. 0 = negative class (absence of something)
   ii. 1 = positive class (presence of something)

Types of Classification Problem:

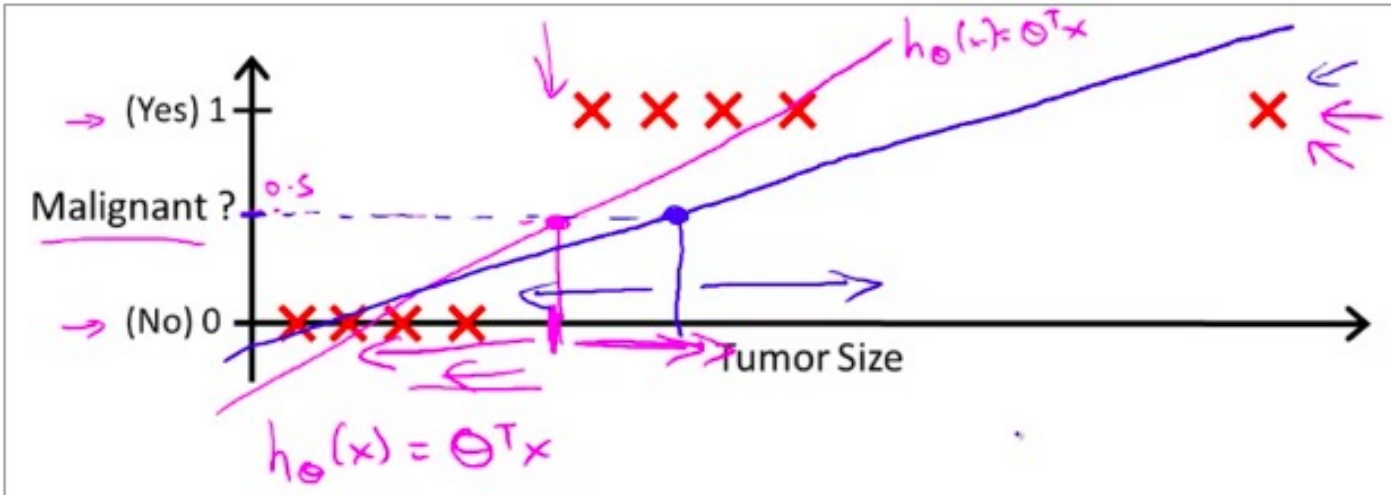
- **binary class problems**
- **multiclass classification problem**



- Threshold classifier output $h_\theta(x)$ at 0.5:
  - If $h_\theta(x) \geq 0.5$, predict "y = 1"
  - If $h_\theta(x) < 0.5$, predict "y = 0"

Andrew Ng

Classification



→ Threshold classifier output $h_\theta(x)$ at 0.5:

→ If $h_\theta(x) \geq 0.5$, predict "y = 1"

If $h_\theta(x) < 0.5$, predict "y = 0"

Andrew Ng

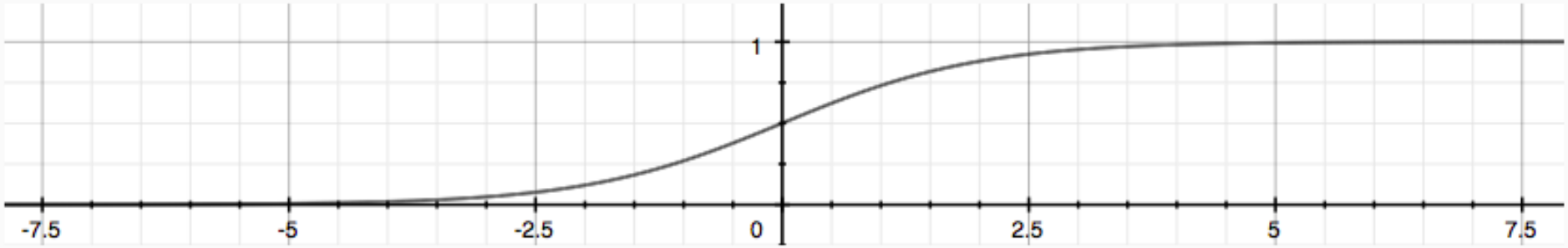
Our new form uses the "Sigmoid Function," also called the "Logistic Function":

$$h_\theta(x) = g(\theta^T x)$$

$$z = \theta^T x$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

The following image shows us what the sigmoid function looks like:



The function g(z), shown here, maps any real number to the (0, 1) interval, making it useful for transforming an arbitrary-valued function into a function better suited for classification.

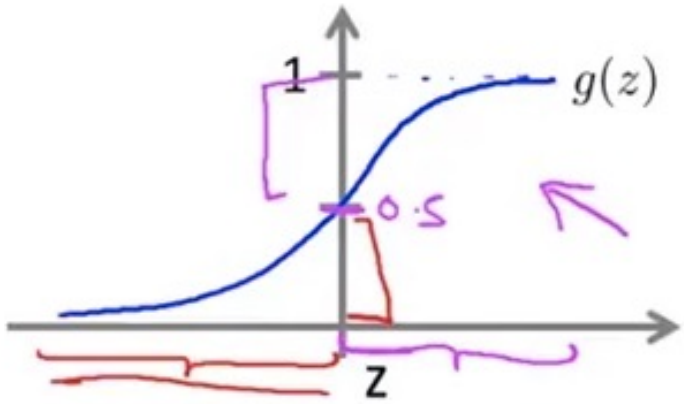
$h_\theta(x)$ will give us the **probability** that our output is 1. For example, $h_\theta(x) = 0.7$ gives us a probability of 70% that our output is 1. Our probability that our prediction is 0 is just the complement of our probability that it is 1 (e.g. if probability that it is 1 is 70%, then the probability that it is 0 is 30%).

$$h_\theta(x) = P(y = 1 | x; \theta) = 1 - P(y = 0 | x; \theta)$$
$$P(y = 0 | x; \theta) + P(y = 1 | x; \theta) = 1$$

Logistic regression

$h_\theta(x) = g(\theta^T x)$ = $P(y=1 \mid x; \theta)$

$g(z) = \frac{1}{1+e^{-z}}$



Suppose predict "$y = 1$" if $h_\theta(x) \geq 0.5$

$\theta^T x \geq 0$

$g(z) \geq 0.5$ when $z \geq 0$

$h_\theta(x) = g(\theta^T x) \geq 0.5$ whenever $\theta^T x \geq 0$ (z)

predict "$y = 0$" if $h_\theta(x) < 0.5$

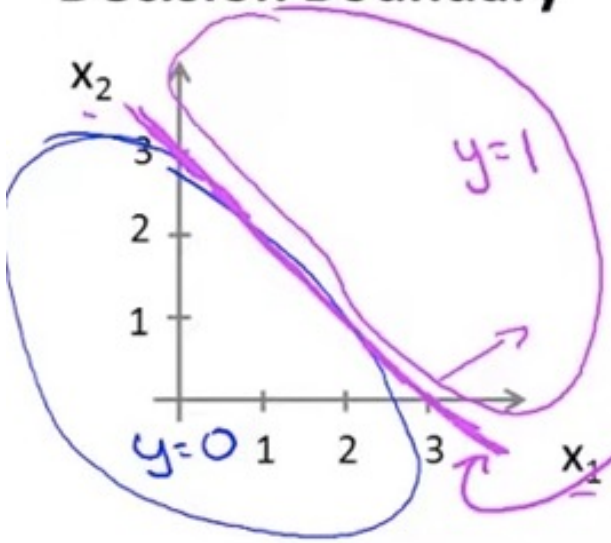
$h_\theta(x) = g(\theta^T x)$

$\theta^T x < 0$

$g(z) < 0.5$

Andrew Ng

Decision Boundary



$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$

$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

$-3 \quad 1 \quad 1$

Decision boundary

Predict "$y = 1$" if $-3 + x_1 + x_2 \geq 0$

$\theta^T x$

x_1, x_2

$h_\theta(x) = 0.5$

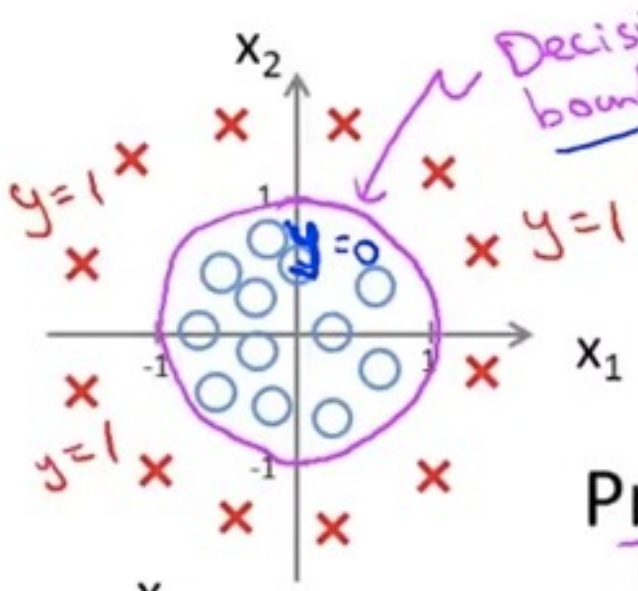
$x_1 + x_2 = 3$

$x_1 + x_2 \geq 3$

$x_1 + x_2 < 3$

$y = 0$

Non-linear decision boundaries



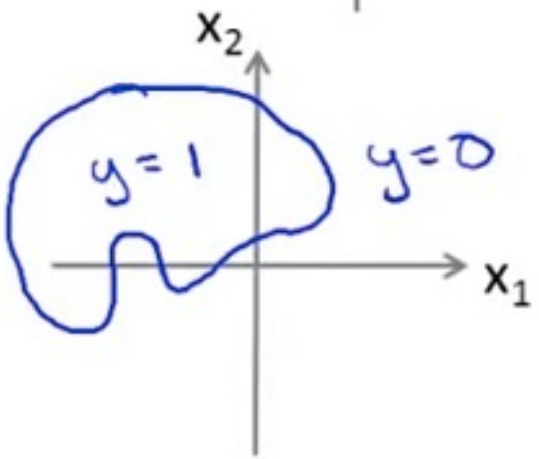
$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Predict "$y = 1$" if $-1 + x_1^2 + x_2^2 \geq 0$

$x_1^2 + x_2^2 = 1$

$x_1^2 + x_2^2 \geq 1$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \theta_6 x_1^3 x_2 + \dots)$$

Cost function

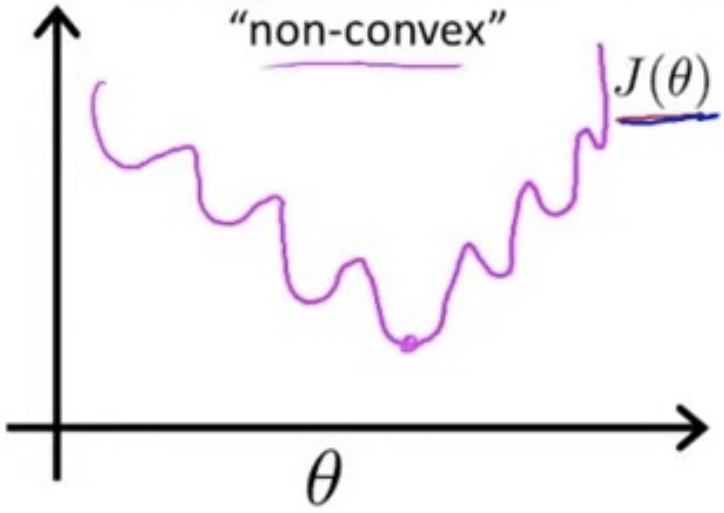
→ ~~Linear~~ regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

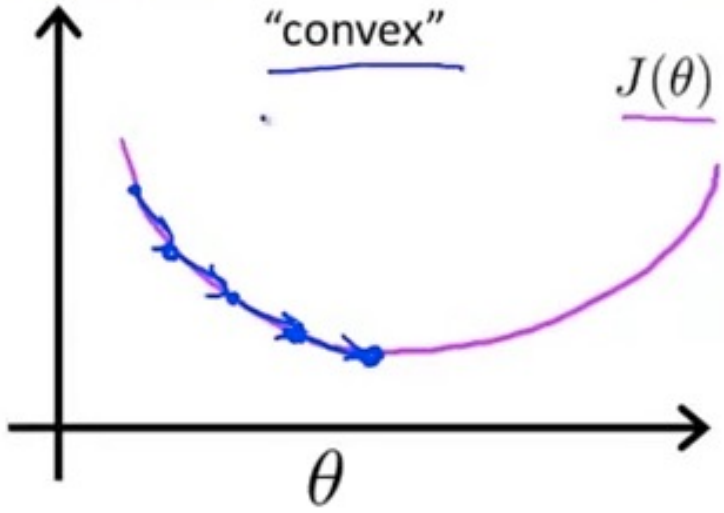
Logistic

$\rightarrow \text{cost}(h_\theta(x^{(i)}), y)$

$\frac{1}{1 + e^{-\theta^T x}}$

$\rightarrow \text{Cost}(h_\theta(x), y) = \frac{1}{2} \left(h_\theta(x) - y\right)^2$ ←

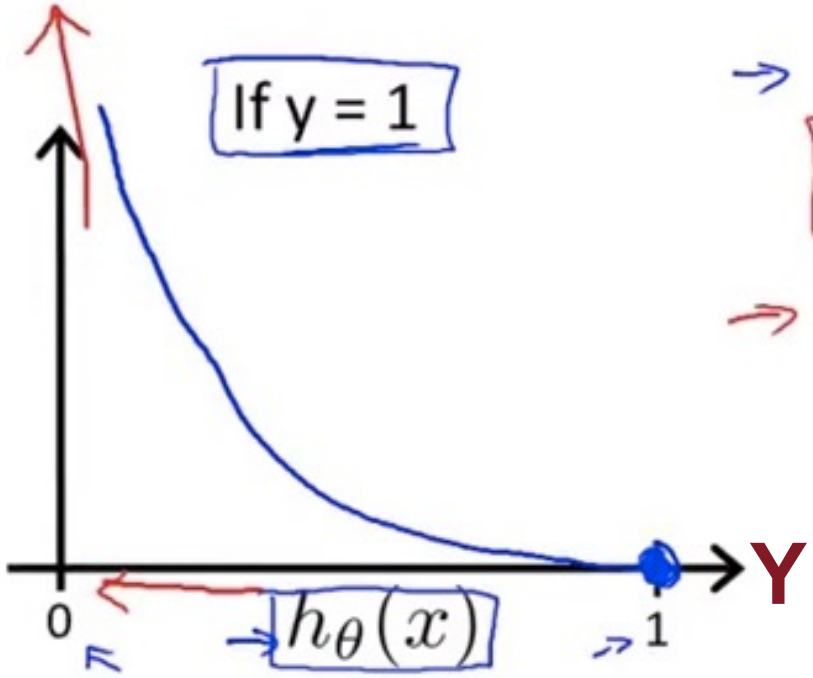




Andrew Ng

Logistic regression cost function

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$



→ Cost $= 0$ if $y = 1, h_\theta(x) = 1$

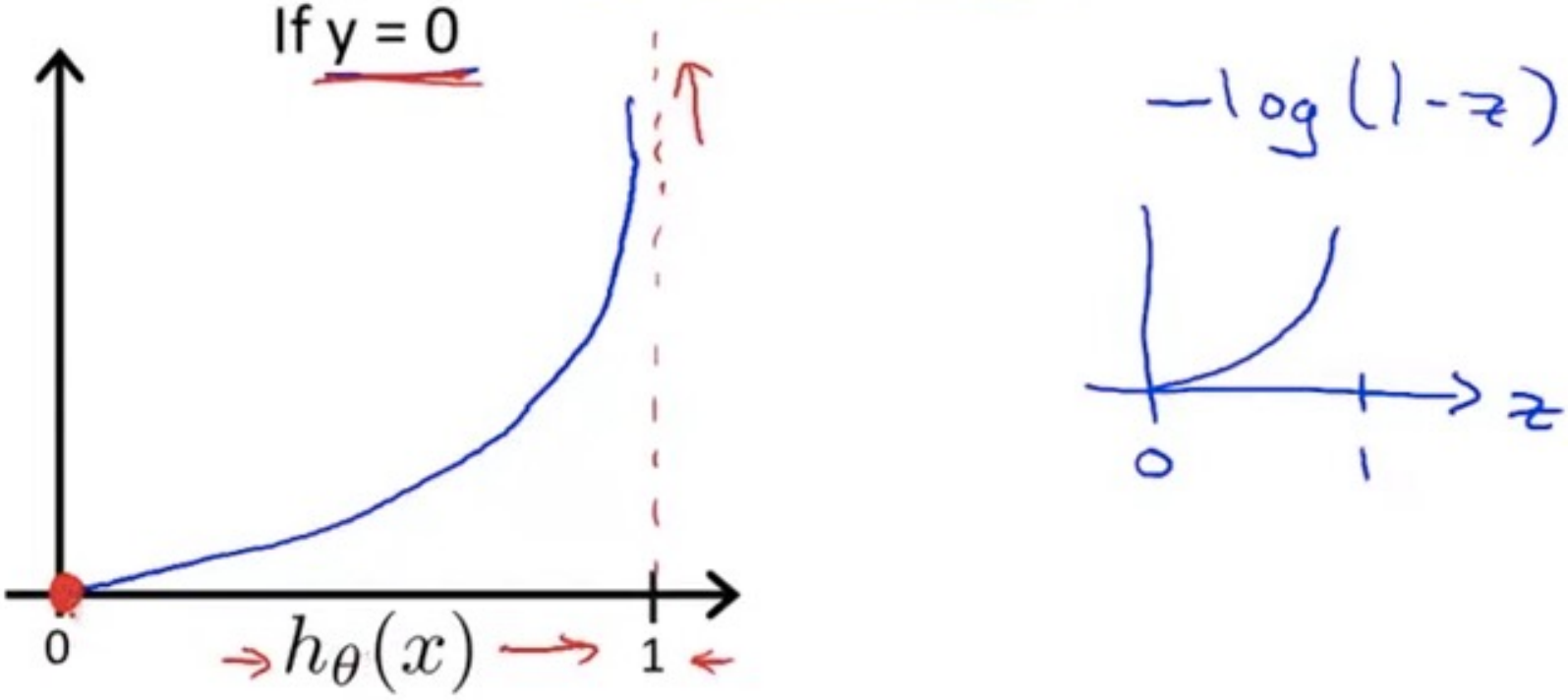
But as $h_\theta(x) \to 0$

$Cost \to \infty$

→ Captures intuition that if $h_\theta(x) = 0$, (predict $P(y = 1|x; \theta) = 0$), but $y = 1$, we'll penalize learning algorithm by a very large cost.

Logistic regression cost function

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$



Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

(simultaneously update all θ_j)

$\}$

Want $\min_\theta J(\theta)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

$\}$

Optimization algorithm

Given θ, we have code that can compute

- $J(\theta)$
- $\frac{\partial}{\partial \theta_j} J(\theta)$ (for $j = 0, 1, \ldots, n$)

Optimization algorithms:

- Gradient descent
- Conjugate gradient
- BFGS
- L-BFGS

Optimizer Comparison Table

Optimizer	Type	Learning Rate	Memory	Convergence	Best Use Case
Batch GD	First-order	Required	O(n)	Linear	Small datasets
SGD	First-order	Schedule needed	O(1)	Sublinear	Large datasets, online
Mini-batch	First-order	Required	O(b)	Linear	Most practical cases
L-BFGS	Quasi-Newton	Not needed	O(m·d)	Superlinear	Small/medium datasets
Newton	Second-order	Not needed	$O(d^2)$	Quadratic	Small, well-conditioned
CG	Krylov subspace	Not needed	O(d)	Linear	Large n_features
SAGA	Variance-reduced	Required	O(n+d)	Linear	Medium datasets

Where: d = number of features, n = number of samples, b = batch size, m = memory size (L-BFGS)

sklearn.linear_model.LogisticRegression

class `sklearn.linear_model.`**`LogisticRegression`**(*penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0, warm_start=False, n_jobs=None, l1_ratio=None*) [source]

Logistic Regression (aka logit, MaxEnt) classifier.

solver : *{'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'}, default='lbfgs'*

Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects:

- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- For multiclass problems, only 'newton-cg', 'sag', 'saga' and 'lbfgs' handle multinomial loss;
- 'liblinear' and is limited to one-versus-rest schemes.
- 'newton-cholesky' is a good choice for `n_samples` >> `n_features`, especially with one-hot encoded categorical features with rare categories. Note that it is limited to binary classification and the one-versus-rest reduction for multiclass classification. Be aware that the memory usage of this solver has a quadratic dependency on `n_features` because it explicitly computes the Hessian matrix.

Warning: The choice of the algorithm depends on the penalty chosen. Supported penalties by solver:

- 'lbfgs' - ['l2', None]
- 'liblinear' - ['l1', 'l2']
- 'newton-cg' - ['l2', None]
- 'newton-cholesky' - ['l2', None]
- 'sag' - ['l2', None]
- 'saga' - ['elasticnet', 'l1', 'l2', None]

ACTIVITIES IN LOGISTIC REGRESSION

Predicting if a person would buy life insurance based on his age using logistic regression

Entrée [1]:

```
import pandas as pd
from matplotlib import pyplot as plt
%matplotlib inline
```

Entrée [2]:

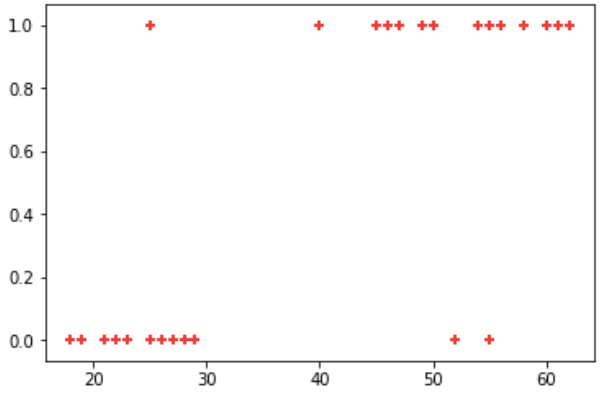
```
df = pd.read_csv("insurance_data.csv")
df.head()
#df
```

Out[2]:

	age	bought_insurance
0	22	0
1	25	0
2	47	1
3	52	0
4	46	1

```
: plt.scatter(df.age,df.bought_insurance,marker='+',color='red')
```

```
<matplotlib.collections.PathCollection at 0x11c1c6c18>
```



[4]:
```
from sklearn.model_selection import train_test_split
```

[20]:
```
X_train, X_test, y_train, y_test = train_test_split(df[['age']],df.bought_insurance,train_size=0.9)
```

[21]:
```
X_test
```

21]:

	age
18	19
17	58
1	25

[16]:
```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(solver='lbfgs')
```

[17]:
```
model.fit(X_train, y_train)
```

17]:
```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                   intercept_scaling=1, l1_ratio=None, max_iter=100,
                   multi_class='warn', n_jobs=None, penalty='l2',
                   random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                   warm_start=False)
```

```
[9]: X_test
```

[9]:

	age
13	29
24	50
21	26

```
[10]: y_predicted = model.predict(X_test)
      y_predicted
```

```
10]: array([0, 1, 0])
```

```
[11]: model.predict_proba(X_test)
```

```
11]: array([[0.51968091, 0.48031909],
            [0.26512076, 0.73487924],
            [0.55864439, 0.44135561]])
```

```
[12]: model.score(X_test,y_test)
```

```
12]: 1.0
```

```
[13]: X_test
```

MULTICLASS CLASSIFICATION

Multiclass classification

Email foldering/tagging: Work, Friends, Family, Hobby

$y=1$ $y=2$ $y=3$ $y=4$

Medical diagrams: Not ill, Cold, Flu

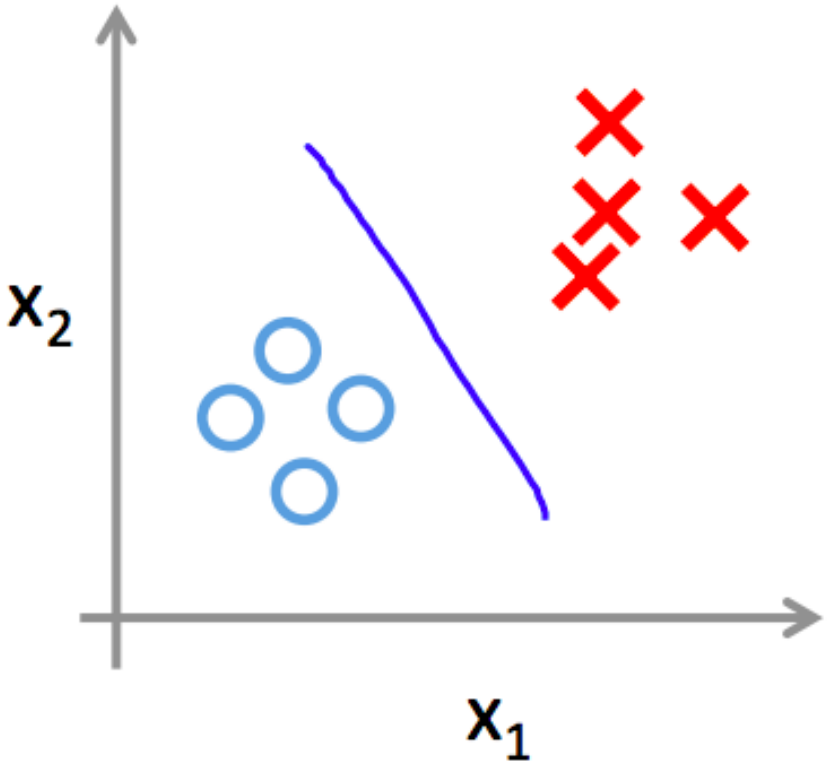
$y=1$ 2 3

Weather: Sunny, Cloudy, Rain, Snow

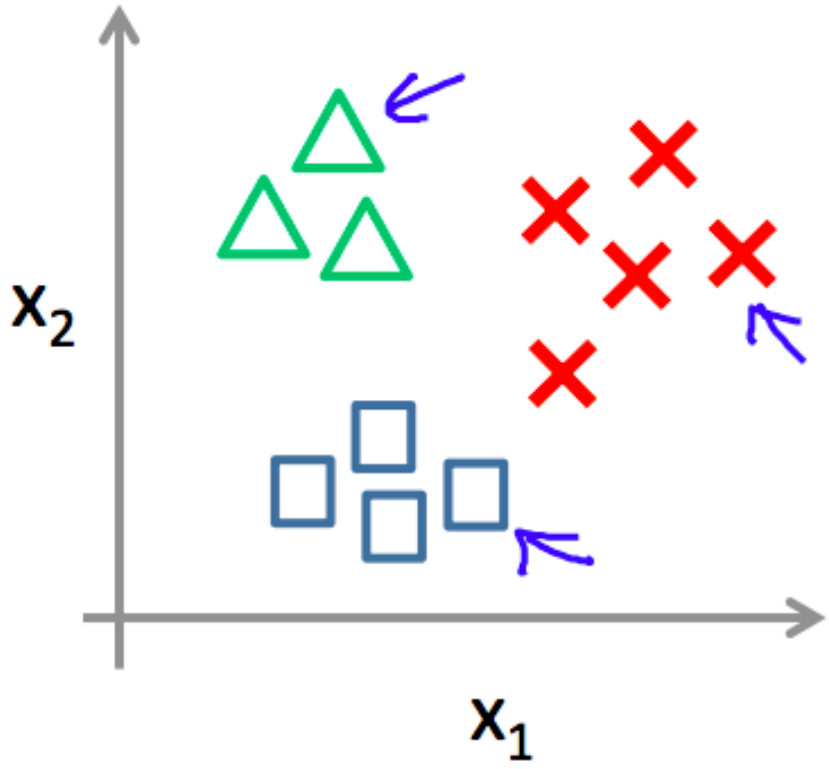
$y=1$ 2 3 4

0 1 2 3

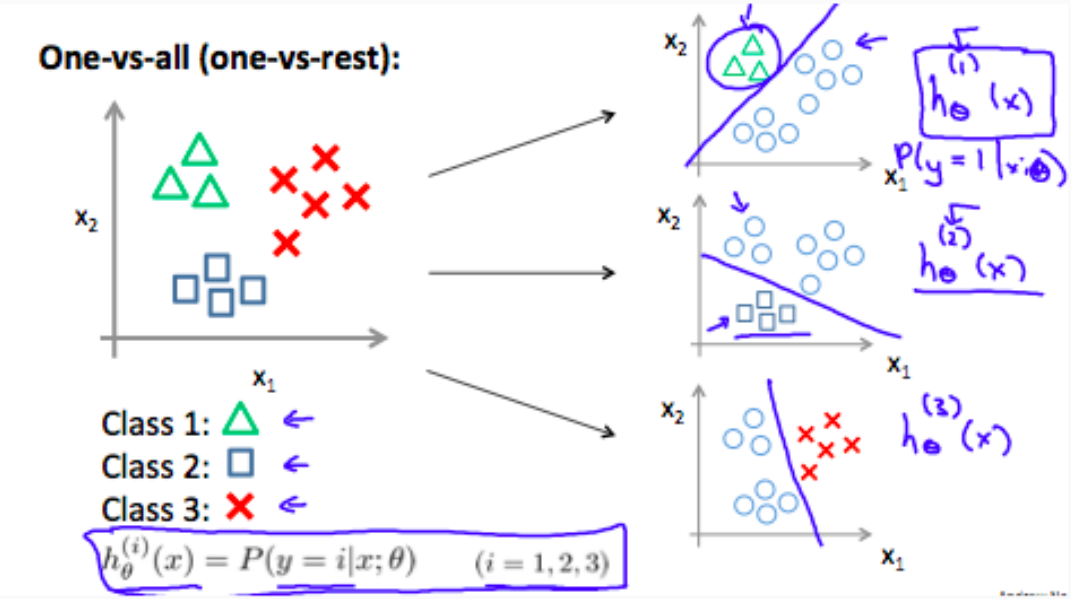
Binary classification:



Multi-class classification:



The following image shows how one could classify 3 classes:



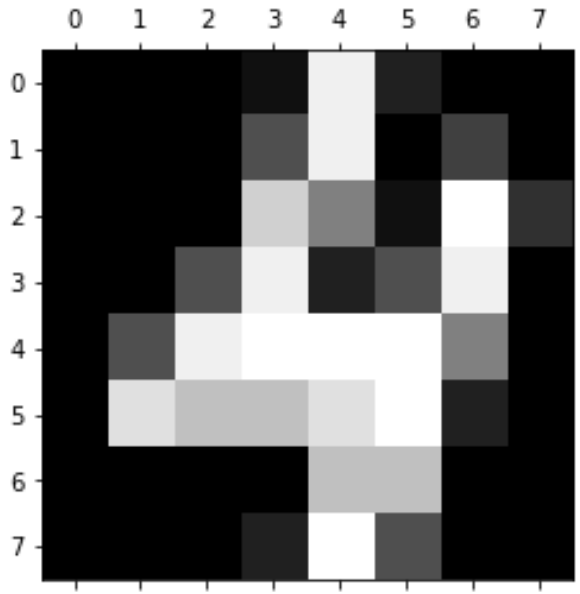
To summarize:

Train a logistic regression classifier $h_\theta(x)$ for each class to predict the probability that y = i .

To make a prediction on a new x, pick the class that maximizes $h_\theta(x)$

Logistic Regression: Multiclass Classification

```
[2]: from sklearn.datasets import load_digits
     %matplotlib inline
     import matplotlib.pyplot as plt
     digits = load_digits()

[6]: digits.target.shape

6]: (1797,)

[ ]:

[3]: plt.gray()
     plt.matshow(digits.images[1001])
     print(digits.target[1001])

     4
```




```
4]: plt.gray()
    for i in range(5):
        plt.matshow(digits.images[i])
        print(digits.target[i])
```

```
: dir(digits)
```

```
['DESCR', 'data', 'images', 'target', 'target_names']
```

```
]: digits.DESCR
```

```
".. _digits_dataset:\n\nOptical recognition of handwritten digits dataset\n---------
--------\n\n**Data Set Characteristics:**\n\n    :Number of Instances: 5620\n    :Nu
ibute Information: 8x8 image of integer pixels in the range 0..16.\n    :Missing Att
r: E. Alpaydin (alpaydin '@' boun.edu.tr)\n    :Date: July; 1998\n\nThis is a copy o
```

Create and train logistic regression model

```
[6]: from sklearn.linear_model import LogisticRegression
     model = LogisticRegression(multi_class='auto',solver='lbfgs',max_iter=10000)
```

```
[7]: from sklearn.model_selection import train_test_split
```

```
[8]: X_train, X_test, y_train, y_test = train_test_split(digits.data,digits.target, test_size=0.2)
```

```
[15]: model.fit(X_train, y_train)
```

```
.5]: LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                        intercept_scaling=1, l1_ratio=None, max_iter=10000,
                        multi_class='auto', n_jobs=None, penalty='l2',
                        random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                        warm_start=False)
```

Measure accuracy of our model

```
]: model.score(X_test, y_test)
```

```
0.975
```

```
]: model.predict(digits.data[0:5])
   #model.predict([digits.data[1]])
```

```
array([0, 1, 2, 3, 4])
```

Confusion Matrix

```
]: y_predicted = model.predict(X_test)
```

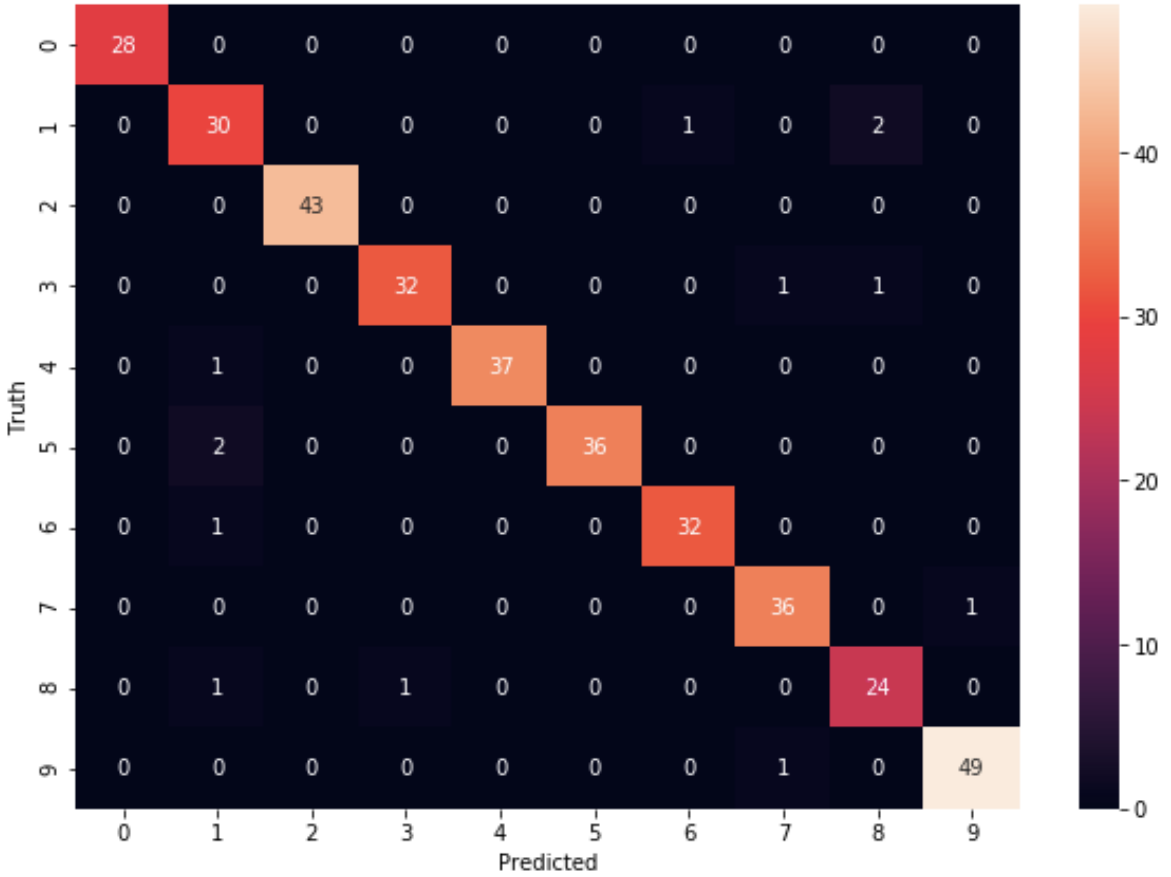
```
]: from sklearn.metrics import confusion_matrix
   cm = confusion_matrix(y_test, y_predicted)
   cm
```

```
array([[28,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0, 30,  0,  0,  0,  0,  1,  0,  2,  0],
       [ 0,  0, 43,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0, 32,  0,  0,  0,  1,  1,  0],
       [ 0,  1,  0,  0, 37,  0,  0,  0,  0,  0],
       [ 0,  2,  0,  0,  0, 36,  0,  0,  0,  0],
       [ 0,  1,  0,  0,  0,  0, 32,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0, 36,  0,  1],
       [ 0,  1,  0,  1,  0,  0,  0,  0, 24,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  1,  0, 49]])
```

Confusion Matrix

```
[13]: import seaborn as sn
      plt.figure(figsize = (10,7))
      sn.heatmap(cm, annot=True)
      plt.xlabel('Predicted')
      plt.ylabel('Truth')
```

```
3]: Text(69.0, 0.5, 'Truth')
```



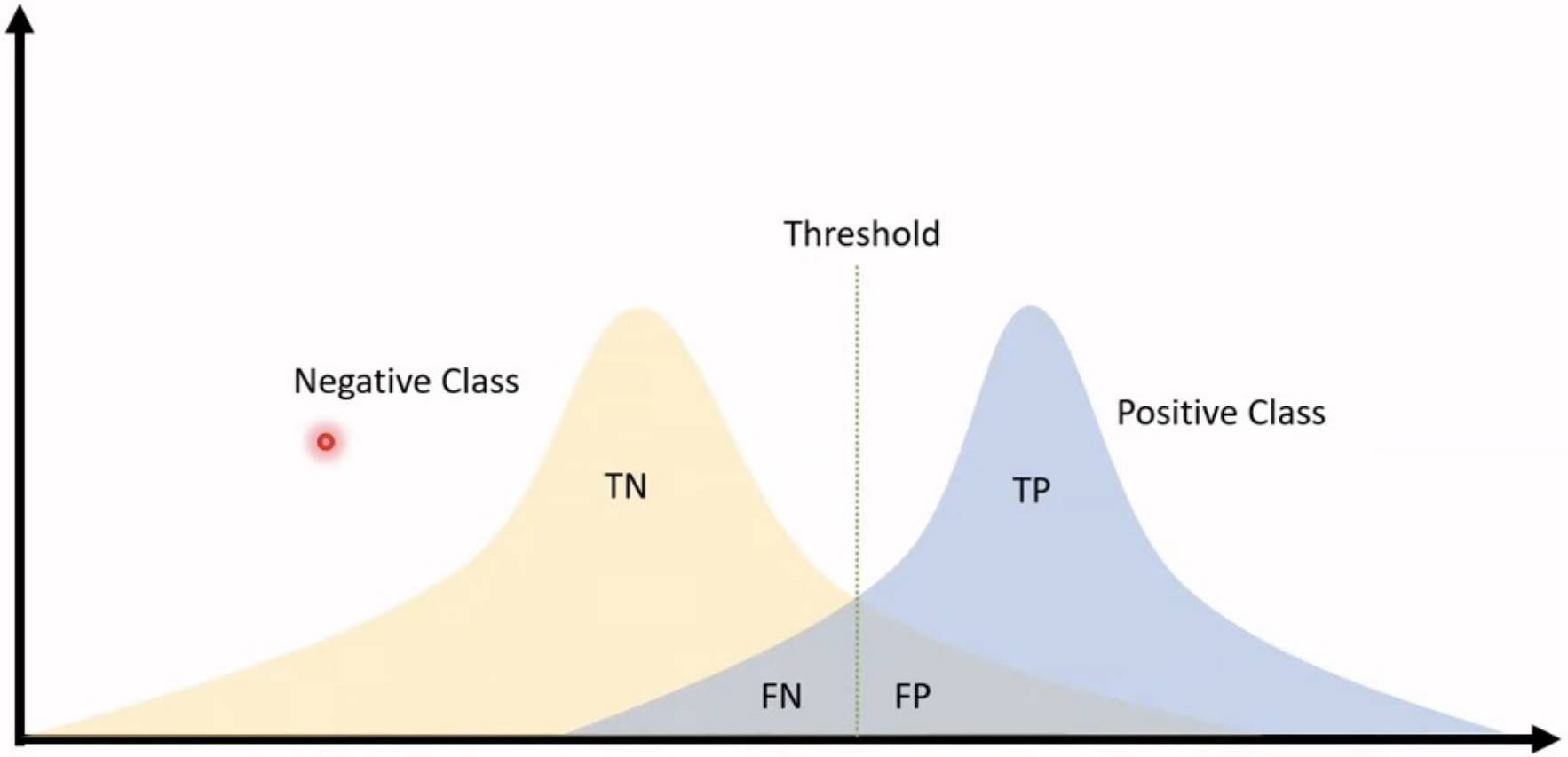
Possible Outcomes

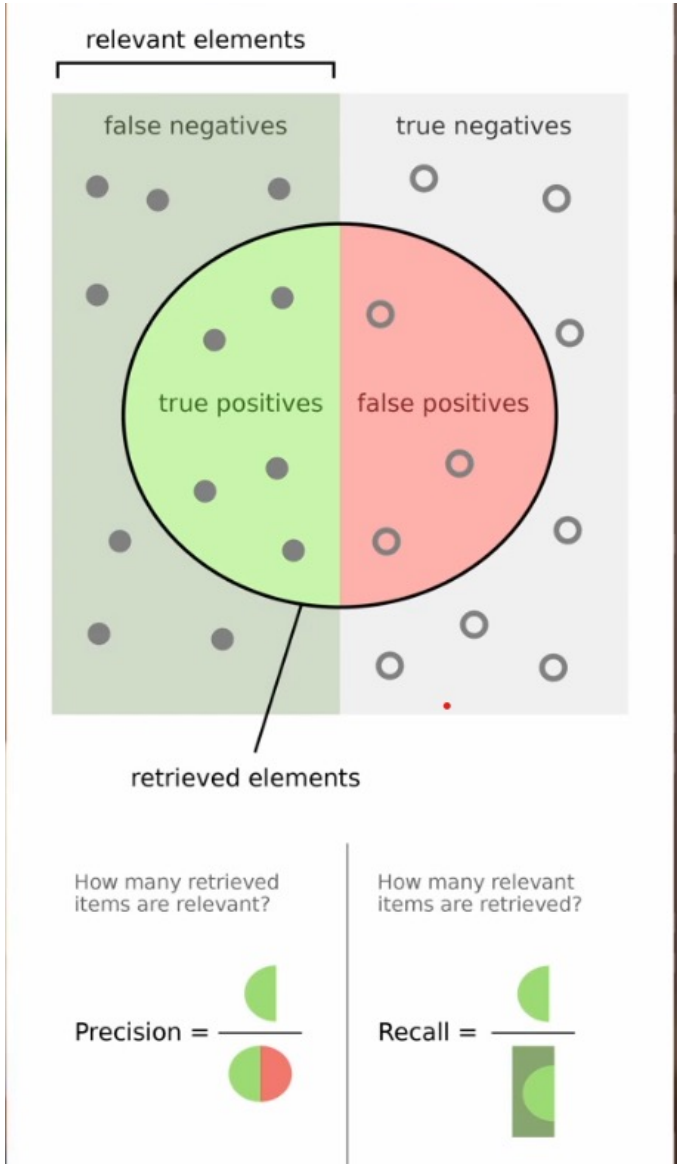
- True Positive (TP)
- False Positive (FP) - Type I Error/False Alarm
- False Negative (FN) - Type II Error
- True Negative (TN)

1. Confusion Matrix Template

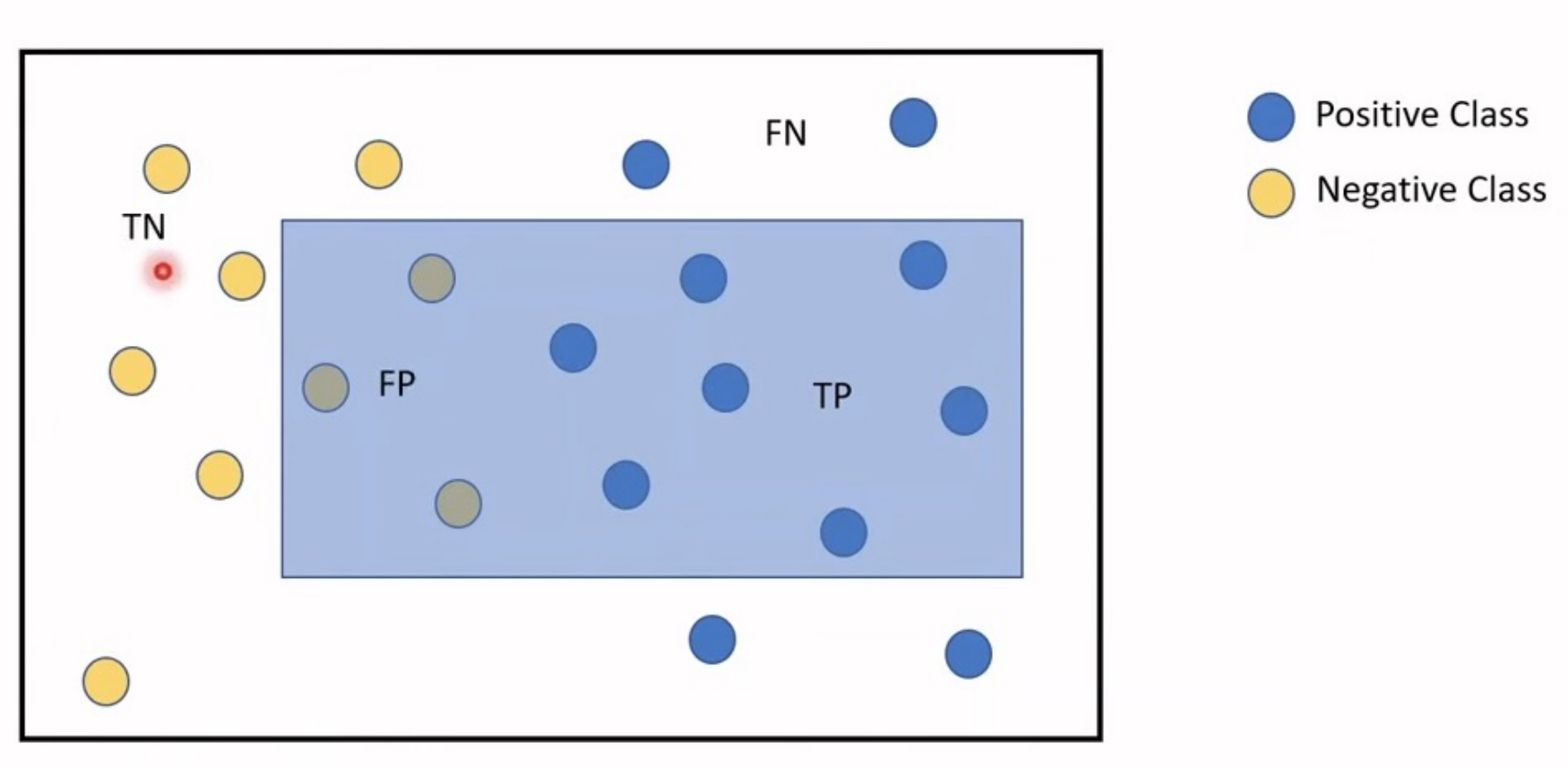
Actual \ Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

Visualisation



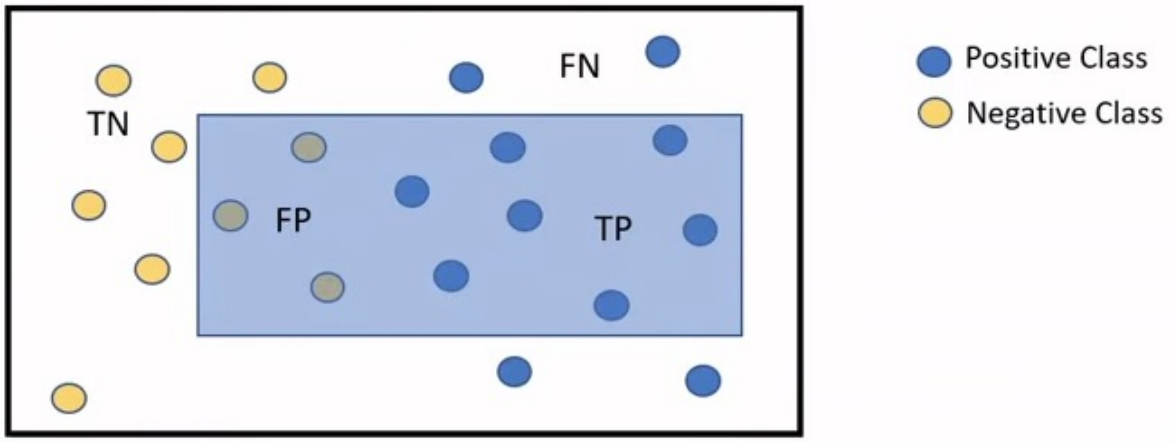

relevant elements
false negatives
true negatives
true positives
false positives
retrieved elements
How many retrieved items are relevant?
Precision =
How many relevant items are retrieved?
Recall =

Visualisation



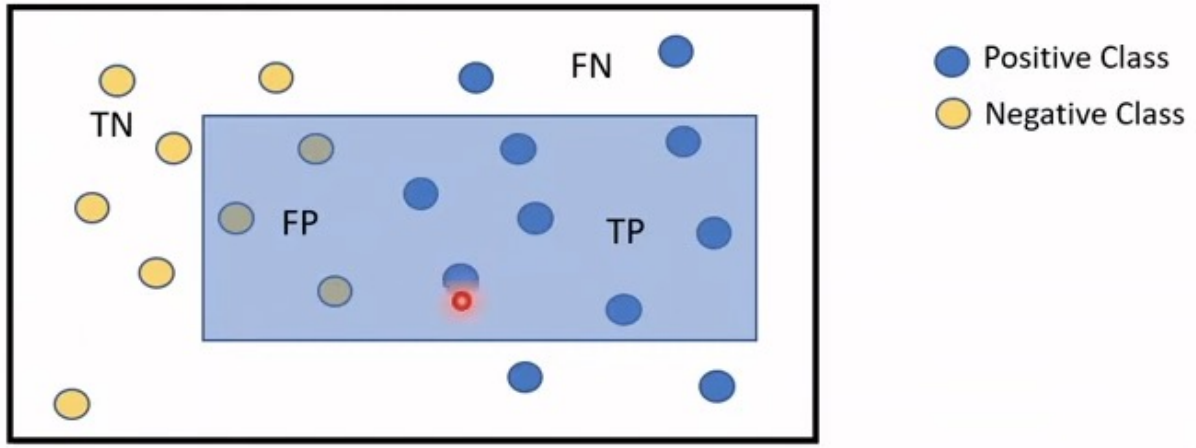
Accuracy

- $Accuracy = \frac{TP+TN}{TP+FP+FN+TN}$



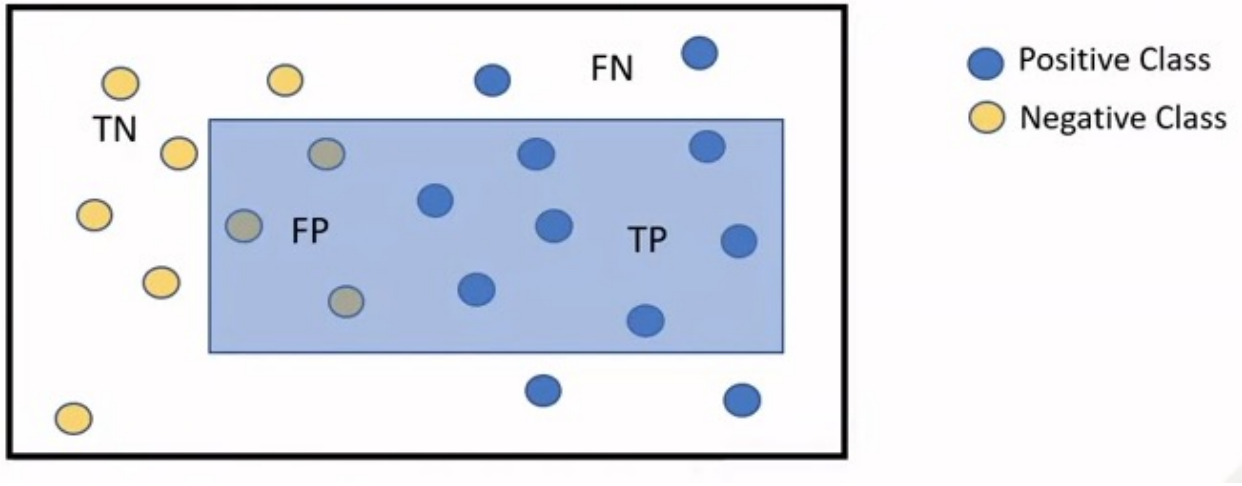
Precision

- $Precision = \frac{TP}{TP+FP}$



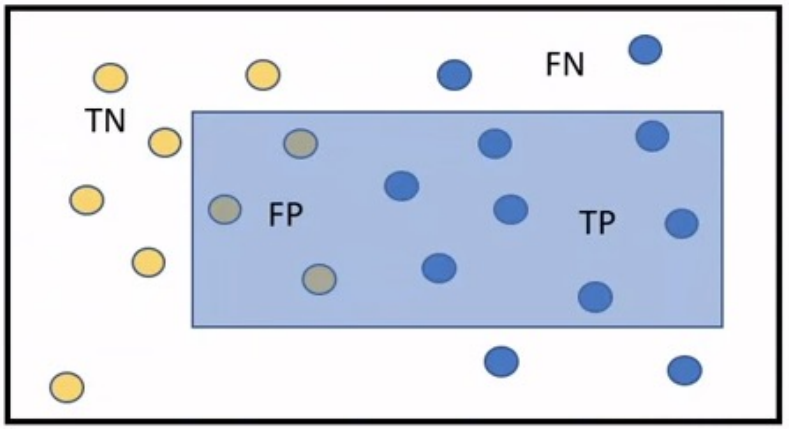
Recall/Sensitivity/TPR (True Positive Rate)

- $Recall = \frac{TP}{TP+FN}$



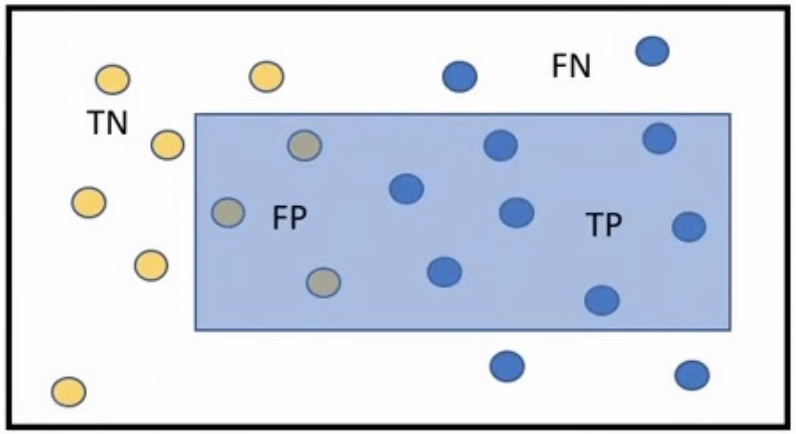
Specificity/TNR

- Specificity is also referred to as: true negative rate (TNR)
- $Specicity = \frac{TN}{TN+FP}$



F1-Score/F-Measure

- $F1 - score = 2 * \frac{(Recall * Precision)}{(Recall + Precision)}$



EXAMPLE

2. Example Values

Assume the following confusion matrix values for calculation:

- **TP = 45** (Correctly predicted positives)
- **TN = 35** (Correctly predicted negatives)
- **FP = 15** (Incorrectly predicted positives)
- **FN = 5** (Incorrectly predicted negatives)

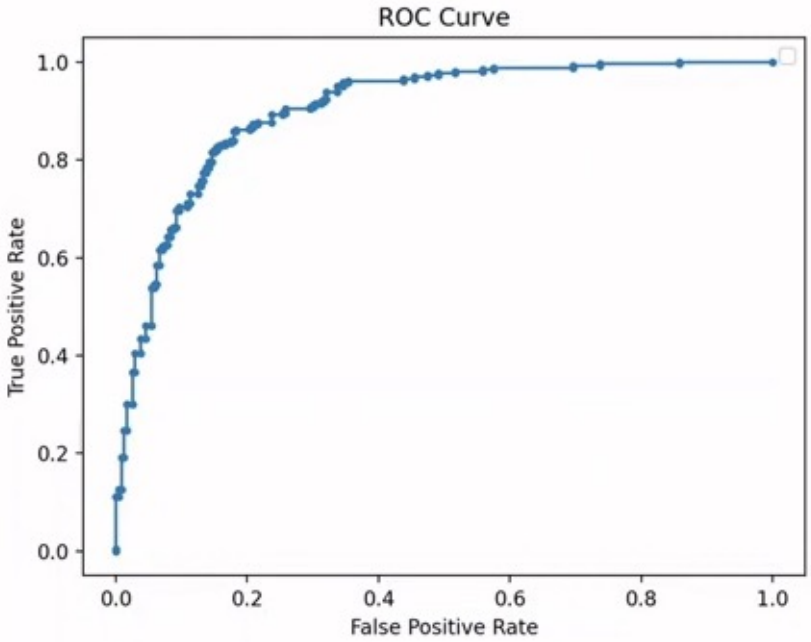
Actual \ Predicted	Positive	Negative
Positive	TP = 45	FN = 5
Negative	FP = 15	TN = 35

3. Metric Formulas & Calculations

Metric	Formula	Calculation	Result	Interpretation
TPR (Recall)	$\frac{TP}{TP+FN}$	$\frac{45}{45+5}$	**0.90**	90% of actual positives are correctly identified.
TNR	$\frac{TN}{TN+FP}$	$\frac{35}{35+15}$	**0.70**	70% of actual negatives are correctly identified.
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	$\frac{45+35}{100}$	**0.80**	80% of all predictions are correct.
Precision	$\frac{TP}{TP+FP}$	$\frac{45}{45+15}$	**0.75**	75% of predicted positives are true positives.

ROC

- ROC stands for receiver operating characteristic curve
- It shows the performance of a classification model at all classification thresholds
- The curve is plotted by two parameters:
  - $true\ positive\ rate = \frac{TP}{TP+FN}$
  - $false\ positive\ rate = \frac{FP}{FP+TN}$

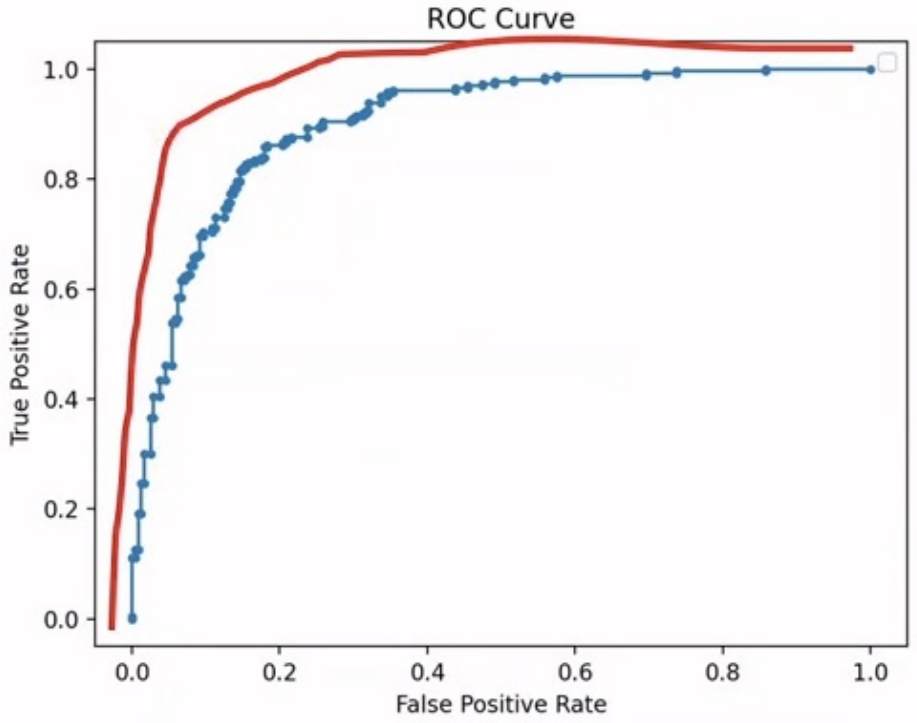


AUC

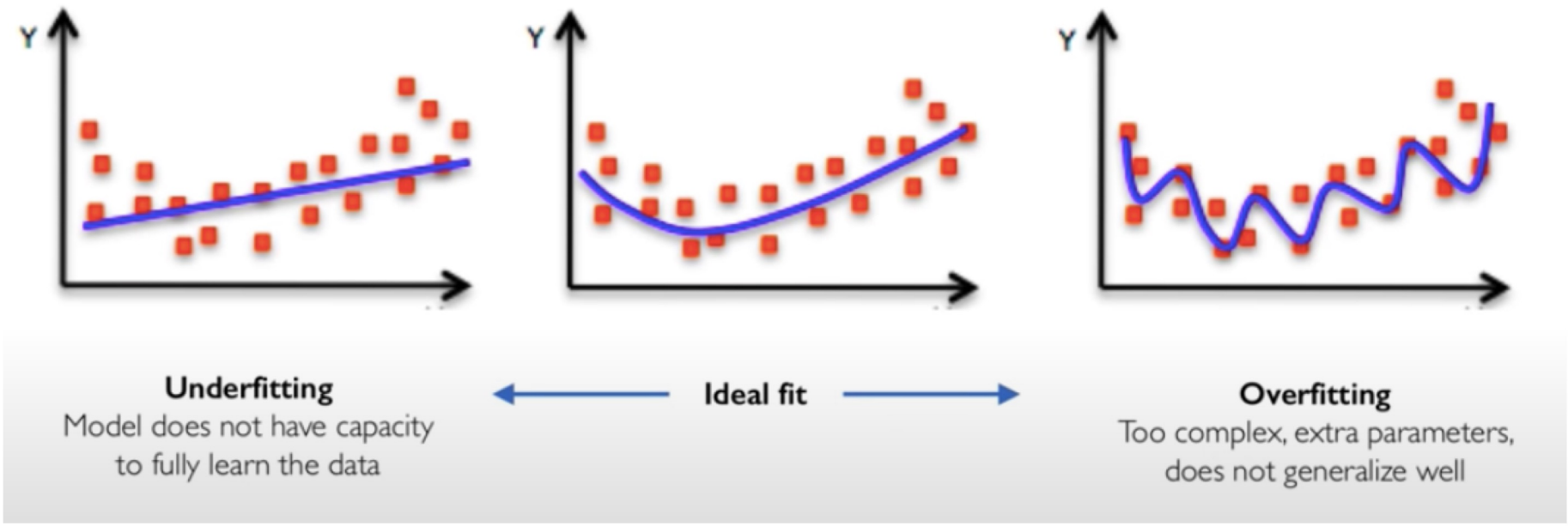
- It helps evaluate several Models

- The one with largest area under the curve is the best (The one in red)

• AUC stands for "Area under the ROC Curve"



UNDERFITTING AND OVERFITTING



Variance Vs Bias

- **High training error, high test error** → High Bias (Underfitting)
- **Low training error, high test error** → High Variance (Overfitting)
- **Add complexity** to reduce bias
- **Add regularization/simplify** to reduce variance
- **More data** helps with variance but not bias

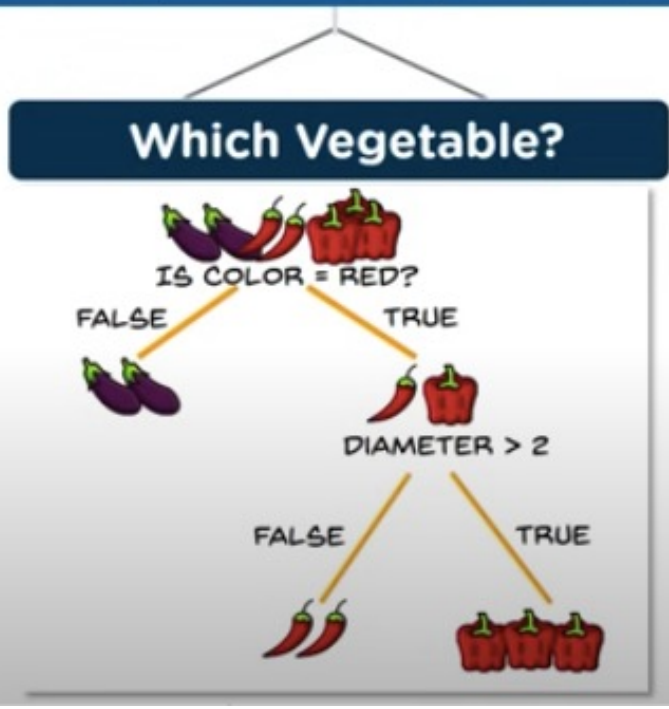
Variance Vs Bias

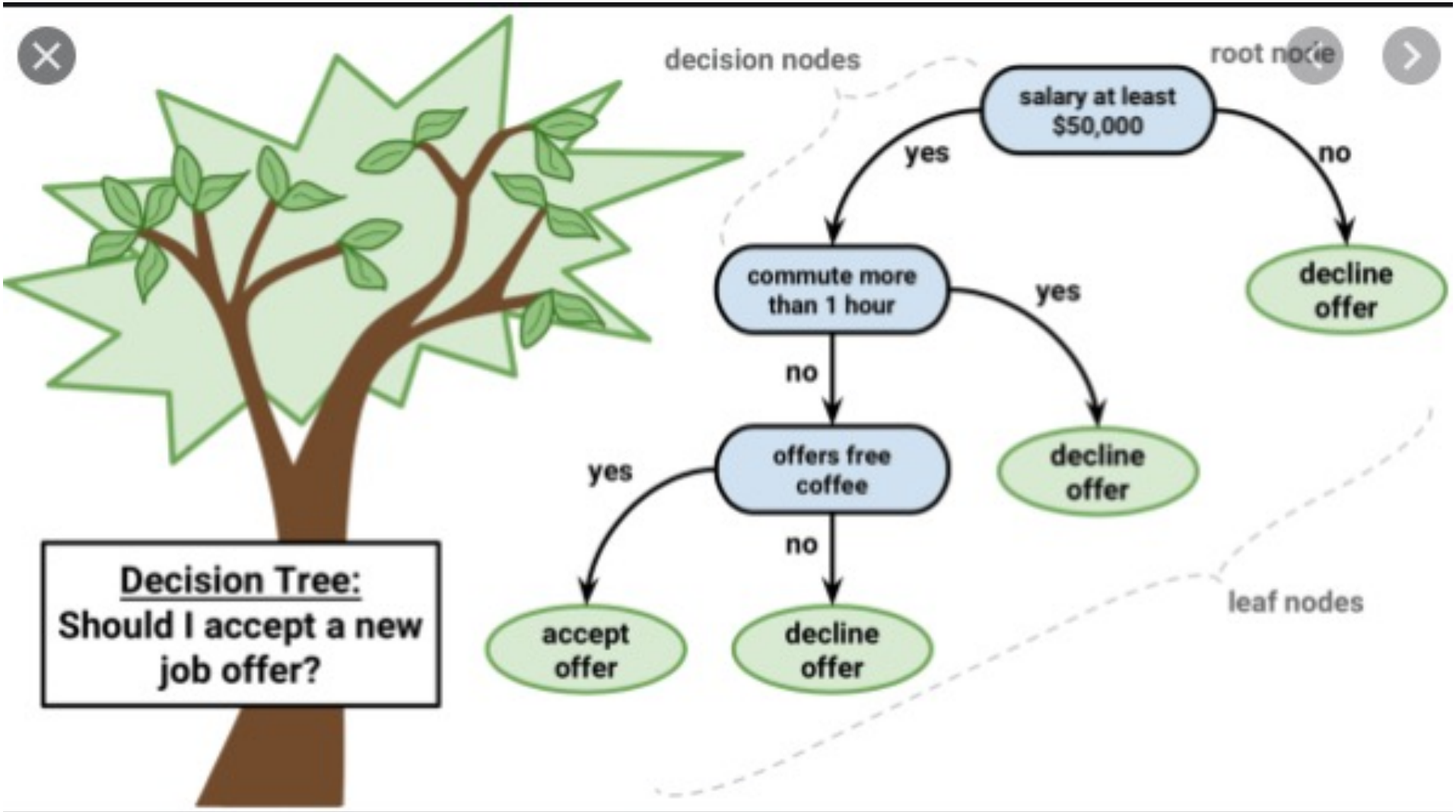
Aspect	High Bias	High Variance
Model Complexity	Too simple	Too complex
Training Performance	Poor	Excellent
Test Performance	Poor	Poor
Error Type	Systematic	Random
Solution	Add features, Use more complex model	Simplify model, Regularization
Example Models	Linear regression on nonlinear data	Deep neural network on small dataset

DECISION TREE CLASSIFIER

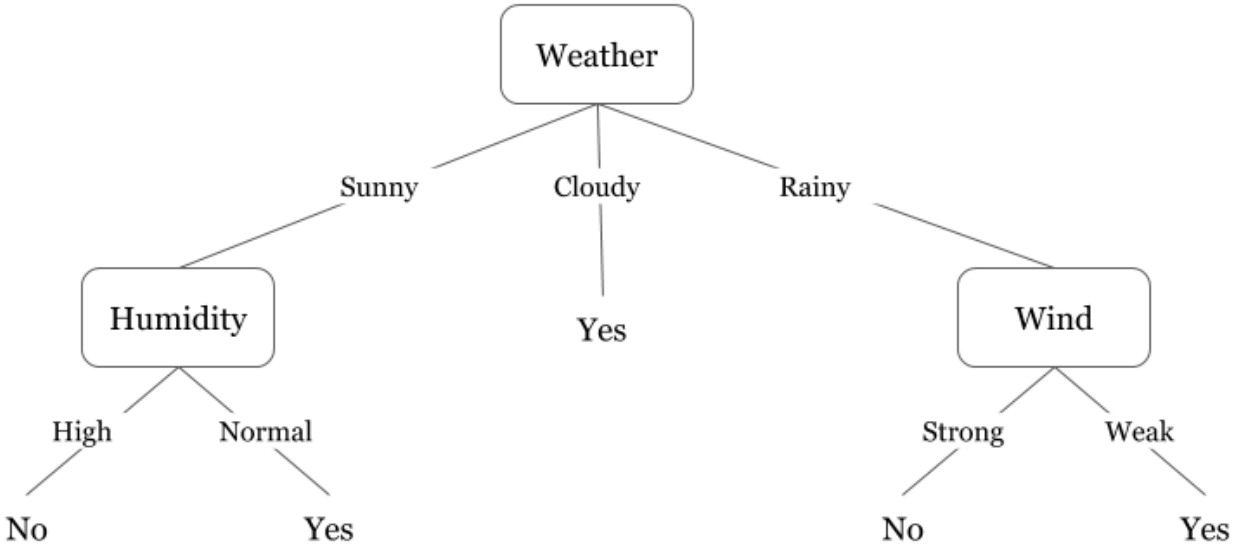
What is Decision Tree?

Decision Tree is a tree shaped diagram used to determine a course of action. Each branch of the tree represents a possible decision, occurrence or reaction




decision nodes
root node
salary at least $50,000
yes
no
commute more than 1 hour
yes
decline offer
no
offers free coffee
yes
decline offer
no
accept offer
decline offer
leaf nodes
Decision Tree:
Should I accept a new job offer?

Base on the weather condition, can we go OUT???
YES OR NO

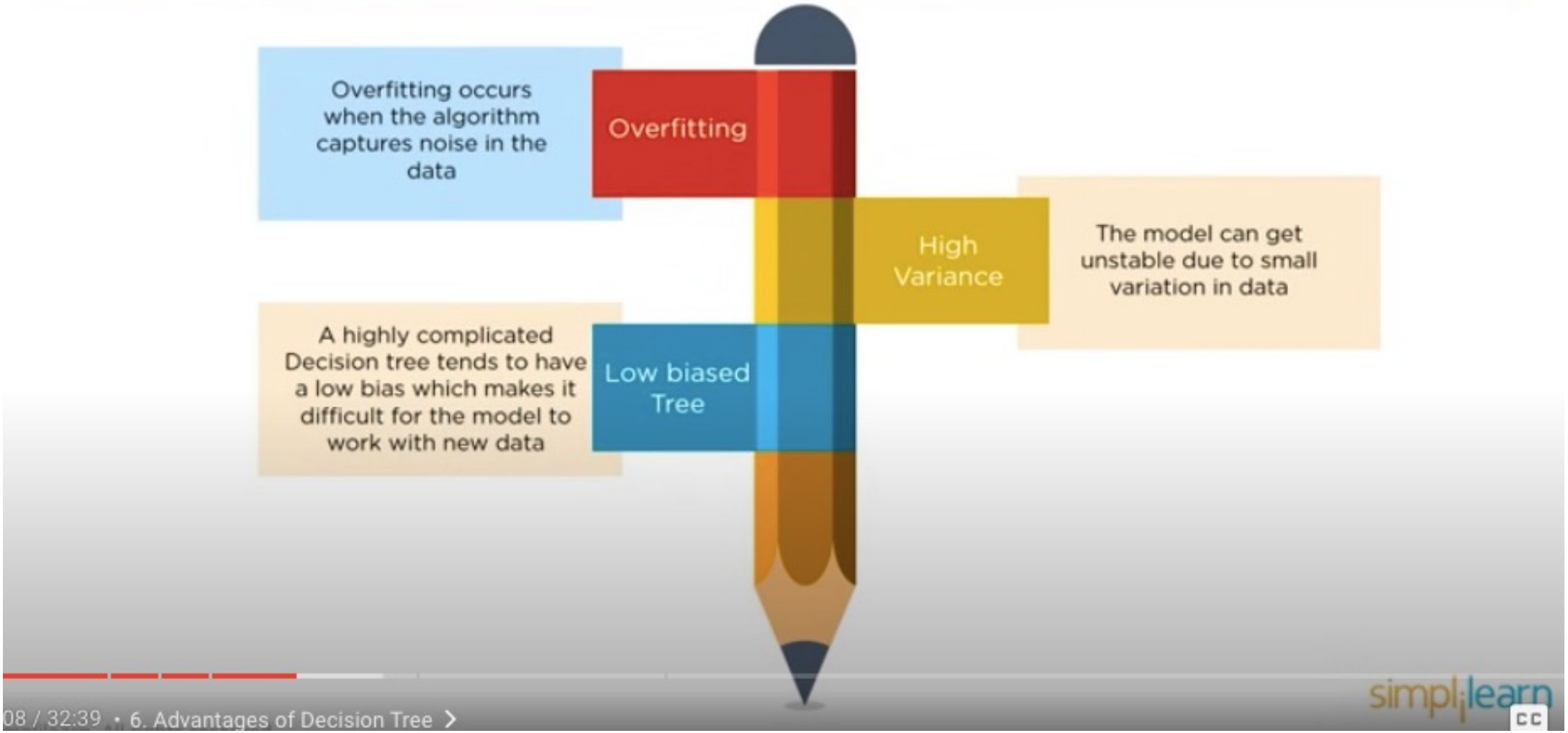


Problems that Decision Tree can solve

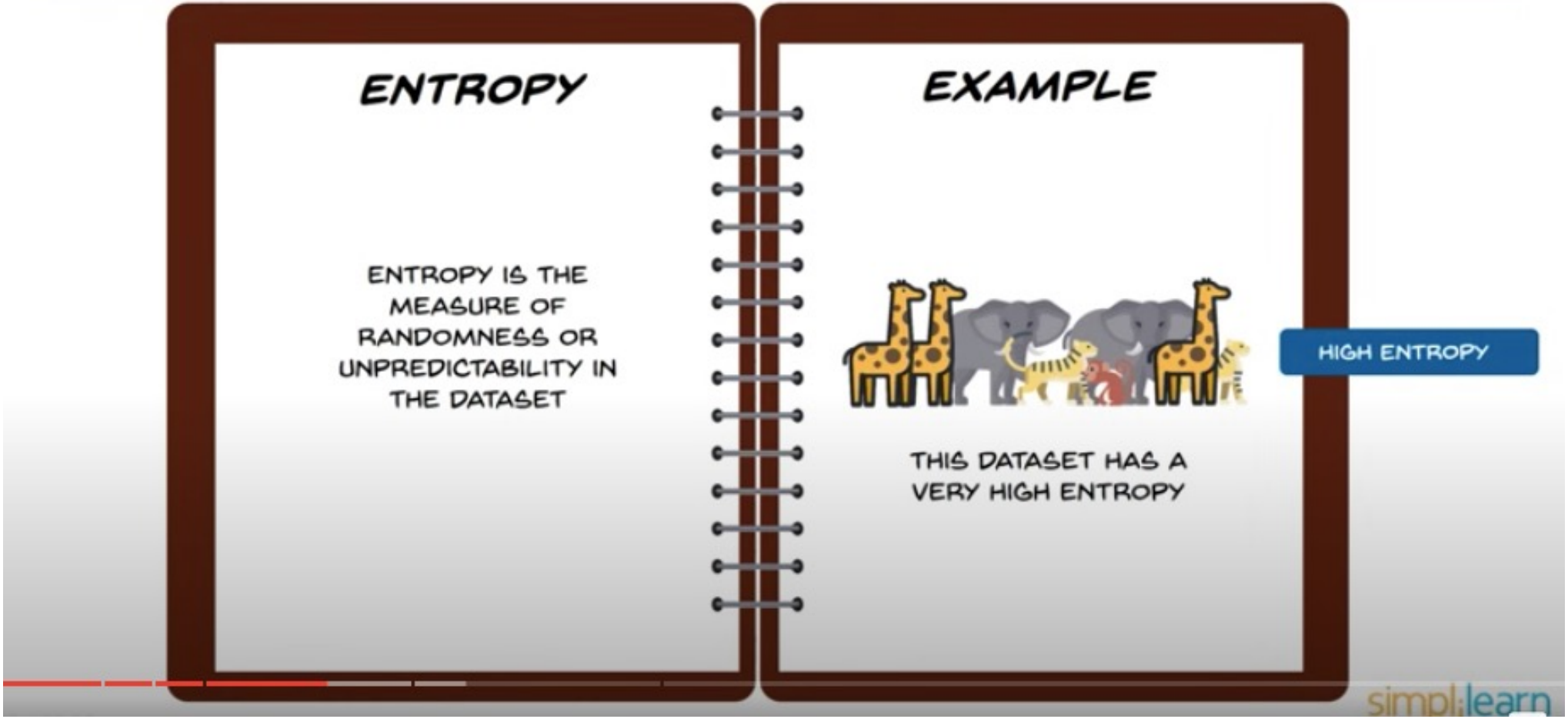
Classification

Regression

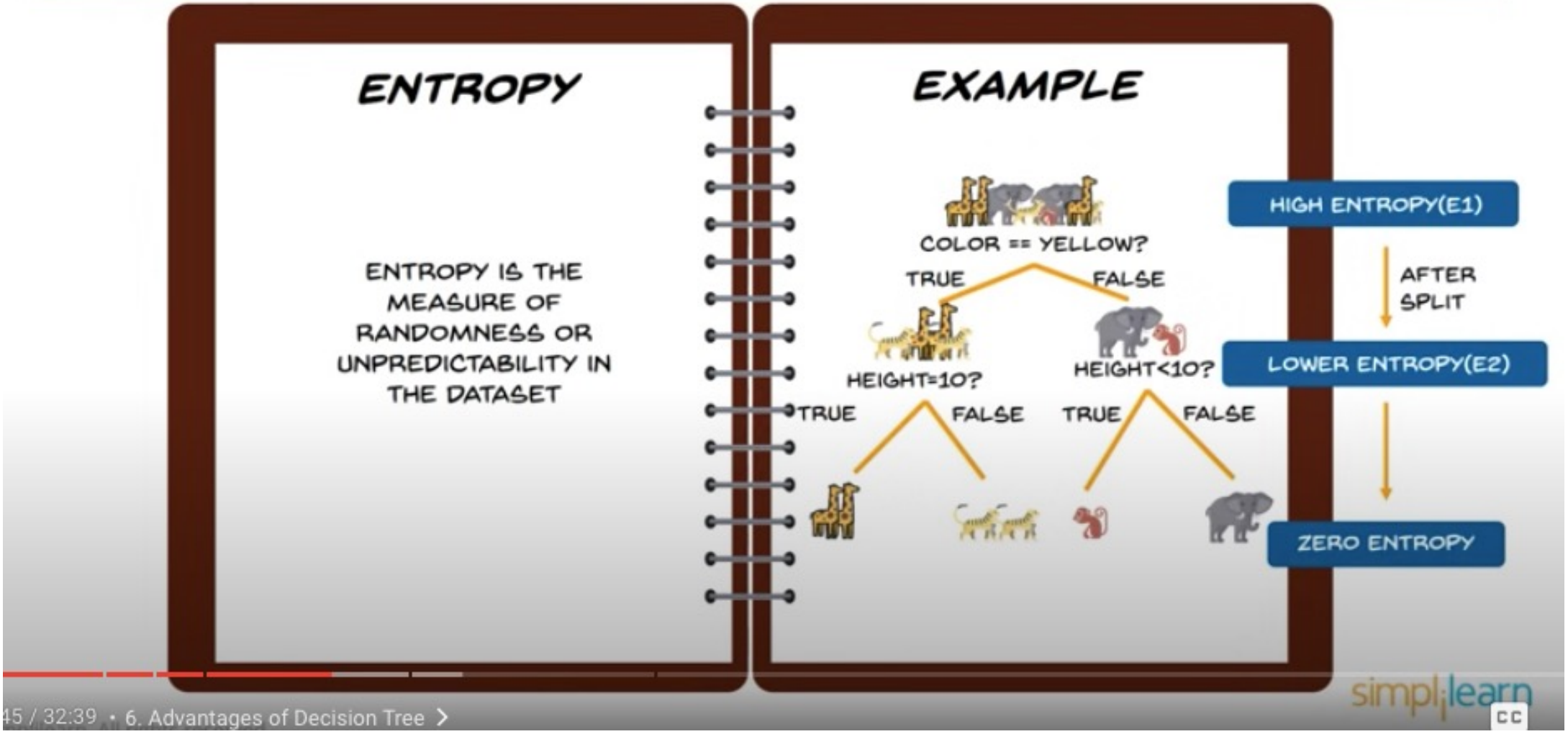
Disadvantages of Decision Tree



Decision Tree – Important Terms

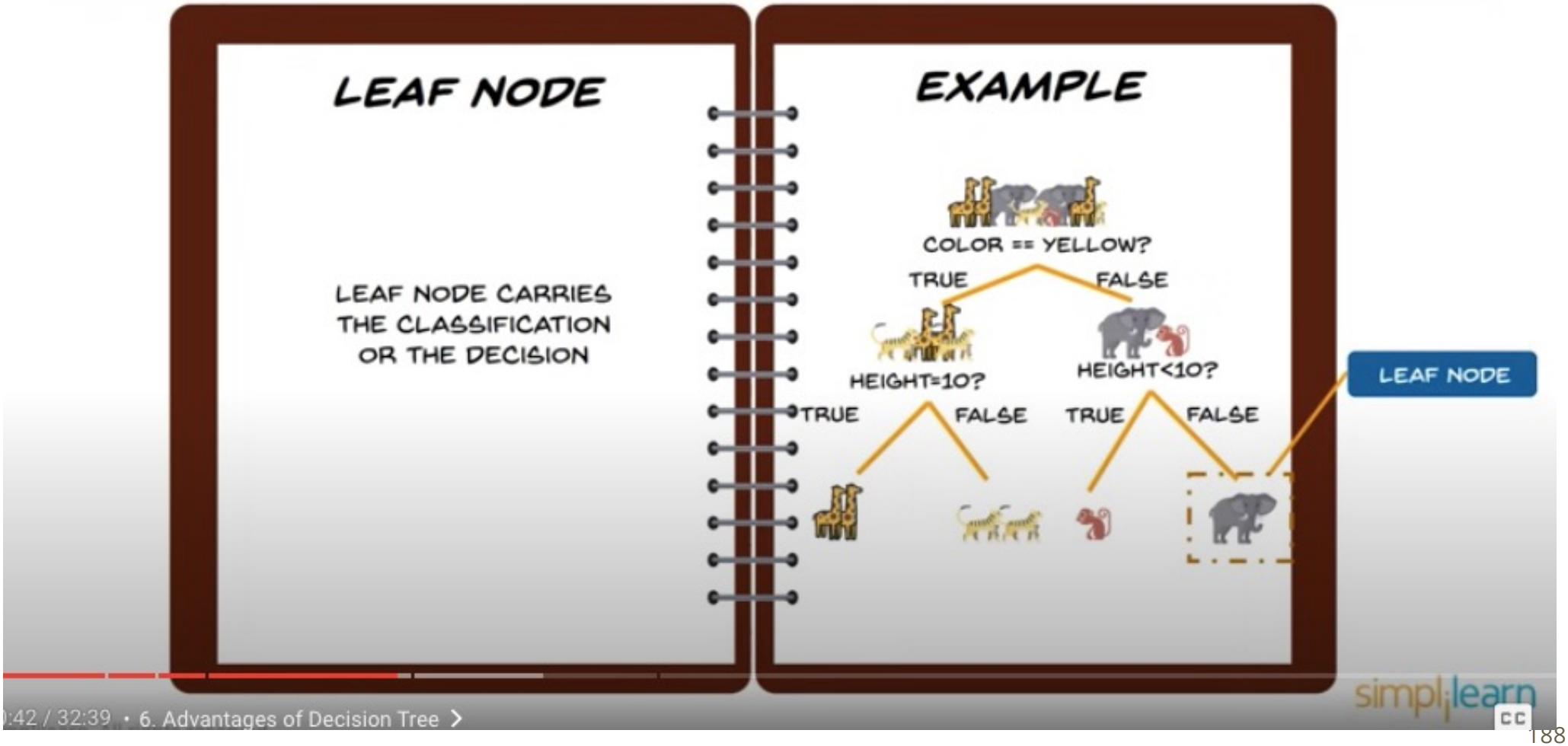


Decision Tree – Important Terms

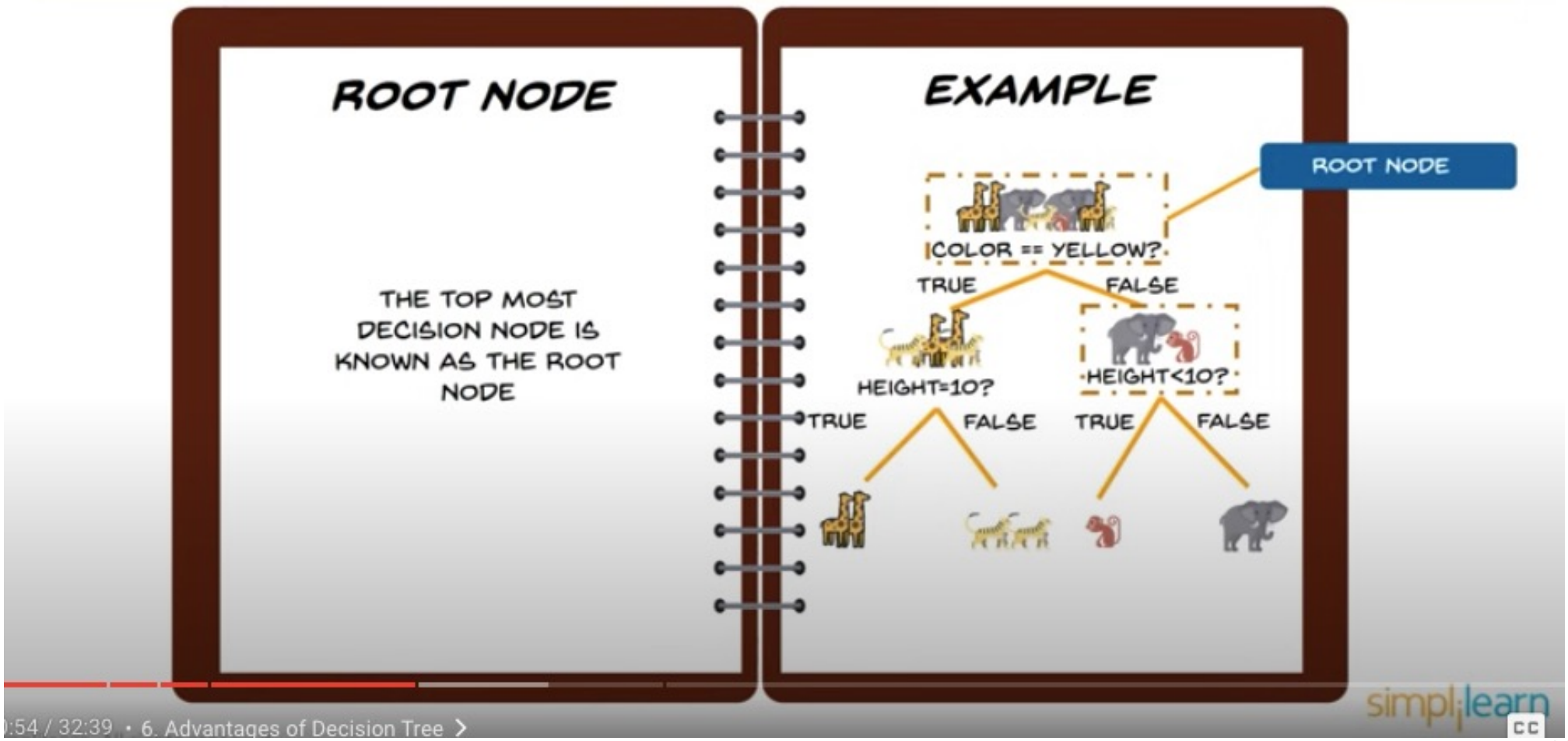


45 / 32:39 • 6. Advantages of Decision Tree >

Decision Tree – Important Terms



Decision Tree – Important Terms



How does a Decision Tree work?

THE DATASET IS LOOKING QUITE MESSY AND THE ENTROPY IS HIGH IN THIS CASE

TRAINING DATASET

COLOR	HEIGHT	LABEL
GREY	10	ELEPHANT
YELLOW	10	GIRAFFE
BROWN	3	MONKEY
GREY	10	ELEPHANT
YELLOW	4	TIGER

How does a Decision Tree work?

HOW TO SPLIT THE DATA

WE HAVE TO FRAME THE CONDITIONS THAT SPLIT THE DATA IN SUCH A WAY THAT THE INFORMATION GAIN IS THE HIGHEST

NOTE

GAIN IS THE MEASURE OF DECREASE IN ENTROPY AFTER SPLITTING

How does a Decision Tree work?

LET'S USE THE FORMULA

$$\sum_{i=1}^{k} P(valuei).log_2(P(value_i)$$

ENTROPY = $(\frac{3}{8})\ log_2(\frac{3}{8}) + (\frac{2}{8})\ log_2(\frac{2}{8}) + (\frac{1}{8})\ log_2(\frac{1}{8}) + (\frac{2}{8})\ log_2(\frac{2}{8})$

ENTROPY=0.571

WE WILL CALCULATE THE ENTROPY OF THE DATASET SIMILARLY AFTER EVERY SPLIT TO CALCULATE THE GAIN

How does a Decision Tree work?

CONDITIONS

COLOR== YELLOW?

HEIGHT>=10

COLOR== BROWN?

COLOR==GREY

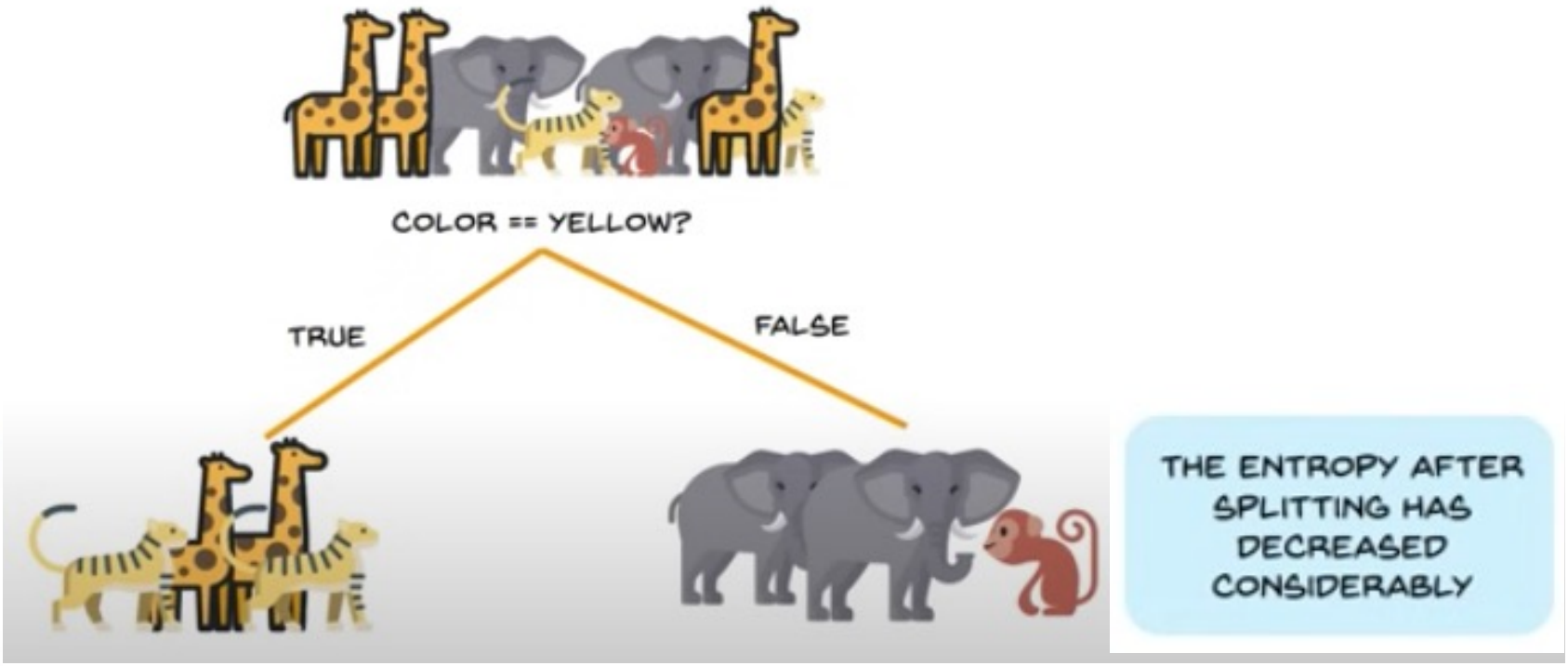
DIAMETER<10

TRAINING DATASET

COLOR	HEIGHT	LABEL
GREY	10	ELEPHANT
YELLOW	10	GIRAFFE
BROWN	3	MONKEY
GREY	10	ELEPHANT
YELLOW	4	TIGER

How does a Decision Tree work?

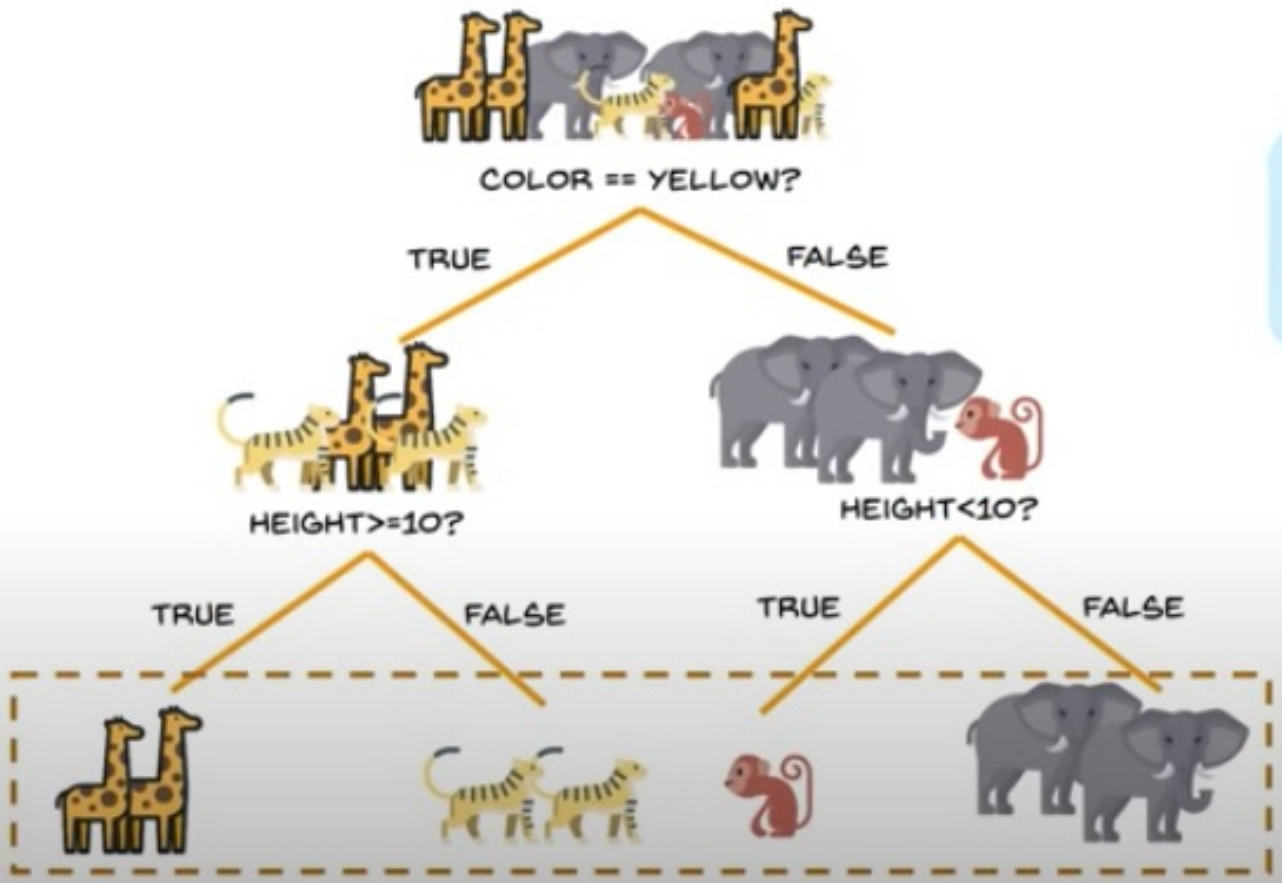
WE SPLIT THE DATA



HOWEVER WE STILL NEED SOME SPLITTING AT BOTH THE BRANCHES TO ATTAIN AN ENTROPY VALUE EQUAL TO ZERO

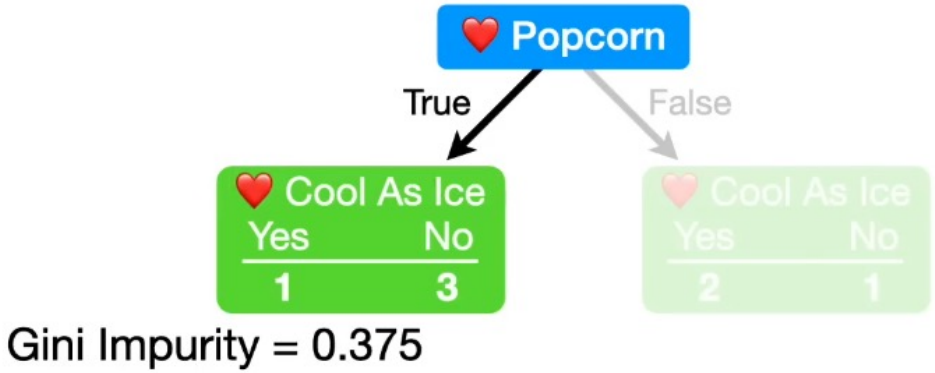
THE ENTROPY AFTER SPLITTING HAS DECREASED CONSIDERABLY

How does a Decision Tree work?



THIS TREE CAN NOW PREDICT ALL THE CLASSES OF ANIMALS PRESENT IN THE DATASET WITH 100% ACCURACY

GINI ENTROPY FOR A LEAF



Gini Impurity for a Leaf = 1 - (the probability of "Yes")^2 - (the probability of "No")^2

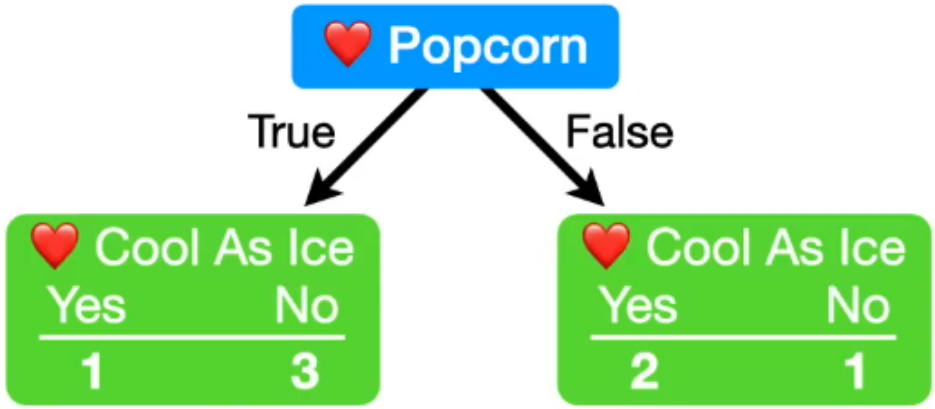
$$= 1 - \left(\frac{1}{1+3}\right)^2 - \left(\frac{3}{1+3}\right)^2$$

$$= 0.375$$

So let's put **0.375** under the **Leaf** on the left so we don't forget it.

Gini entropy

And when we do the math, we get **0.405**.



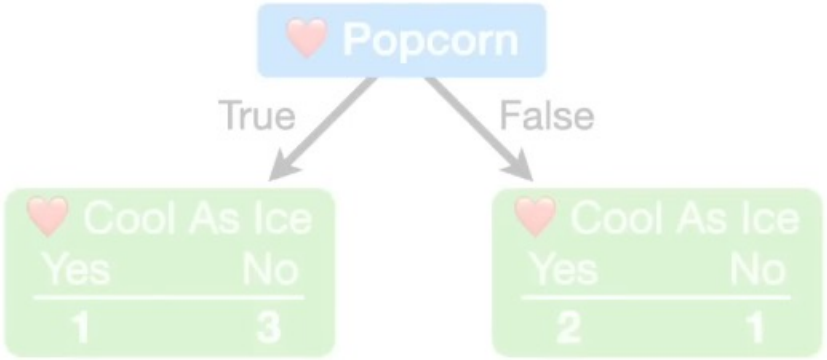
Gini Impurity = 0.375

0.444

Total **Gini Impurity** = weighted average of **Gini Impurities** for the **Leaves**

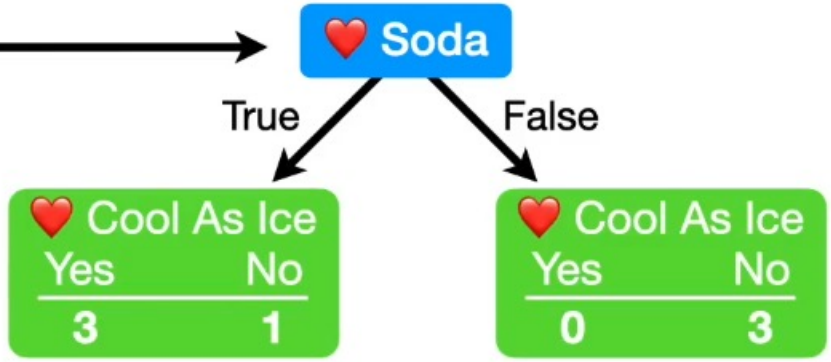
$$= \left(\frac{4}{4+3}\right) 0.375 + \left(\frac{3}{4+3}\right) 0.444$$

Gini Impurity for Loves Popcorn = 0.405



Likewise, the **Gini Impurity** for **Loves Soda** is **0.214**.

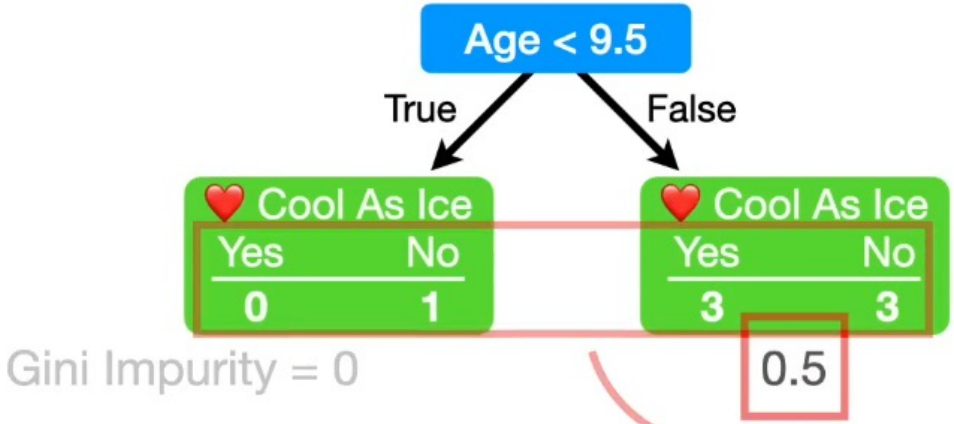
Gini Impurity for Loves Soda = 0.214



ENTROPY FOR NUMERAL FEATURE

Age	Loves Cool As Ice
7	No
12	No
18	Yes
35	Yes
38	Yes
50	No
83	No

9.5, 15, 26.5, 36.5, 44, 66.5



$$\text{Total Gini Impurity} = \left(\frac{1}{1+6}\right) 0 + \left(\frac{6}{1+6}\right) 0.5$$

Now we calculate the **Weighted Average** of the two **Impurities** to get the **Total Gini Impurity**…

sklearn.tree.DecisionTreeClassifier

class `sklearn.tree.` **`DecisionTreeClassifier`** (*criterion='gini', splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None, random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, class_weight=None, presort=False*) [source]

A decision tree classifier.

Read more in the User Guide.

Parameters:

criterion : *string, optional (default="gini")*

The function to measure the quality of a split. Supported criteria are “gini” for the Gini impurity and “entropy” for the information gain.

splitter : *string, optional (default="best")*

The strategy used to choose the split at each node. Supported strategies are “best” to choose the best split and “random” to choose the best random split.

max_depth : *int or None, optional (default=None)*

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : *int, float, optional (default=2)*

The minimum number of samples required to split an internal node:

- If int, then consider `min_samples_split` as the minimum number.
- If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

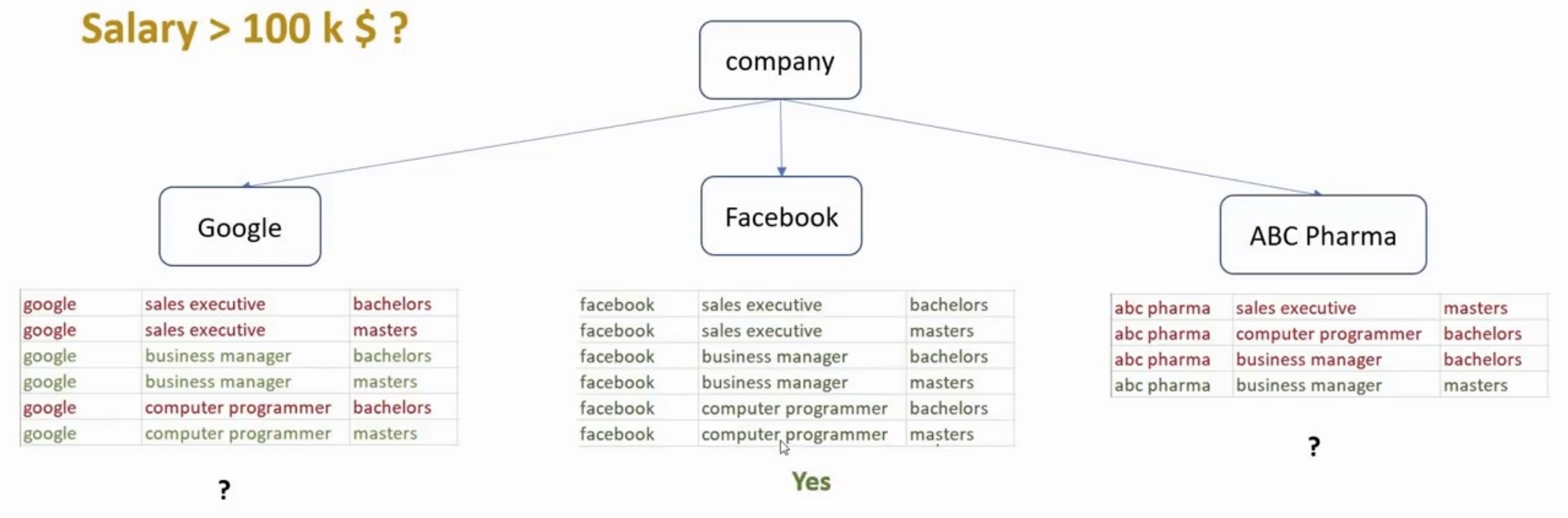
Changed in version 0.18: Added float values for fractions.

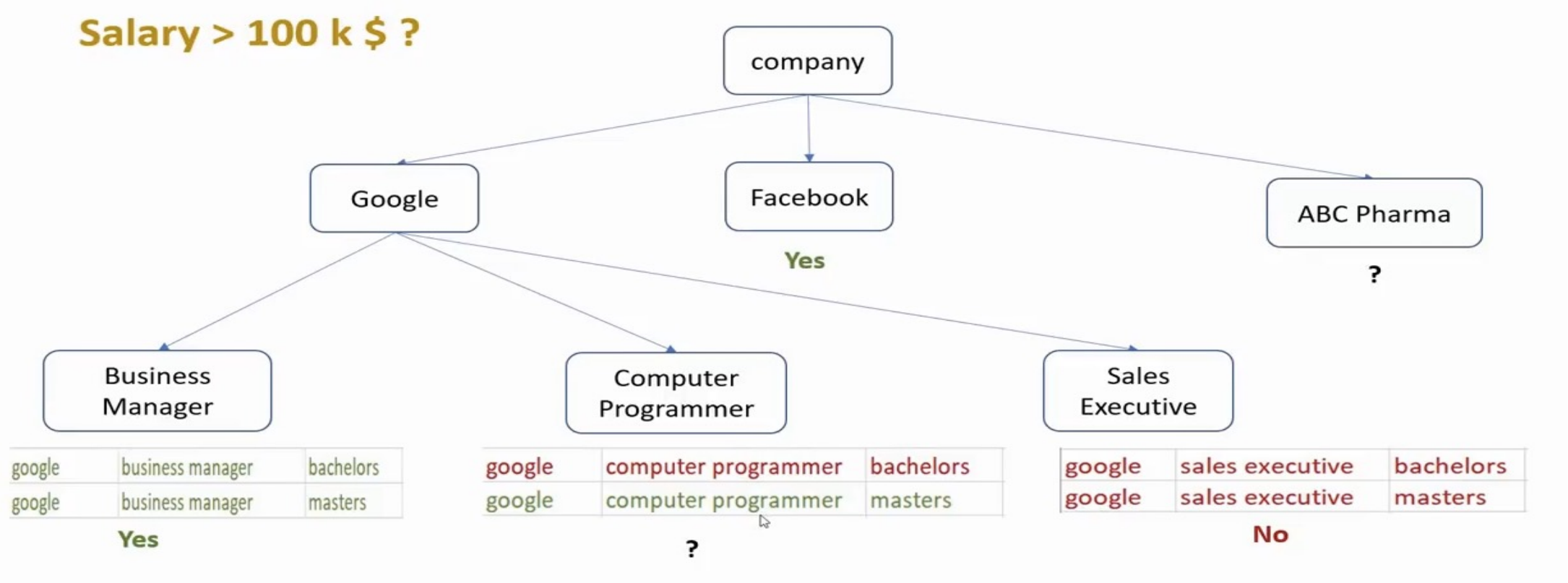
min_samples_leaf : *int, float, optional (default=1)*

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

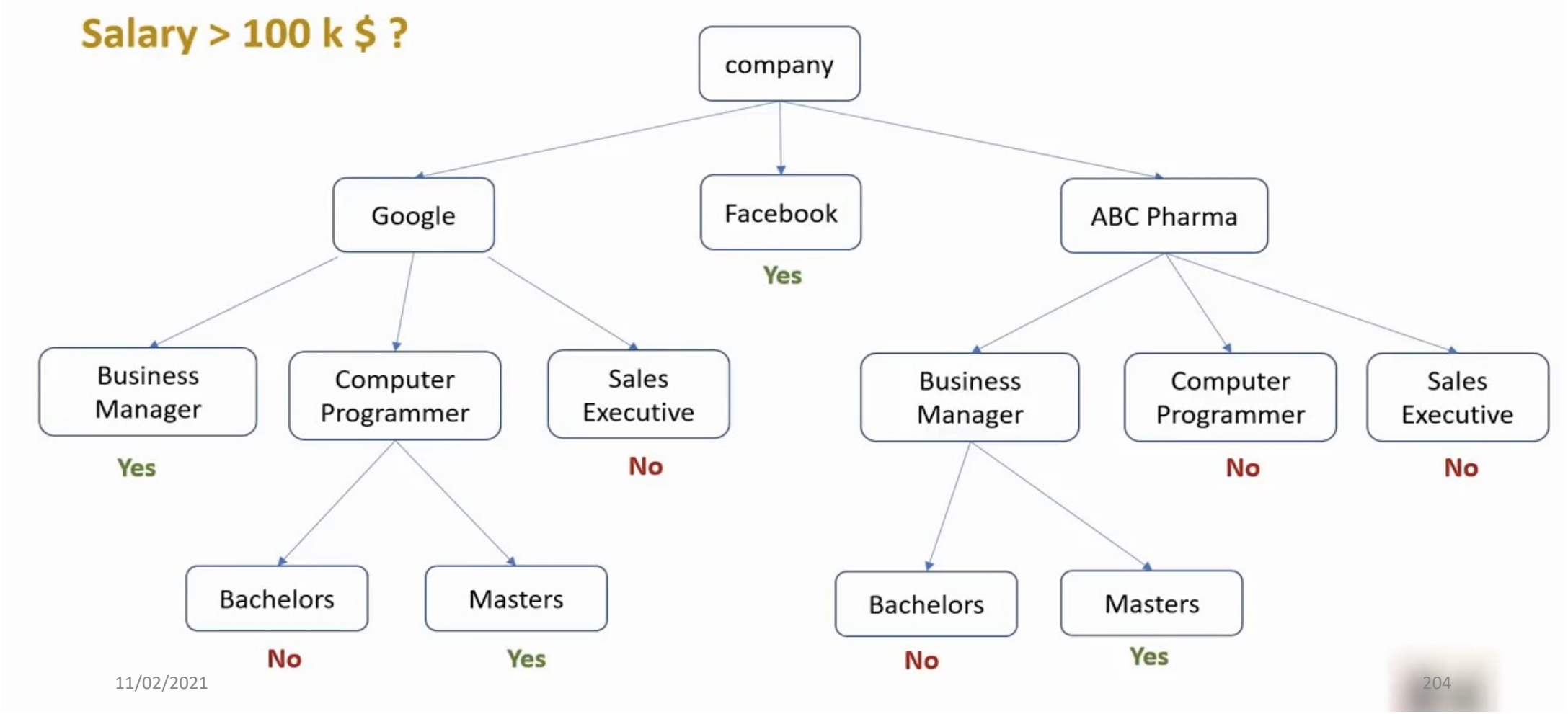
Company	Job	Degree	Salary_more_then_100k
google	sales executive	bachelors	0
google	sales executive	masters	0
google	business manager	bachelors	1
google	business manager	masters	1
google	computer programmer	bachelors	0
google	computer programmer	masters	1
abc pharma	sales executive	masters	0
abc pharma	computer programmer	bachelors	0
abc pharma	business manager	bachelors	0
abc pharma	business manager	masters	1
facebook	sales executive	bachelors	1
facebook	sales executive	masters	1
facebook	business manager	bachelors	1
facebook	business manager	masters	1
facebook	computer programmer	bachelors	1
facebook	computer programmer	masters	1

Based Company Approach:
ONLY FACEBOOK GIVES THE ANSWER (YES/NO)

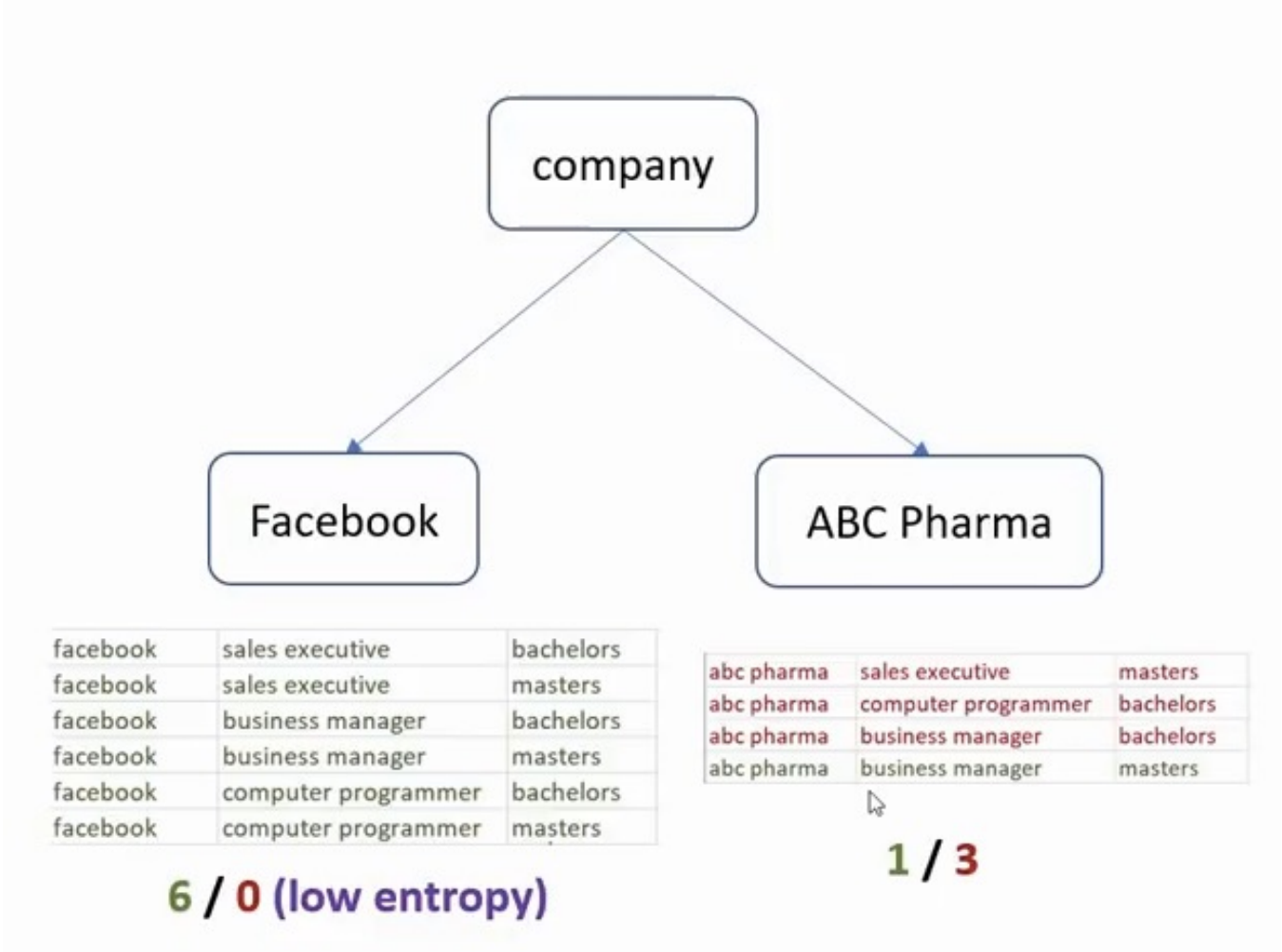


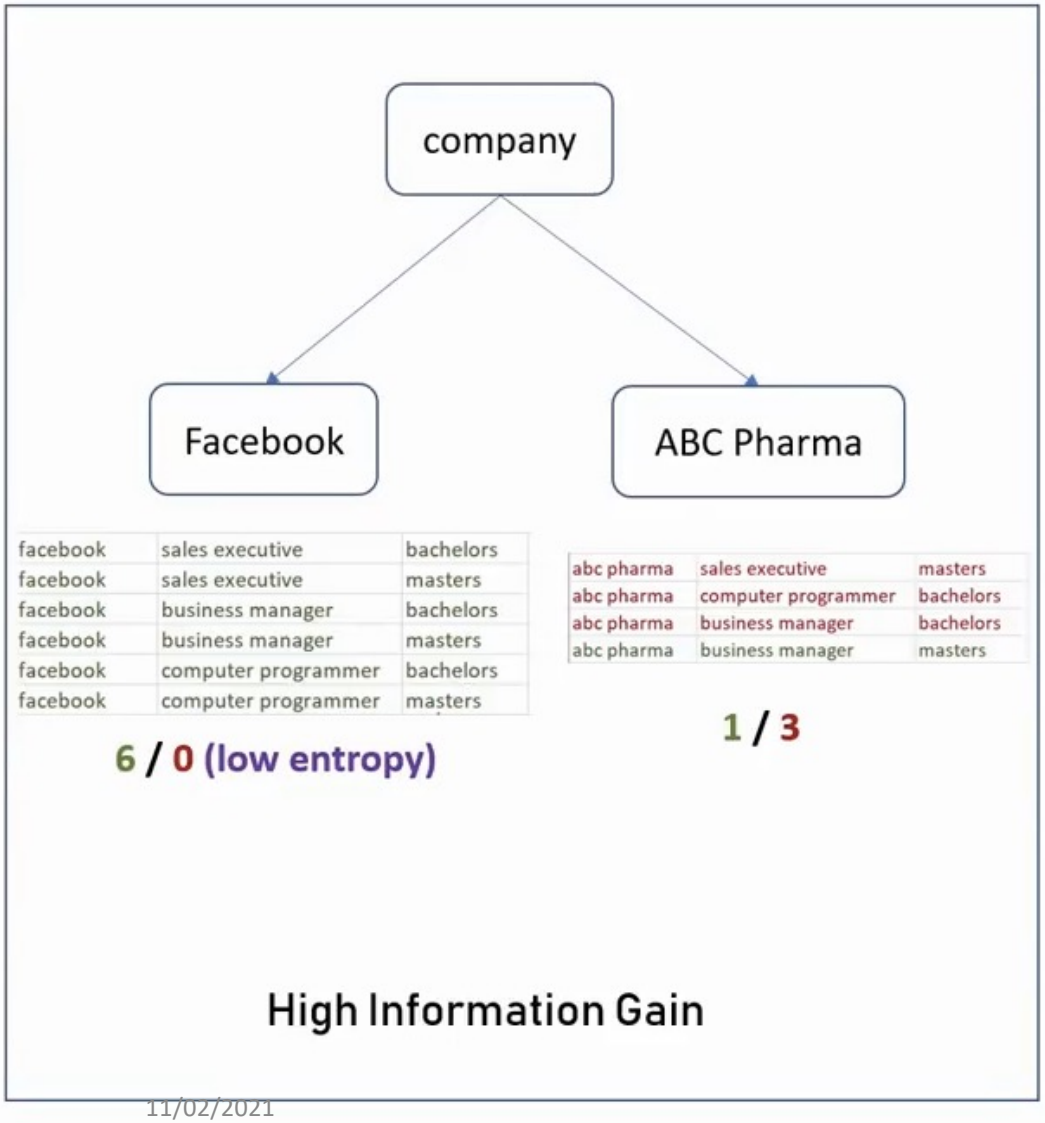

Salary > 100 k $?
company
Google
Facebook
ABC Pharma
Yes
?
Business Manager
Computer Programmer
Sales Executive
google business manager bachelors
google business manager masters
Yes
google computer programmer bachelors
google computer programmer masters
?
google sales executive bachelors
google sales executive masters
No

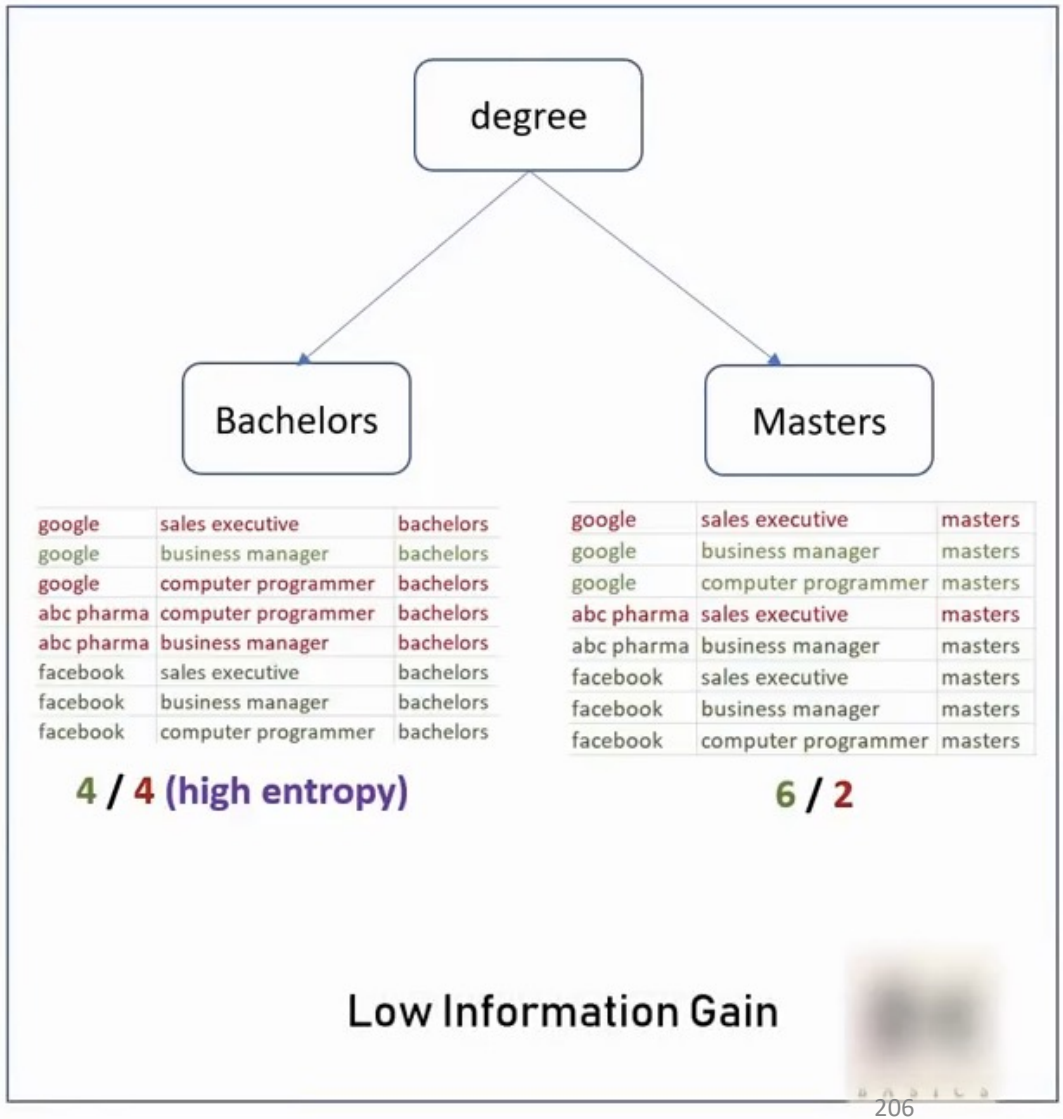
Iterativlly this is the graph, but you can have in real live scenario much more



Entropy: The amount of imputity




company
Facebook
ABC Pharma
facebook sales executive bachelors
facebook sales executive masters
facebook business manager bachelors
facebook business manager masters
facebook computer programmer bachelors
facebook computer programmer masters
6 / 0 (low entropy)
abc pharma sales executive masters
abc pharma computer programmer bachelors
abc pharma business manager bachelors
abc pharma business manager masters
1 / 3
High Information Gain


degree
Bachelors
Masters
google sales executive bachelors
google business manager bachelors
google computer programmer bachelors
abc pharma computer programmer bachelors
abc pharma business manager bachelors
facebook sales executive bachelors
facebook business manager bachelors
facebook computer programmer bachelors
4 / 4 (high entropy)
google sales executive masters
google business manager masters
google computer programmer masters
abc pharma sales executive masters
abc pharma business manager masters
facebook sales executive masters
facebook business manager masters
facebook computer programmer masters
6 / 2
Low Information Gain

Decision Tree Classification

```
[2]: import pandas as pd
```

```
[3]: df = pd.read_csv("salaries.csv")
     df
```

3]:

	company	job	degree	salary_more_then_100k
0	google	sales executive	bachelors	0
1	google	sales executive	masters	0
2	google	business manager	bachelors	1
3	google	business manager	masters	1
4	google	computer programmer	bachelors	0
5	google	computer programmer	masters	1
6	abc pharma	sales executive	masters	0
7	abc pharma	computer programmer	bachelors	0
8	abc pharma	business manager	bachelors	0
9	abc pharma	business manager	masters	1
10	facebook	sales executive	bachelors	1
11	facebook	sales executive	masters	1

APPLICATION LABEL INCODING

```
Entrée [1]: import pandas as pd

Entrée [2]: df = pd.read_csv("salaries.csv")
            df
```

Out[2]:

	company	job	degree	salary_more_then_100k
0	google	sales executive	bachelors	0
1	google			
2	google	bu		

```
Entrée [7]: inputs
```

Out[7]:

	company	job	degree	company_n	job_n	degree_n
0	google	sales executive	bachelors	2	2	0
1	google	sales executive	masters	2	2	1
2	google	business manager	bachelors	2	0	0
3	google	business manager	masters	2	0	1
4	google	computer programmer	bachelors	2	1	0
5	google	computer programmer	masters	2	1	1
6	abc pharma	sales executive	masters	0	2	1
7	abc pharma	computer programmer	bachelors	0	1	0
8	abc pharma	business manager	bachelors	0	0	0
9	abc pharma	business manager	masters	0	0	1
10	facebook	sales executive	bachelors	1	2	0

Entrée [ ]:
```
import pandas as pd
```

Entrée [ ]:
```
df = pd.read_csv("salaries.csv")
df
```

Entrée [ ]:
```
inputs = df.drop('salary_more_then_100k',axis='columns')
```

Entrée [ ]:
```
target = df['salary_more_then_100k']
```

Entrée [ ]:
```
from sklearn.preprocessing import LabelEncoder
le_company = LabelEncoder()
le_job = LabelEncoder()
le_degree = LabelEncoder()
```

Entrée [ ]:
```
inputs['company_n'] = le_company.fit_transform(inputs['company'])
inputs['job_n'] = le_job.fit_transform(inputs['job'])
inputs['degree_n'] = le_degree.fit_transform(inputs['degree'])
```

Entrée [ ]:
```
inputs
```

Entrée [ ]:
```
inputs_n = inputs.drop(['company','job','degree'],axis='columns')
```

Entrée [ ]:
```
inputs_n
```

Entrée [3]:
```
inputs = df.drop('salary_more_then_100k',axis='columns')
```

Entrée [4]:
```
target = df['salary_more_then_100k']
```

Entrée [5]:
```
from sklearn.preprocessing import LabelEncoder
le_company = LabelEncoder()
le_job = LabelEncoder()
le_degree = LabelEncoder()
```

Entrée [6]:
```
inputs['company_n'] = le_company.fit_transform(inputs['company'])
inputs['job_n'] = le_job.fit_transform(inputs['job'])
inputs['degree_n'] = le_degree.fit_transform(inputs['degree'])
```

Entrée [7]:
```
inputs
```

Out[7]:

	company	job	degree	company_n	job_n	degree_n

Entrée [8]:
```
inputs_n = inputs.drop(['company','job','degree'],axis='columns')
```

Entrée [9]:
```
inputs_n
```

Out[9]:

	company_n	job_n	degree_n
0	2	2	0
1	2	2	1

```
Entrée [8]: inputs_n = inputs.drop(['company','job','degree'],axis='columns')
```

```
Entrée [11]: from sklearn import tree
             model = tree.DecisionTreeClassifier()
```

```
Entrée [12]: model.fit(inputs_n, target)

Out[12]: DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
                     max_features=None, max_leaf_nodes=None,
                     min_impurity_decrease=0.0, min_impurity_split=None,
                     min_samples_leaf=1, min_samples_split=2,
                     min_weight_fraction_leaf=0.0, presort=False, random_state=None,
                     splitter='best')
```

```
Entrée [13]: model.score(inputs_n,target)

Out[13]: 1.0
```

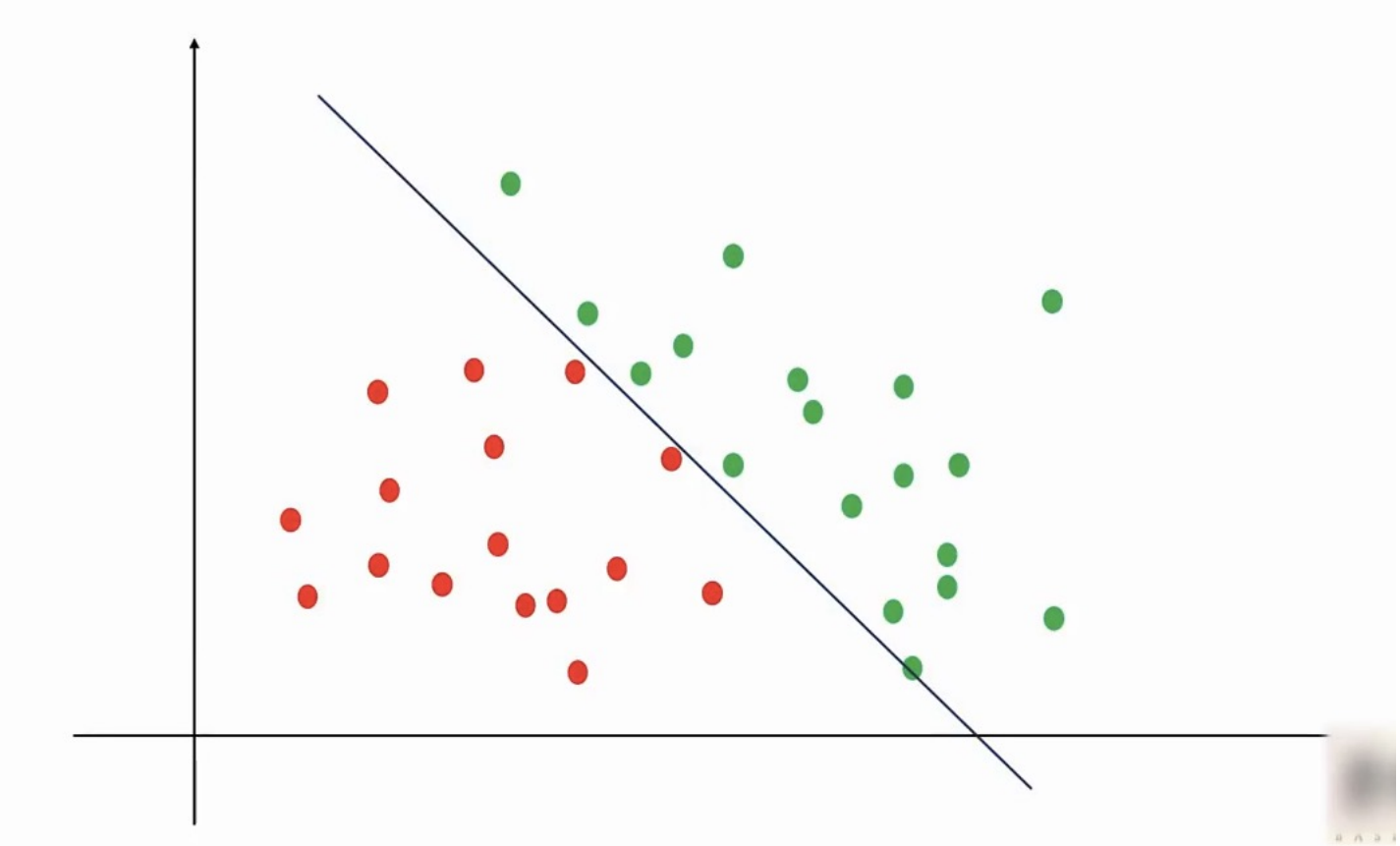
Is salary of Google, Computer Engineer, Bachelors degree > 100 k ?

```
Entrée [14]: model.predict([[2,1,0]])

Out[14]: array([0])
```

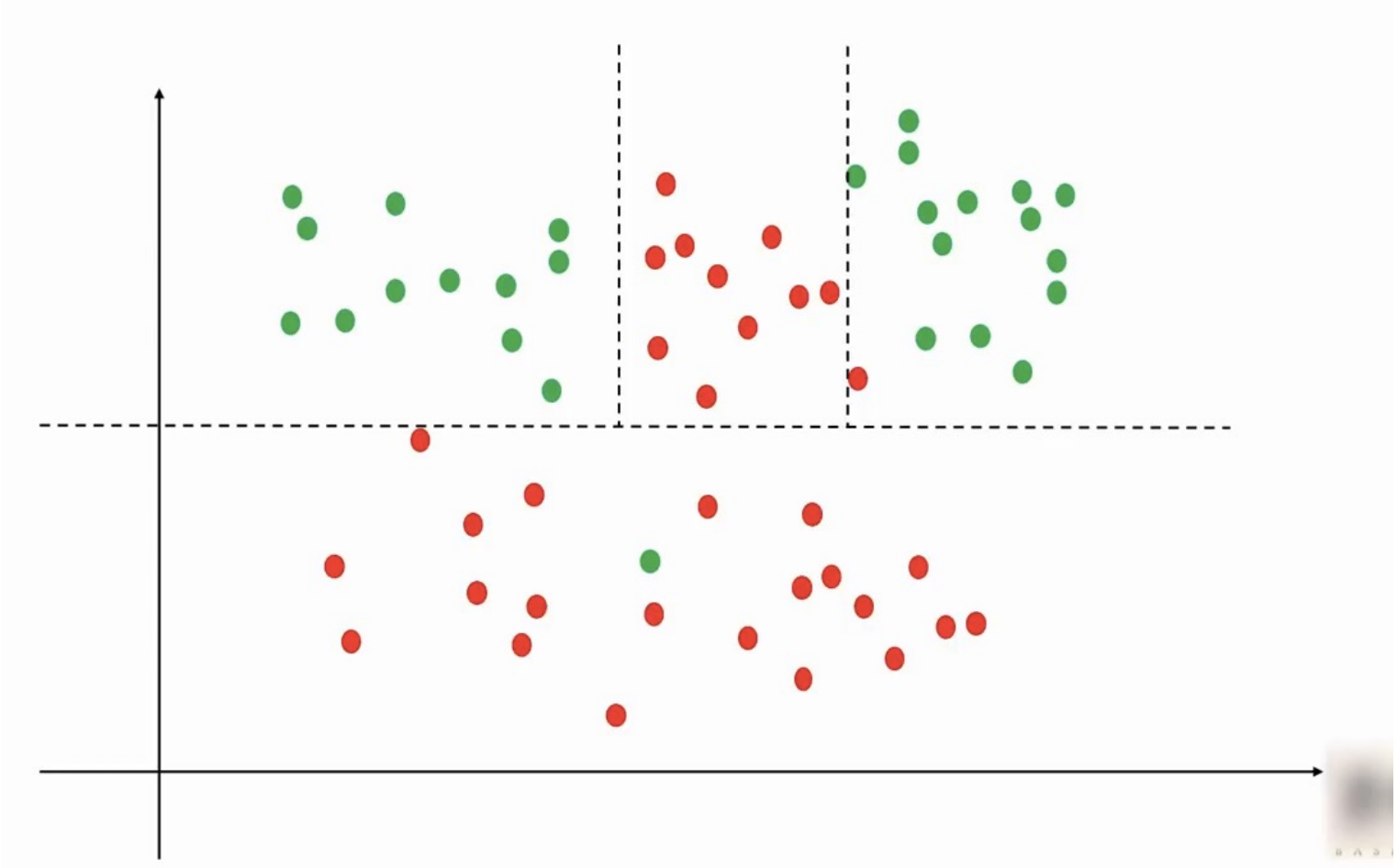
Is salary of Google, Computer Engineer, Masters degree > 100 k ?

```
Entrée [15]: model.predict([[2,1,1]])
```

Commun Decision Boundary



SEQUENTIAL CLASSIFICATION



Random Forest Algorithm

RANDOM FOREST
Random Forest and Decision Tree
simplilearn

Why Random Forest?

No overfitting

Use of multiple trees reduce the risk of overfitting

Training time is less

High accuracy

Runs efficiently on large database

For large data, it produces highly accurate predictions

Estimates missing data

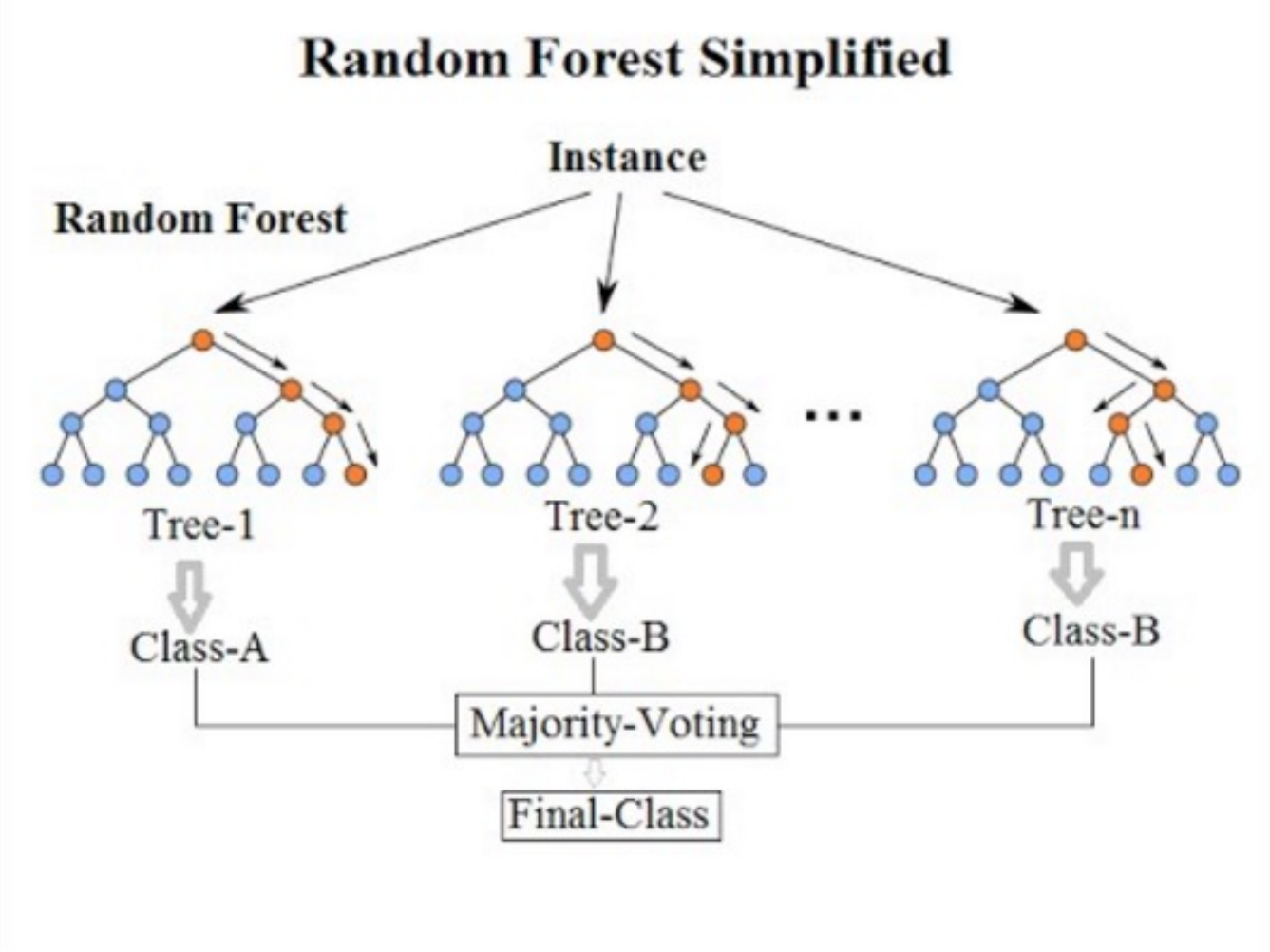
Random Forest can maintain accuracy when a large proportion of data is missing

mplilearn. All rights reserved

What is Random Forest?

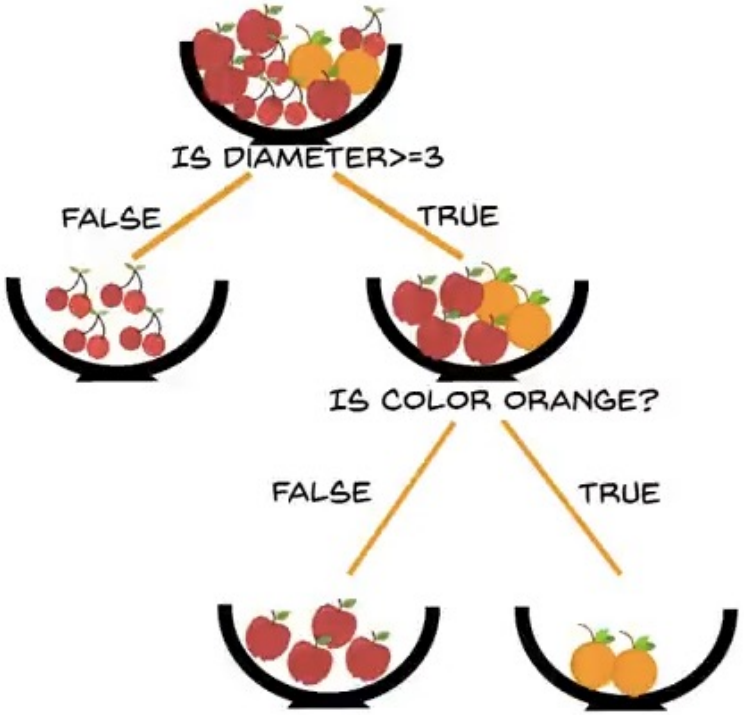
Random forest or Random Decision Forest is a method that operates by constructing multiple Decision Trees during training phase.
The Decision of the majority of the trees is chosen by the random forest as the final decision

mplilearn. All rights reserved.


Random Forest Simplified
Instance
Random Forest
Tree-1
Tree-2
Tree-n
Class-A
Class-B
Class-B
Majority-Voting
Final-Class

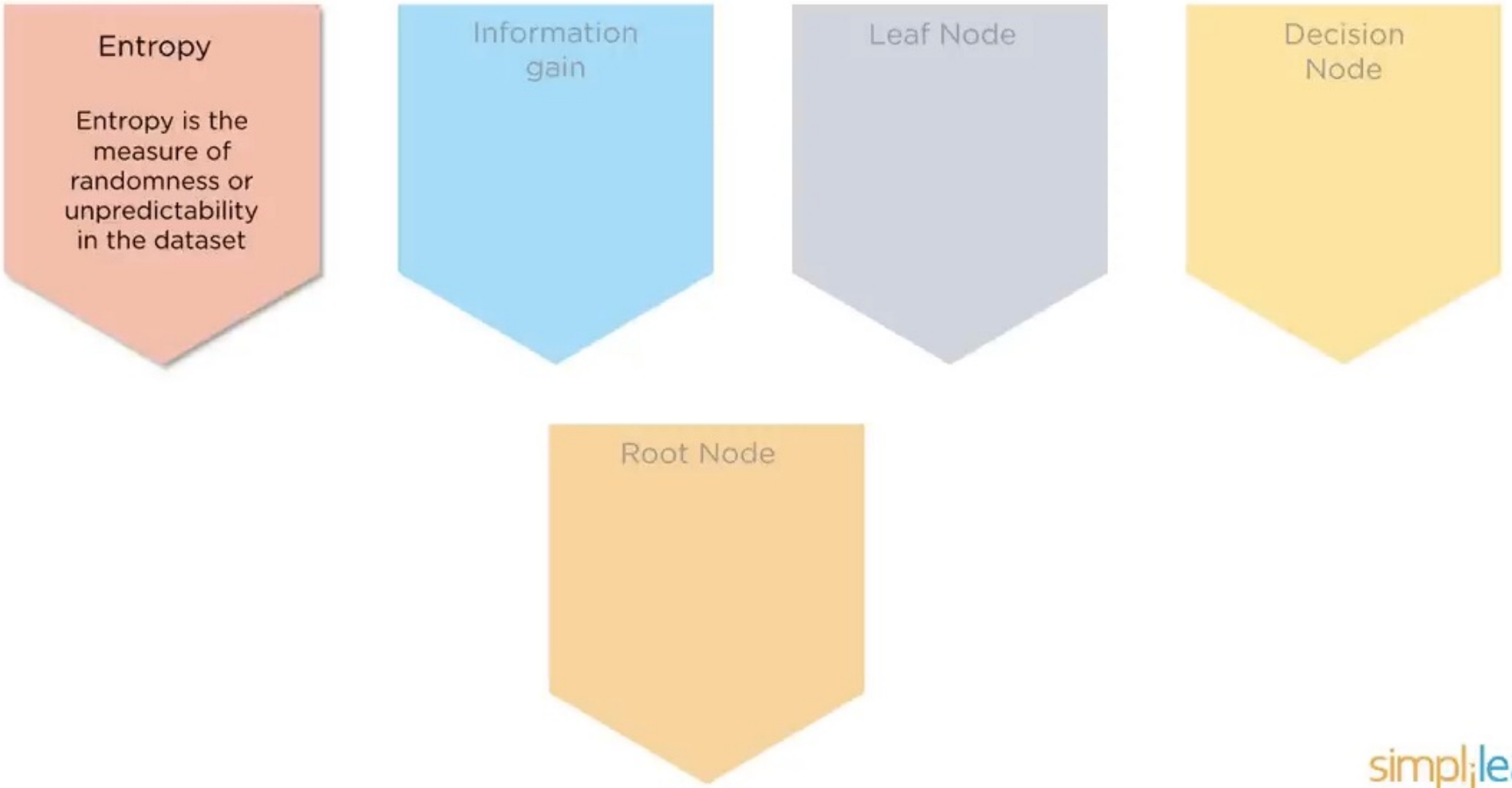
Decision Tree

Decision Tree is a tree shaped diagram used to determine a course of action. Each branch of the tree represents a possible decision, occurrence or reaction



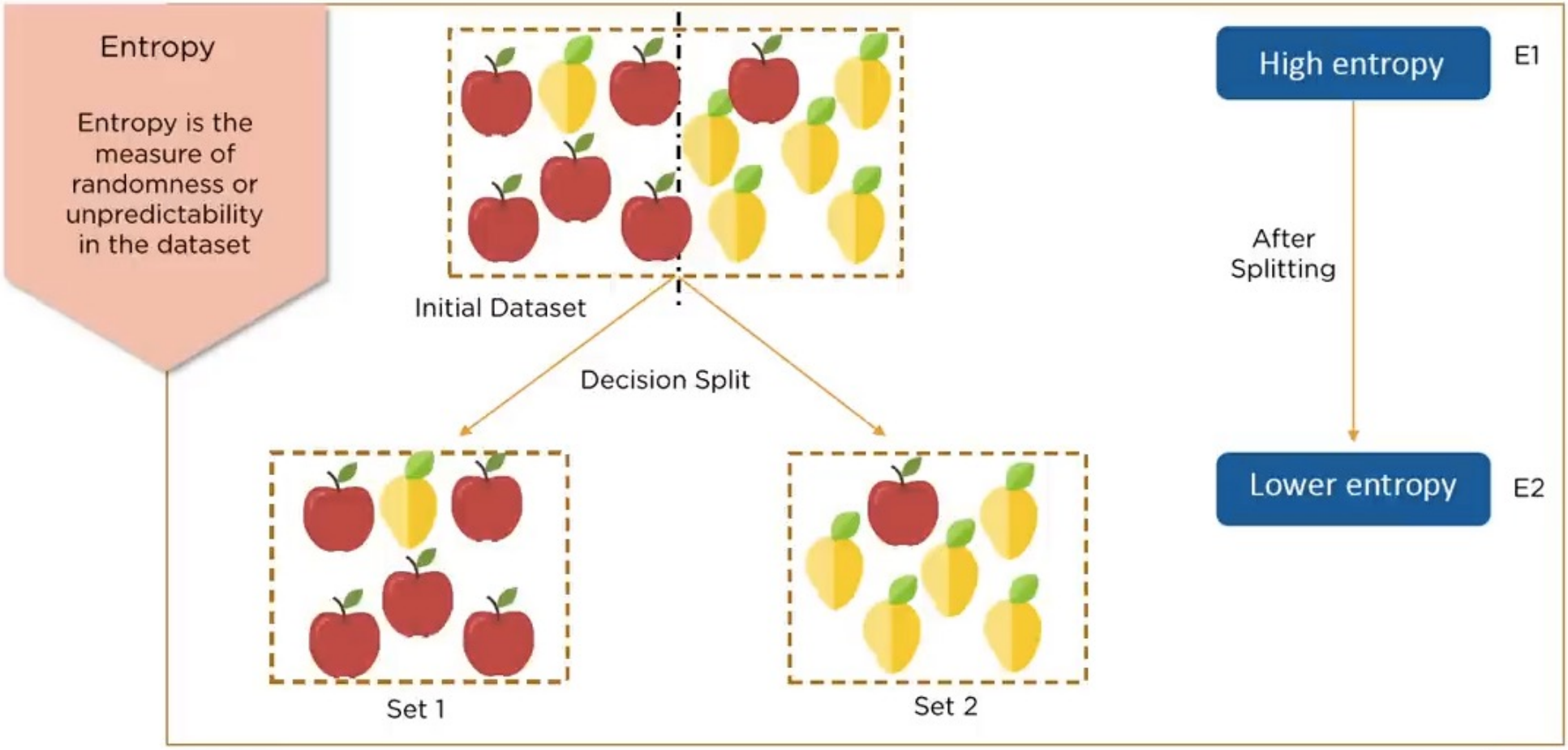
simplilearn

Decision Tree- Important Terms



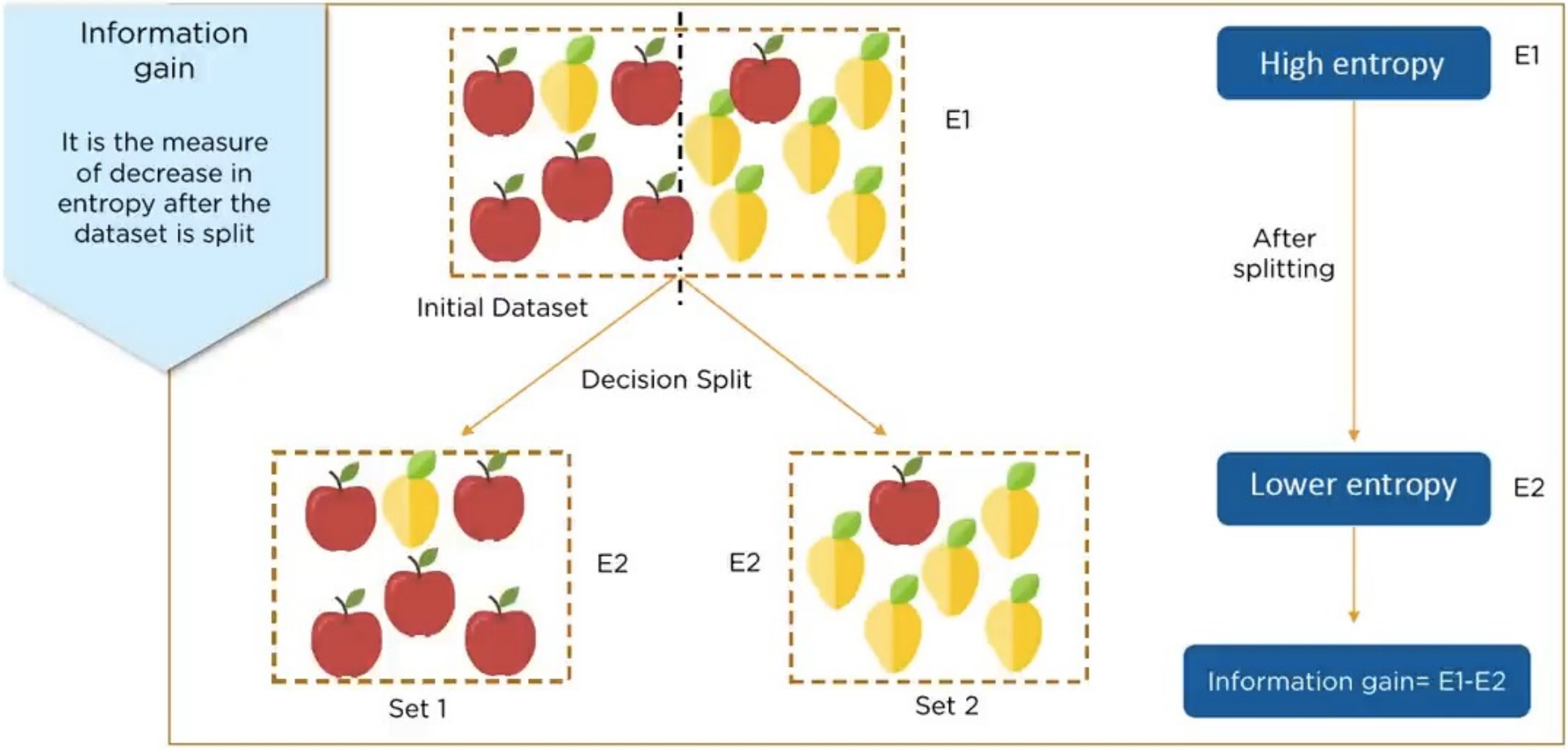
mplilearn. All rights reserved.

Decision Tree - Important Terms



nplilearn. All rights reserved.

Decision Tree - Important Terms



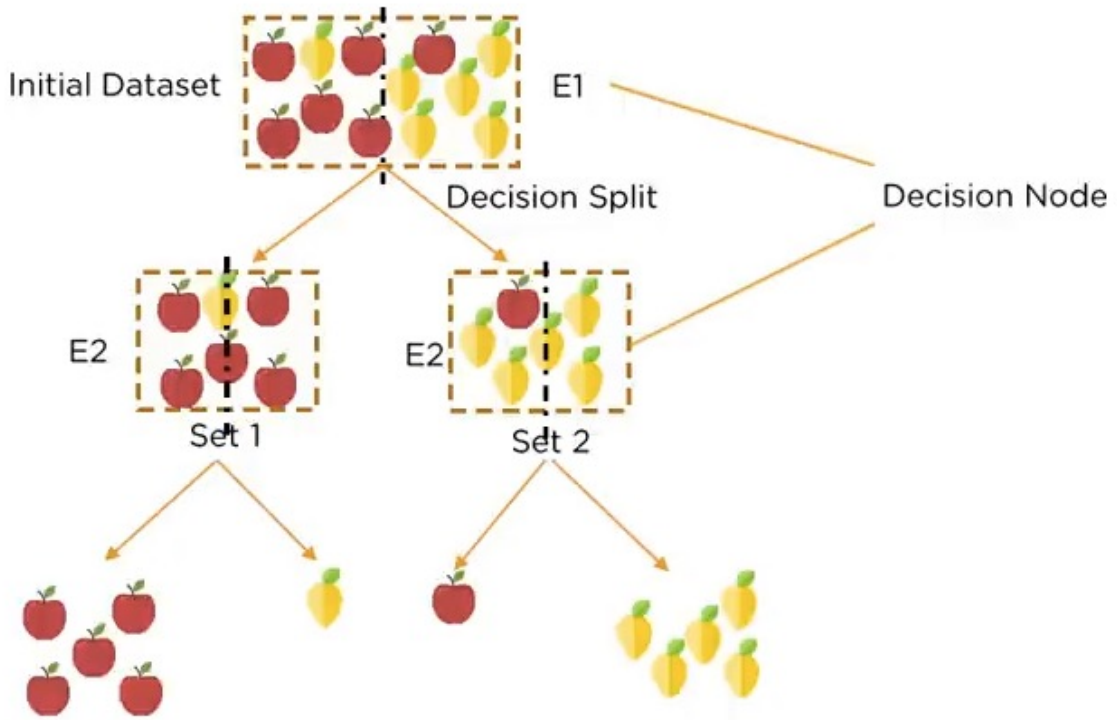
simplilearn

mplilearn. All rights reserved

Decision Tree - Important Terms

Decision Node

Decision node has two or more branches



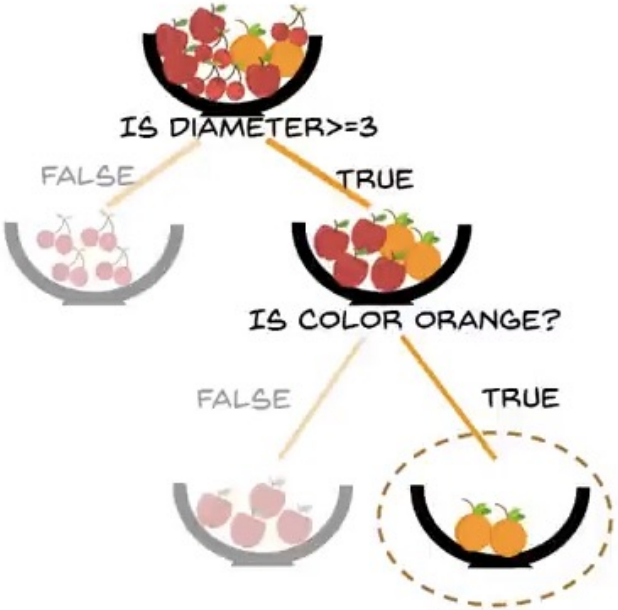
simplilearn

nplilearn. All rights reserved.

How does a Random Forest work?

TREE 1 CLASSIFIES
IT AS AN ORANGE

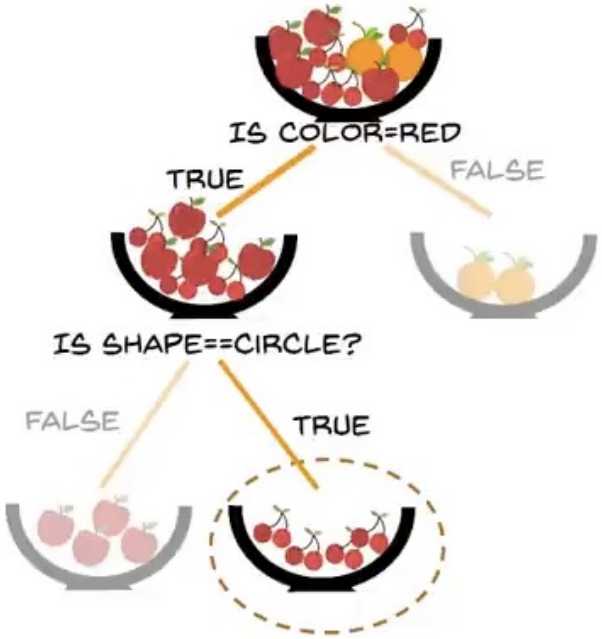
DIAMETER = 3
COLOUR = ORANGE
GROWS IN SUMMER = YES
SHAPE = CIRCLE



How does a Random Forest work?

TREE 2 CLASSIFIES IT AS CHERRIES

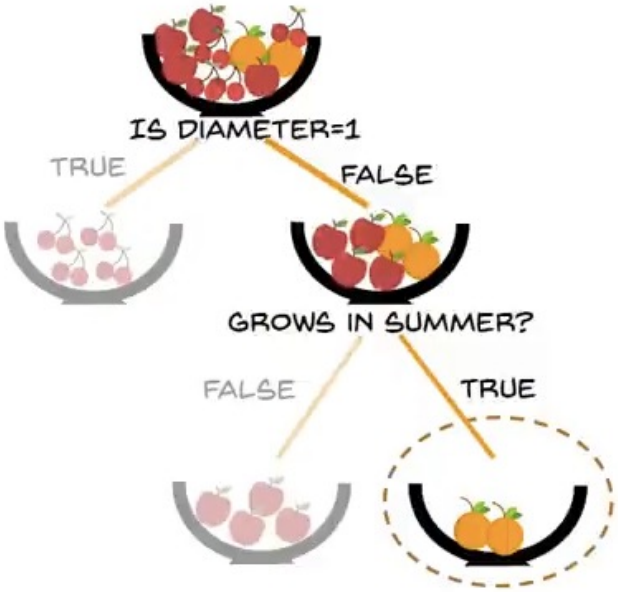
DIAMETER = 3
COLOUR = ORANGE
GROWS IN SUMMER = YES
SHAPE = CIRCLE



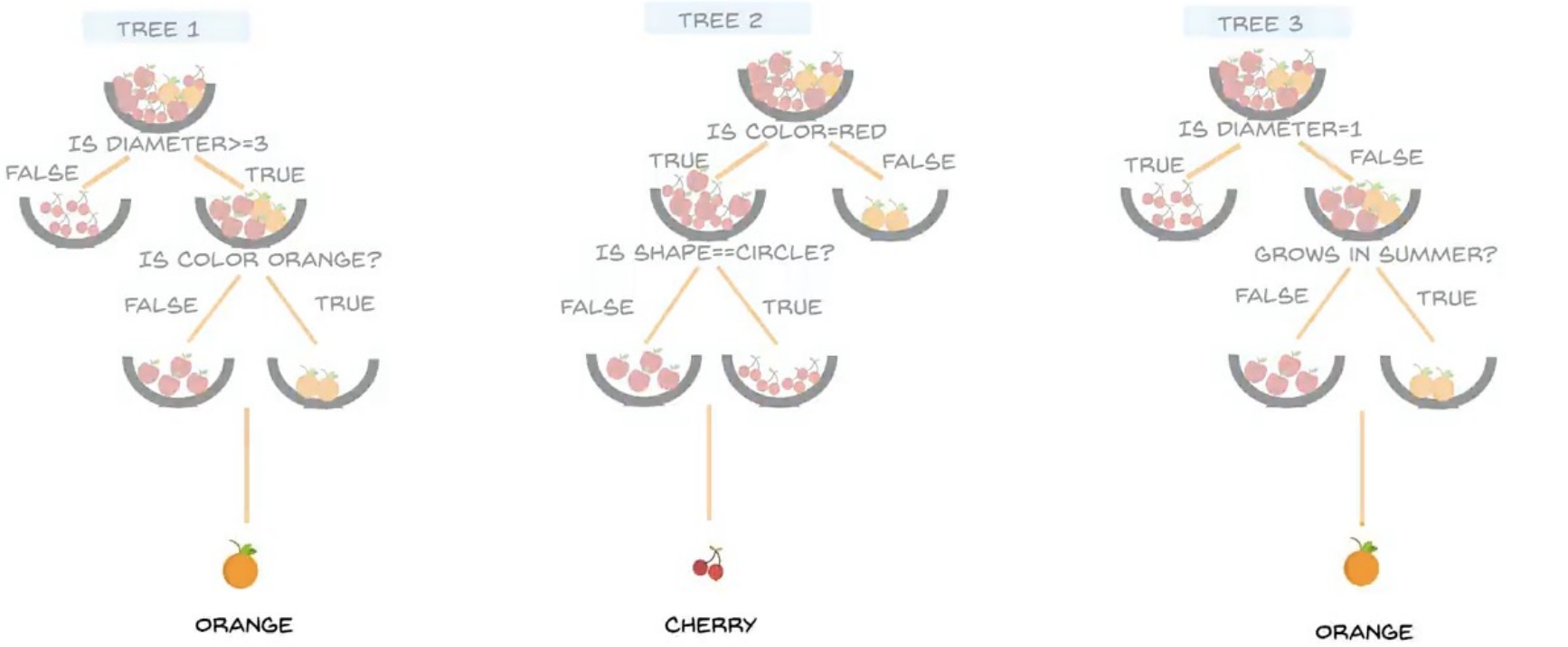
How does a Random Forest work?

TREE 3 CLASSIFIES IT AS ORANGE

DIAMETER = 3
COLOUR = ORANGE
GROWS IN SUMMER = YES
SHAPE = CIRCLE



How does a Random Forest work?

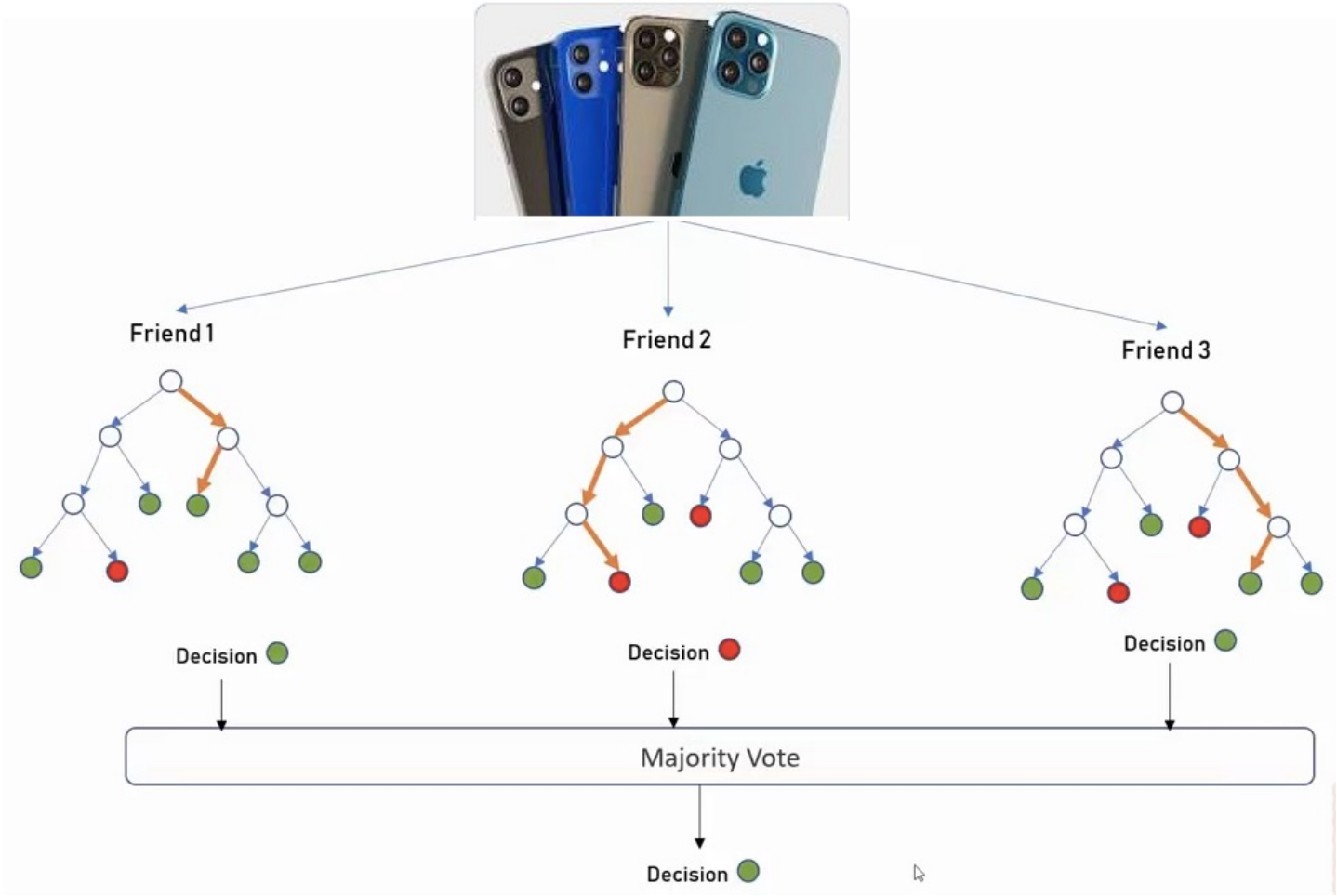


simplilearn

Simplilearn. All rights reserved.

EMULATE DIFFERENT SCENARIOS WITH DIFFERENT PARAMETERS AND OPERATE THE MAJORITY VOTE

Taking opinoin of freinds making a final decision



CONCLUSION

But as stated, a random forest is a collection of decision trees. ... With that said, random forests are a strong modeling technique and much more robust than a single decision tree. They aggregate many decision trees to limit overfitting as well as error due to bias and therefore yield useful results

Identify hand written digits recognition

0 1 2 3 4 5 6 7 8

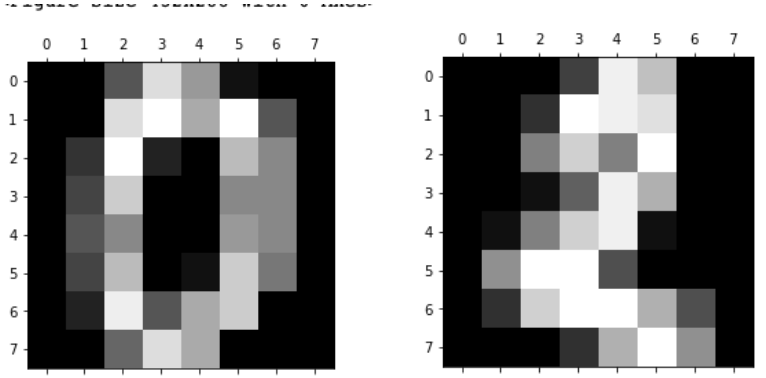
Entrée [2]:
```
import pandas as pd
from sklearn.datasets import load_digits
digits = load_digits()
```

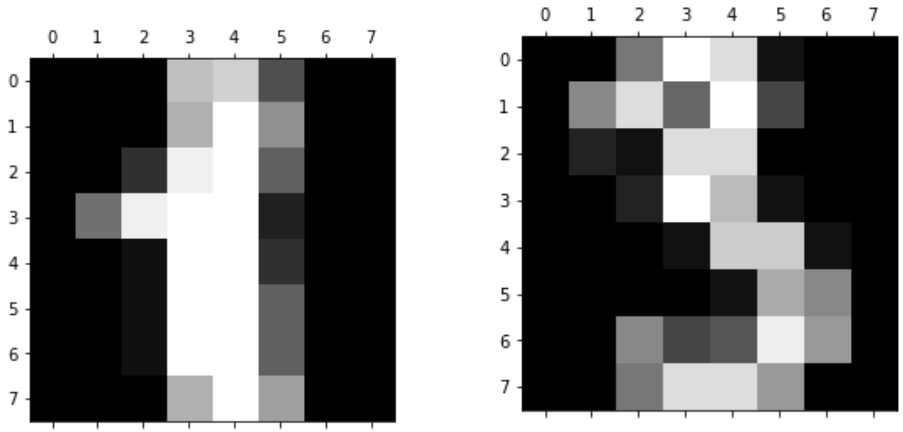
Entrée [3]:
```
dir(digits)
```

Out[3]:
```
['DESCR', 'data', 'images', 'target', 'target_names']
```

Entrée [4]:
```
%matplotlib inline
import matplotlib.pyplot as plt
```

Entrée [5]:
```
plt.gray()
for i in range(4):
    plt.matshow(digits.images[i])
```





Entrée [6]:
```
df = pd.DataFrame(digits.data)
df.head()
```

Out[6]:

	0	1	2	3	4	5	6	7	8	9	...	54	55	56	57	58	59	60	61	62	63
0	0.0	0.0	5.0	13.0	9.0	1.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	6.0	13.0	10.0	0.0	0.0	0.0
1	0.0	0.0	0.0	12.0	13.0	5.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	11.0	16.0	10.0	0.0	0.0
2	0.0	0.0	0.0	4.0	15.0	12.0	0.0	0.0	0.0	0.0	...	5.0	0.0	0.0	0.0	0.0	3.0	11.0	16.0	9.0	0.0
3	0.0	0.0	7.0	15.0	13.0	1.0	0.0	0.0	0.0	8.0	...	9.0	0.0	0.0	0.0	7.0	13.0	13.0	9.0	0.0	0.0
4	0.0	0.0	0.0	1.0	11.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	2.0	16.0	4.0	0.0	0.0

5 rows × 64 columns

Entrée [7]:
```
df['target'] = digits.target
```

Entrée [8]:
```
df[0:12]
```

Out[8]:

	0	1	2	3	4	5	6	7	8	9	...	55	56	57	58	59	60	61	62	63	target
0	0.0	0.0	5.0	13.0	9.0	1.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	6.0	13.0	10.0	0.0	0.0	0.0	0
1	0.0	0.0	0.0	12.0	13.0	5.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	11.0	16.0	10.0	0.0	0.0	1
2	0.0	0.0	0.0	4.0	15.0	12.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	3.0	11.0	16.0	9.0	0.0	2

Entrée [9]:
```
X = df.drop('target',axis='columns')
y = df.target
```

Entrée [10]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2)
```

Entrée [11]:
```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=40)
model.fit(X_train, y_train)
```

Out[11]:
```
RandomForestClassifier(bootstrap=True, class_weight=None, criterion='gini',
                       max_depth=None, max_features='auto', max_leaf_nodes=None,
                       min_impurity_decrease=0.0, min_impurity_split=None,
                       min_samples_leaf=1, min_samples_split=2,
                       min_weight_fraction_leaf=0.0, n_estimators=40,
                       n_jobs=None, oob_score=False, random_state=None,
                       verbose=0, warm_start=False)
```

Entrée [12]:
```
model.score(X_test, y_test)
```

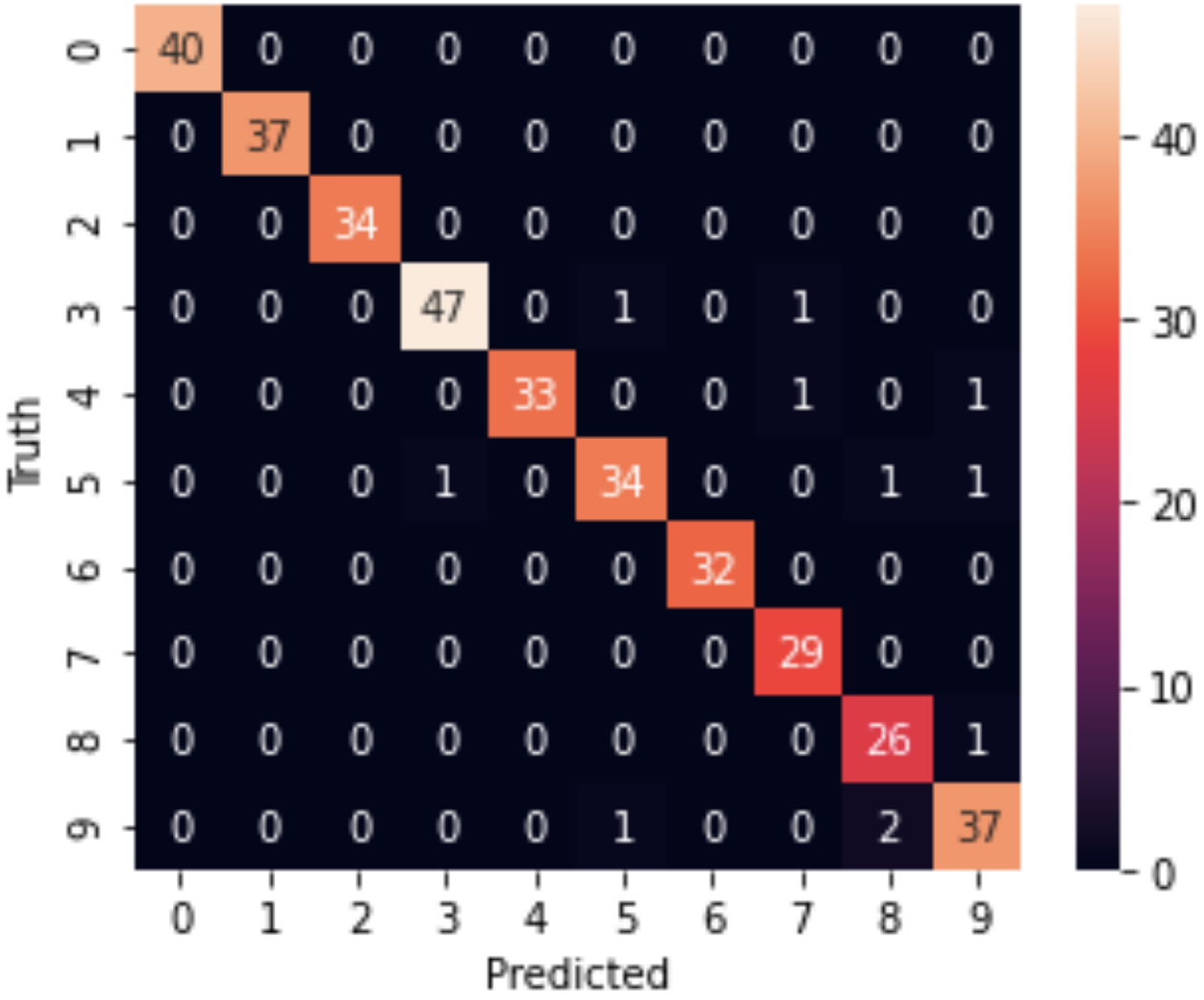
Out[12]:
```
0.9722222222222222
```

Entrée [13]:
```
y_predicted = model.predict(X_test)
```

Entrée [14]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_predicted)
cm
```

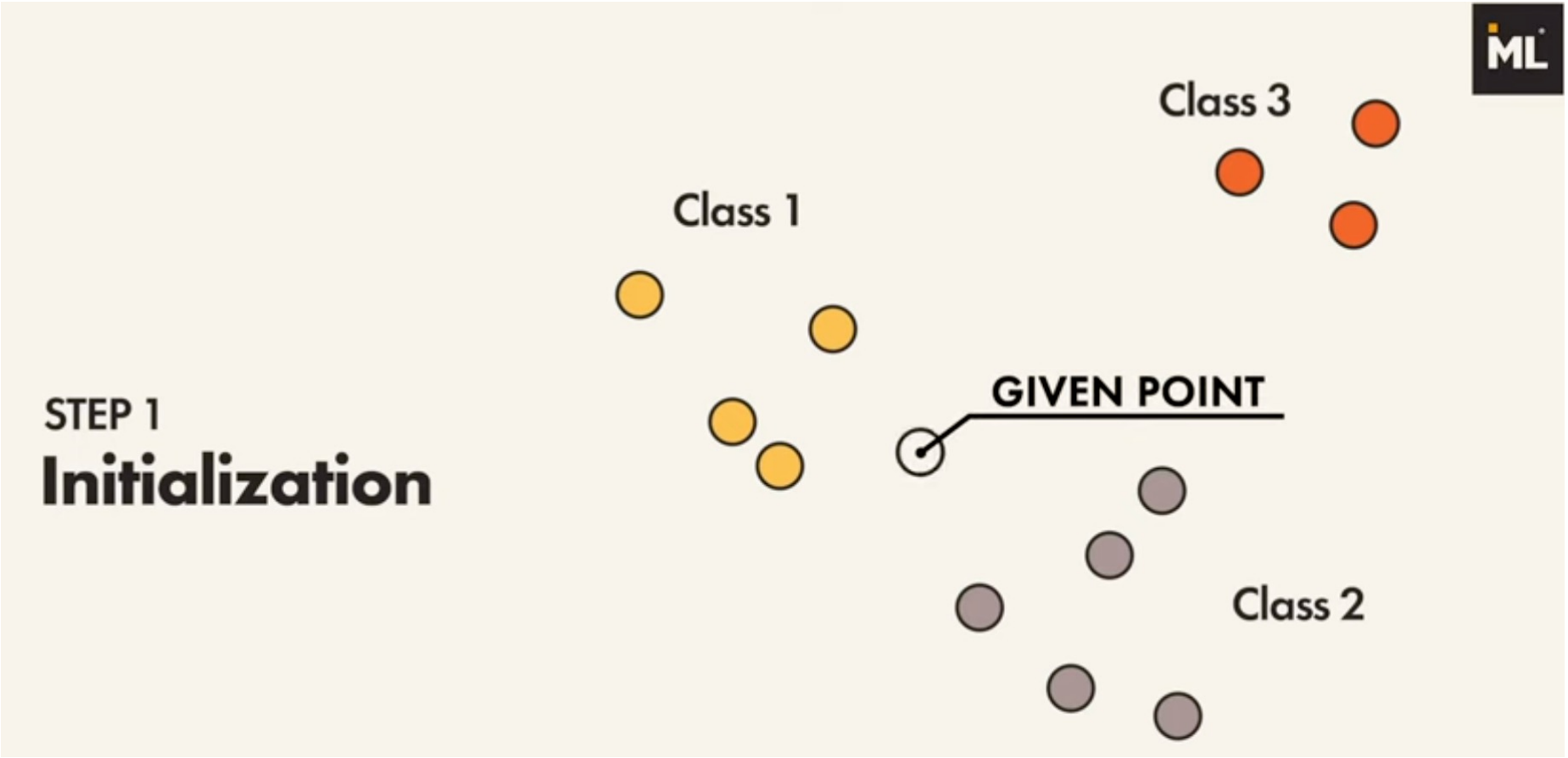
Out[14]:
```
array([[40,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0, 31,  0,  0,  0,  1,  0,  0,  0,  0],
       [ 0,  0, 37,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  1, 33,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0,  0, 31,  0,  0,  1,  0,  0],
       [ 0,  0,  0,  0,  1, 38,  0,  0,  0,  1],
       [ 0,  0,  0,  0,  0,  0, 37,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0, 38,  0,  1],
       [ 0,  0,  1,  1,  0,  0,  0,  0, 33,  0],
       [ 0,  1,  0,  1,  0,  0,  0,  0,  0, 32]])
```

Entrée [15]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sn
plt.figure(figsize=(5,4))
sn.heatmap(cm, annot=True)
plt.xlabel('Predicted')
plt.ylabel('Truth')
```


Truth
Predicted
0 1 2 3 4 5 6 7 8 9
40 0 0 0 0 0 0 0 0 0
0 37 0 0 0 0 0 0 0 0
0 0 34 0 0 0 0 0 0 0
0 0 0 47 0 1 0 1 0 0
0 0 0 0 33 0 0 1 0 1
0 0 0 1 0 34 0 0 1 1
0 0 0 0 0 0 32 0 0 0
0 0 0 0 0 0 0 29 0 0
0 0 0 0 0 0 0 0 26 1
0 0 0 0 0 1 0 0 2 37
40
30
20
10
0

K-Nearest Neighbors

KNN


ML
Class 3
Class 1
GIVEN POINT
STEP 1
Initialization
Class 2

STEP 2

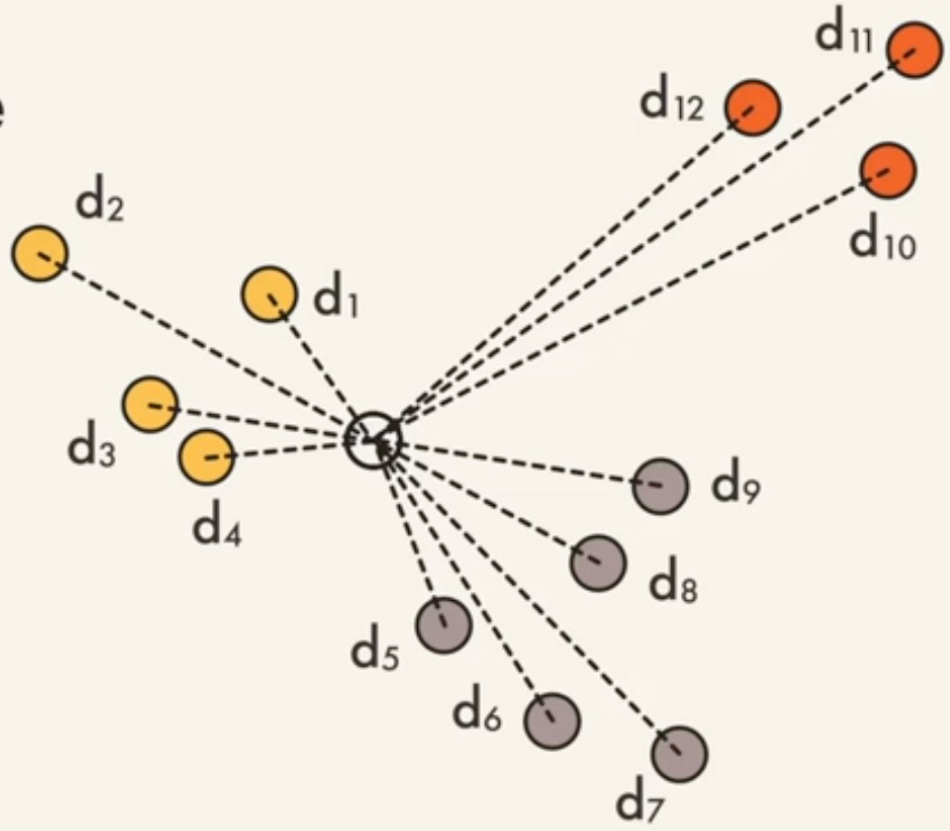
Calculate Distance

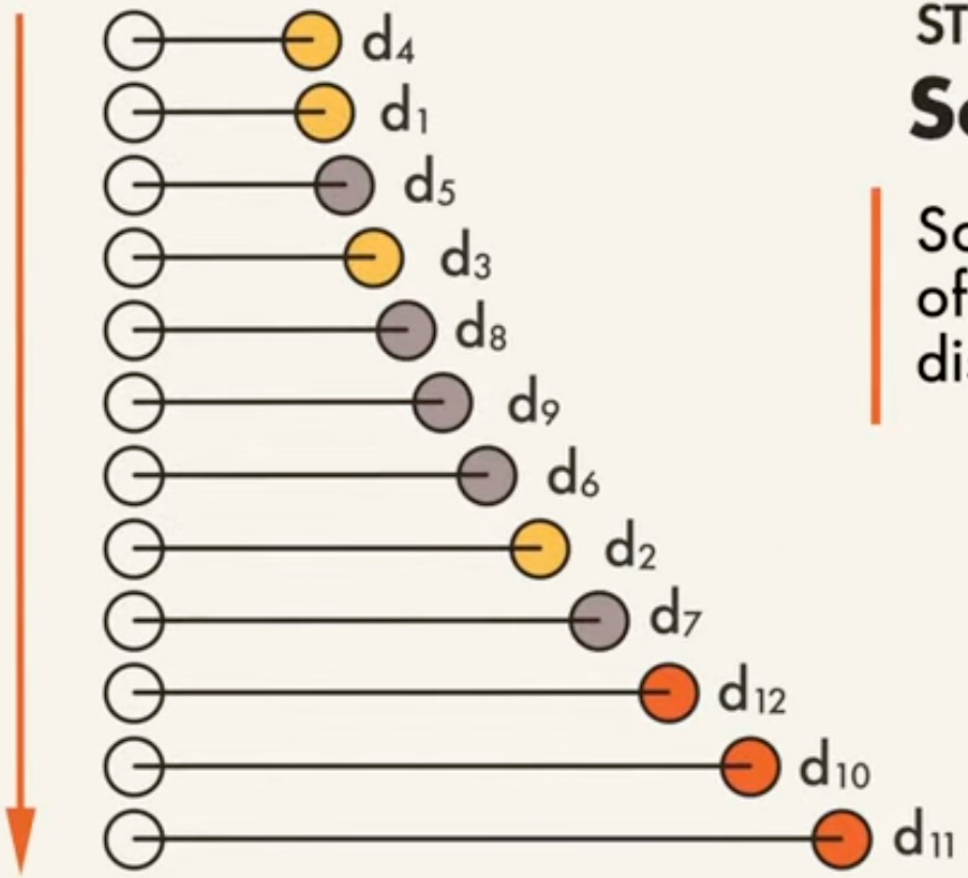
Euclidean:

$$d(x,y) = \sqrt{\sum_{i=1}^{m} (x_i - y_i)^2}$$

Manhattan/city - block:

$$d(x,y) = \sum_{i=1}^{m} |x_i - y_i|$$

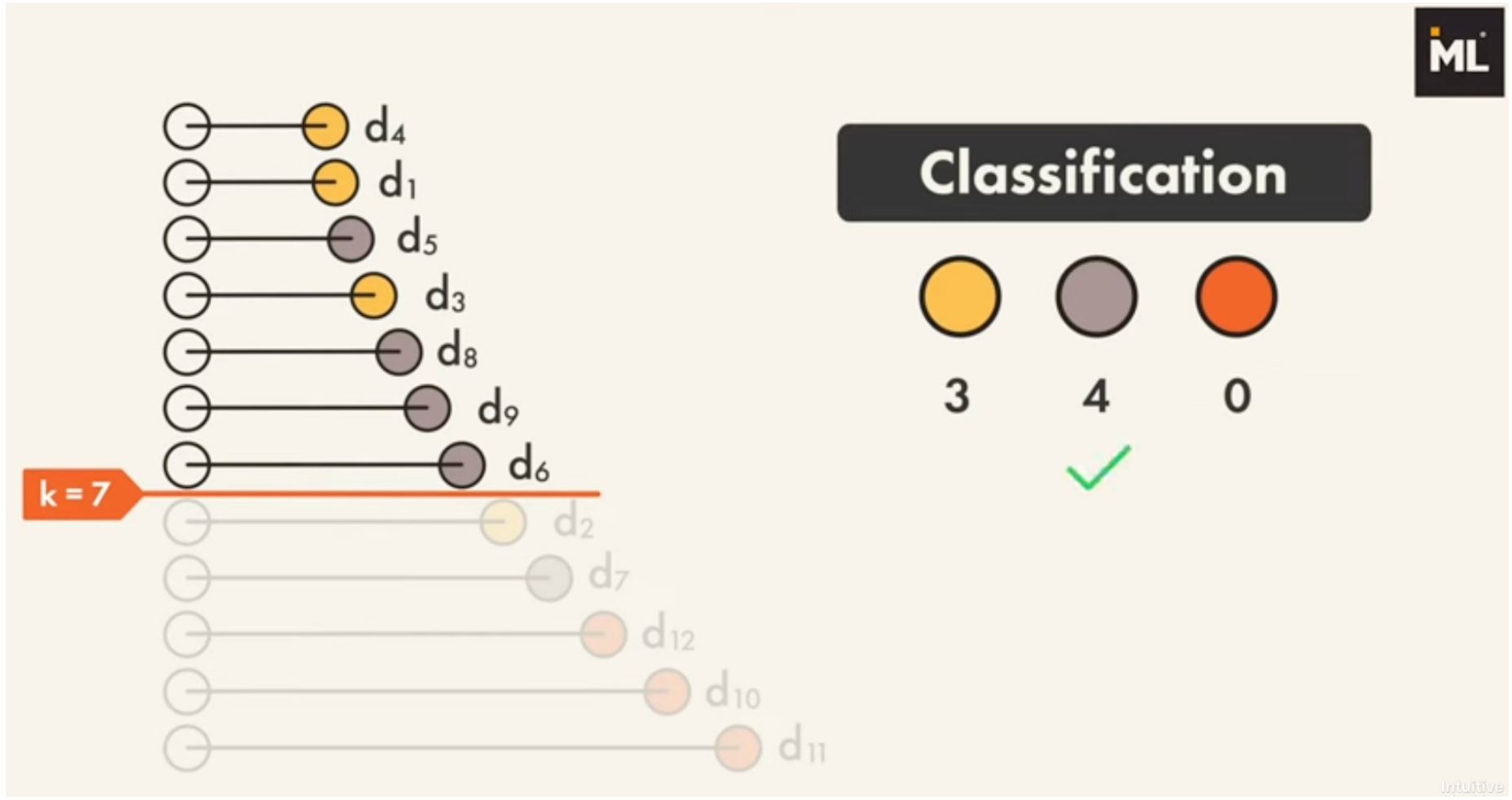


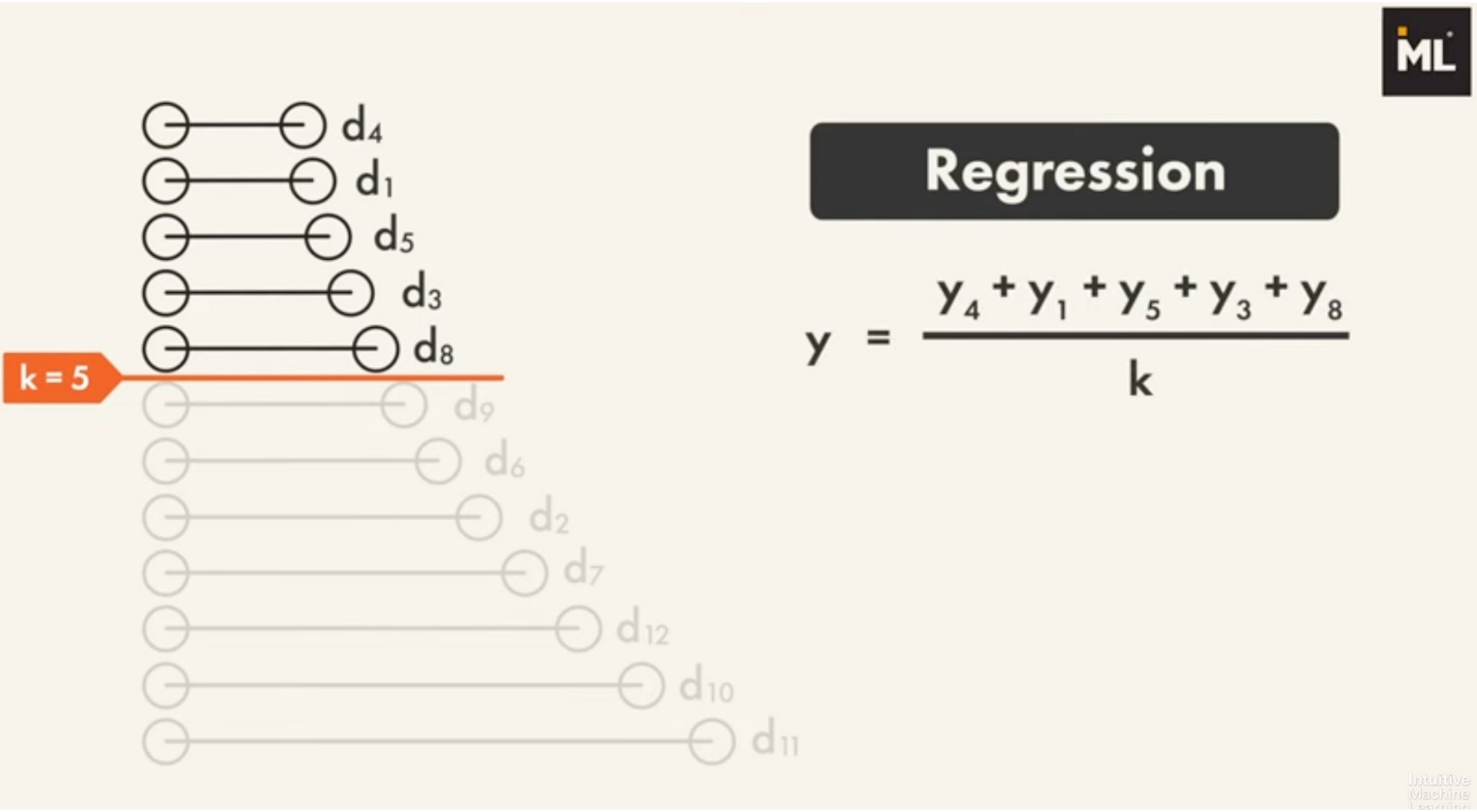


STEP 3

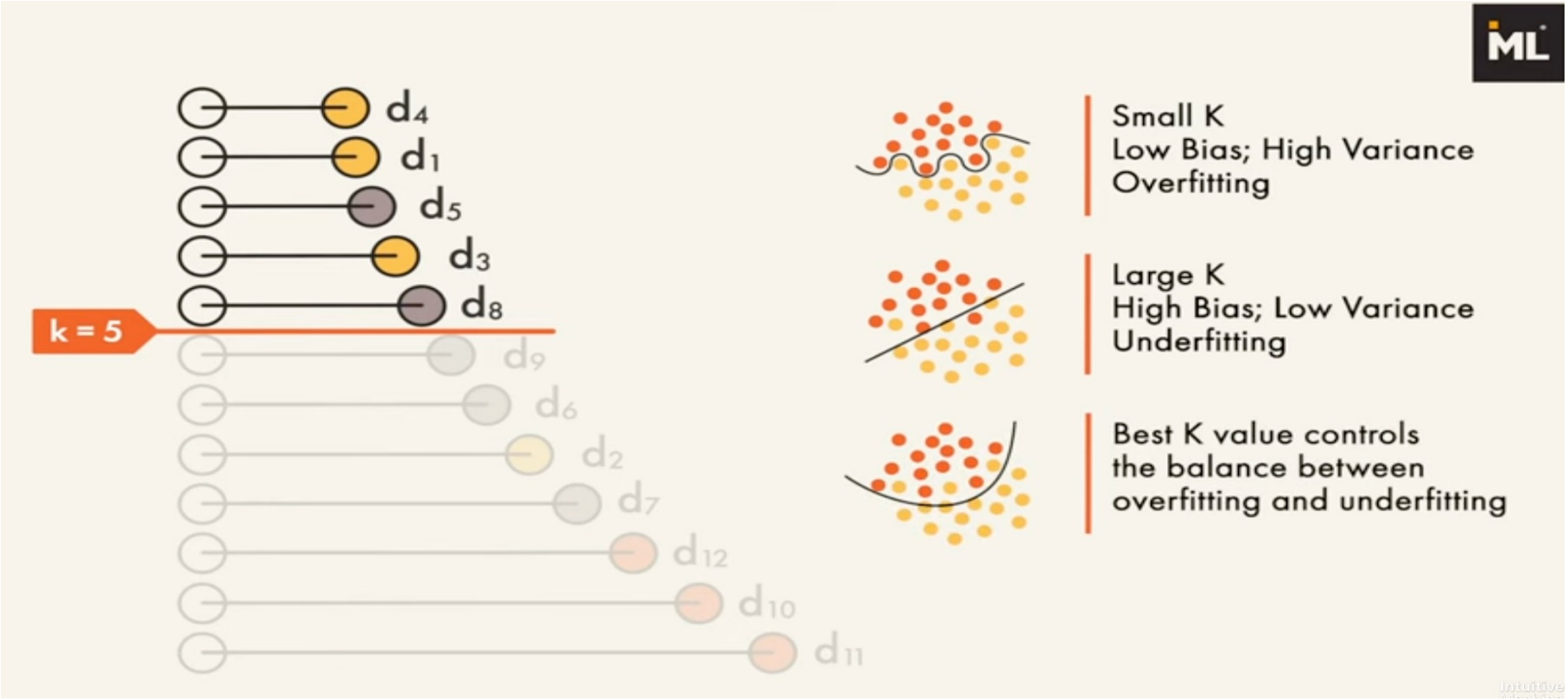
Sort Distance

Sort the nearest neighbors of the given point by the distances in increasing order


ML
d4
d1
d5
d3
d8
d9
d6
k = 7
d2
d7
d12
d10
d11
Classification
3
4
0
Intuitive


ML
d4
d1
d5
d3
d8
k = 5
d9
d6
d2
d7
d12
d10
d11
Regression
y = (y4 + y1 + y5 + y3 + y8) / k
Intuitive Machine

Choice for K


ML
d4
d1
d5
d3
d8
k = 5
d9
d6
d2
d7
d12
d10
d11
Small K
Low Bias; High Variance
Overfitting
Large K
High Bias; Low Variance
Underfitting
Best K value controls
the balance between
overfitting and underfitting

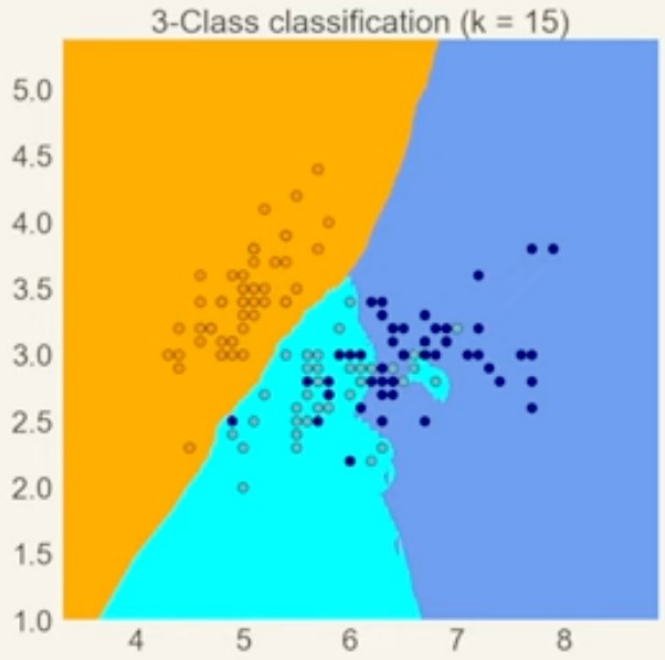
CODE

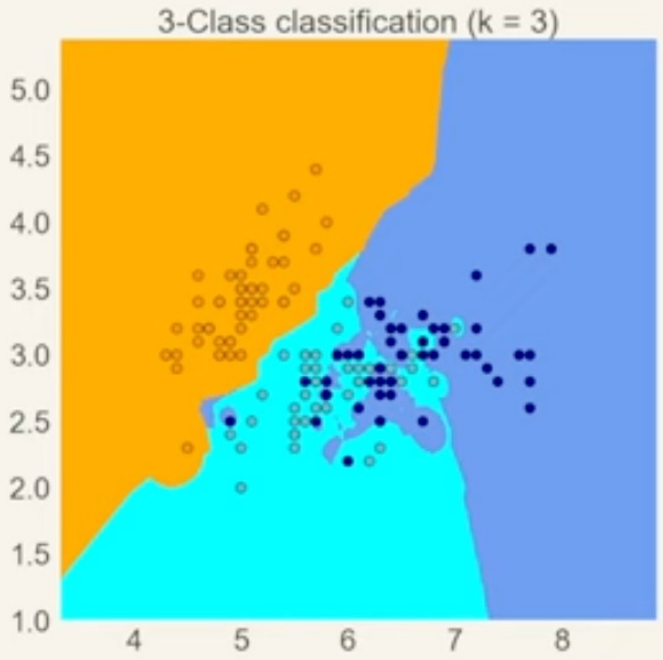
EXAMPLE

```
# knn from sklearn
from sklearn import neighbors, datasets
# import some data to play with
iris = datasets.load_iris()
# we only take the first two features for demonstration
X = iris.data[:, :2]
y = iris.target
clf = neighbors.KNeighborsClassifier(n_neighbors=15)
clf.fit(X, y)
```

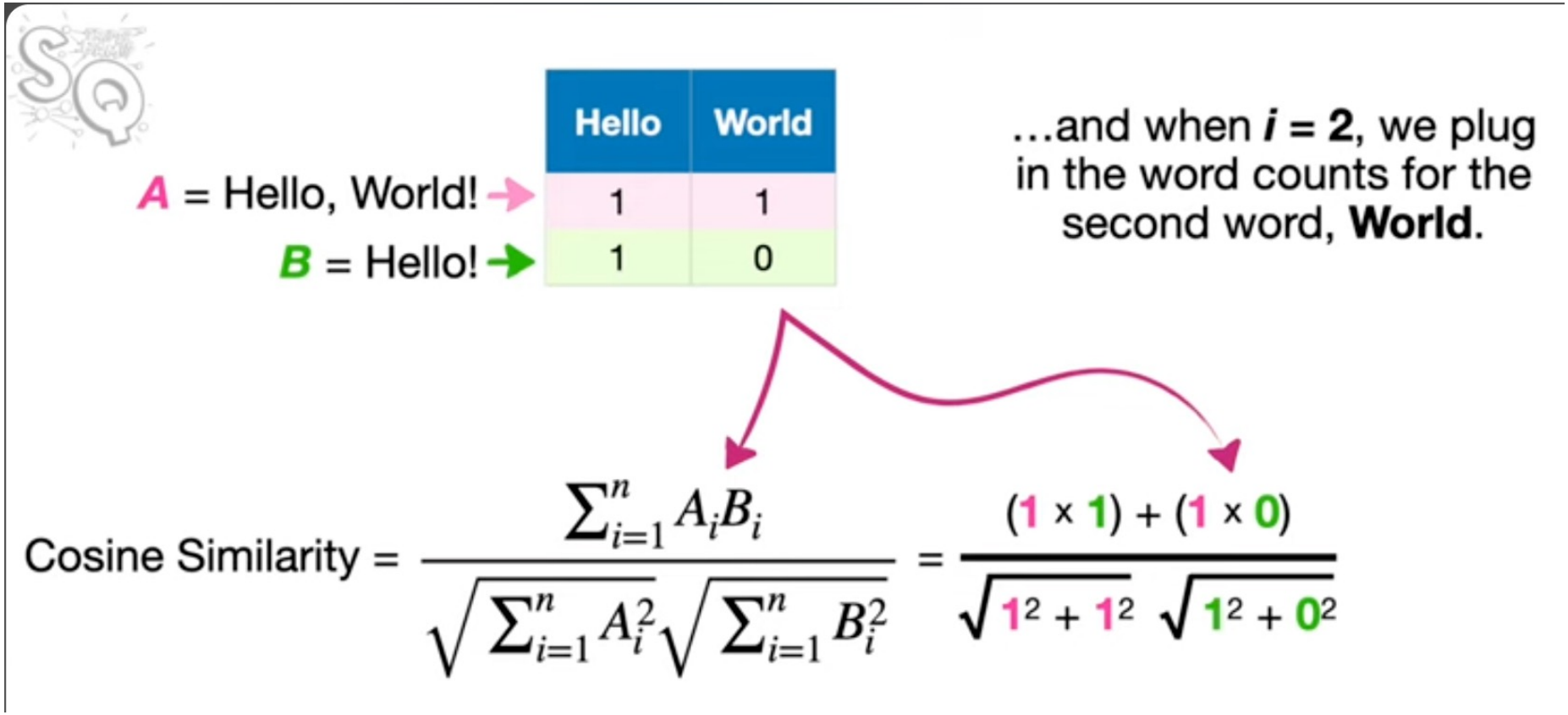
CODE

VISUAL


3-Class classification (k = 15)
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
4
5
6
7
8


3-Class classification (k = 3)
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
4
5
6
7
8

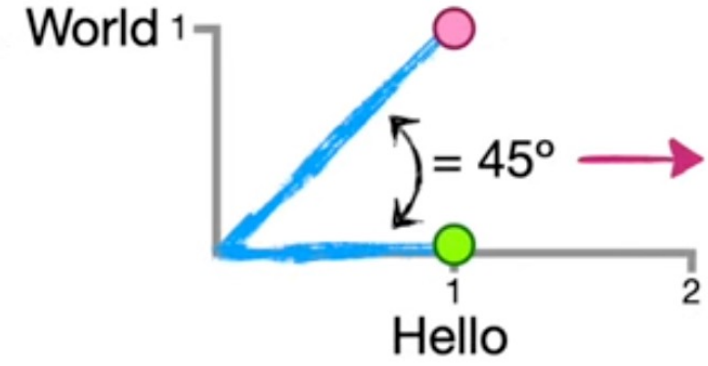
COSINE SIMILARITY


Hello
World
A = Hello, World!
1
1
B = Hello!
1
0
…and when i = 2, we plug in the word counts for the second word, World.
Cosine Similarity = $\frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}} = \frac{(1 \times 1) + (1 \times 0)}{\sqrt{1^2 + 1^2}\sqrt{1^2 + 0^2}}$

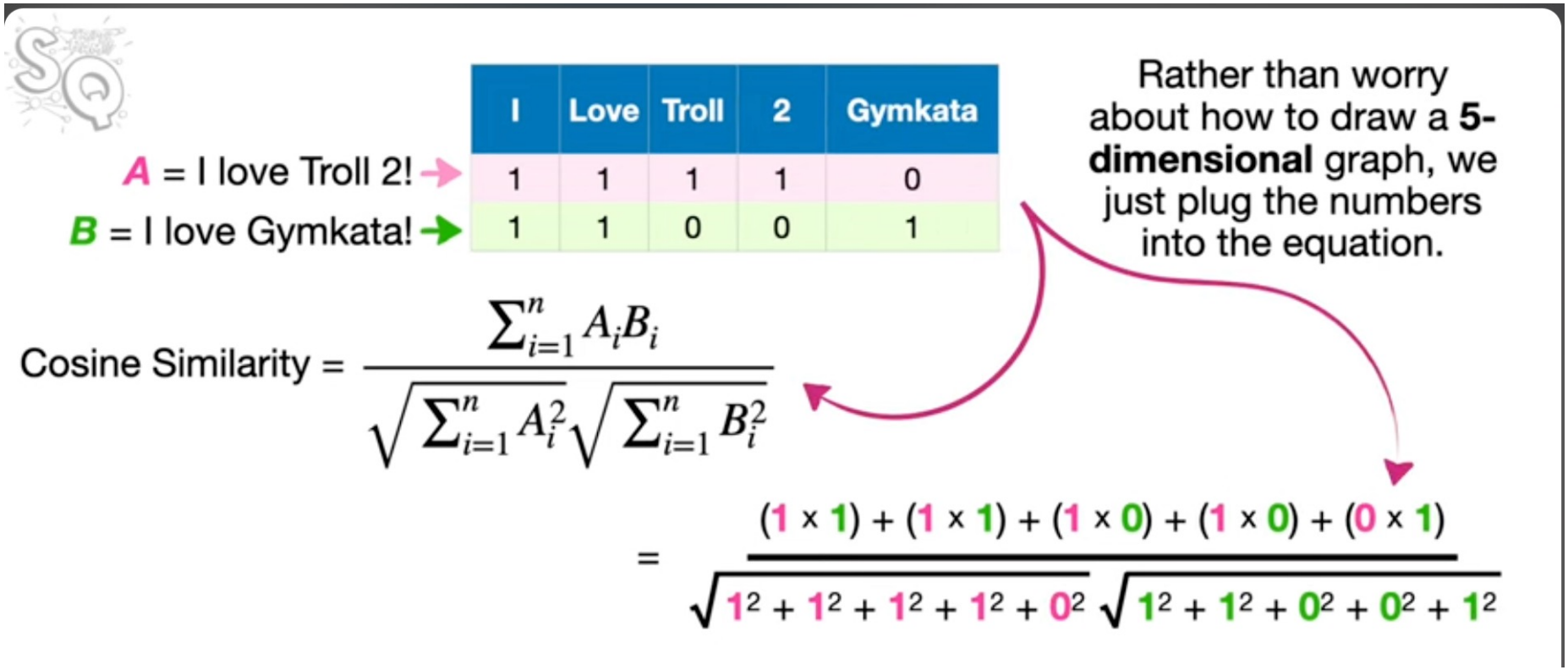
	Hello	World
A = Hello, World! →	1	1
B = Hello! →	1	0

…which is the same value we got when we took the **cos(45°)**.

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}} = \frac{(1 \times 1) + (1 \times 0)}{\sqrt{1^2 + 1^2}\ \sqrt{1^2 + 0^2}} = 0.71$$

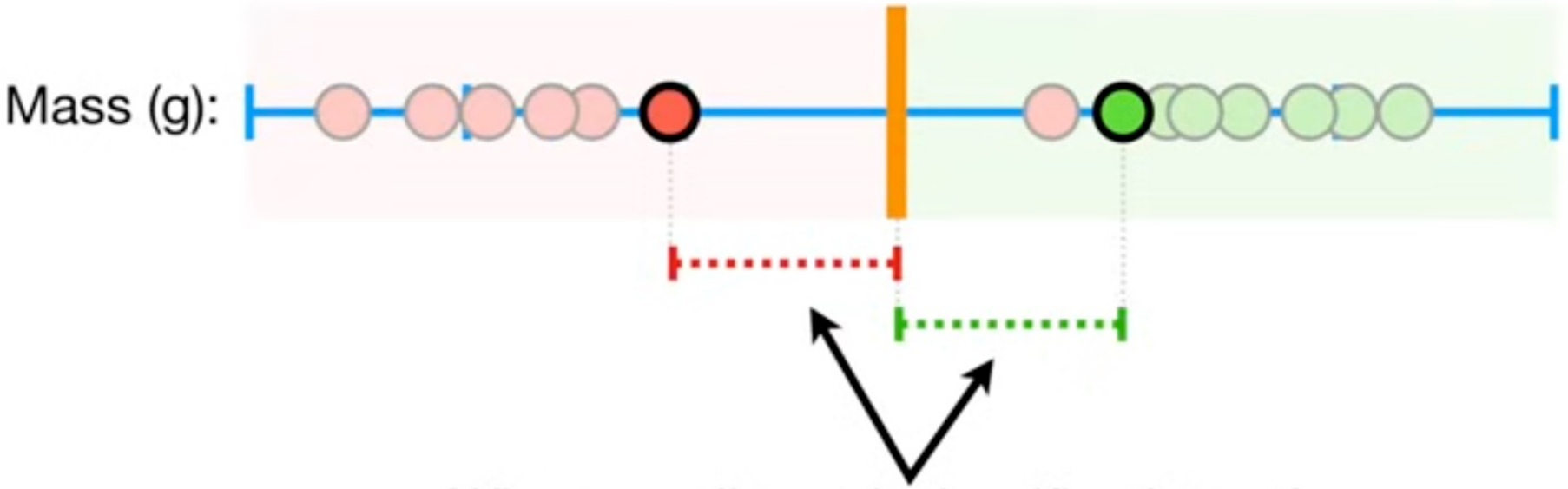


cos(45°) = 0.71 = Cosine Similarity

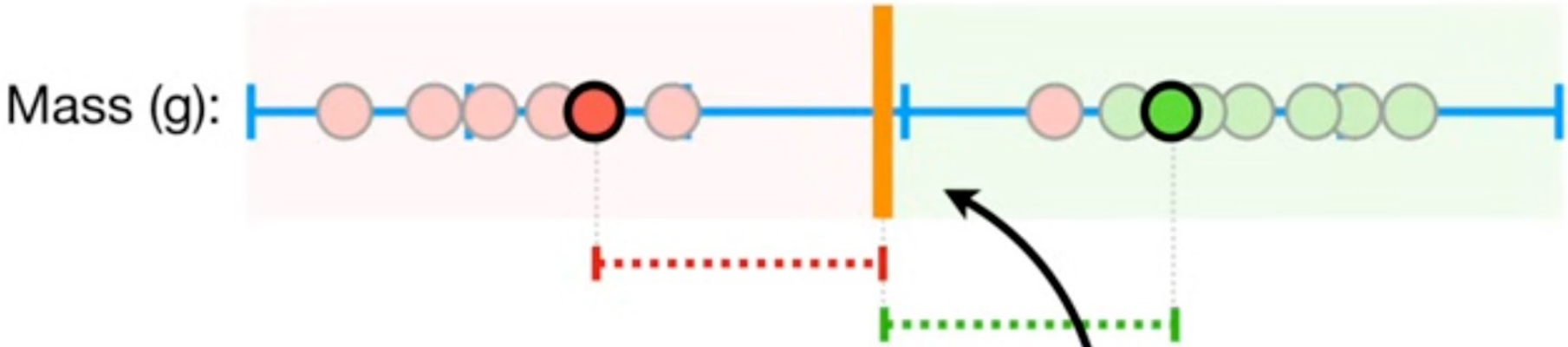

I
Love
Troll
2
Gymkata
A = I love Troll 2!
1 1 1 1 0
B = I love Gymkata!
1 1 0 0 1
Rather than worry about how to draw a 5-dimensional graph, we just plug the numbers into the equation.
Cosine Similarity = $\frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$
$= \frac{(1 \times 1) + (1 \times 1) + (1 \times 0) + (1 \times 0) + (0 \times 1)}{\sqrt{1^2 + 1^2 + 1^2 + 1^2 + 0^2}\sqrt{1^2 + 1^2 + 0^2 + 0^2 + 1^2}}$

Support Vector Machines

SVM



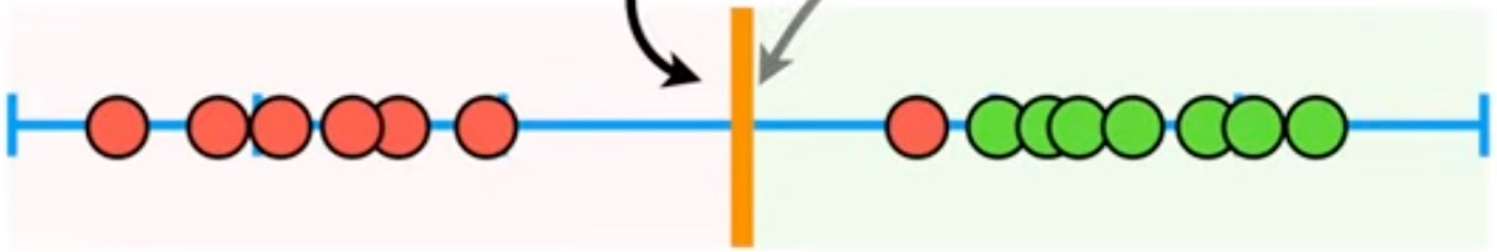
When we allow misclassifications, the distance between the observations and the threshold is called a **Soft Margin**.

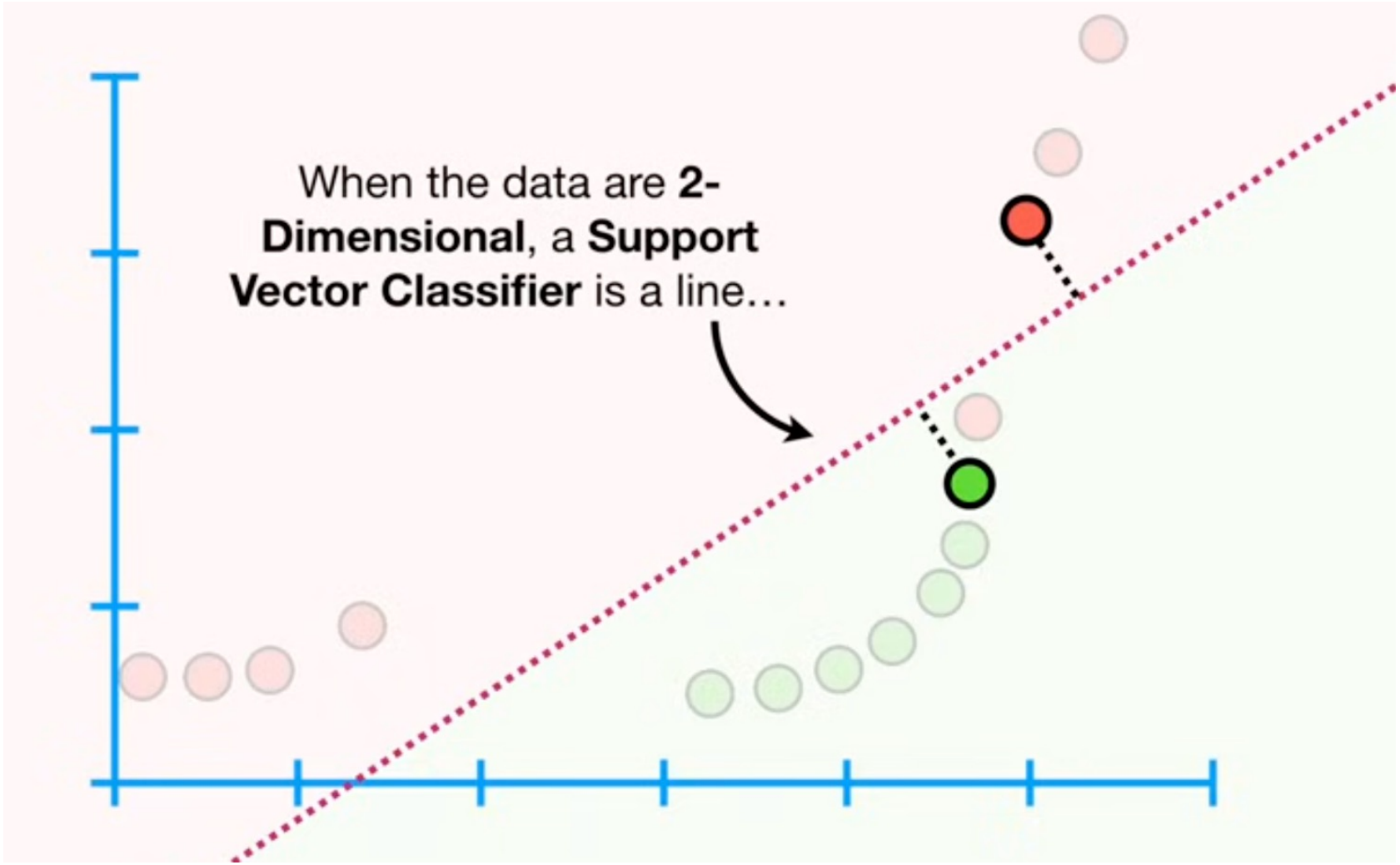


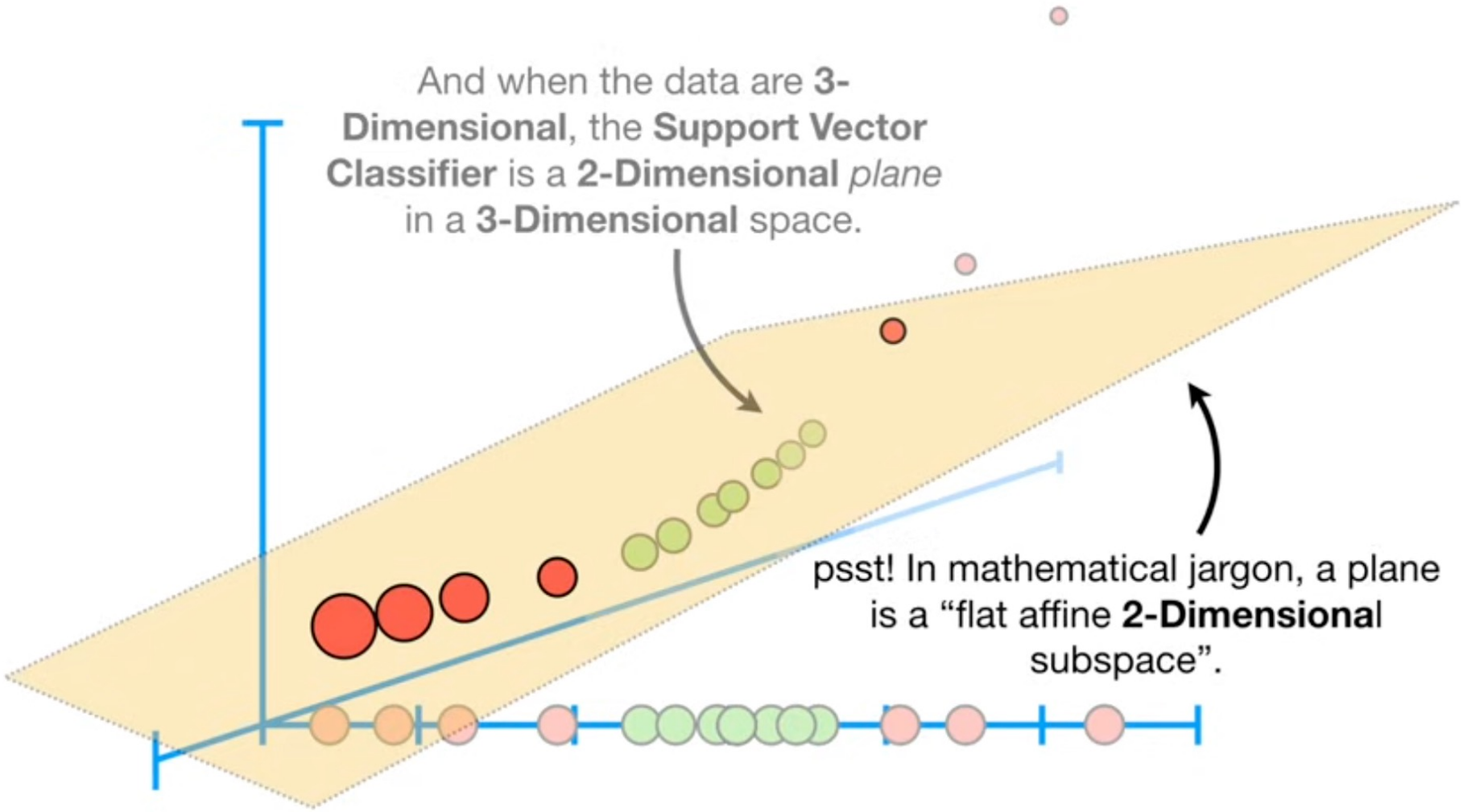
…then we are using a **Soft Margin Classifier** aka a **Support Vector Classifier** to classify observations.

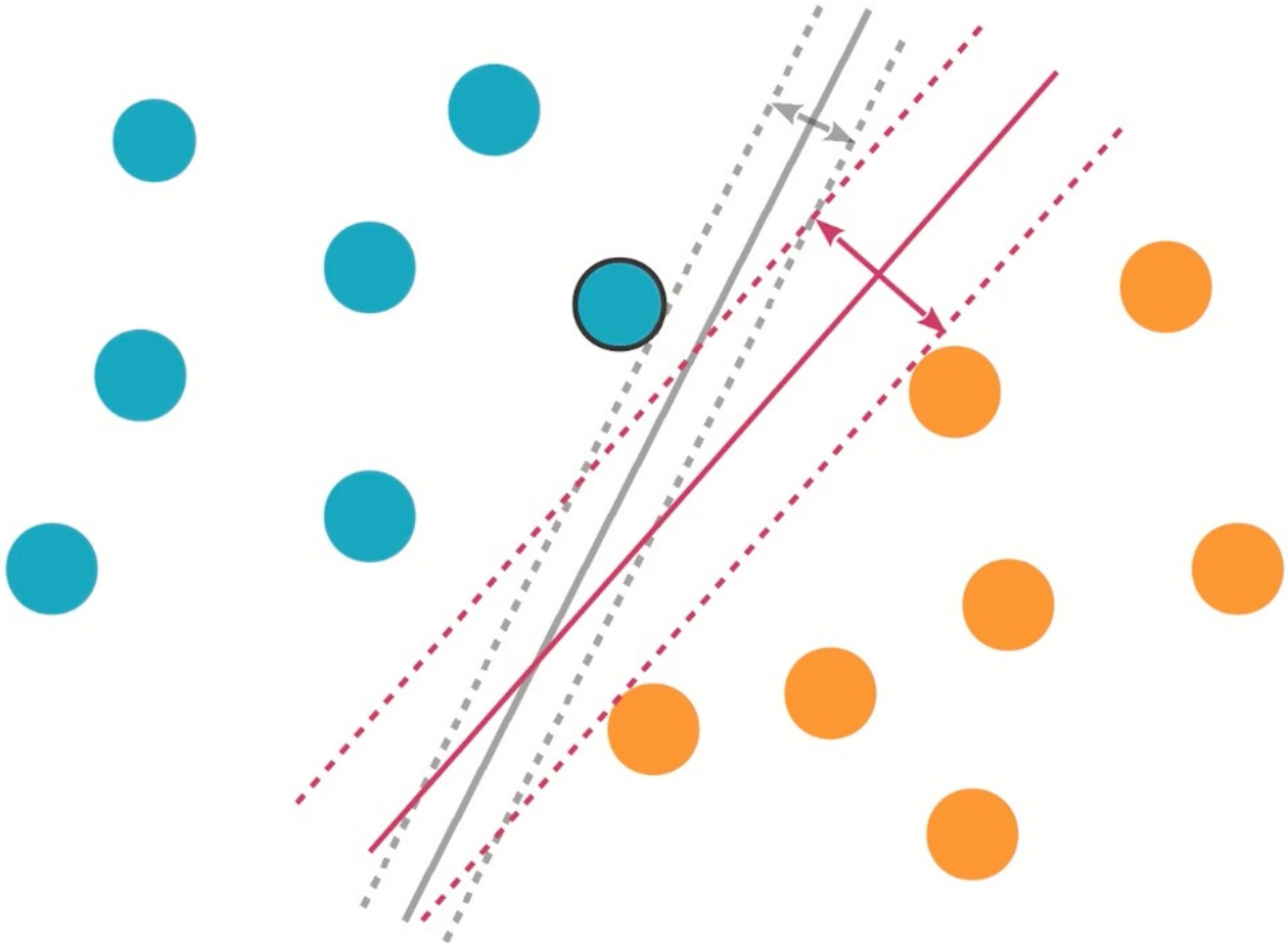
But we know that when the data are **1-Dimensional**, the **Support Vector Classifier** is a single *point* on a **1-Dimensional** number line.

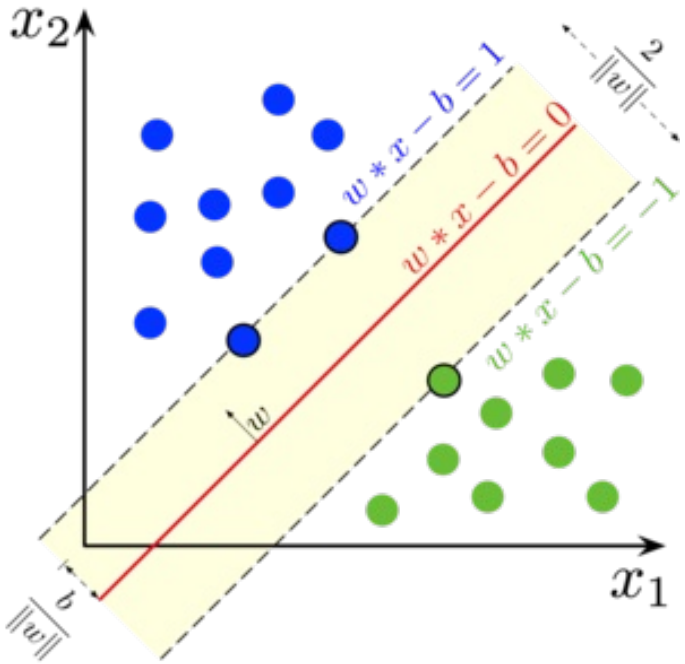
psst! In mathematical jargon, a point is a "flat affine **0-Dimensional** subspace".



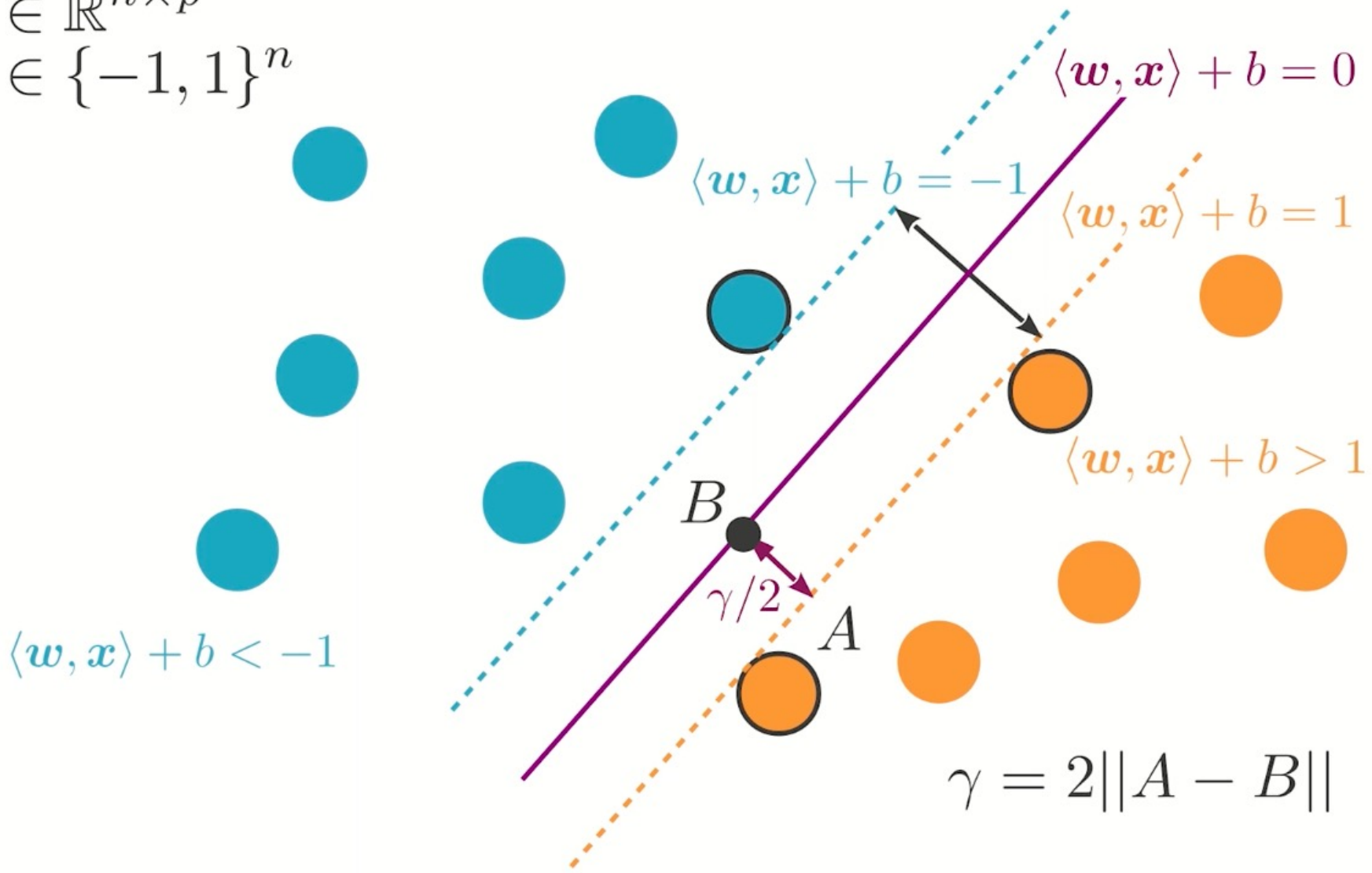

When the data are **2-Dimensional**, a **Support Vector Classifier** is a line…


And when the data are **3-Dimensional**, the **Support Vector Classifier** is a **2-Dimensional** *plane* in a **3-Dimensional** space.
psst! In mathematical jargon, a plane is a “flat affine **2-Dimensional** subspace”.




x_2
x_1
$w * x - b = 1$
$w * x - b = 0$
$w * x - b = -1$
w
$\frac{2}{\|w\|}$
$\frac{b}{\|w\|}$

$$X \in \mathbb{R}^{n \times p}$$
$$y \in \{-1, 1\}^n$$



$$X \in \mathbb{R}^{n \times p}$$
$$y \in \{-1, 1\}^n$$

$$\min_{\boldsymbol{w} \in \mathbb{R}^p} \frac{1}{2}||\boldsymbol{w}||^2 \quad \text{t.q.} \quad y^{(i)}.\left(\langle \boldsymbol{w}, \boldsymbol{x}^{(i)} \rangle + b\right) \geq 1 \quad \forall i \in \{1, \ldots, n\}$$

$$\max_{\alpha \in \mathbb{R}^n_+} \min_{\substack{\boldsymbol{w} \in \mathbb{R}^p \\ b \in \mathbb{R}}} \boxed{\frac{1}{2}||\boldsymbol{w}||^2 - \sum_{i=1}^{n} \alpha_i \left(y^{(i)}.\left(\langle \boldsymbol{w}, \boldsymbol{x}^{(i)} \rangle + b\right) - 1\right)}$$

$$L_p$$

$$\alpha_i = 0 \;\; \text{ou} \;\; \left(y^{(i)}.\left(\langle \boldsymbol{w}, \boldsymbol{x}^{(i)} \rangle + b\right) - 1\right) = 0$$

$$\nabla_{\boldsymbol{w}} L_p = 0 \longrightarrow \boldsymbol{w} = \sum_{i=1}^{n} \alpha_i y^{(i)} \boldsymbol{x}^{(i)} \qquad \nabla_b L_p = 0 \longrightarrow \boxed{\sum_{i=1}^{n} \alpha_i y^{(i)} = 0}$$

$$\max_{\alpha \in \mathbb{R}^n_+} -\frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)} \rangle + \sum_{i=1}^{n} \alpha_i \quad \text{t.q.} \quad \boxed{\sum_{i=1}^{n} \alpha_i y^{(i)} = 0}$$

$X \in \mathbb{R}^{n \times p}$

$y \in \{-1, 1\}^n$

primal

$$\min_{\boldsymbol{w} \in \mathbb{R}^p} \frac{1}{2}||\boldsymbol{w}||^2 \quad \text{t.q.} \quad y^{(i)}.\left(\langle \boldsymbol{w}, \boldsymbol{x}^{(i)} \rangle + b\right) \geq 1 \quad \forall i \in \{1, \ldots, n\}$$

$$\max_{\alpha \in \mathbb{R}^n_+} \min_{\substack{\boldsymbol{w} \in \mathbb{R}^p \\ b \in \mathbb{R}}} \frac{1}{2}||\boldsymbol{w}||^2 - \sum_{i=1}^{n} \alpha_i \left(y^{(i)}.\left(\langle \boldsymbol{w}, \boldsymbol{x}^{(i)} \rangle + b\right) - 1\right)$$

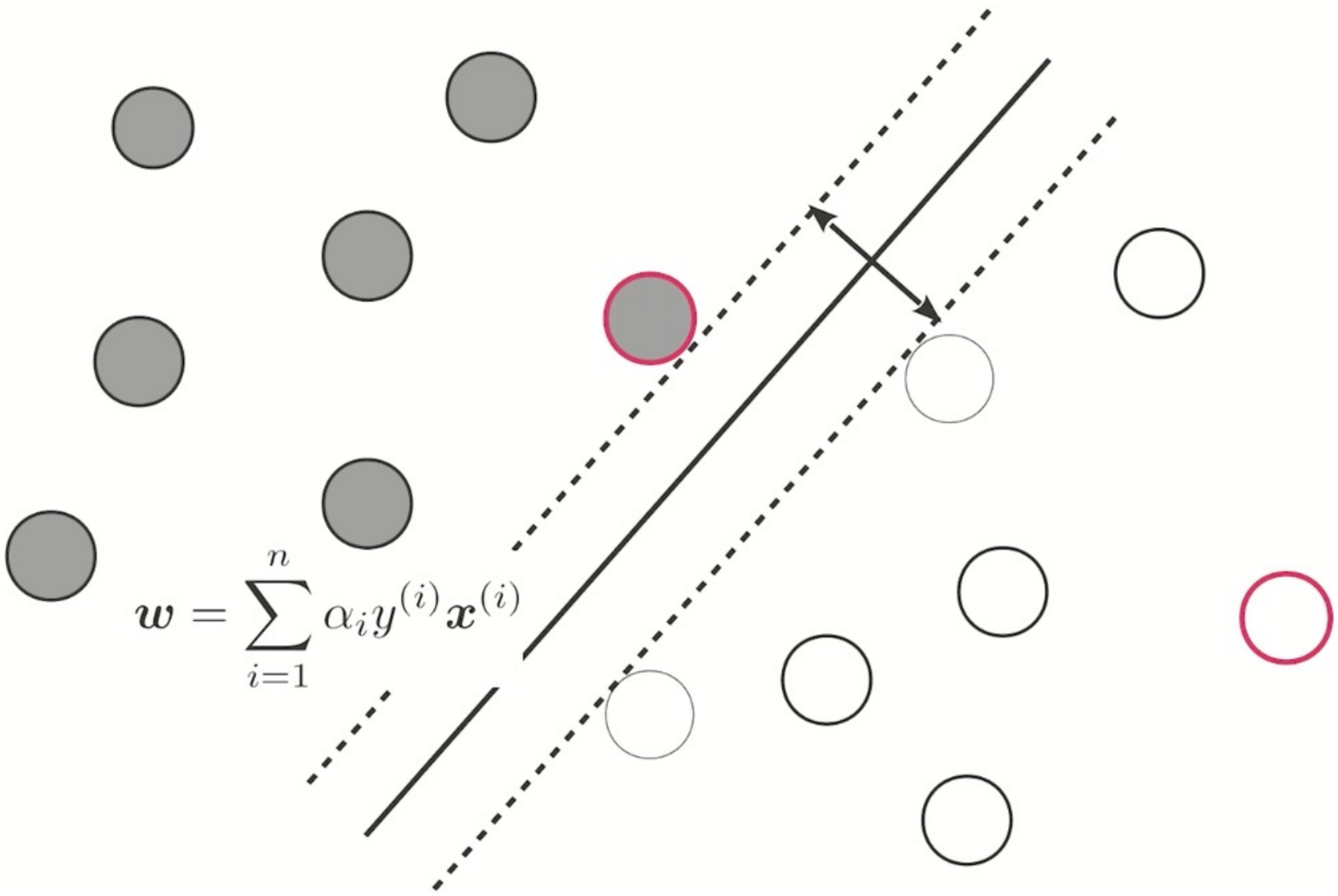
L_p

$$\alpha_i = 0 \text{ ou } \left(y^{(i)}.\left(\langle \boldsymbol{w}, \boldsymbol{x}^{(i)} \rangle + b\right) - 1\right) = 0$$

$$\nabla_{\boldsymbol{w}} L_p = 0 \longrightarrow \boldsymbol{w} = \sum_{i=1}^{n} \alpha_i y^{(i)} \boldsymbol{x}^{(i)} \qquad \nabla_b L_p = 0 \longrightarrow \sum_{i=1}^{n} \alpha_i y^{(i)} = 0$$

dual

$$\max_{\alpha \in \mathbb{R}^n_+} -\frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)} \rangle + \sum_{i=1}^{n} \alpha_i \quad \text{t.q.} \quad \sum_{i=1}^{n} \alpha_i y^{(i)} = 0$$


$\boldsymbol{w} = \sum_{i=1}^{n} \alpha_i y^{(i)} \boldsymbol{x}^{(i)}$

primal

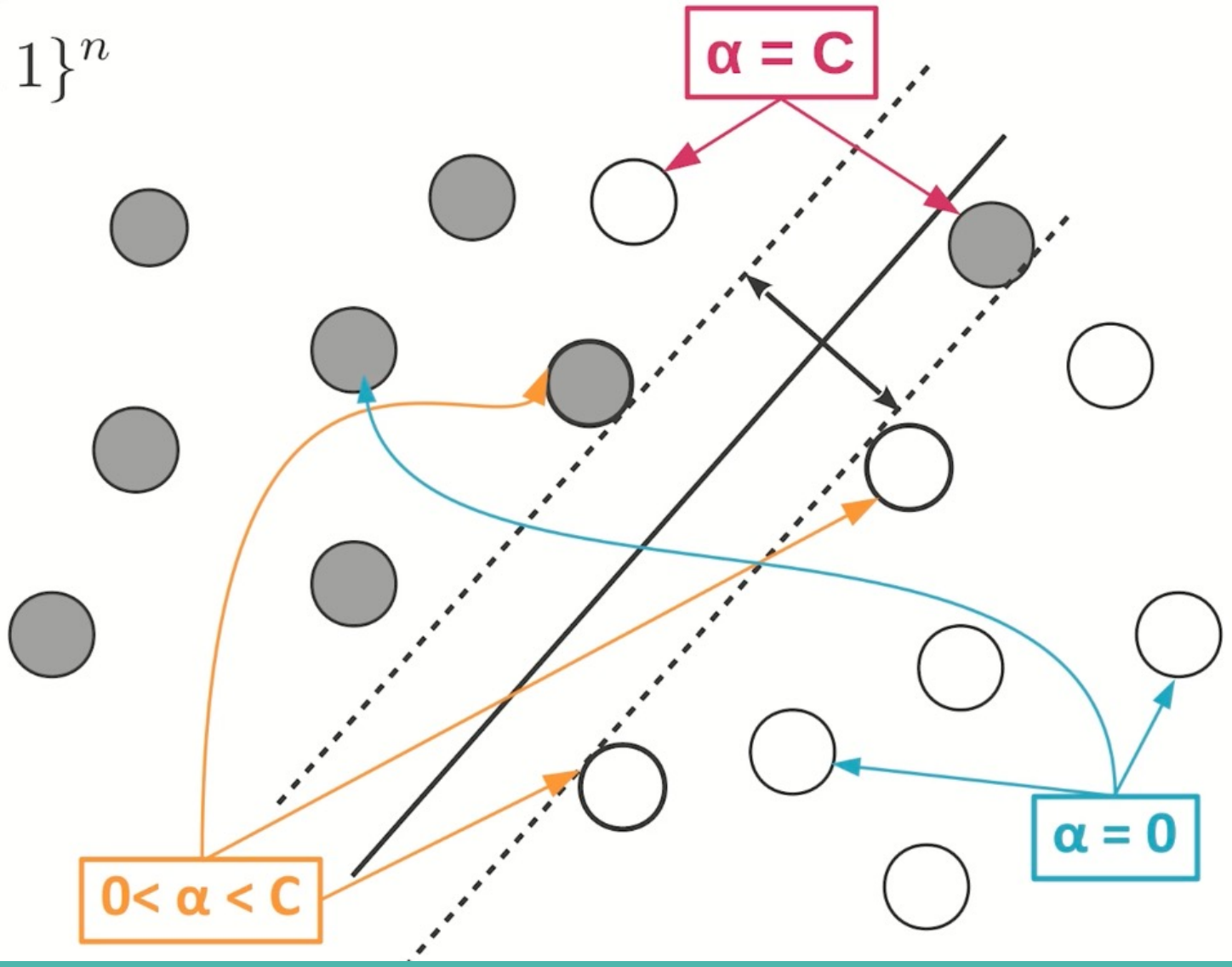
$$\min_{\substack{\boldsymbol{w}\in\mathbb{R}^p \\ b\in\mathbb{R} \\ \boldsymbol{\xi}\in\mathbb{R}^n_+}} \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_{i=1}^{n}\xi_i \quad \text{t.q. } y^{(i)}.\left(\langle \boldsymbol{w}, \boldsymbol{x}^{(i)}\rangle + b\right) \geq 1-\xi_i \quad \forall i \in \{1,\dots,n\}$$

- $\alpha_i = 0$
- $\alpha_i = C$
- $0 < \alpha_i < C$

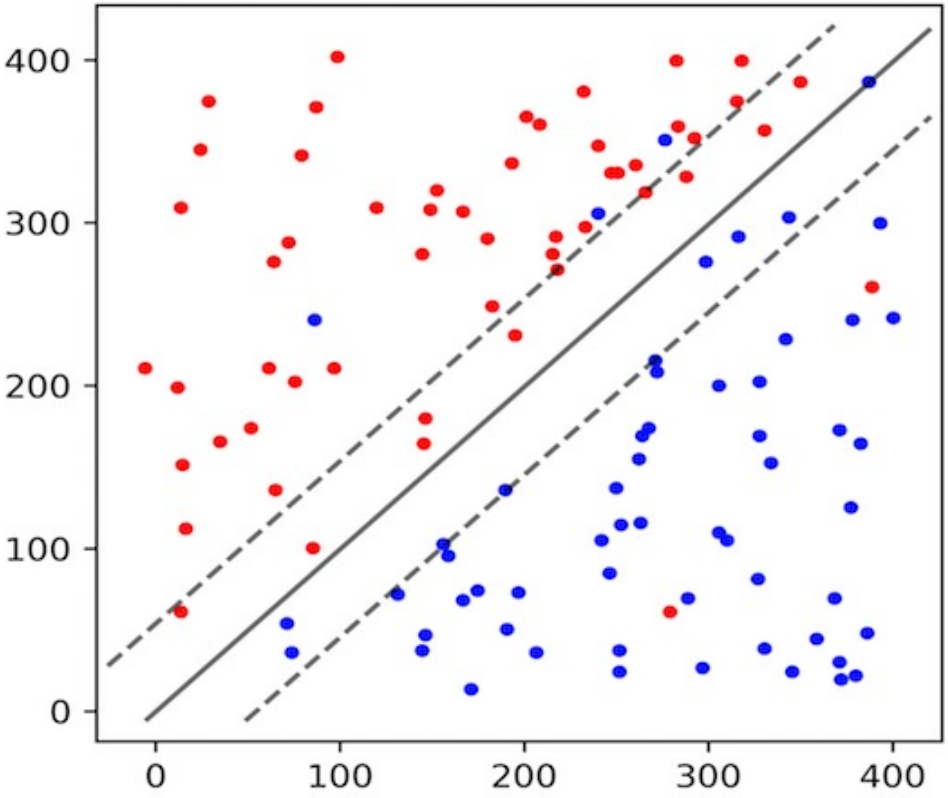
dual

$$\max_{\alpha\in\mathbb{R}^n_+} -\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y^{(i)}y^{(j)}\langle \boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\rangle + \sum_{i=1}^{n}\alpha_i \quad \text{t.q.} \sum_{i=1}^{n}\alpha_i y^{(i)} = 0 \text{ and } 0 \leq \alpha_i \leq C$$

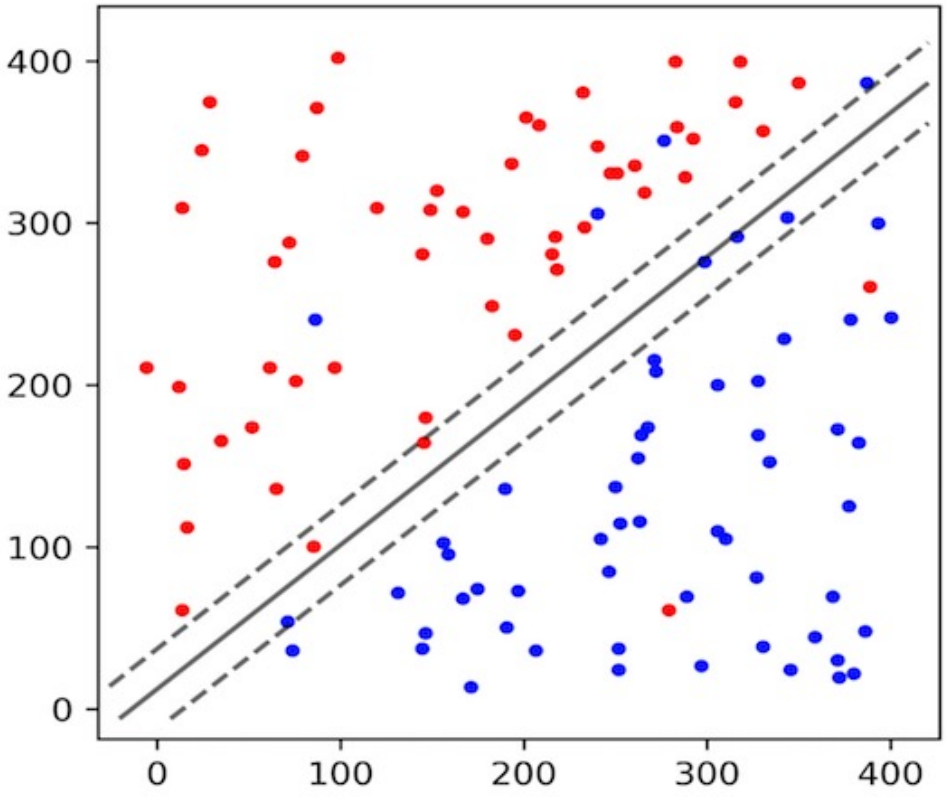
$$X \in \mathbb{R}^{n \times p}$$
$$y \in \{-1, 1\}^n$$



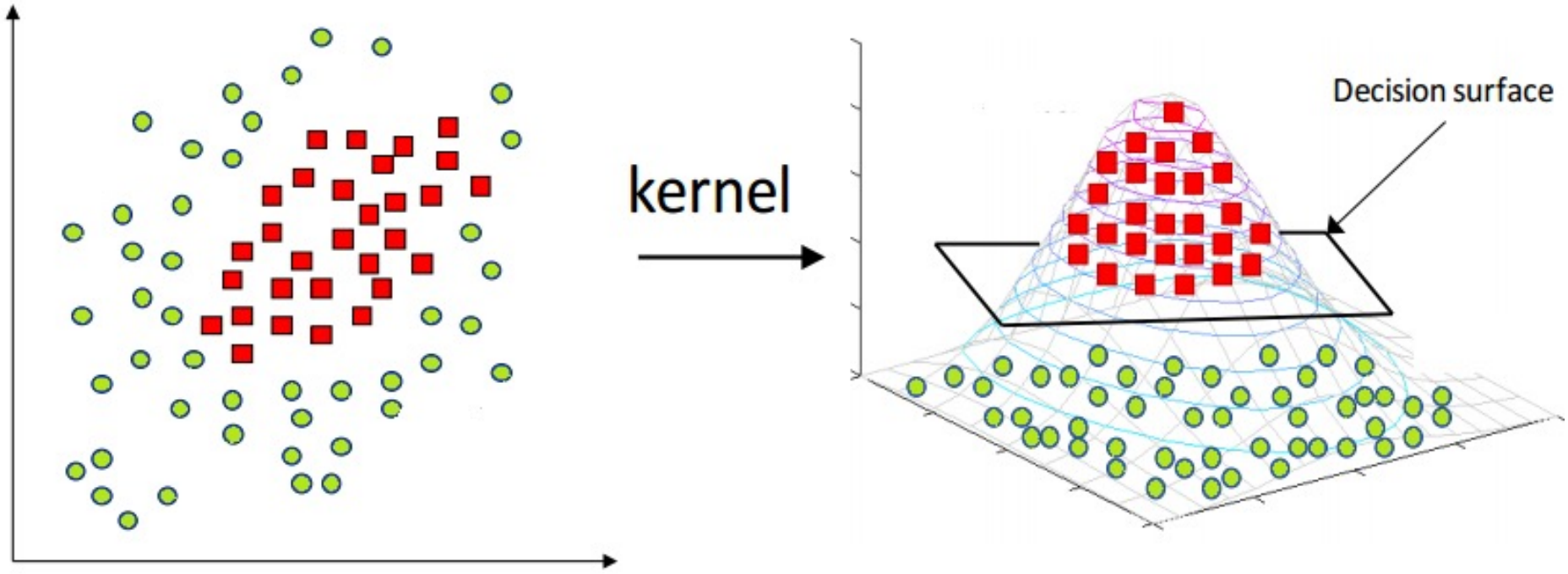
SVM Parameter C



C = 1



C = 100


kernel
Decision surface

sklearn.svm.SVC

class sklearn.svm. **SVC** (*C=1.0, kernel='rbf', degree=3, gamma='auto_deprecated', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr', random_state=None*) [source]

C-Support Vector Classification.

The implementation is based on libsvm. The fit time scales at least quadratically with the number of samples and may be impractical beyond tens of thousands of samples. For large datasets consider using **sklearn.linear_model.LinearSVC** or **sklearn.linear_model.SGDClassifier** instead, possibly after a **sklearn.kernel_approximation.Nystroem** transformer.

The multiclass support is handled according to a one-vs-one scheme.

For details on the precise mathematical formulation of the provided kernel functions and how `gamma`, `coef0` and `degree` affect each other, see the corresponding section in the narrative documentation: Kernel functions.

Read more in the User Guide.

Parameters:

C : *float, optional (default=1.0)*

Penalty parameter C of the error term.

kernel : *string, optional (default='rbf')*

Specifies the kernel type to be used in the algorithm. It must be one of 'linear', 'poly', 'rbf', 'sigmoid', 'precomputed' or a callable. If none is given, 'rbf' will be used. If a callable is given it is used to pre-compute the kernel matrix from data matrices; that matrix should be an array of shape `(n_samples, n_samples)`.

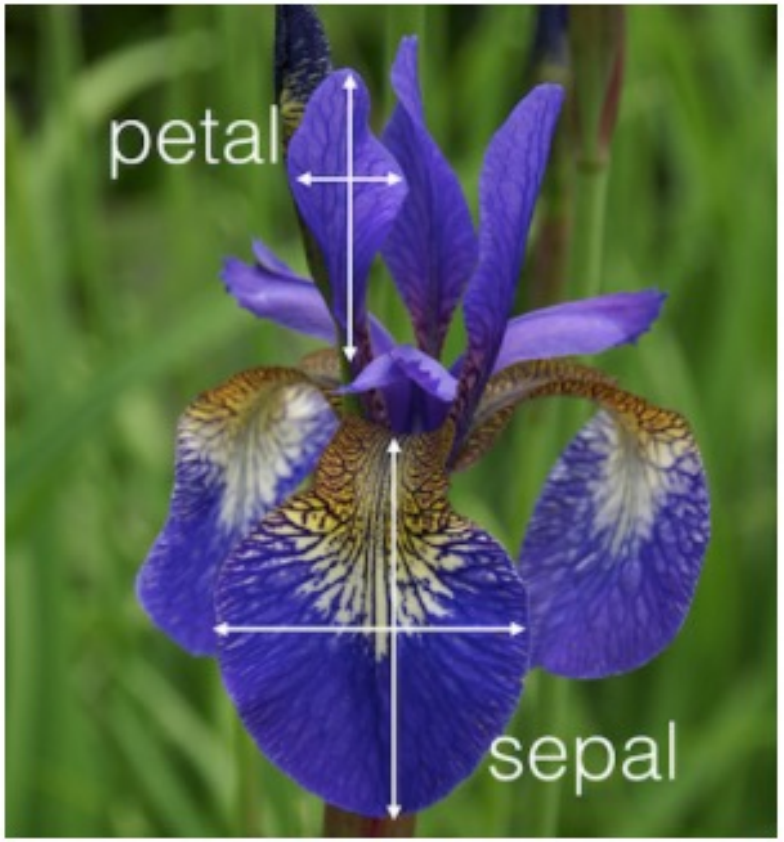
degree : *int, optional (default=3)*

Degree of the polynomial kernel function ('poly'). Ignored by all other kernels.

gamma : *float, optional (default='auto')*

Entrée [1]:
```
import pandas as pd
from sklearn.datasets import load_iris
iris = load_iris()
dir(iris)
```

Out[1]: ['DESCR', 'data', 'feature_names', 'filename', 'target', 'target_names']



Entrée [2]:
```
iris.feature_names
```

Out[2]:
```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

Entrée [3]:
```
iris.target_names
```

Out[3]:
```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

Entrée [4]:
```
df = pd.DataFrame(iris.data,columns=iris.feature_names)
df.head()
```

Out[4]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Entrée [5]:
```
df['target'] = iris.target
df['target'].head()
```

Out[5]:
```
0    0
1    0
2    0
3    0
4    0
Name: target, dtype: int64
```

Entrée [6]:
```
df[df.target==0].head()
```

Out[6]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Entrée [7]:
```
df[df.target==2].head()
```

Out[7]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
100	6.3	3.3	6.0	2.5	2

Entrée [8]:
```
df['flower_name'] =df.target.apply(lambda x: iris.target_names[x])
df.head()
```

Out[8]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	flower_name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

Entrée [9]:
```
df[97:102]
```

Out[9]:

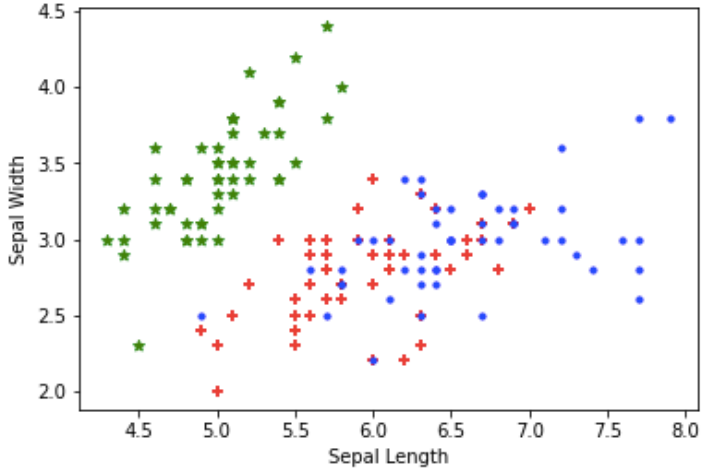
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	flower_name
97	6.2	2.9	4.3	1.3	1	versicolor
98	5.1	2.5	3.0	1.1	1	versicolor
99	5.7	2.8	4.1	1.3	1	versicolor
100	6.3	3.3	6.0	2.5	2	virginica
101	5.8	2.7	5.1	1.9	2	virginica

Entrée [10]:
```
df0 = df[:50]
df1 = df[50:100]
df2 = df[100:]
```

Entrée [11]:
```
import matplotlib.pyplot as plt
%matplotlib inline
```

Sepal length vs Sepal Width (Setosa vs Versicolor)

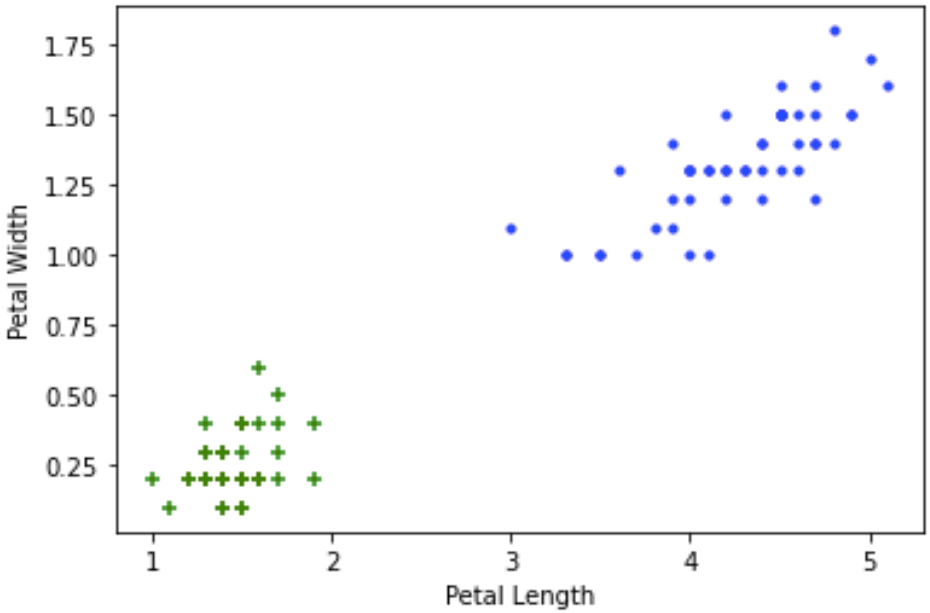
Entrée [12]:
```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.scatter(df0['sepal length (cm)'], df0['sepal width (cm)'],color="green",marker='*')
plt.scatter(df1['sepal length (cm)'], df1['sepal width (cm)'],color="red",marker='+')
plt.scatter(df2['sepal length (cm)'], df2['sepal width (cm)'],color="blue",marker='.')
```



Petal length vs Pepal Width (Setosa vs Versicolor)

Entrée [13]:
```
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
plt.scatter(df0['petal length (cm)'], df0['petal width (cm)'],color="green",marker='+')
plt.scatter(df1['petal length (cm)'], df1['petal width (cm)'],color="blue",marker='.')
```

Out[13]: `<matplotlib.collections.PathCollection at 0x7fd3035b3710>`



Train Using Support Vector Machine (SVM)

```
Entrée [14]: from sklearn.model_selection import train_test_split

Entrée [15]: X = df.drop(['target','flower_name'], axis='columns')
             y = df.target

Entrée [16]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

Entrée [17]: len(X_train)

  Out[17]: 120

Entrée [18]: len(X_test)

  Out[18]: 30

Entrée [19]: from sklearn.svm import SVC
             model = SVC(gamma='scale')

Entrée [20]: model.fit(X_train, y_train)

  Out[20]: SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,
               decision_function_shape='ovr', degree=3, gamma='scale', kernel='rbf',
               max_iter=-1, probability=False, random_state=None, shrinking=True,
               tol=0.001, verbose=False)
```

```
Entrée [21]: model.score(X_test, y_test)

  Out[21]: 0.9666666666666667

Entrée [22]: model.predict([[4.8,3.0,1.5,0.3]])

  Out[22]: array([0])
```

Tune parameters

1. Regularization (C)

Entrée [23]:
```
model_C = SVC(C=1, gamma='scale')
model_C.fit(X_train, y_train)
model_C.score(X_test, y_test)
```

Out[23]: 0.9666666666666667

Entrée [24]:
```
model_C = SVC(C=10)
model_C.fit(X_train, y_train)
model_C.score(X_test, y_test)
```

```
/Users/christian/anaconda3/lib/python3.6/s
amma will change from 'auto' to 'scale' in
to 'auto' or 'scale' to avoid this warning
  "avoid this warning.", FutureWarning)
```

Out[24]: 1.0

2. Gamma

Entrée [25]:
```
model_g = SVC(gamma=10)
model_g.fit(X_train, y_train)
model_g.score(X_test, y_test)
```

Out[25]: 1.0

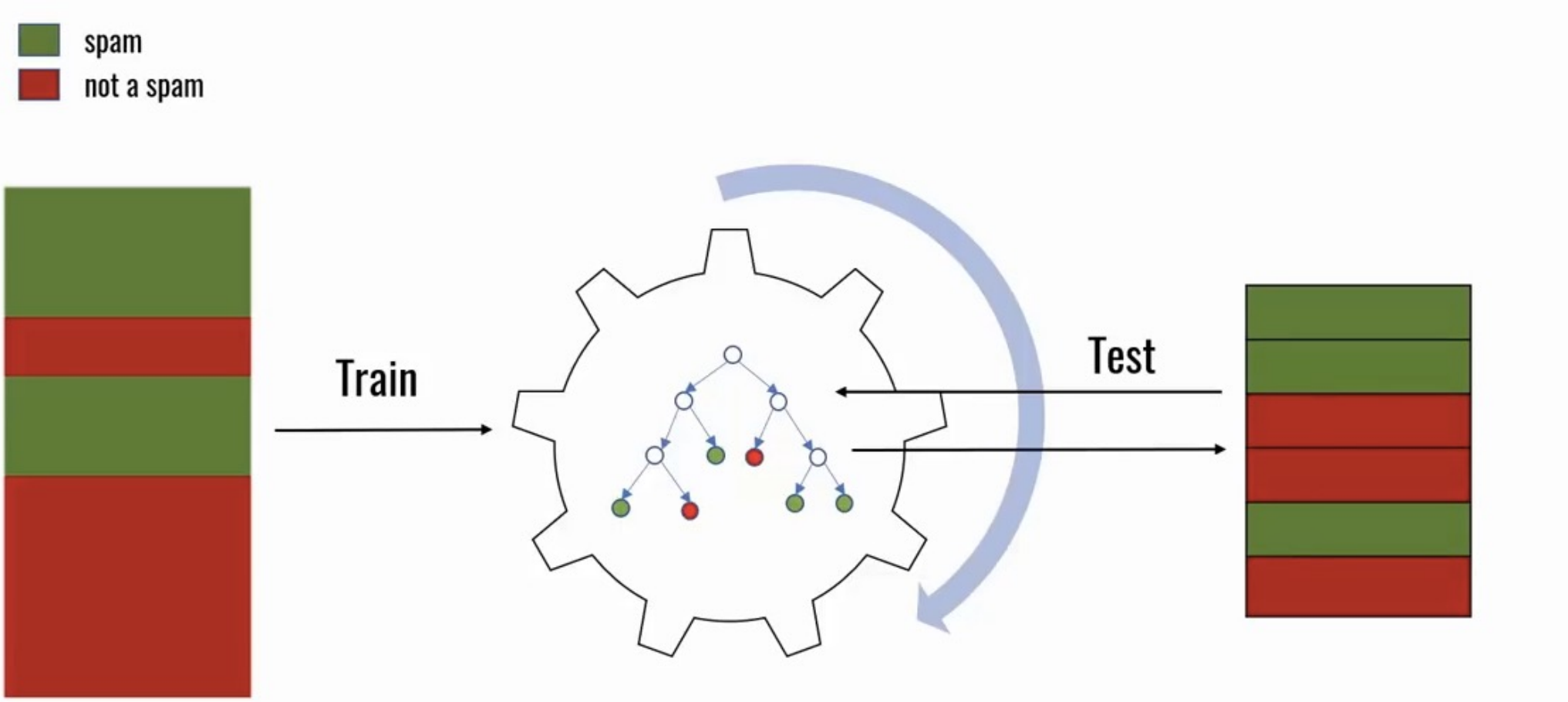
3. Kernel

Entrée [26]:
```
model_linear_kernal = SVC(kernel='linear')
model_linear_kernal.fit(X_train, y_train)
```

Out[26]:
```
SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,
  decision_function_shape='ovr', degree=3, gamma='auto_deprecated',
  kernel='linear', max_iter=-1, probability=False, random_state=None,
  shrinking=True, tol=0.001, verbose=False)
```

Entrée [27]:
```
model_linear_kernal.score(X_test, y_test)
```

Out[27]:
```
0.9666666666666667
```

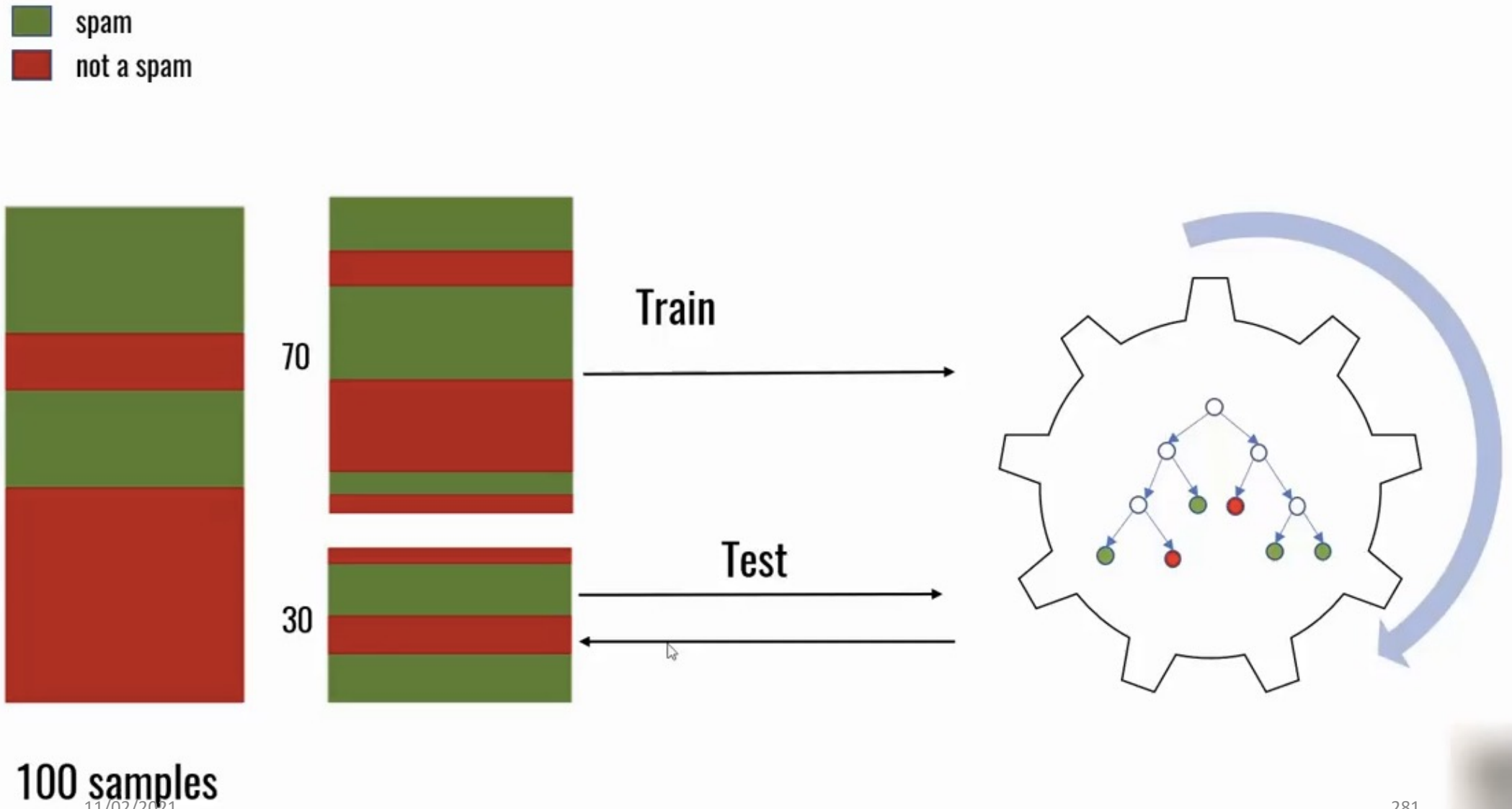

spam
not a spam
Train
Test

Option 1

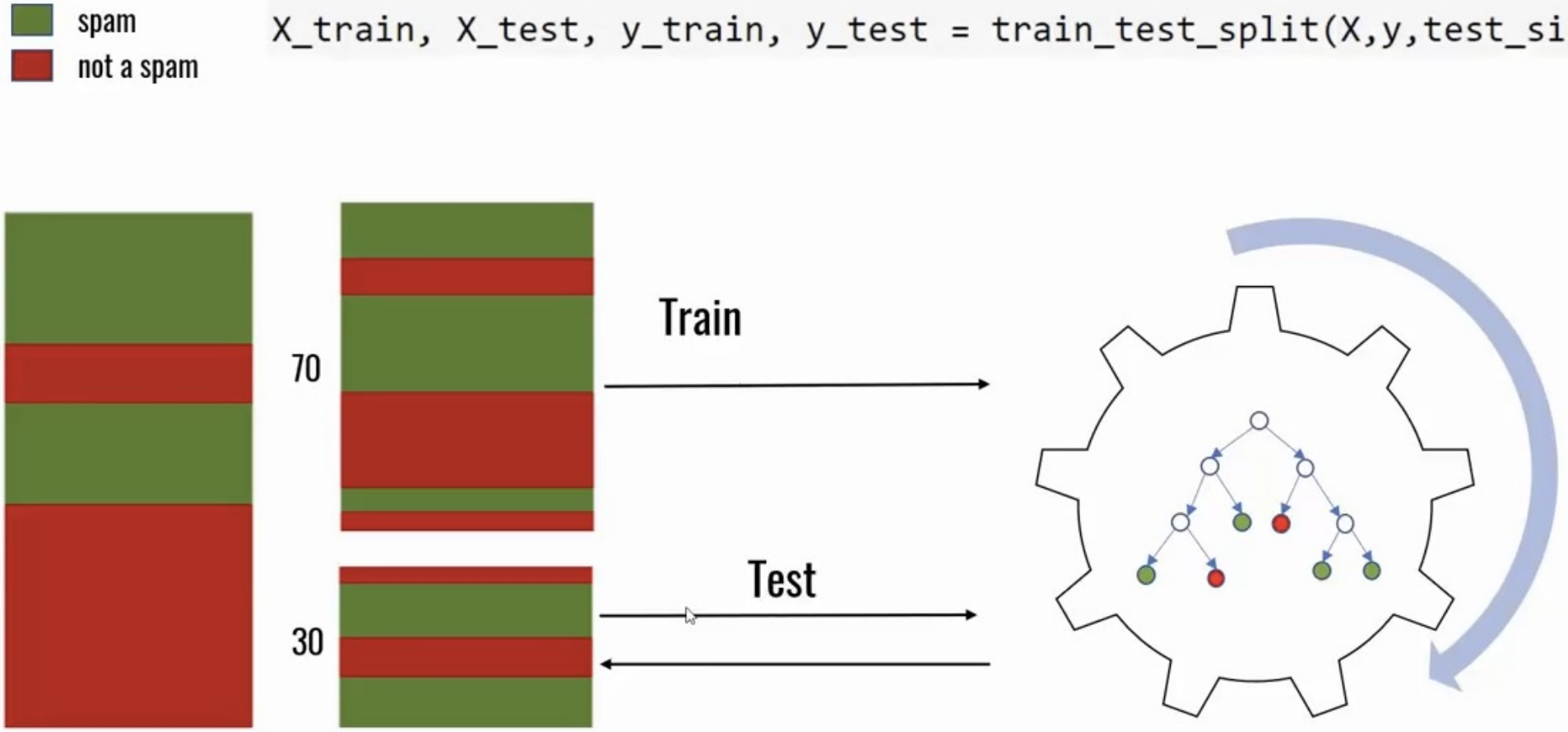
Use all available data for training and test on same dataset

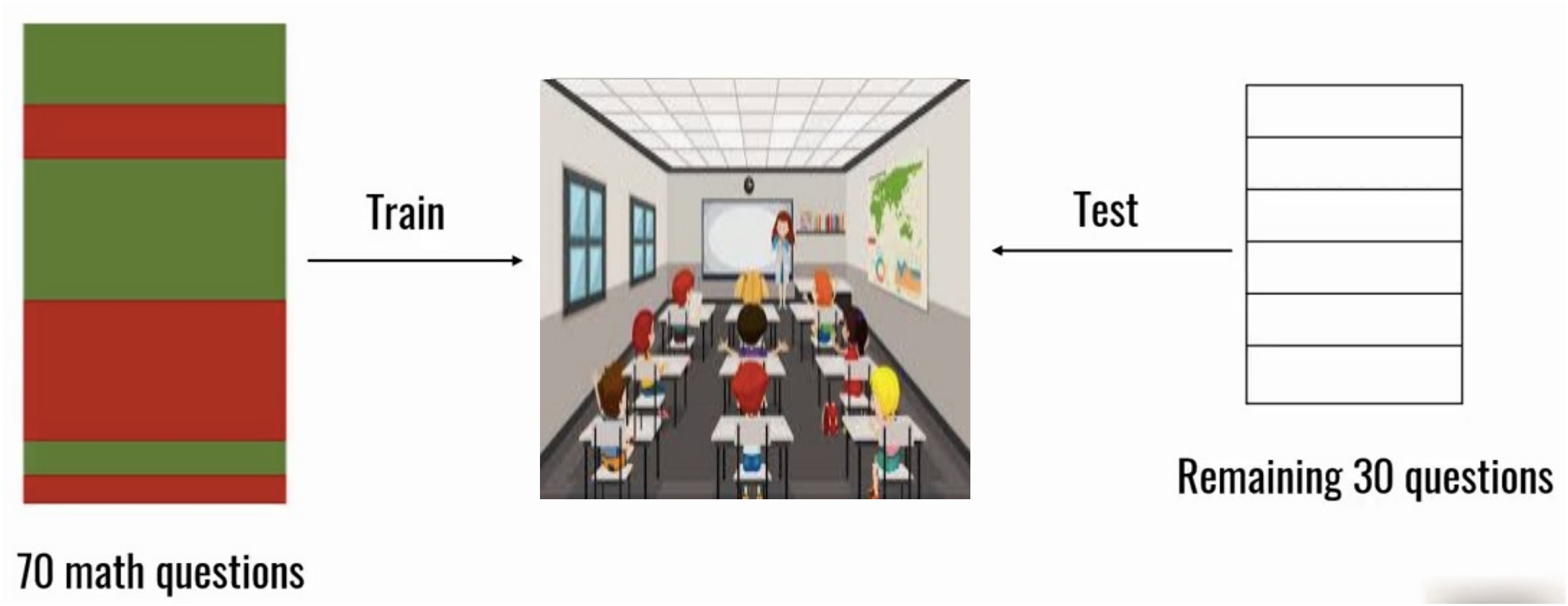
Option 2

Split available dataset into training and test sets


spam
not a spam
70
30
Train
Test
100 samples

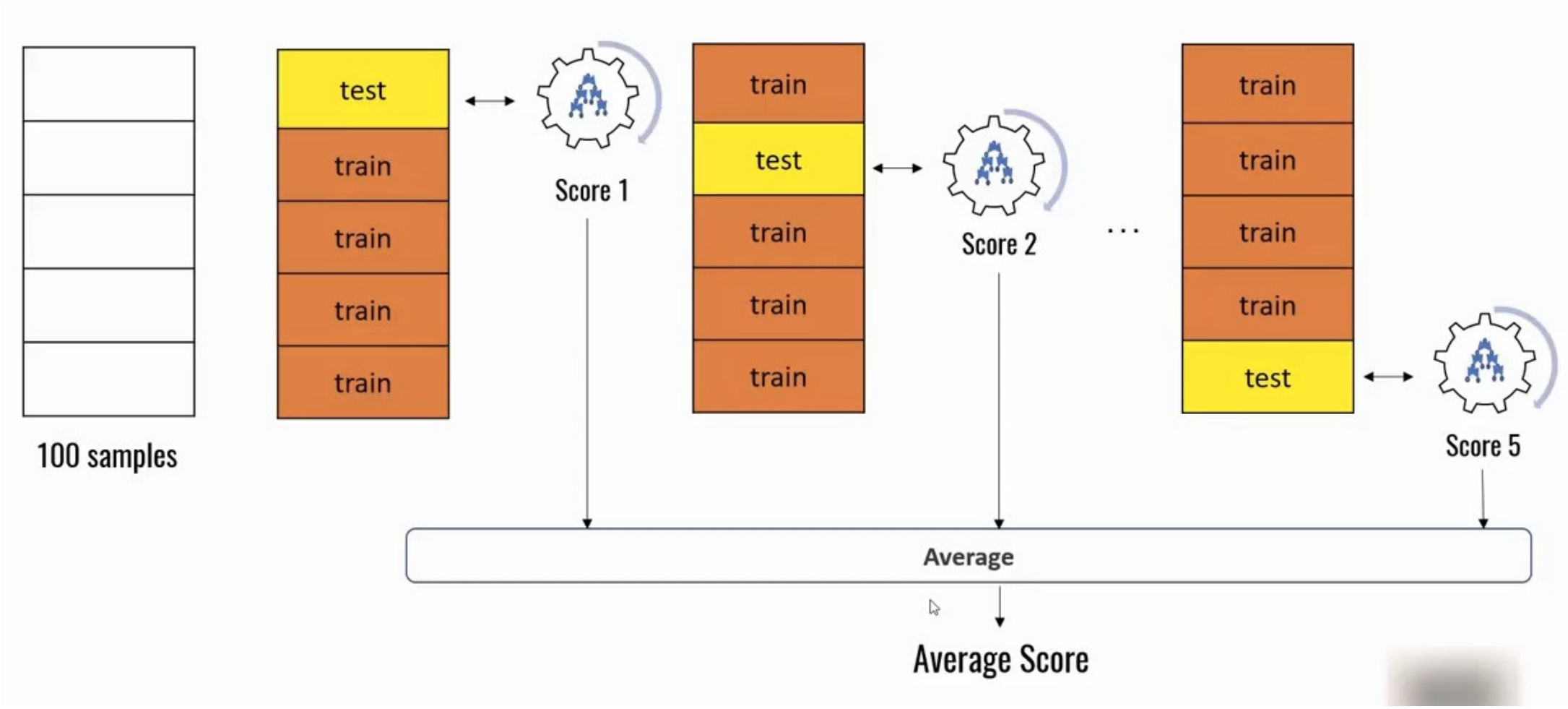
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3)
```



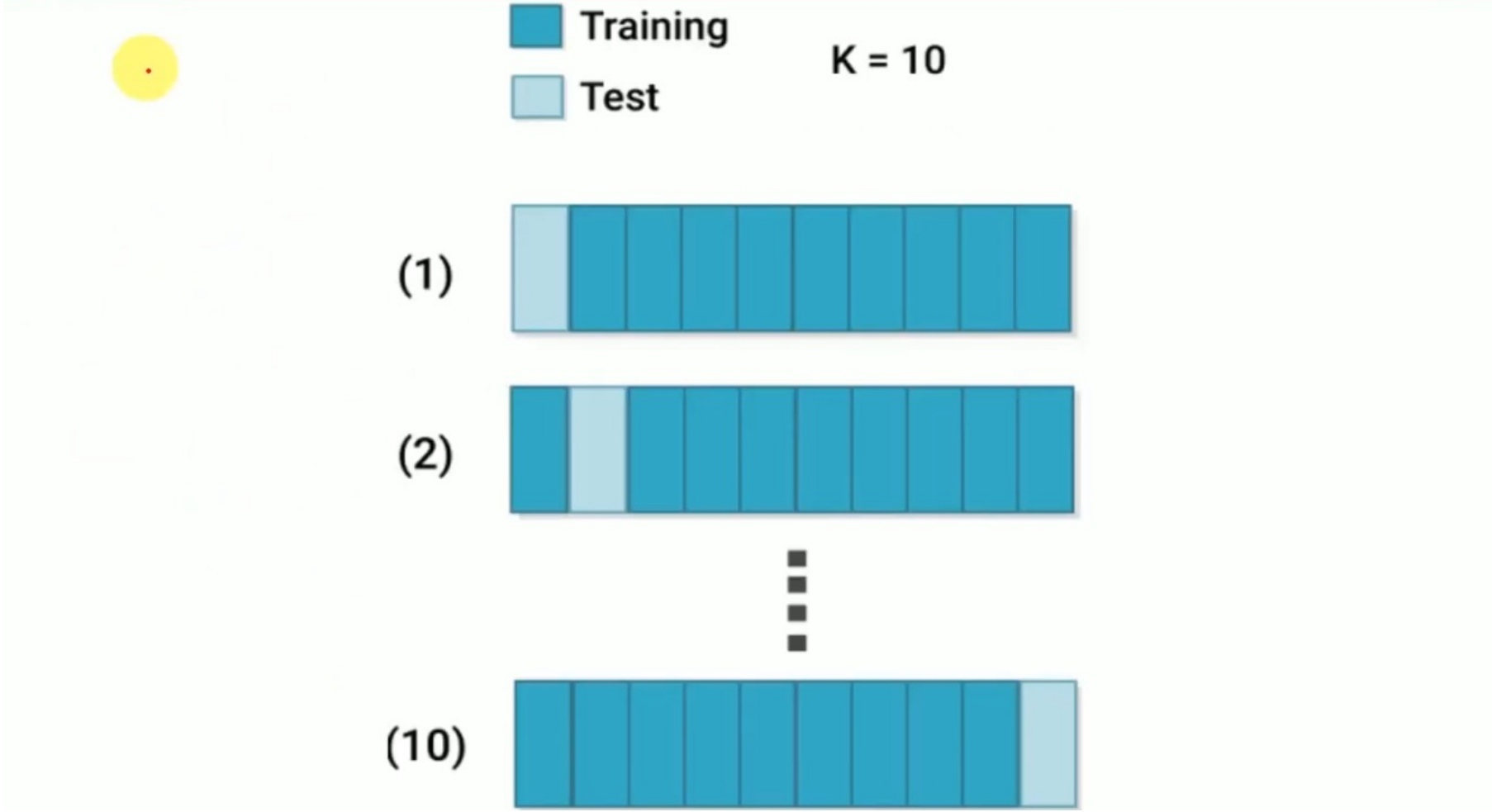

Train
Test
70 math questions
Remaining 30 questions

Option 3

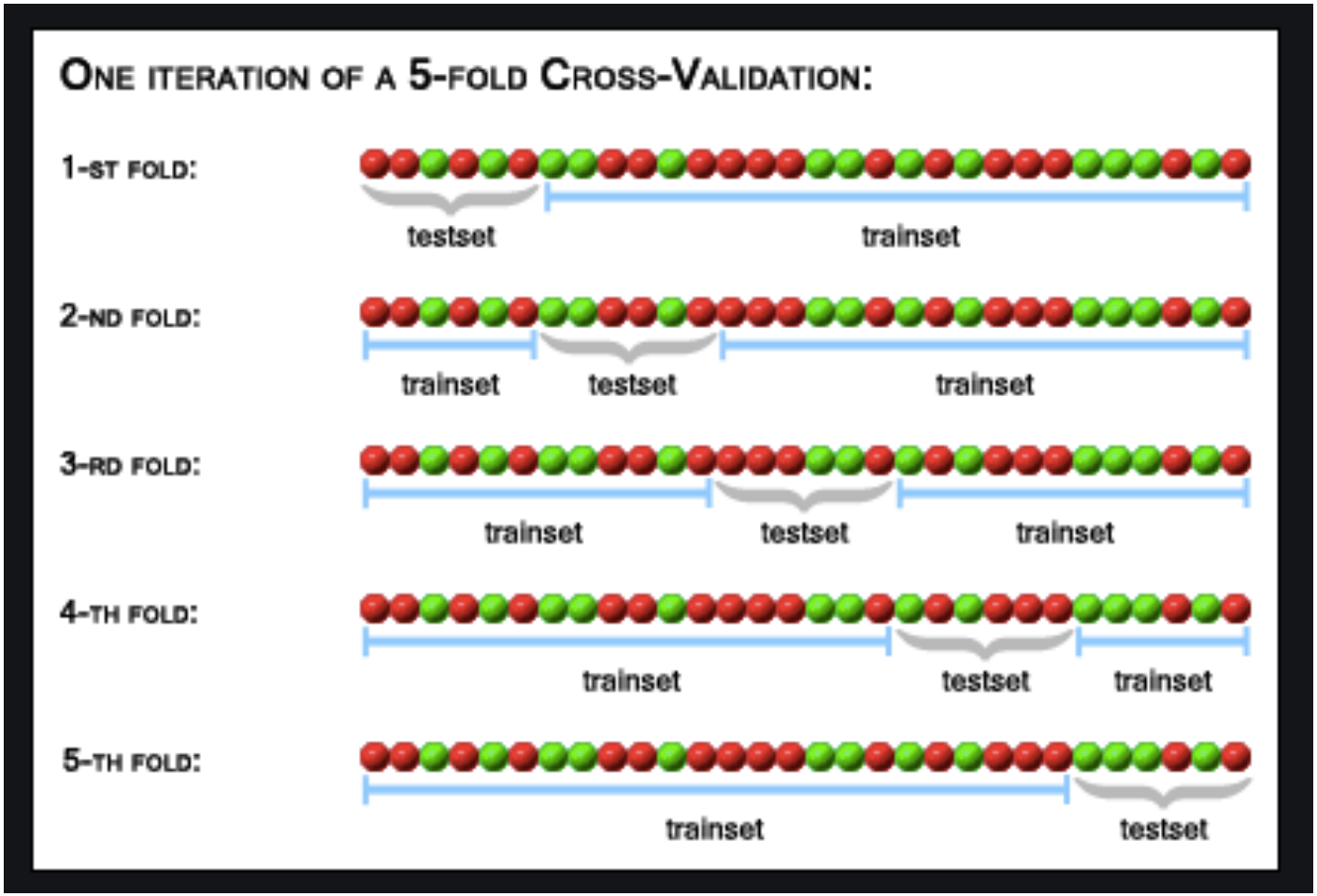
K fold cross validation


100 samples
test
train
train
train
train
Score 1
train
test
train
train
train
Score 2
...
train
train
train
train
test
Score 5
Average
Average Score

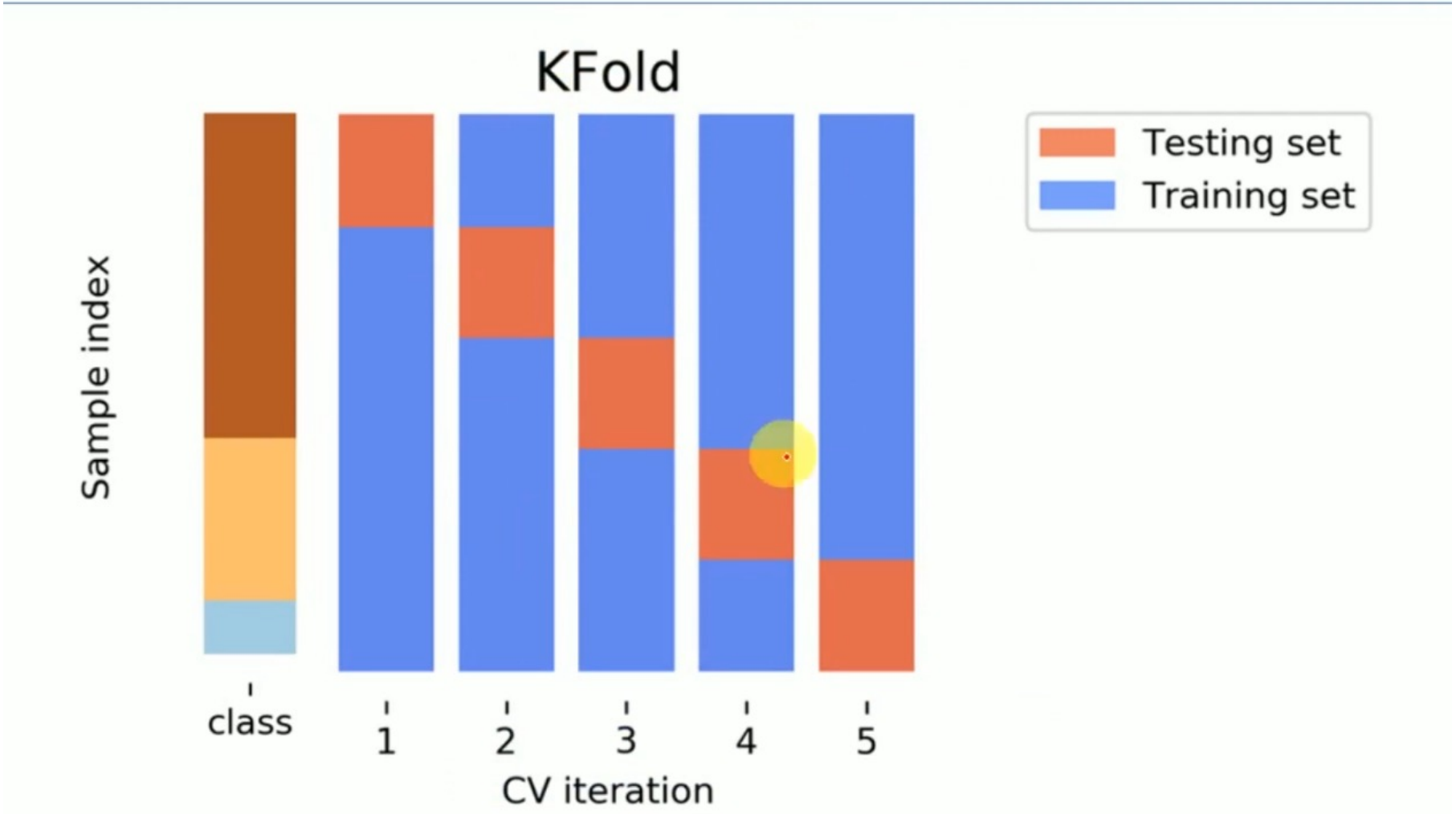
K-Fold Cross Validation in Machine Learning



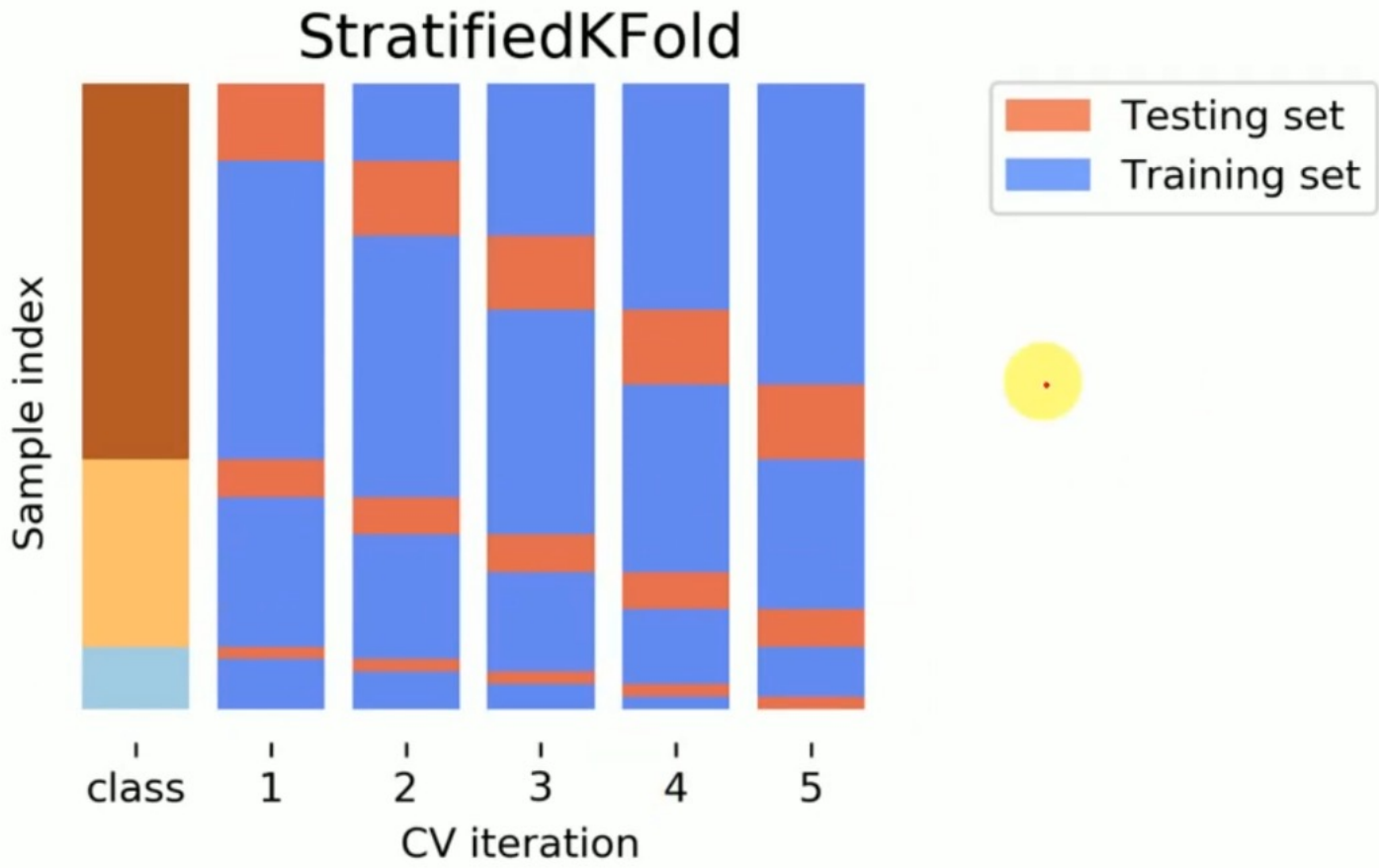
K-FOLD CROSS VALIDATION



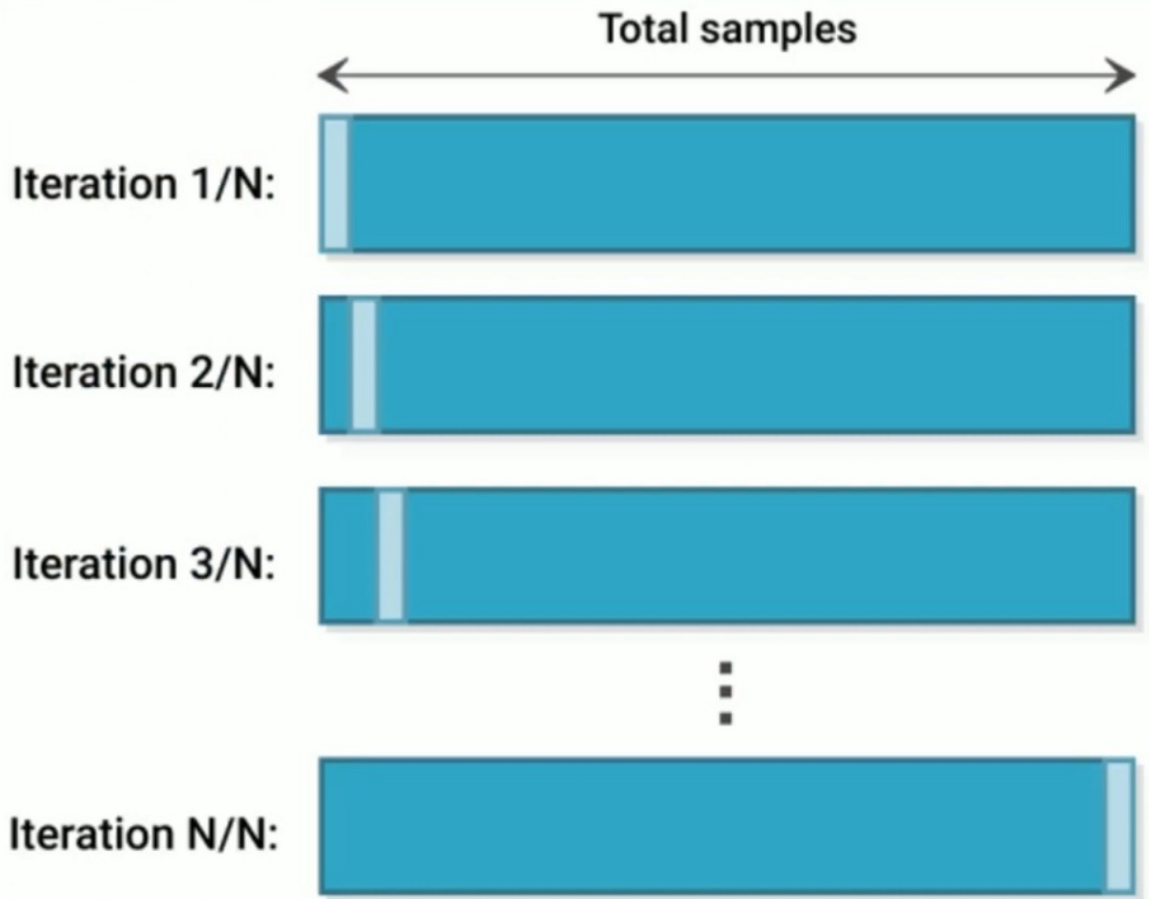
K-folds Cross Validation in Machine Learning



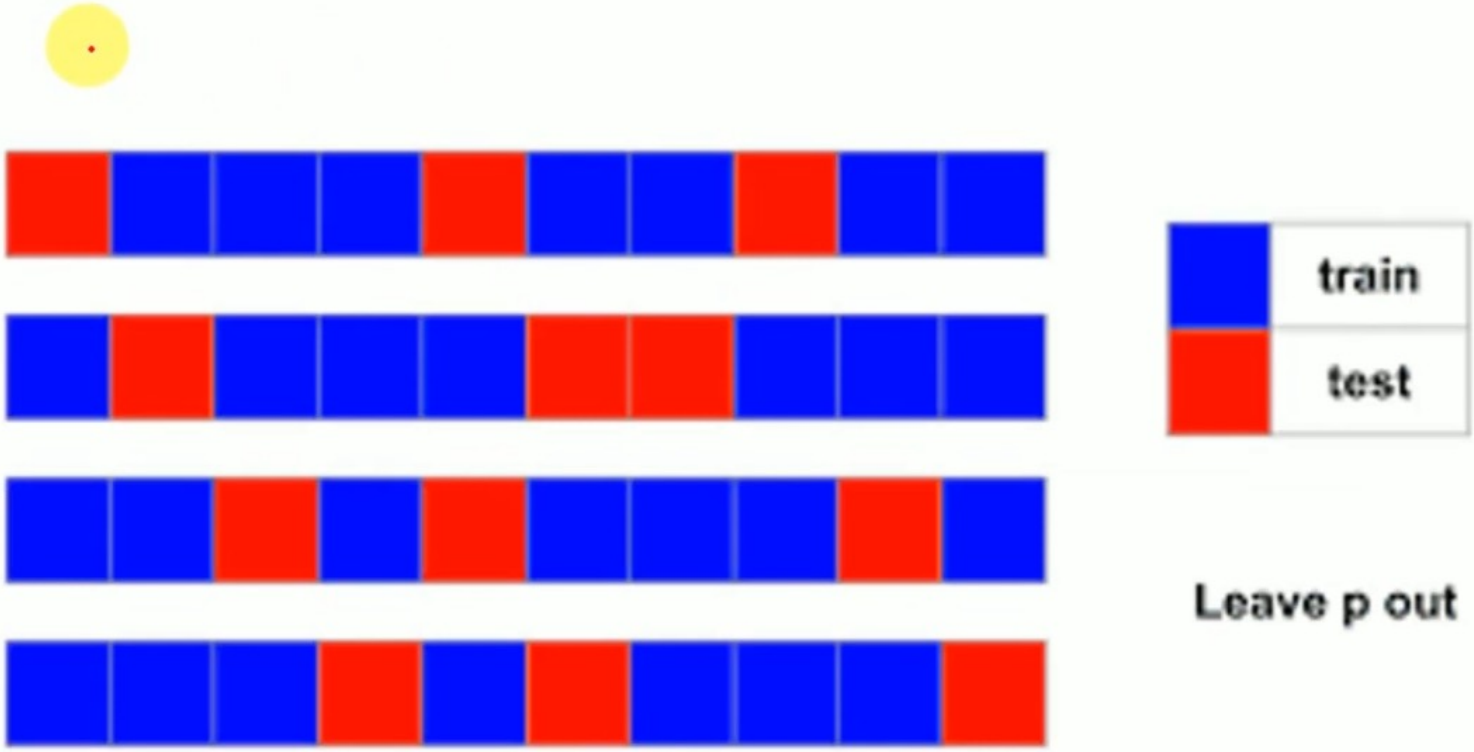
Stratified K-folds Cross Validation in Machine Learning



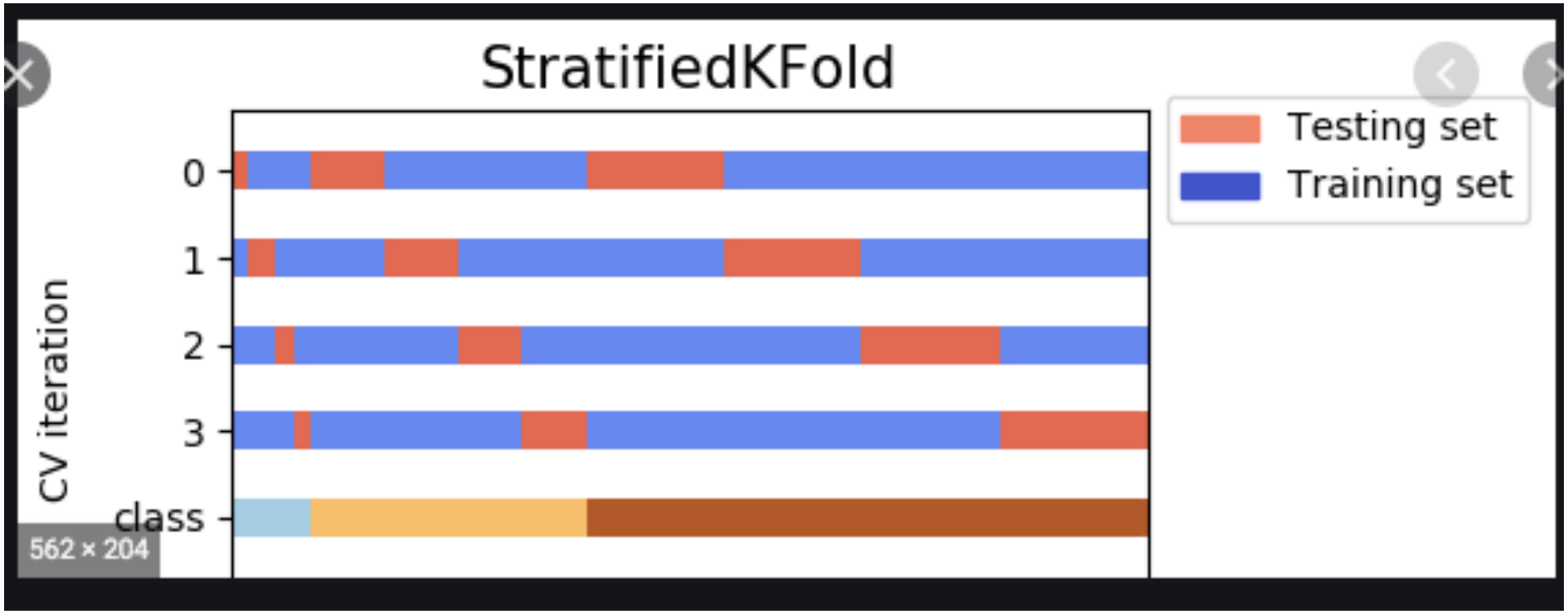
Leave-one-out Cross Validation in Machine Learning



Leave-P-out Cross Validation in Machine Learning



Spitting a data set with StratifiedKFOLD



LET'S CONSIDER TREE DIFFERNT MODELS

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
import numpy as np
from sklearn.datasets import load_digits

digits = load_digits()
```

LET'S RUN ALL SEVERAL TIMES

```
In [17]: from sklearn.model_selection import train_test_split
         X_train, X_test, y_train, y_test = train_test_split(digits.data,digits.target,test_size=0.3)

In [18]: lr = LogisticRegression()
         lr.fit(X_train, y_train)
         lr.score(X_test, y_test)

Out[18]: 0.953703703703703 72

In [19]: svm = SVC()
         svm.fit(X_train, y_train)
         svm.score(X_test, y_test)

Out[19]: 0.6277777777777777

In [20]: rf = RandomForestClassifier(n_estimators=40)
         rf.fit(X_train, y_train)
         rf.score(X_test, y_test)

Out[20]: 0.98148148148148151
```

In [21]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(digits.data,digits.target,test_size=0.3)
```

In [22]:
```
lr = LogisticRegression()
lr.fit(X_train, y_train)
lr.score(X_test, y_test)
```

Out[22]: 0.96296296296296291

In [23]:
```
svm = SVC()
svm.fit(X_train, y_train)
svm.score(X_test, y_test)
```

Out[23]: 0.3925925925925926

In [24]:
```
rf = RandomForestClassifier(n_estimators=40)
rf.fit(X_train, y_train)
rf.score(X_test, y_test)
```

Out[24]: 0.97037037037037033

Spitting a data set with K-FOLD

```
: from sklearn.model_selection import KFold
  kf = KFold(n_splits=3)
  kf
```

```
: KFold(n_splits=3, random_state=None, shuffle=False)
```

```
: for train_index, test_index in kf.split([1,2,3,4,5,6,7,8,9]):
      print(train_index, test_index)
```

```
[3 4 5 6 7 8] [0 1 2]
[0 1 2 6 7 8] [3 4 5]
[0 1 2 3 4 5] [6 7 8]
```

```
: def get_score(model, X_train, X_test, y_train, y_test):
      model.fit(X_train, y_train)
      return model.score(X_test, y_test)
```

```
: get_score(LogisticRegression(),X_train, X_test, y_train, y_test)
```

```
: 0.962962962962962 91
```

```
from sklearn.model_selection import StratifiedKFold
folds = StratifiedKFold(n_splits=3)
```

```
scores_l = []
scores_svm = []
scores_rf = []

for train_index,test_index in kf.split(digits.data):
    X_train, X_test, y_train, y_test = digits.data[train_index], digits.data[test_index], \
                                       digits.target[train_index], digits.target[test_index]
    print(get_score(LogisticRegression(), X_train, X_test, y_train, y_test))
    print(get_score(SVC(), X_train, X_test, y_train, y_test))
    print(get_score(RandomForestClassifier(), X_train, X_test, y_train, y_test))
```

```
scores_l = []
scores_svm = []
scores_rf = []

for train_index,test_index in kf.split(digits.data):
    X_train, X_test, y_train, y_test = digits.data[train_index], digits.data[test_index], \
                                       digits.target[train_index], digits.target[test_index]
    scores_l.append(get_score(LogisticRegression(), X_train, X_test, y_train, y_test))
    scores_svm.append(get_score(SVC(), X_train, X_test, y_train, y_test))
    scores_rf.append(get_score(RandomForestClassifier(), X_train, X_test, y_train, y_test))
```

```
scores_l
```

```
[0.89649415692821366, 0.95158597662771283, 0.91151919866444076]
```

```
scores_svm
```

```
[0.41068447412353926, 0.41569282136894825, 0.42737896494156929]
```

```
scores_rf
```

```
[0.90818030050083476, 0.91151919866444076, 0.91819699499165275]
```

IMPLEMENTATION OF CROSS VALIDATION SCORE

```
scores_l = []
scores_svm = []
scores_rf = []

for train_index,test_index in kf.split(digits.data):
    X_train, X_test, y_train, y_test = digits.data[train_index], digits.data[test_index], \
                                       digits.target[train_index], digits.target[test_index]
    scores_l.append(get_score(LogisticRegression(), X_train, X_test, y_train, y_test))
    scores_svm.append(get_score(SVC(), X_train, X_test, y_train, y_test))
    scores_rf.append(get_score(RandomForestClassifier(n_estimators=40), X_train, X_test, y_train, y_test))
```

```
scores_l
```

```
[0.89649415692821366, 0.95158597662771283, 0.91151919866444076]
```

```
scores_svm
```

```
[0.41068447412353926, 0.41569282136894825, 0.42737896494156929]
```

```
scores_rf
```

```
[0.92654424040066774, 0.95325542570951582, 0.92988313856427374]
```

```
from sklearn.model_selection import cross_val_score
```

```
cross_val_score(LogisticRegression(), digits.data, digits.target)
```

```
In [38]: scores_l
Out[38]: [0.89649415692821366, 0.95158597662771283, 0.91151919866444076]

In [39]: scores_svm
Out[39]: [0.41068447412353926, 0.41569282136894825, 0.42737896494156929]

In [40]: scores_rf
Out[40]: [0.92654424040066774, 0.95325542570951582, 0.92988313856427374]

In [41]: from sklearn.model_selection import cross_val_score

In [42]: cross_val_score(LogisticRegression(), digits.data, digits.target)
Out[42]: array([ 0.89534884,  0.94991653,  0.90939597])

In [43]: cross_val_score(SVC(), digits.data, digits.target)
Out[43]: array([ 0.39368771,  0.41068447,  0.45973154])

In [44]: cross_val_score(RandomForestClassifier(n_estimators=40), digits.data, digits.target)
Out[44]: array([ 0.92026578,  0.94156928,  0.91946309])
```

PARAMETER TUNNING WITH CROSS VALIDATION

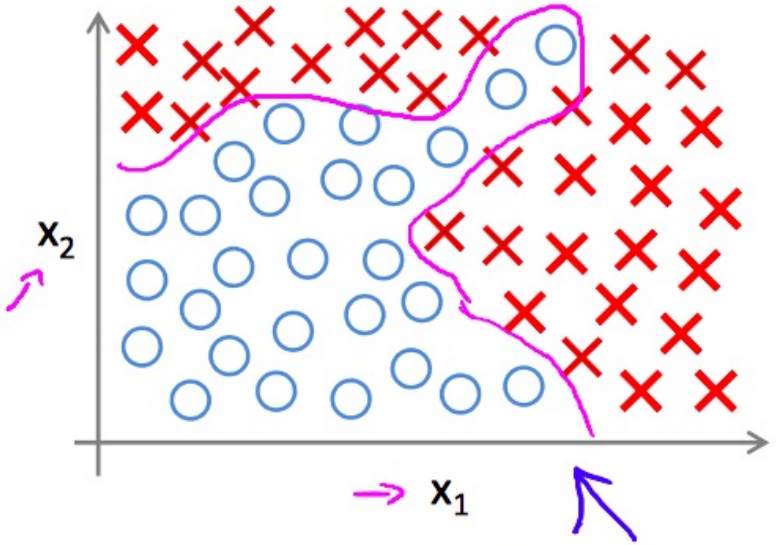
```
In [44]: cross_val_score(RandomForestClassifier(n_estimators=40), digits.data, digits.target)
Out[44]: array([ 0.92026578,  0.94156928,  0.91946309])

In [45]: cross_val_score(RandomForestClassifier(n_estimators=5), digits.data, digits.target)
Out[45]: array([ 0.86212625,  0.85308848,  0.86241611])

In [46]: cross_val_score(RandomForestClassifier(n_estimators=15), digits.data, digits.target)
Out[46]: array([ 0.90365449,  0.92821369,  0.91946309])

In [48]: cross_val_score(RandomForestClassifier(n_estimators=60), digits.data, digits.target)
Out[48]: array([ 0.94352159,  0.94657763,  0.93456376])
```

Neural Networks: Representation

Non-linear hypotheses

Machine Learning

Non-linear Classification



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

$x_1^2, x_1x_2, x_1x_3, x_1x_4 \dots x_1x_{100}$
$x_2^2, x_2x_3 \dots$
≈ 5000 feature $O(n^2)$

$x_1 =$ size
$x_2 =$ # bedrooms
$x_3 =$ # floors
$x_4 =$ age
$\dots$
x_{100}

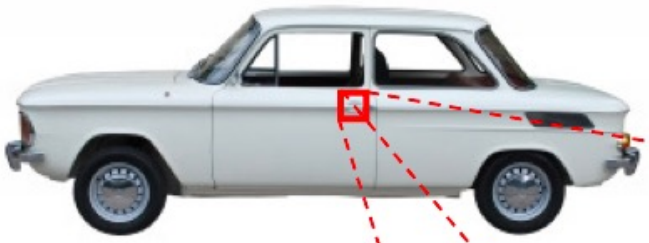
$n = 100$

$x_1^2, x_2^2, x_3^2, \dots, x_{100}^2$ $\approx \frac{n^2}{2}$

$x_1x_2x_3, x_1^2x_2, x_{10}x_{11}x_{17}, \dots$
$O(n^3)$ 170,000

What is this?

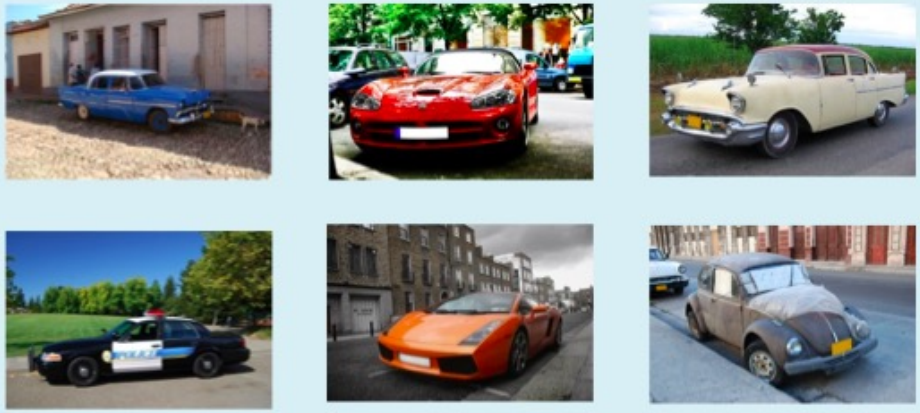
You see this:



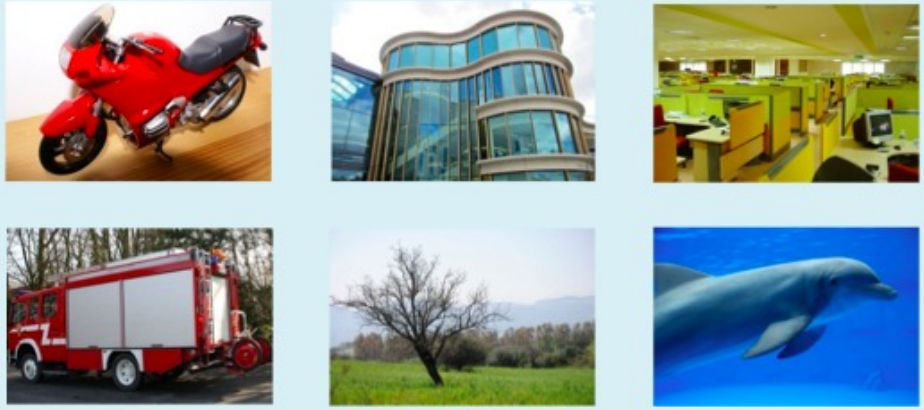
But the camera sees this:

194	210	201	212	199	213	215	195	178	158	182	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	188	176	165	152	140	170	106	78	88
87	103	115	154	143	142	149	153	173	101	57	57
102	112	106	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	68	99	63	45	60	82	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50

Computer Vision: Car detection



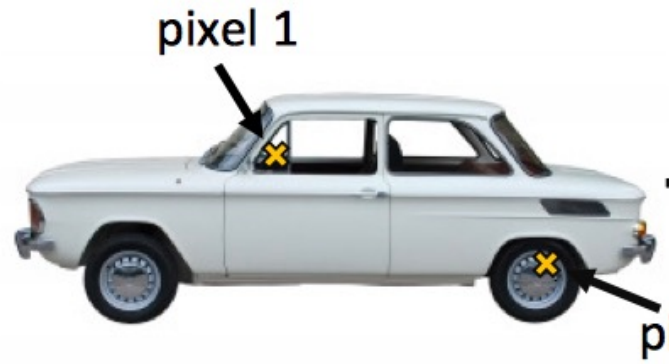
Cars



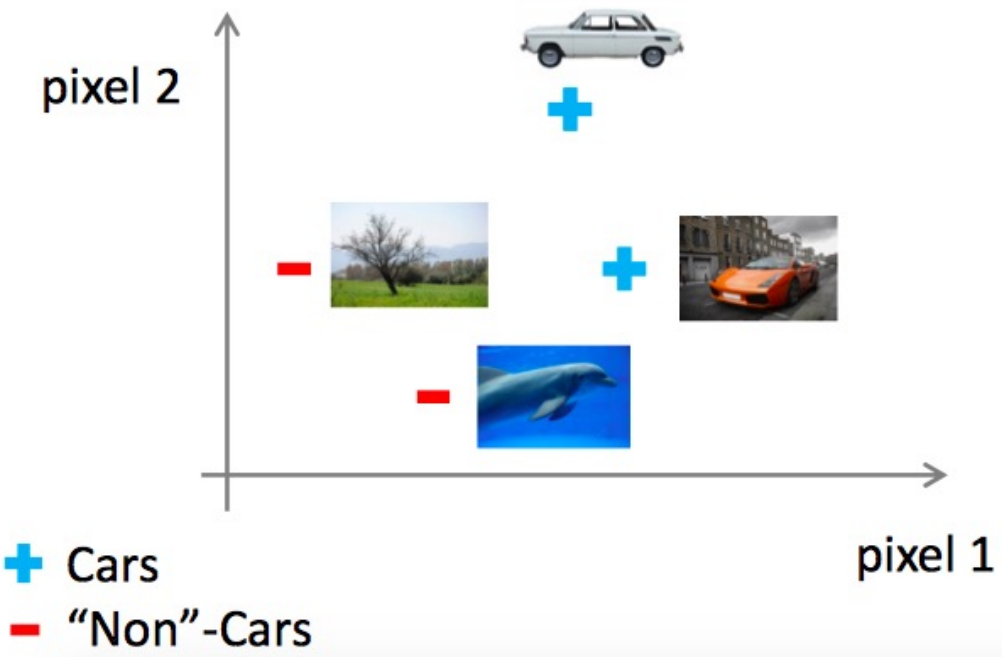
Not a car

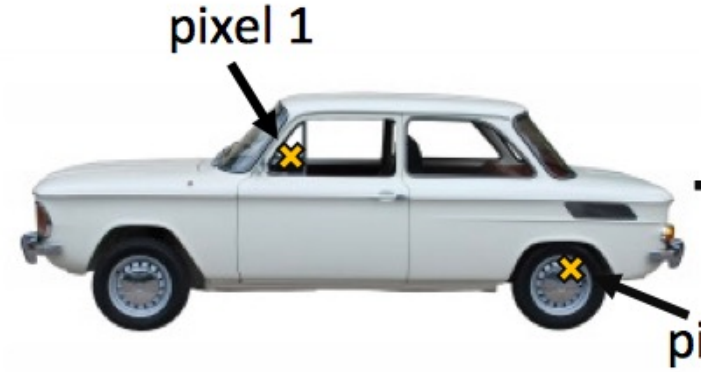
Testing:

What is this?

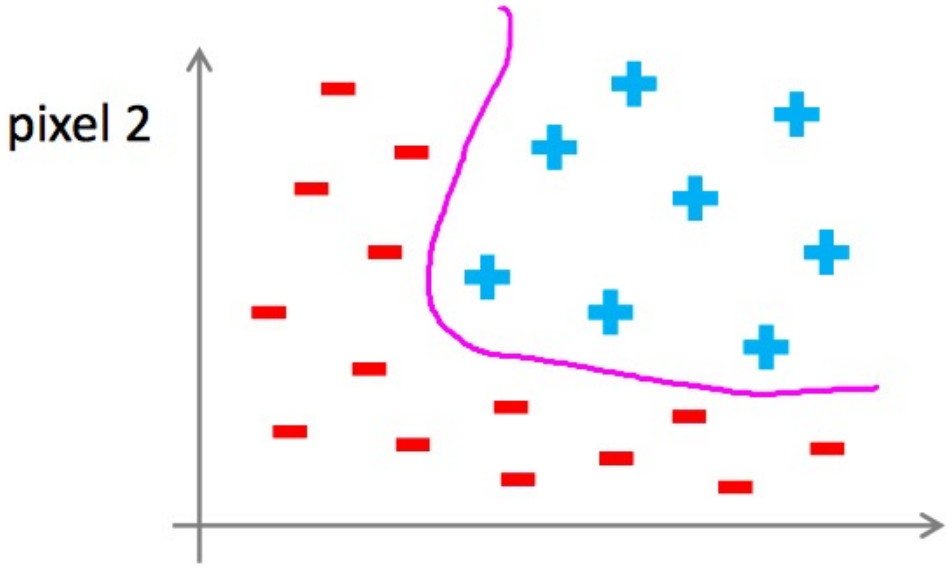


Learning Algorithm





Learning Algorithm



+ Cars
- "Non"-Cars

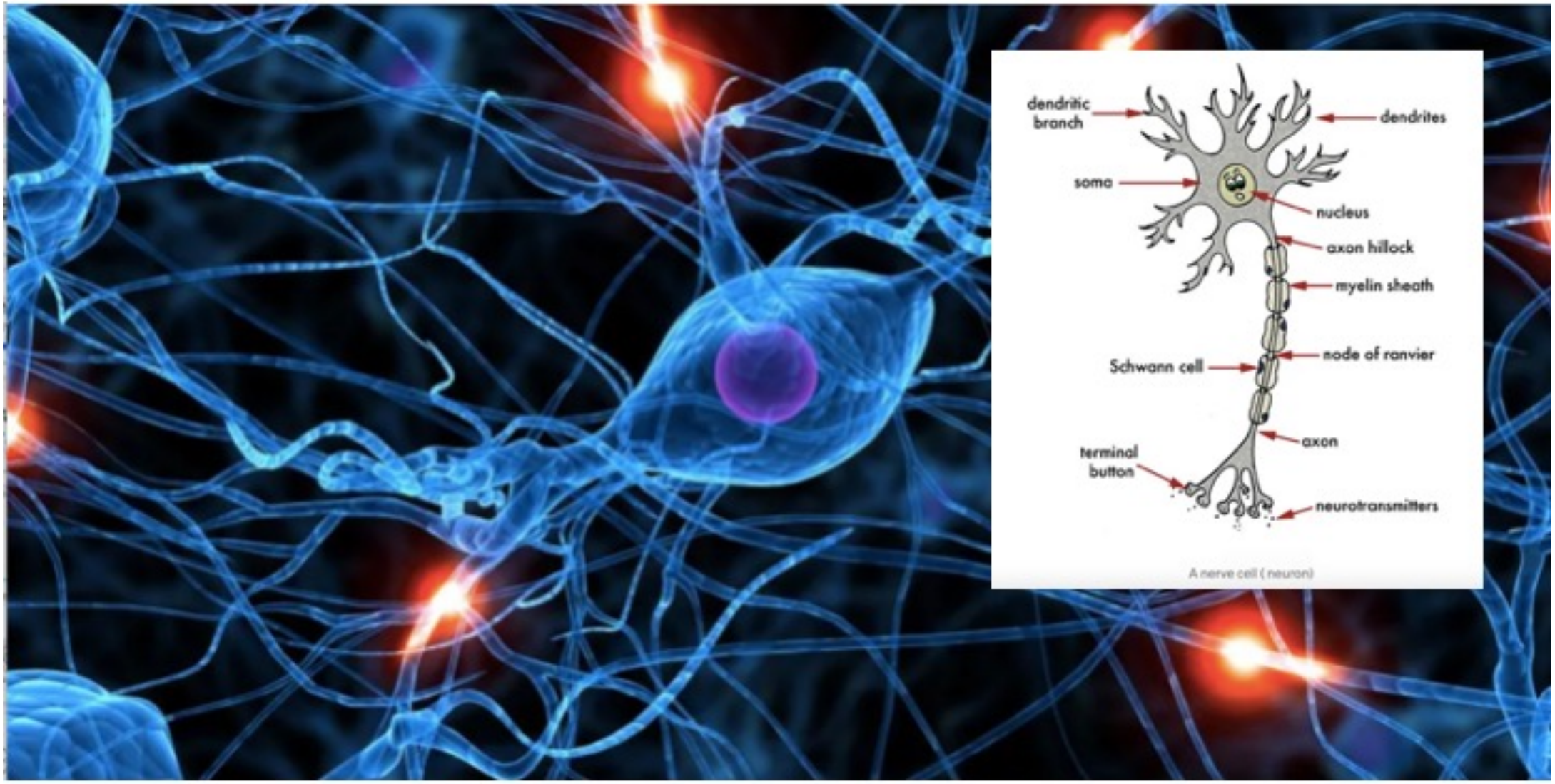
50 x 50 pixel images → 2500 pixels

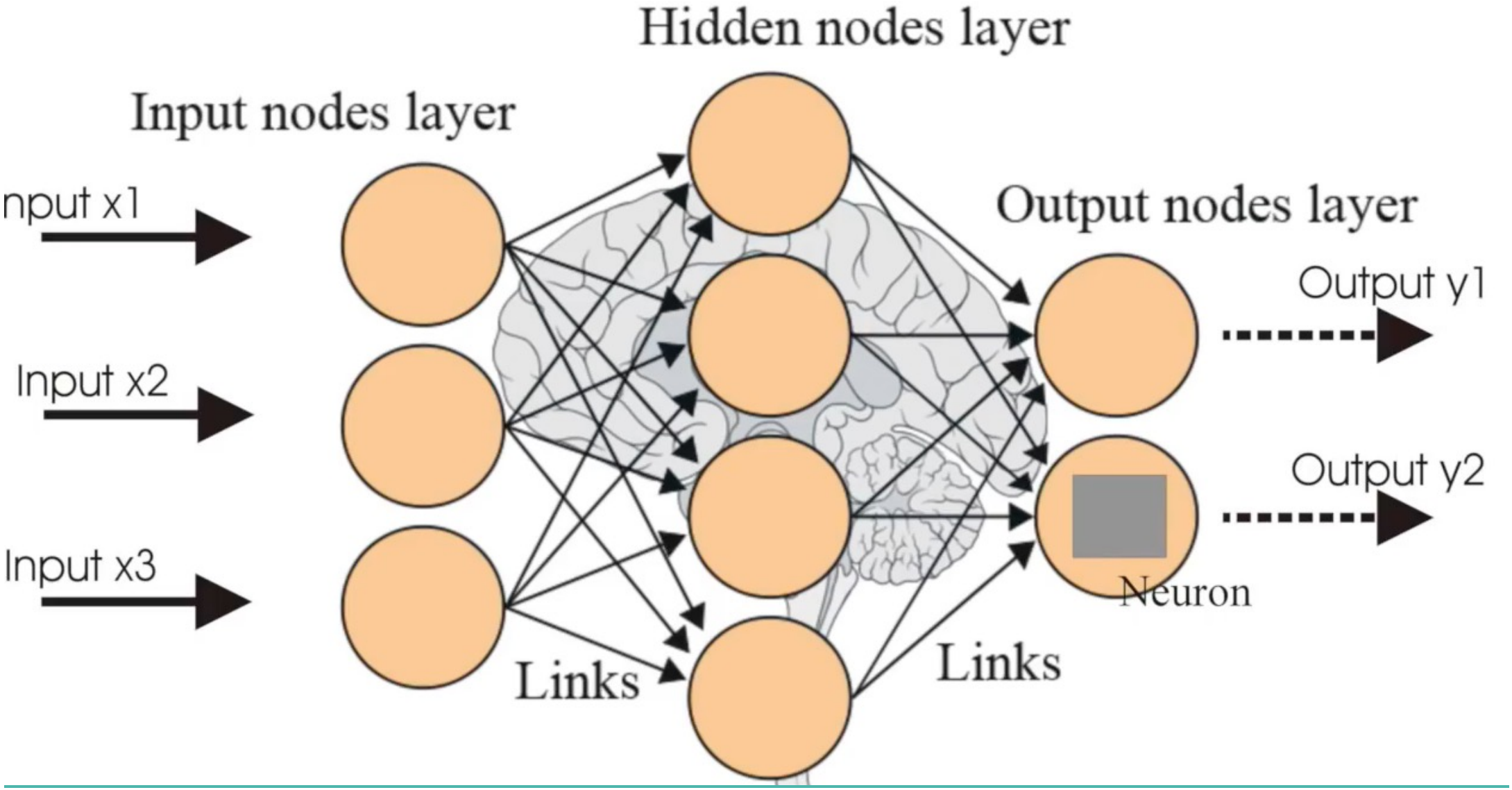
$n = 2500$ (7500 if RGB)

$$x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500 intensity} \end{bmatrix}$$

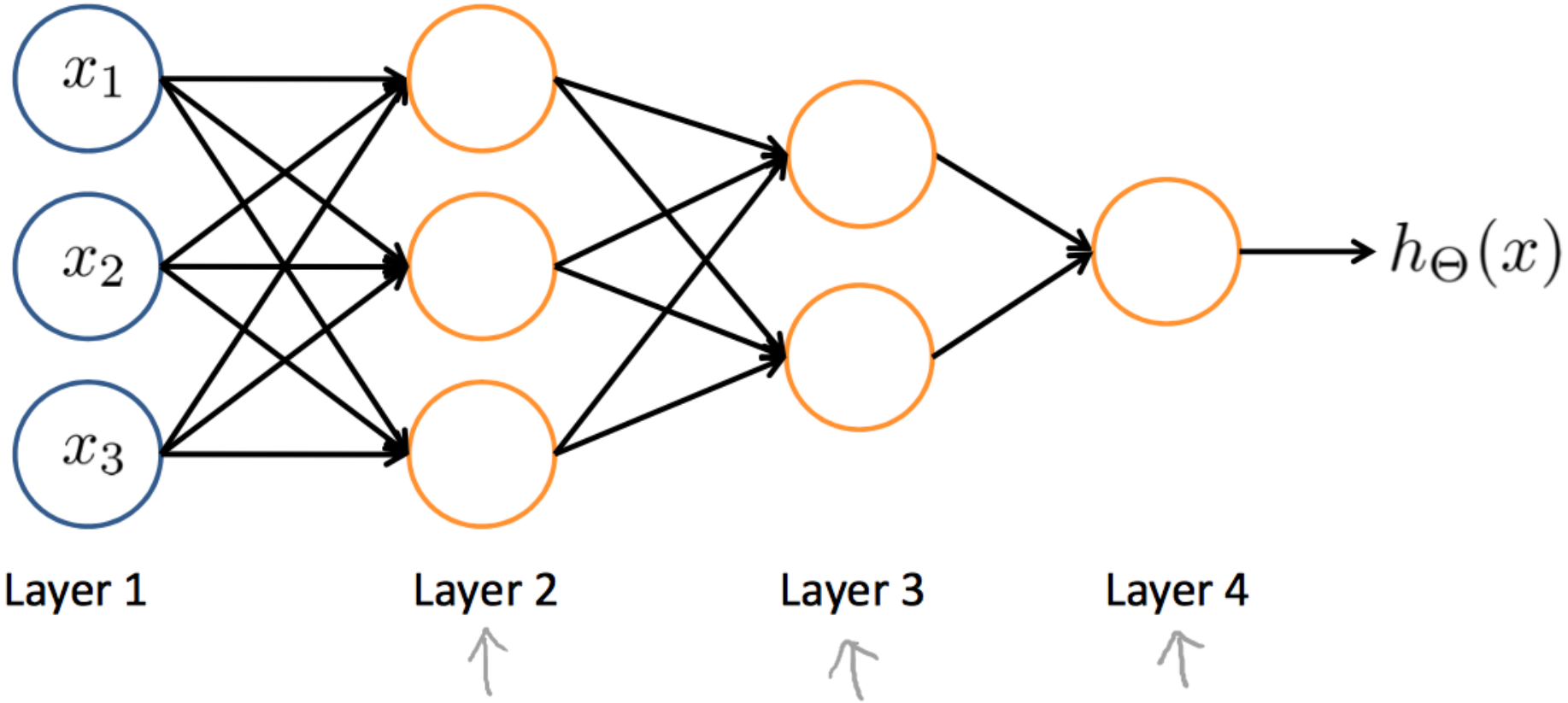
0-255

Quadratic features ($x_i \times x_j$): ≈3 million features


dendritic branch
dendrites
soma
nucleus
axon hillock
myelin sheath
Schwann cell
node of ranvier
axon
terminal button
neurotransmitters
A nerve cell (neuron)


Hidden nodes layer
Input nodes layer
nput x1
Input x2
Input x3
Output nodes layer
Output y1
Output y2
Neuron
Links
Links

Neural Network intuition



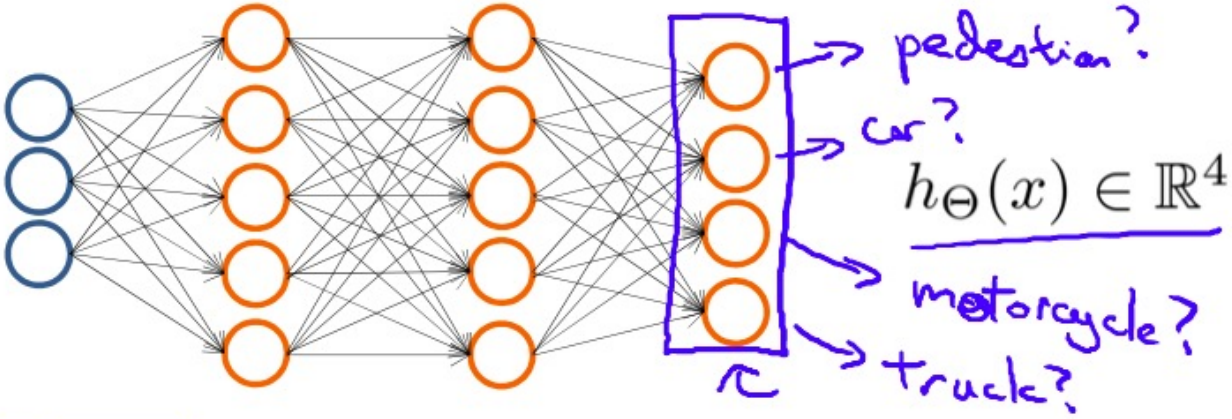
Multiple output units: One-vs-all.

Pedestrian

Car

Motorcycle

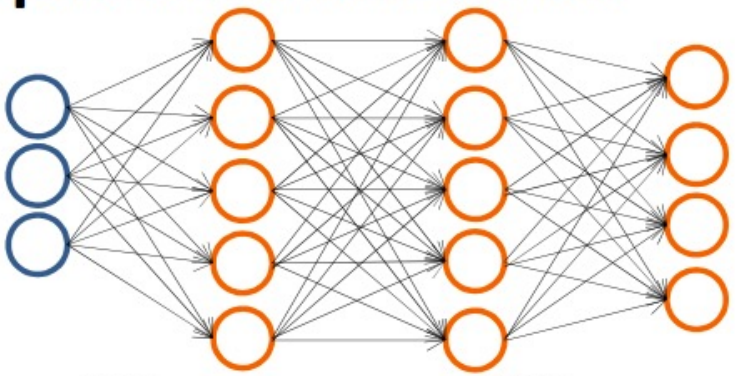
Truck



Want $h_\Theta(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, etc.

when pedestrian when car when motorcycle

Multiple output units: One-vs-all.



$h_\Theta(x) \in \mathbb{R}^4$

Want $h_\Theta(x) \approx \begin{bmatrix}1\\0\\0\\0\end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix}0\\1\\0\\0\end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix}0\\0\\1\\0\end{bmatrix}$, etc.

when pedestrian when car when motorcycle

Training set: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})$

$y^{(i)}$ one of $\begin{bmatrix}1\\0\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\0\\1\\0\end{bmatrix}, \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$

pedestrian car motorcycle truck

Previously: $y \in \{1, 2, 3, 4\}$

$(x^{(i)}, y^{(i)})$

$h_\Theta(x^{(i)}) \approx y^{(i)}$

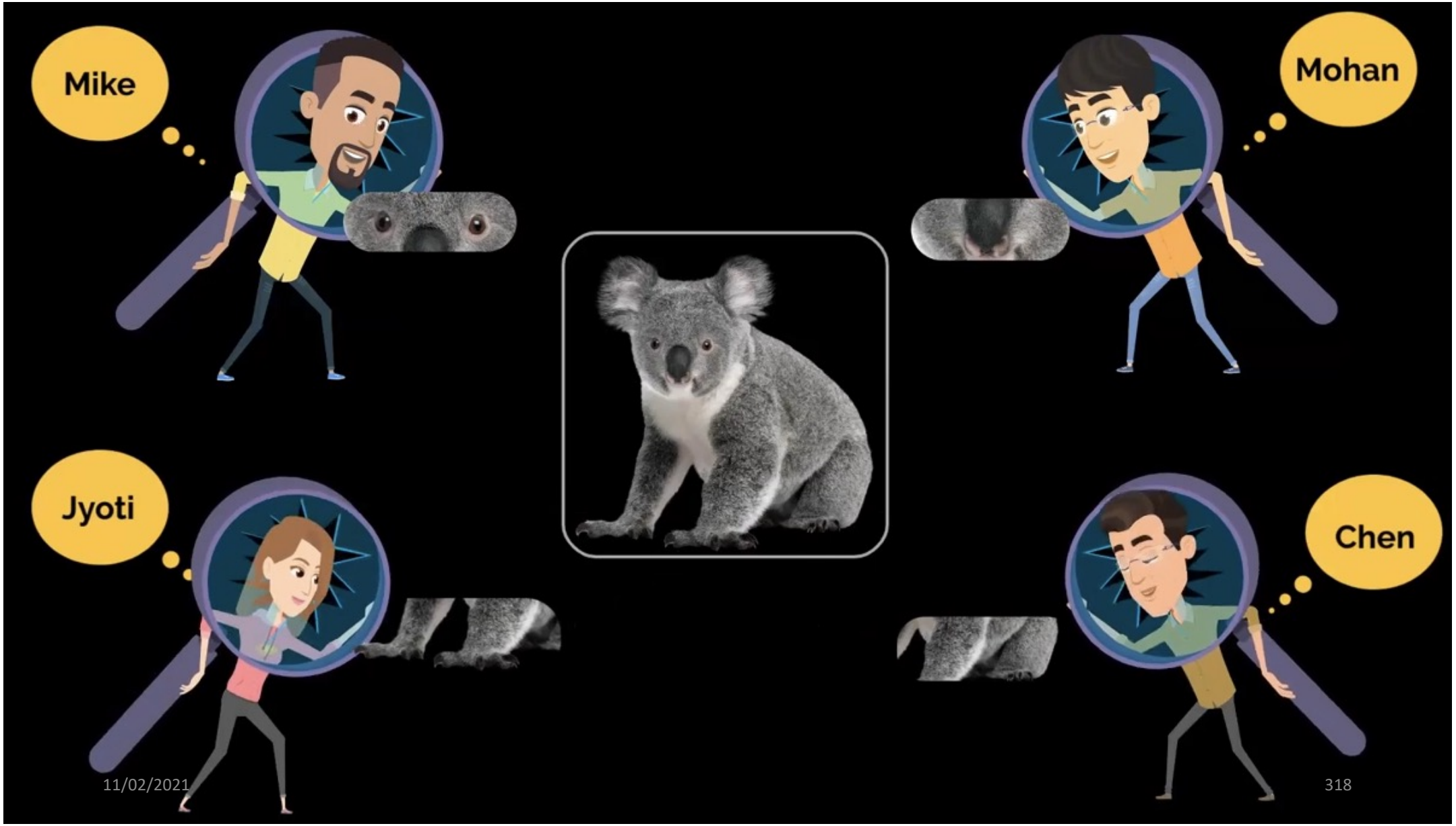
$\mathbb{R}^4$

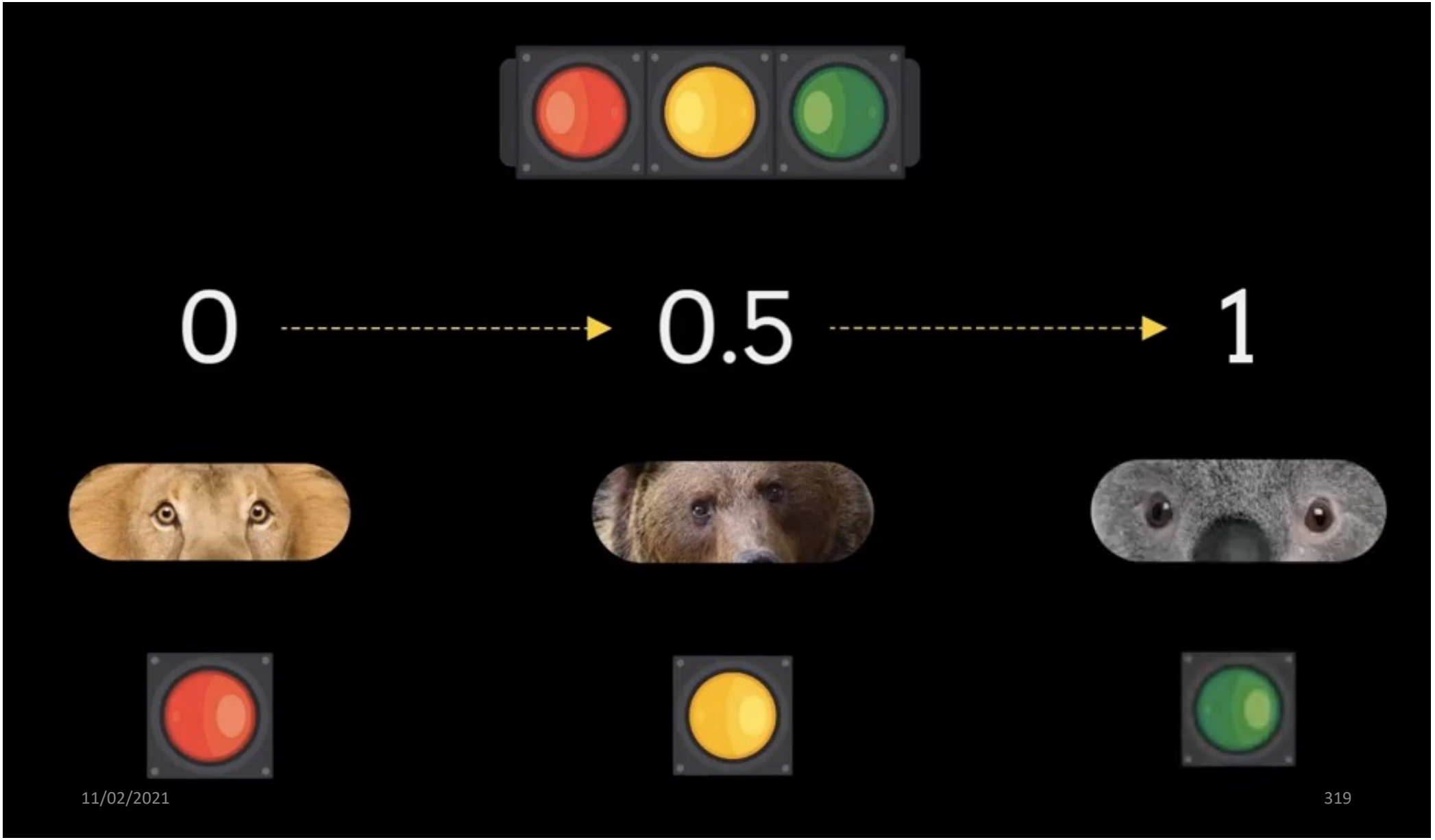
Andrew Ng

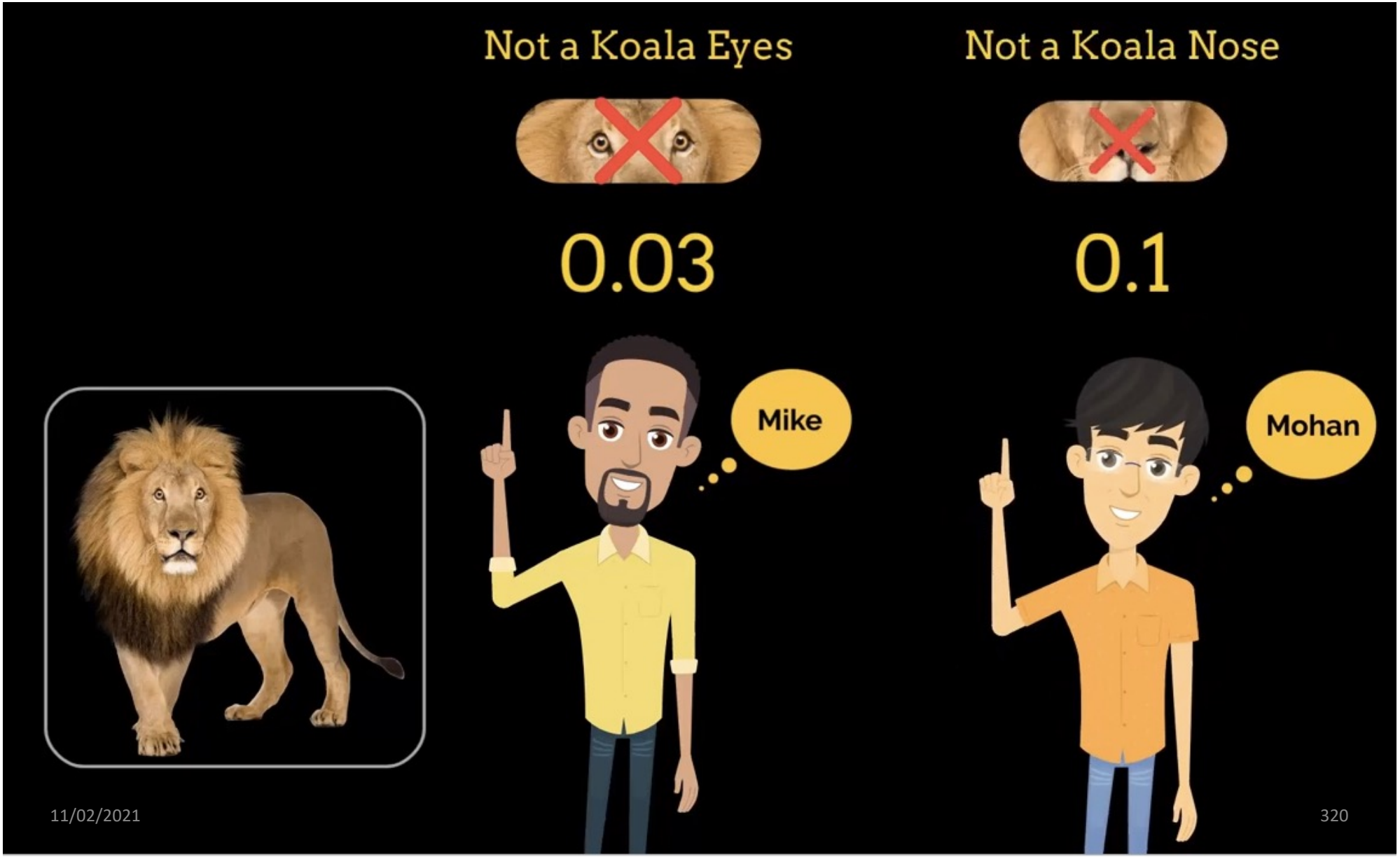
Multi Layer Perceptron
&
Chain Rule

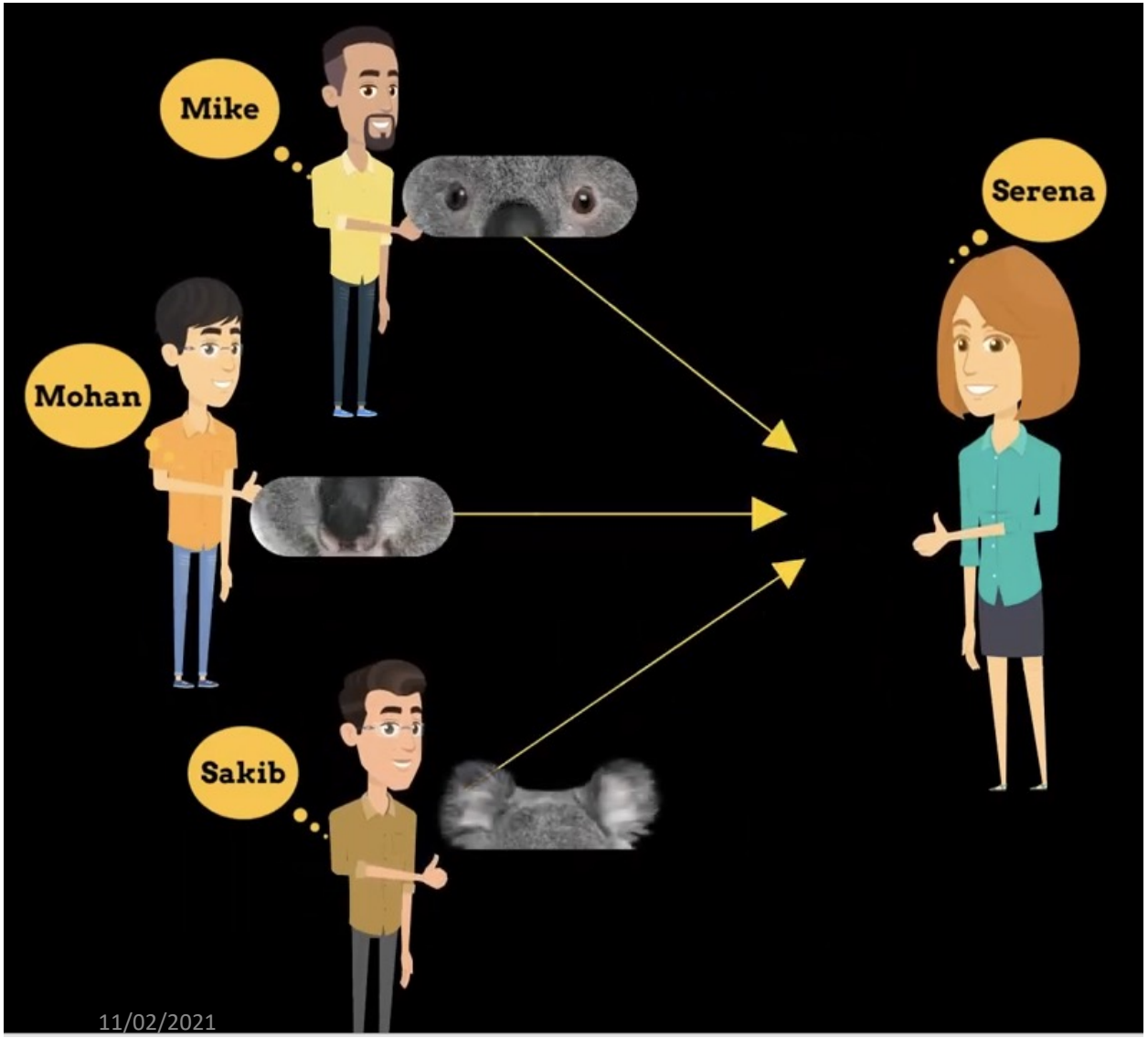
3brown 1Blue

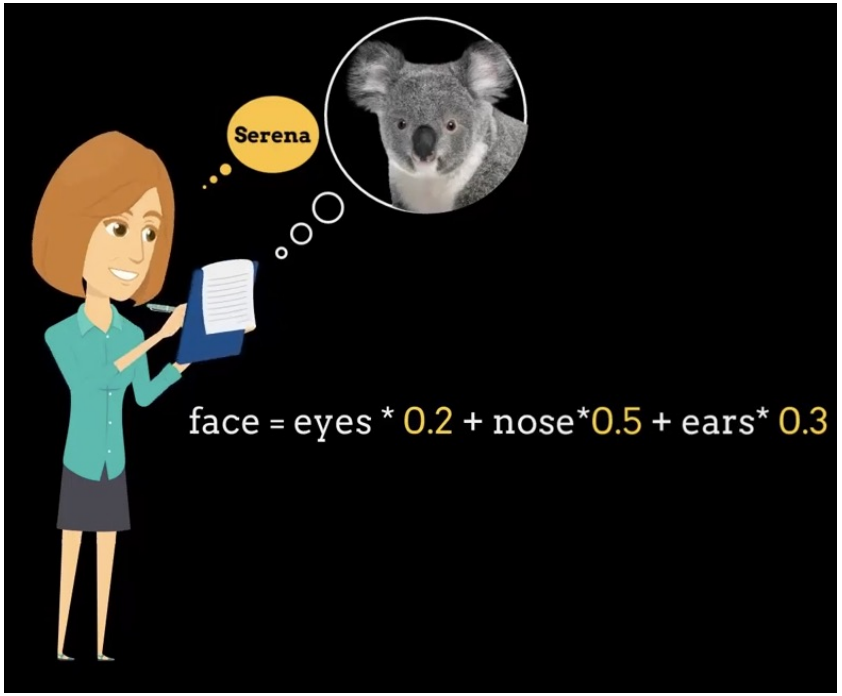
Mike
Mohan
Nidhi
Is this Koala?
Yes
No

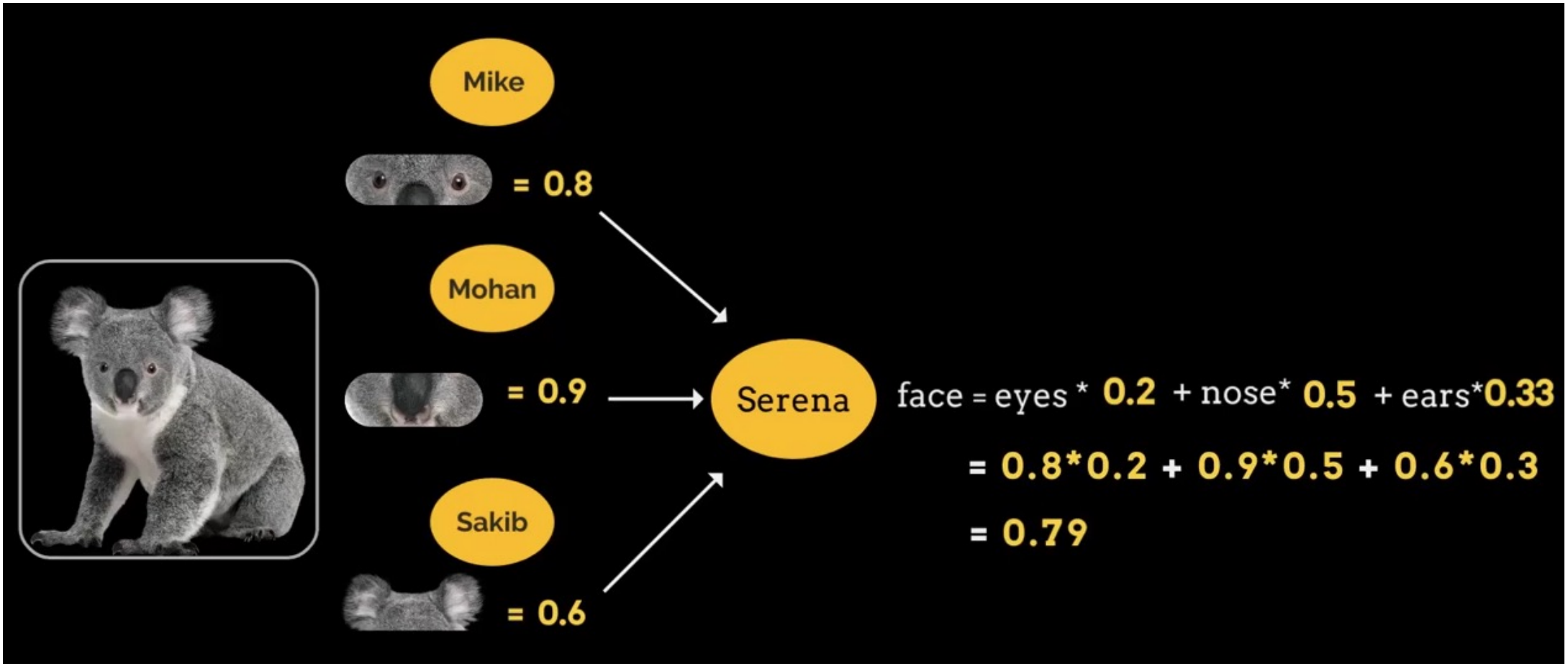

Mike
Mohan
Jyoti
Chen

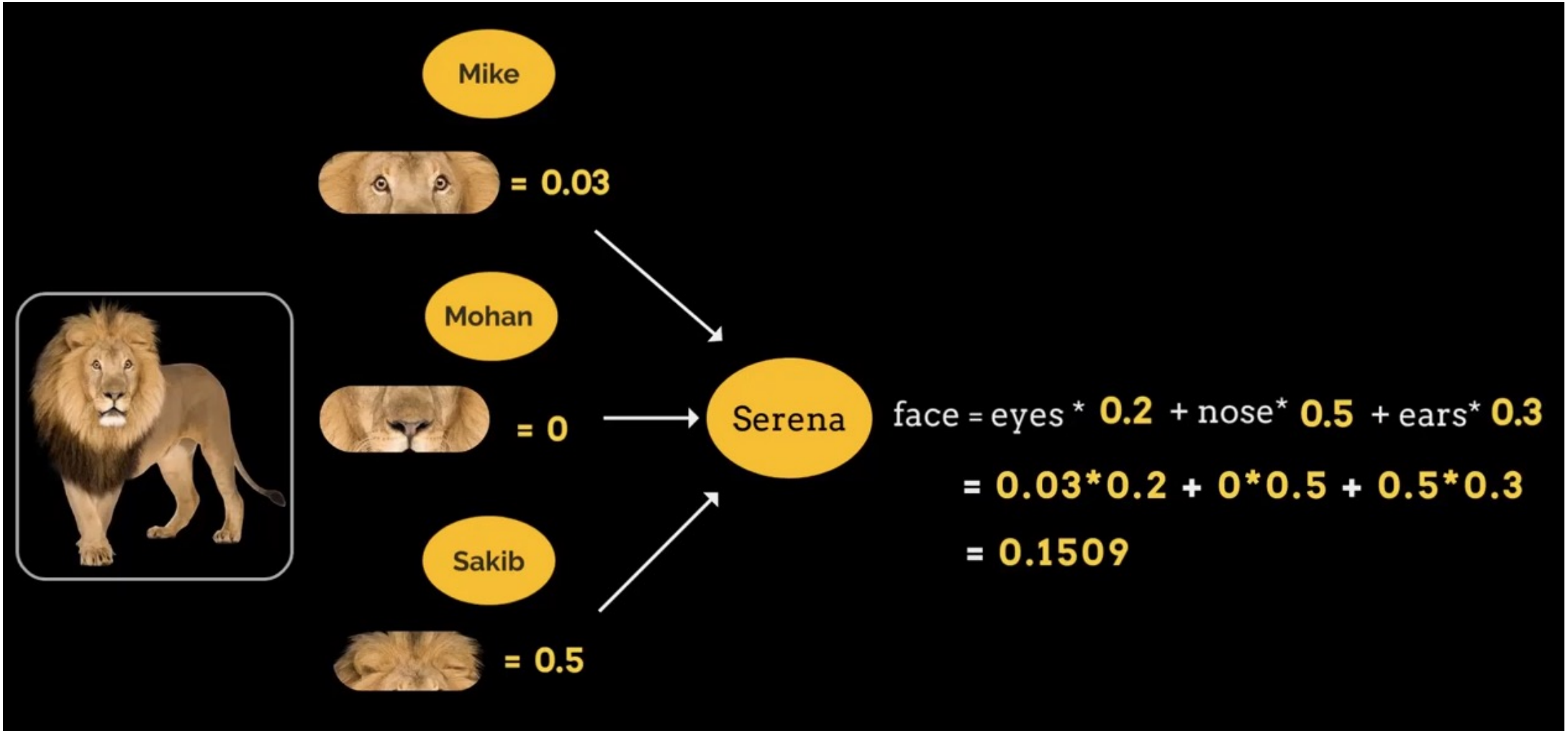

0
0.5
1

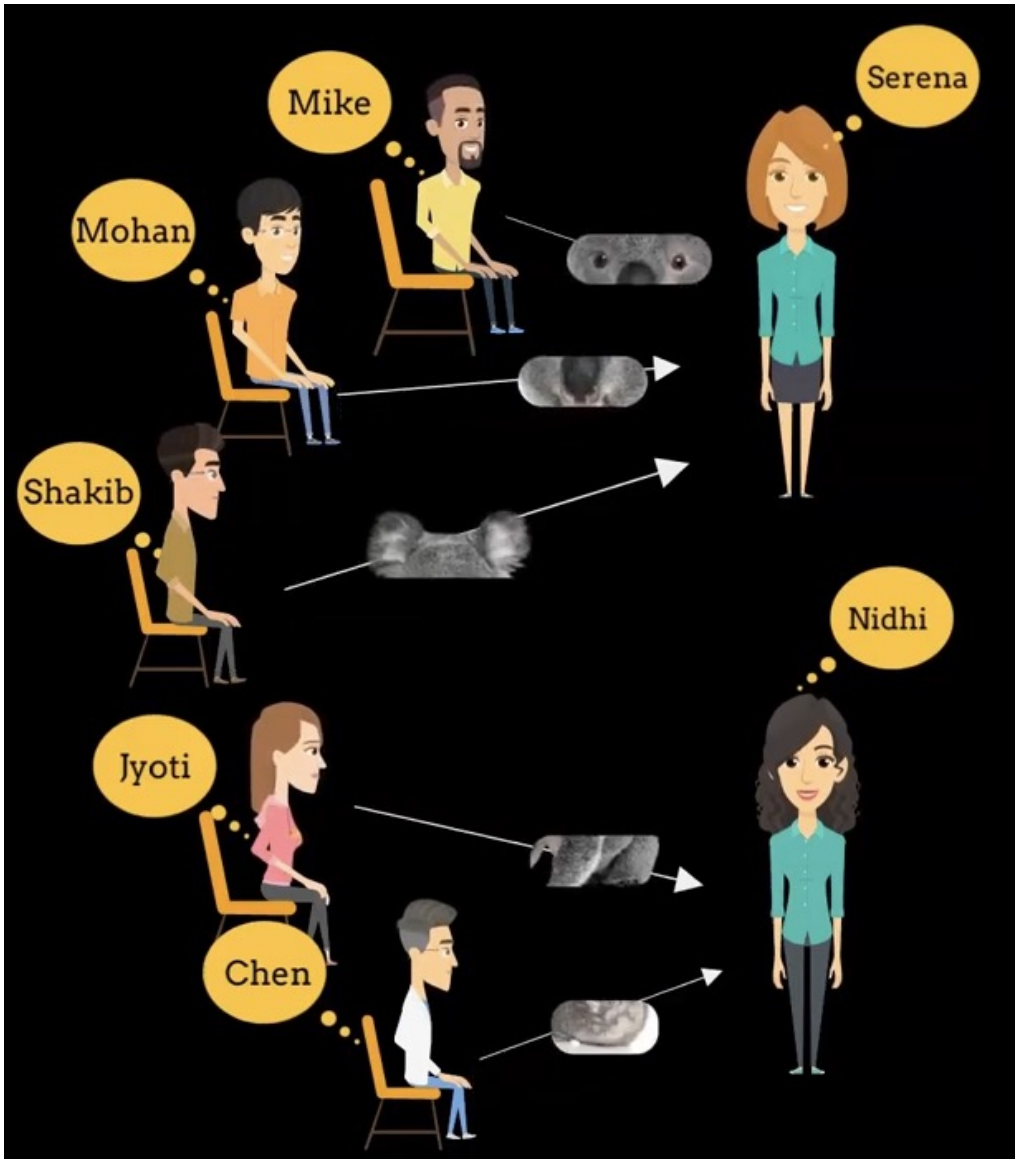

Not a Koala Eyes
Not a Koala Nose
0.03
0.1
Mike
Mohan

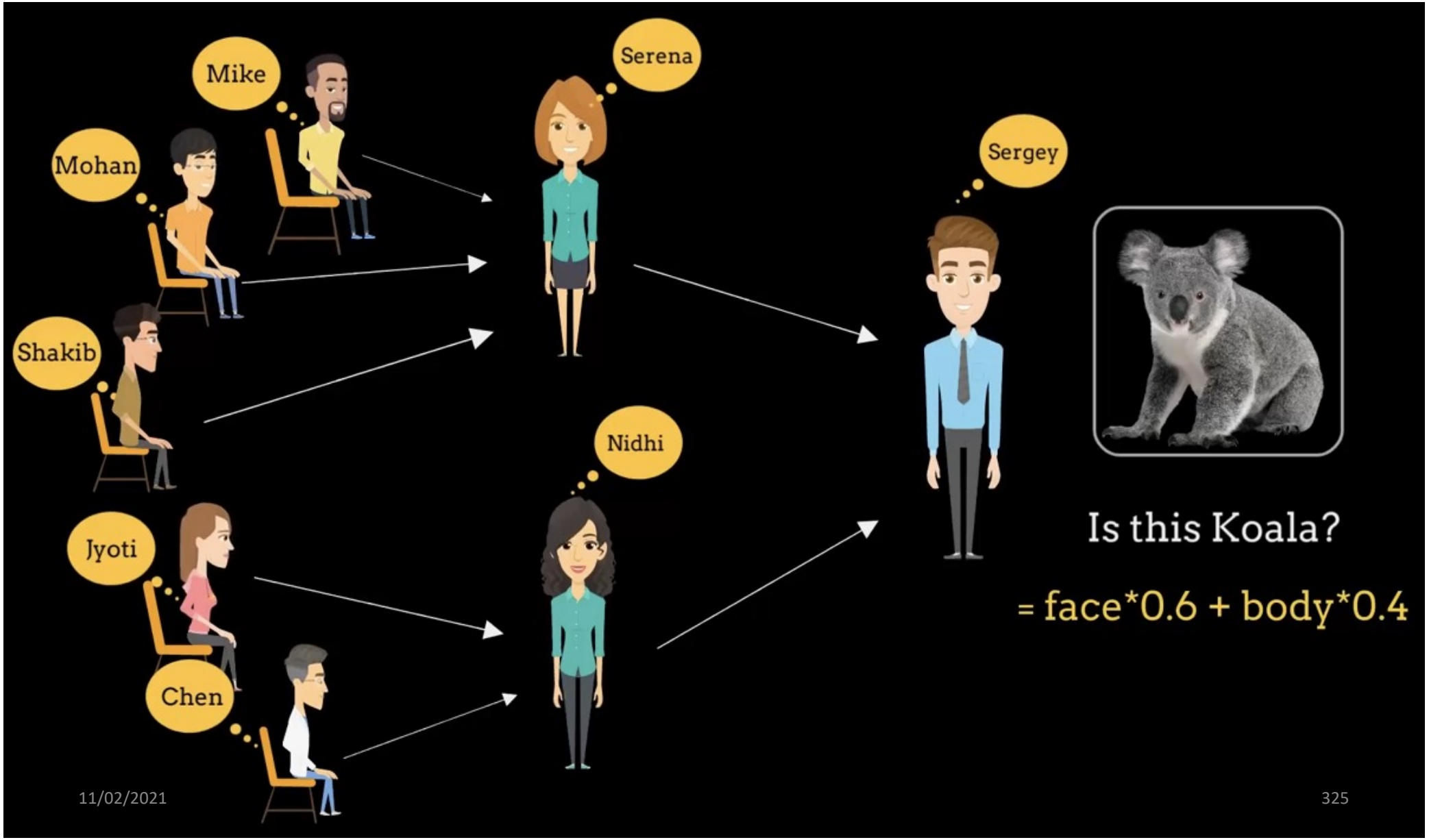

Mike
Mohan
Sakib
Serena


Serena
face = eyes * 0.2 + nose*0.5 + ears* 0.3


Mike
= 0.8
Mohan
= 0.9
Sakib
= 0.6
Serena
face = eyes * 0.2 + nose* 0.5 + ears*0.33
= 0.8*0.2 + 0.9*0.5 + 0.6*0.3
= 0.79


Mike
= 0.03
Mohan
= 0
Sakib
= 0.5
Serena
face = eyes * 0.2 + nose* 0.5 + ears* 0.3
= 0.03*0.2 + 0*0.5 + 0.5*0.3
= 0.1509

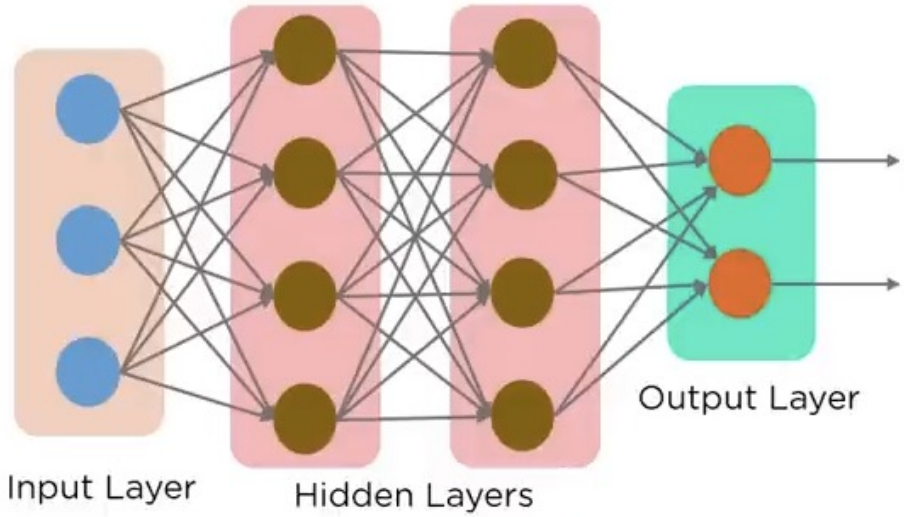

Mike
Serena
Mohan
Shakib
Nidhi
Jyoti
Chen

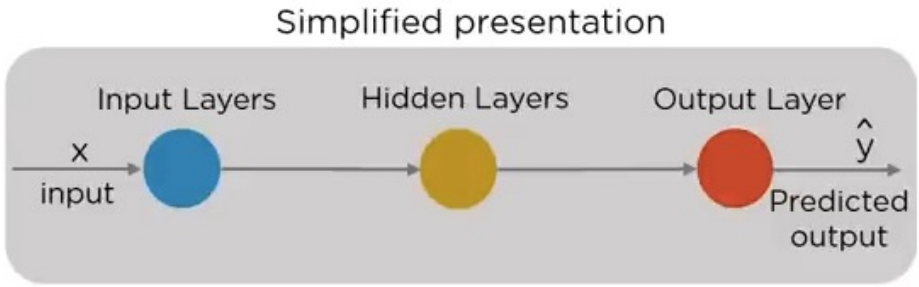

Mike
Mohan
Shakib
Jyoti
Chen
Serena
Nidhi
Sergey
Is this Koala?
= face*0.6 + body*0.4

FEED FORWARD NEURAL NETWORK

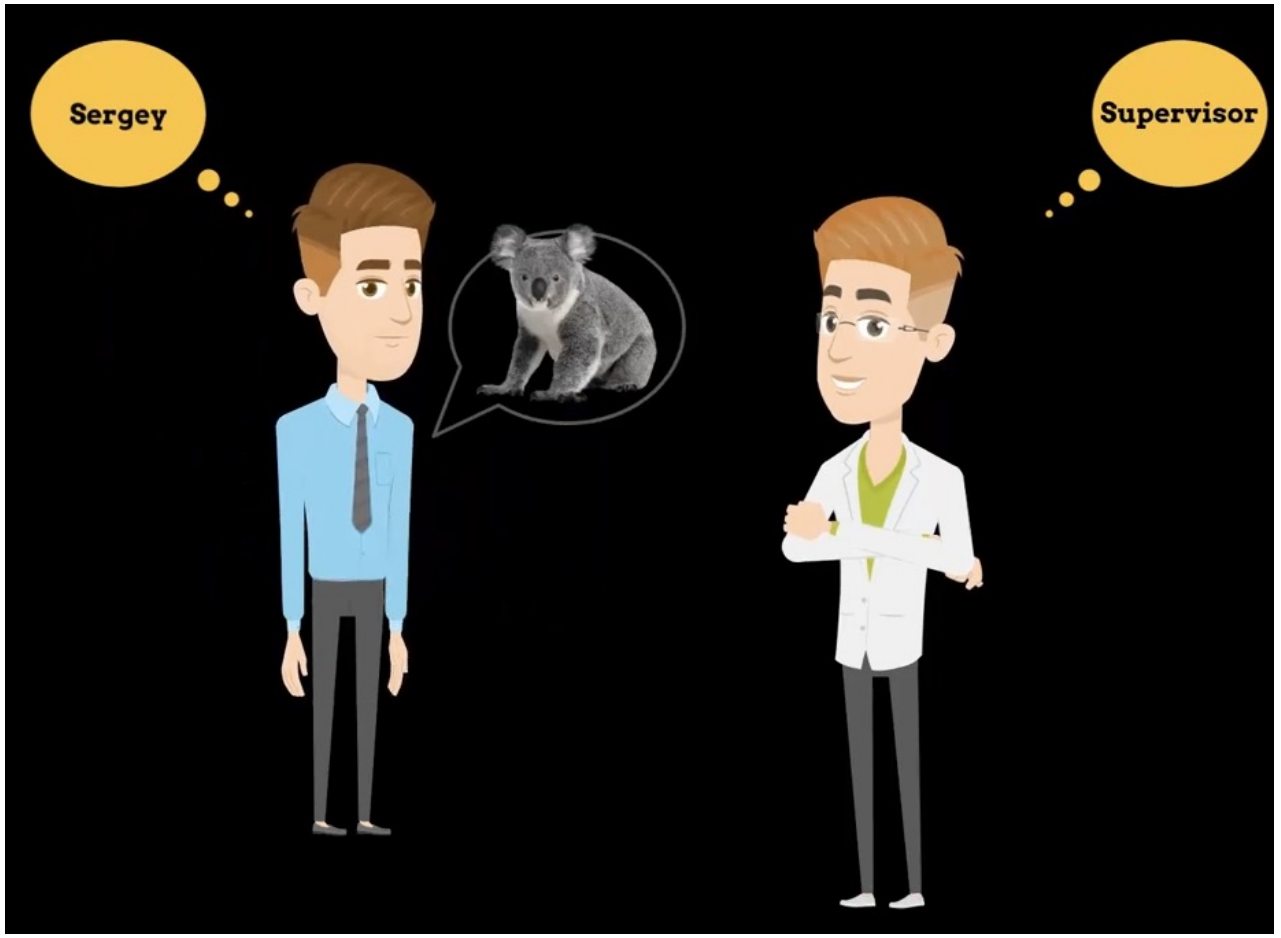
Feed Forward Neural Network

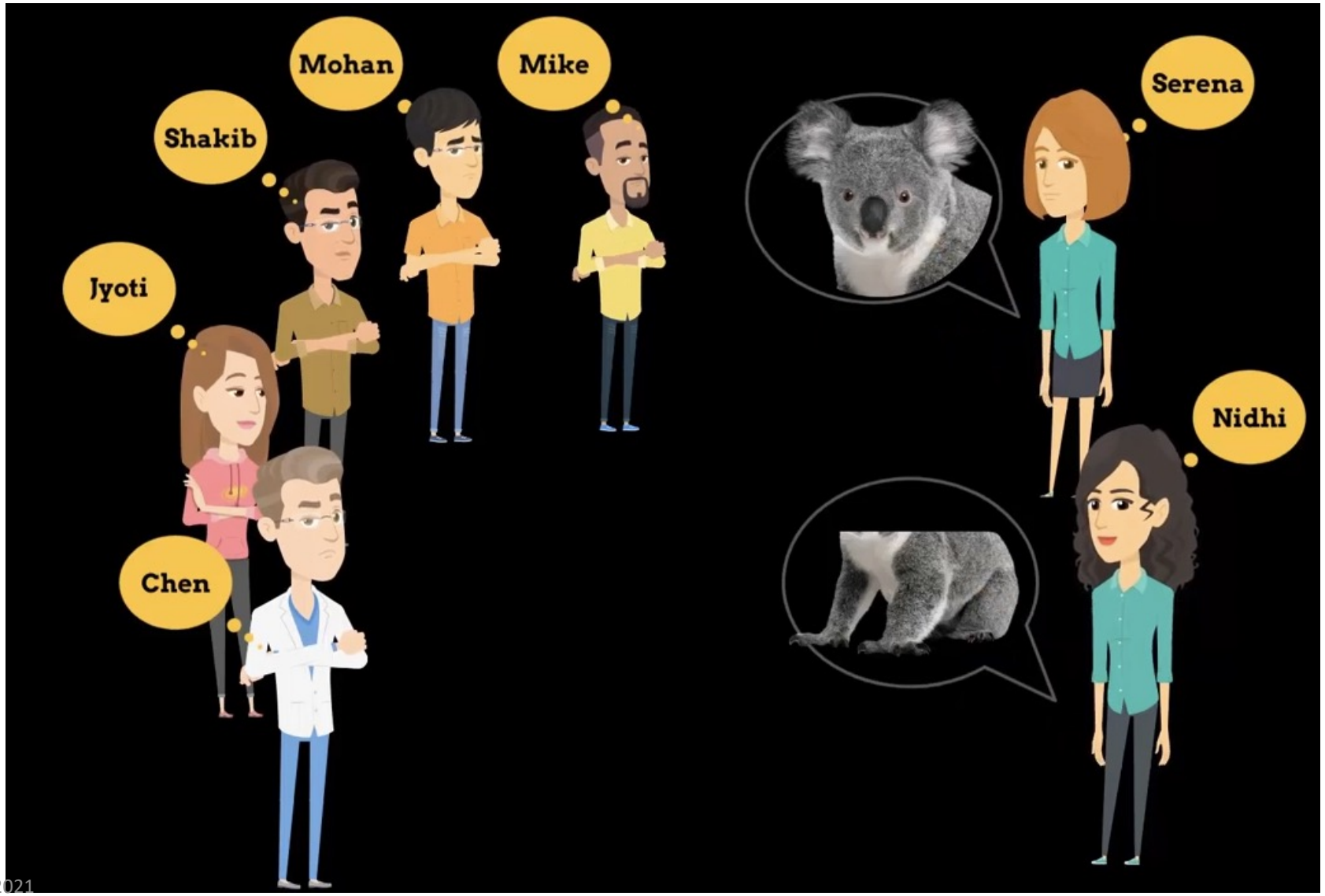
In a Feed-Forward Network , information flows only in forward direction, from the input nodes, through the hidden layers (if any) and to the output nodes. There are no cycles or loops in the network.

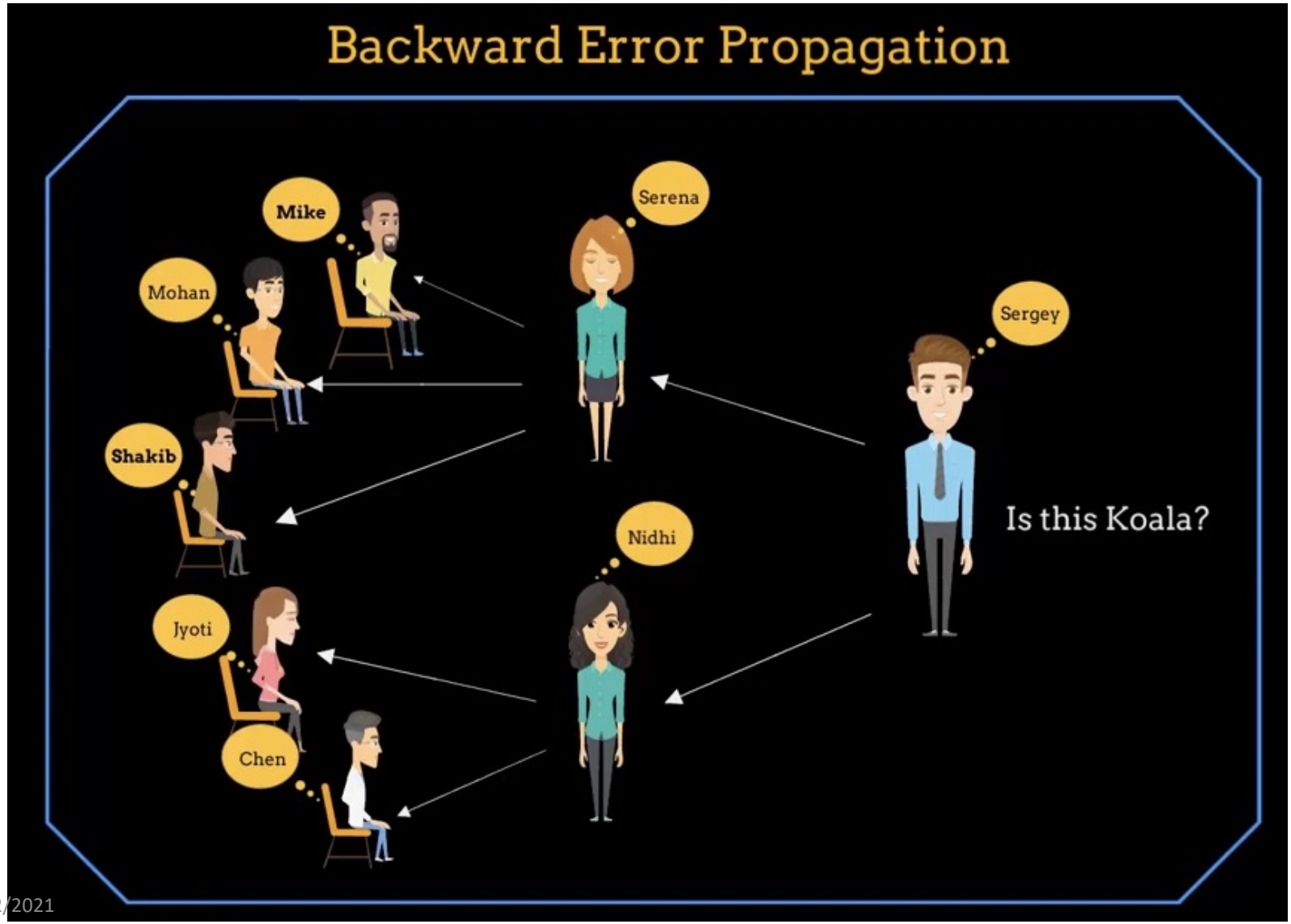


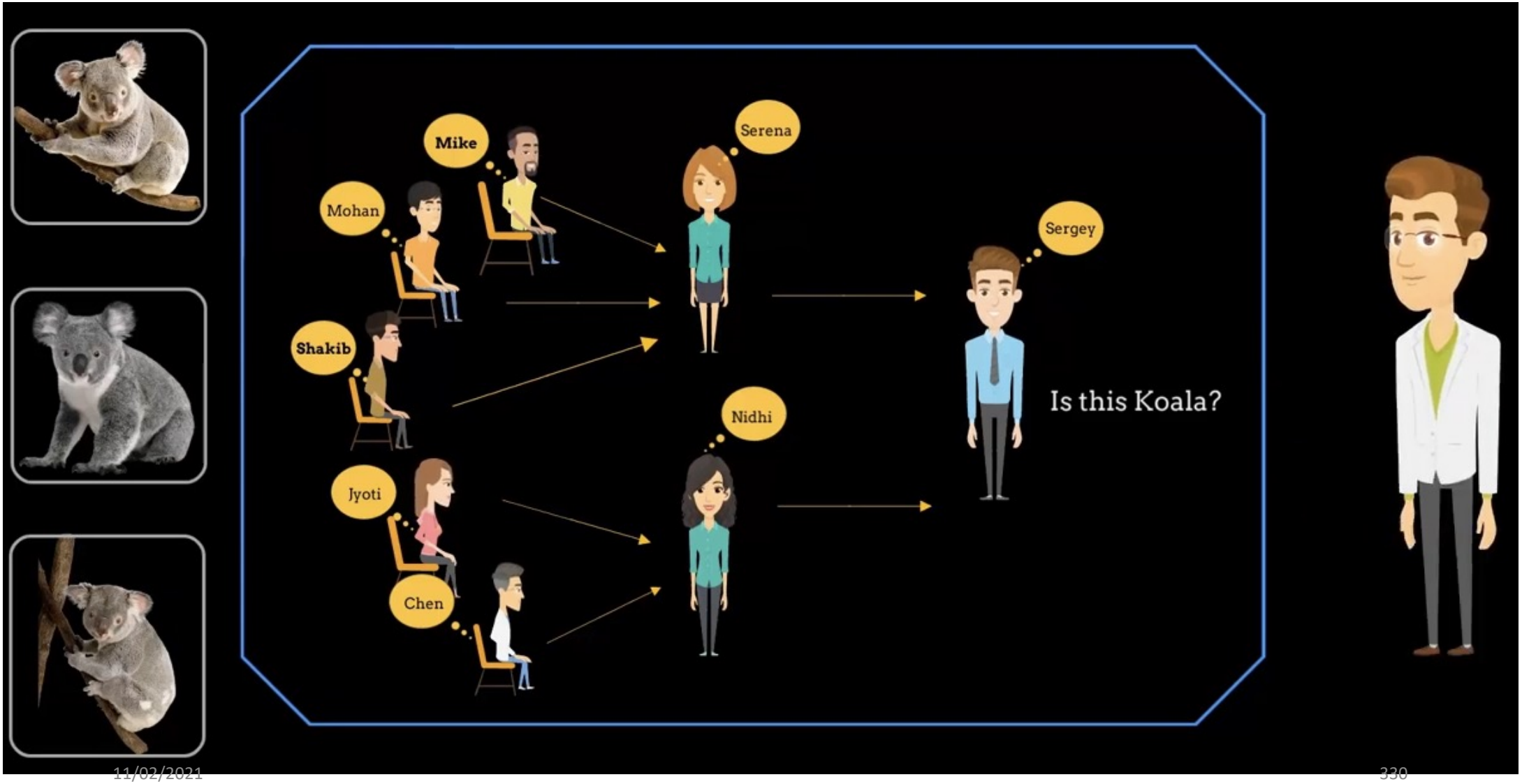


TRAINING IN ANN

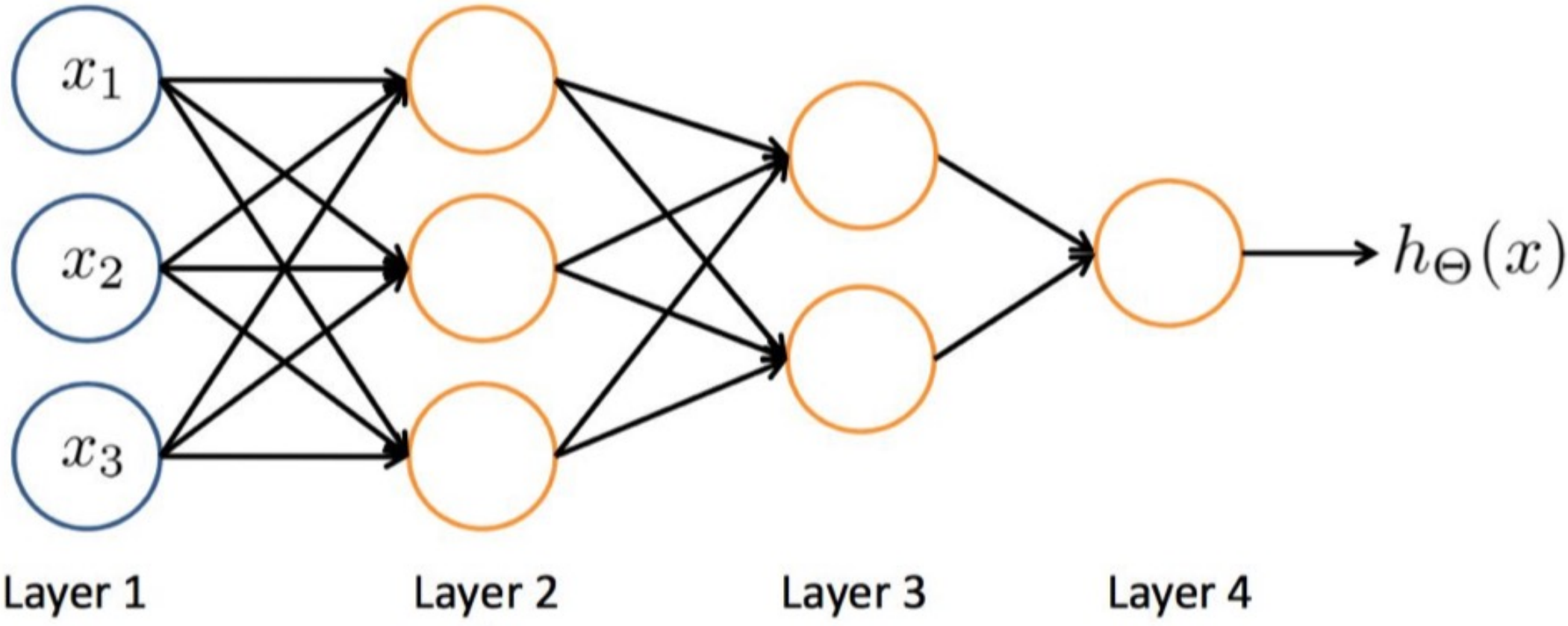



Mohan
Mike
Shakib
Jyoti
Chen
Serena
Nidhi

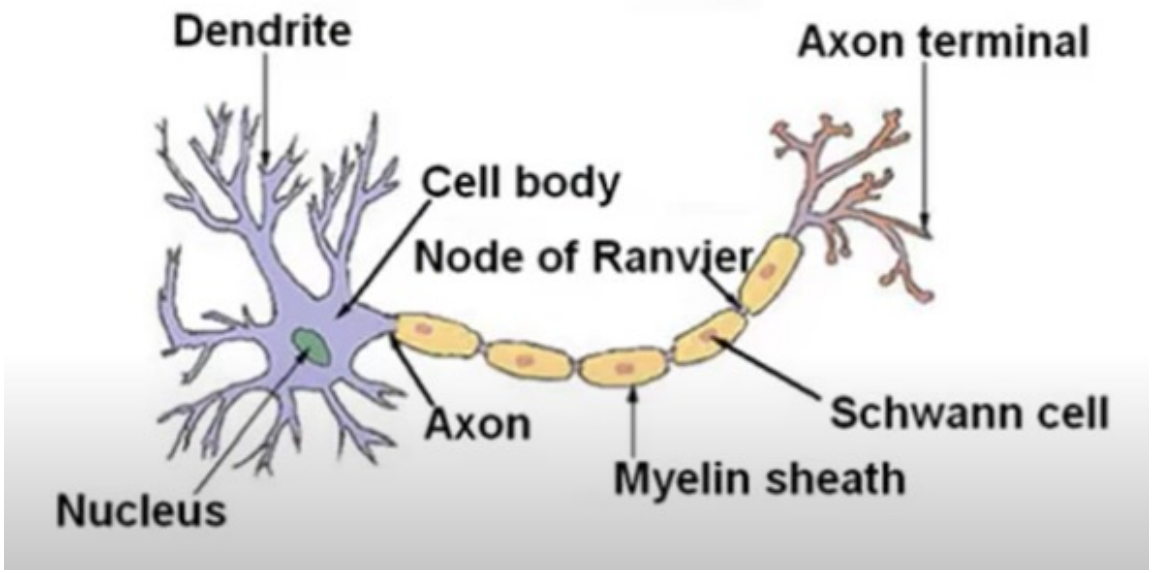

Backward Error Propagation
Mike
Serena
Mohan
Sergey
Shakib
Is this Koala?
Nidhi
Jyoti
Chen

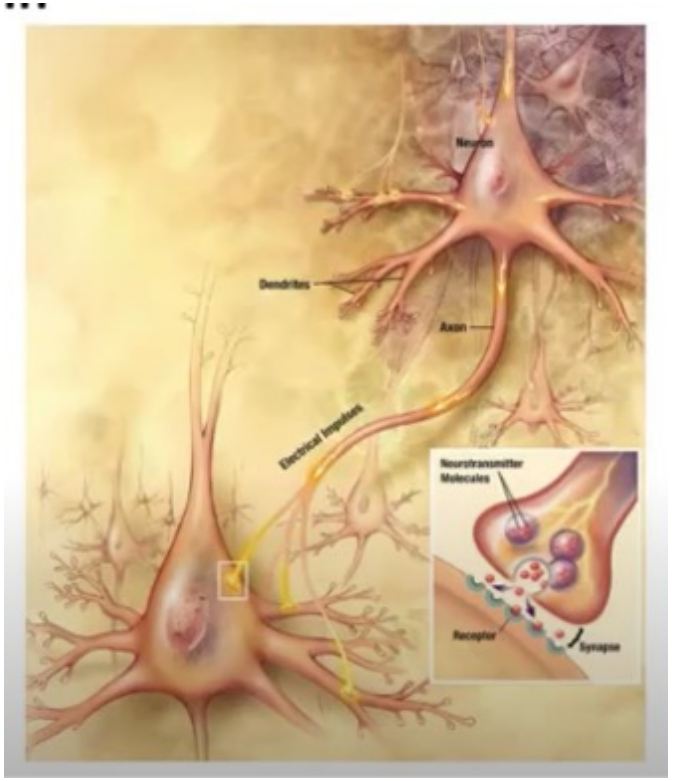

Mike
Mohan
Shakib
Jyoti
Chen
Serena
Nidhi
Sergey
Is this Koala?

Artificial Neural Network

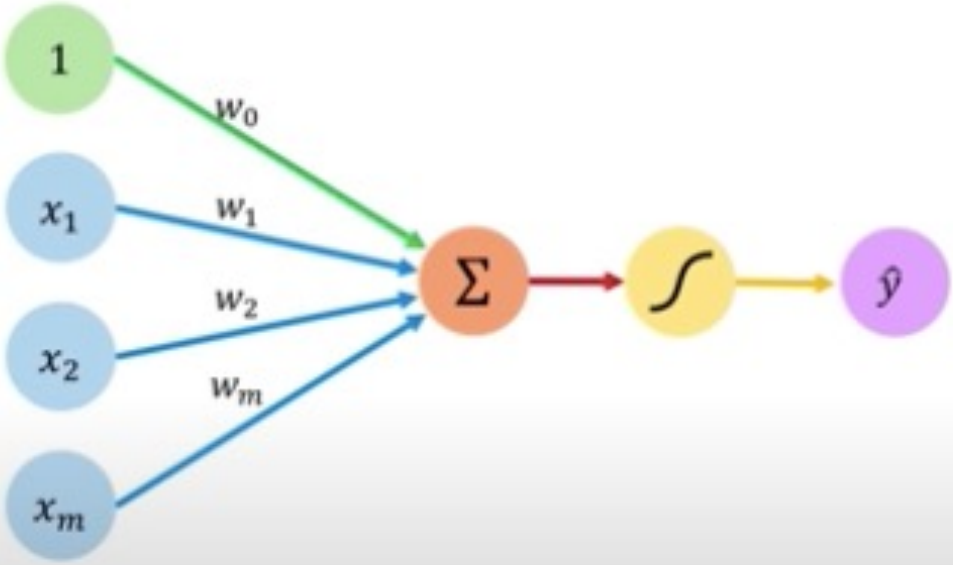


Neurons in the brain





The Perceptron: Forward Propagation



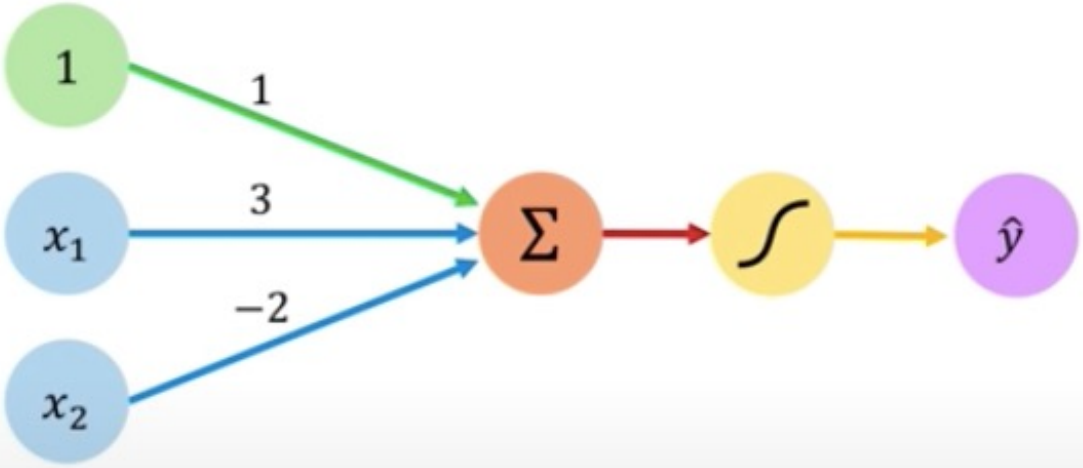
Activation Functions

$$\hat{y} = g(w_0 + \boldsymbol{X}^T \boldsymbol{W})$$

- Example: sigmoid function

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The Perceptron: Example



We have: $w_0 = 1$ and $\boldsymbol{W} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$

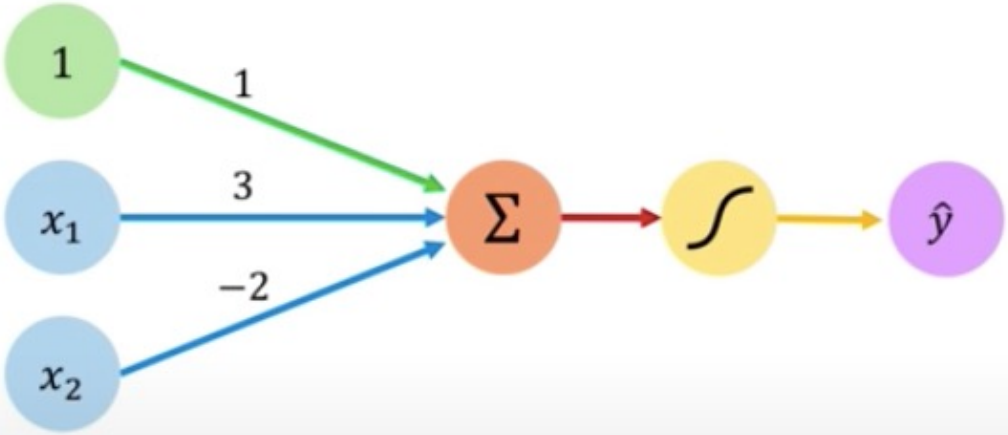
$$\hat{y} = g(w_0 + \boldsymbol{X}^T \boldsymbol{W})$$

$$= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right)$$

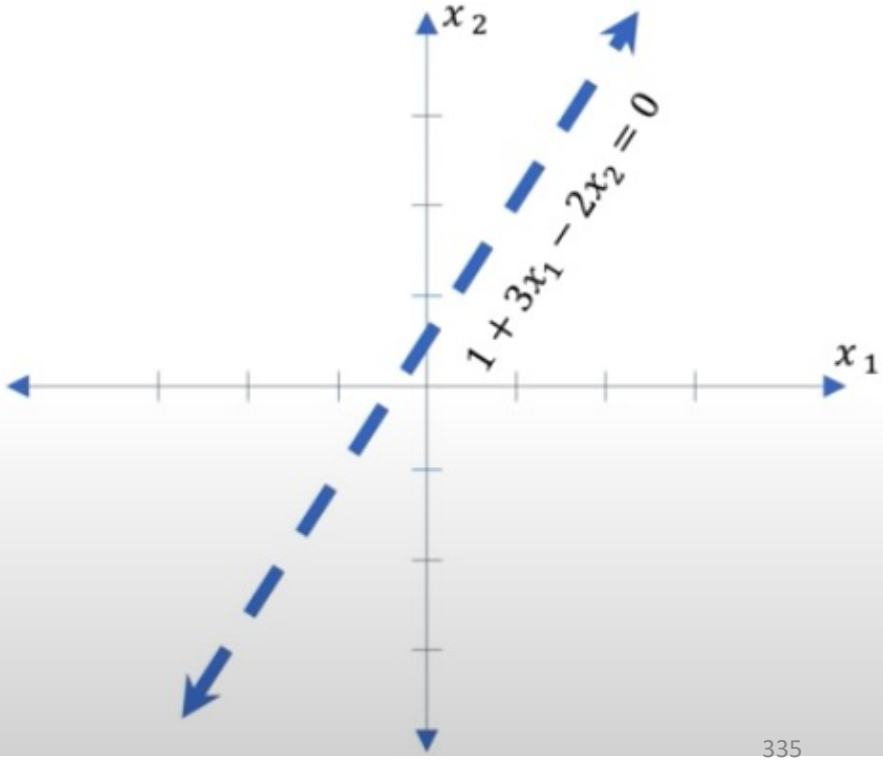
$$\hat{y} = g(1 + 3x_1 - 2x_2)$$

This is just a line in 2D!

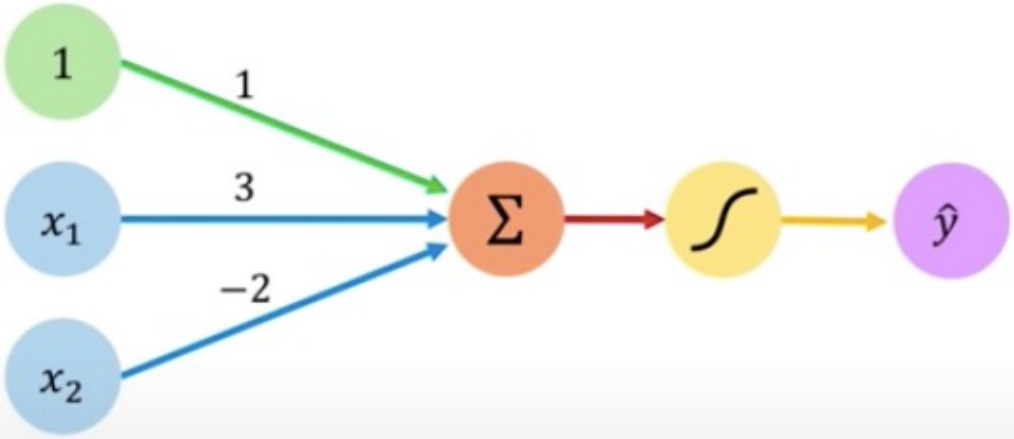
The Perceptron: Example



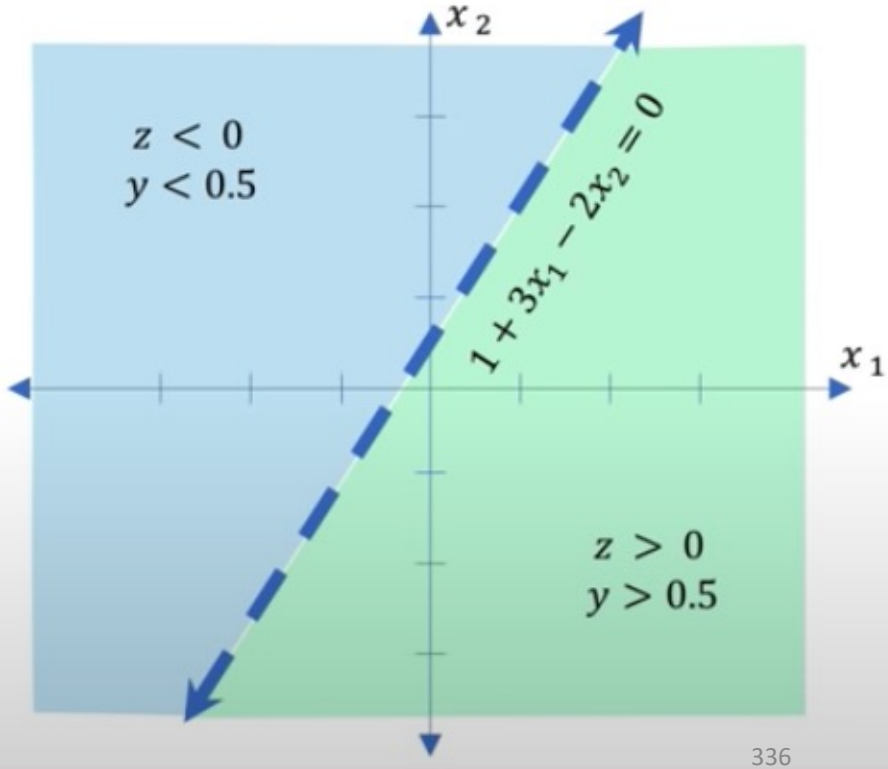
$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



The Perceptron: Example

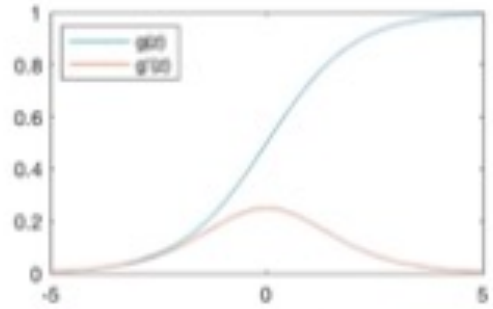


$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



Common Activation Functions

Sigmoid Function

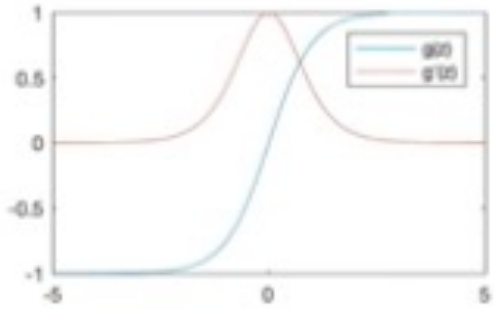


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

`tf.math.sigmoid(z)`

Hyperbolic Tangent

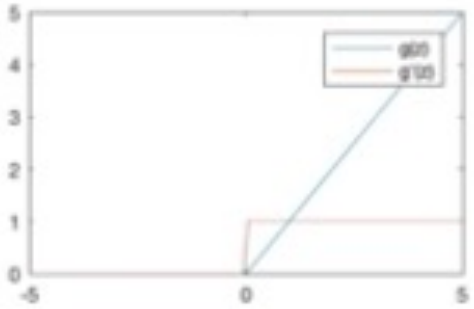


$$g(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

`tf.math.tanh(z)`

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

`tf.nn.relu(z)`

age	have_insurance
22	0
25	0
47	1
52	0
46	1
56	1
55	0
60	1
62	1
61	1
18	0
28	0
27	0
29	0
49	1

APPLICATION EXAMPLE

Step 1

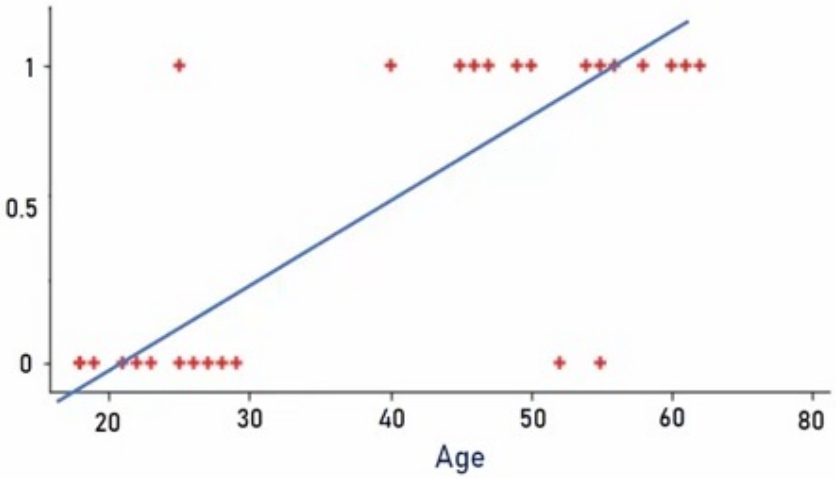
$$y = m * x + b$$

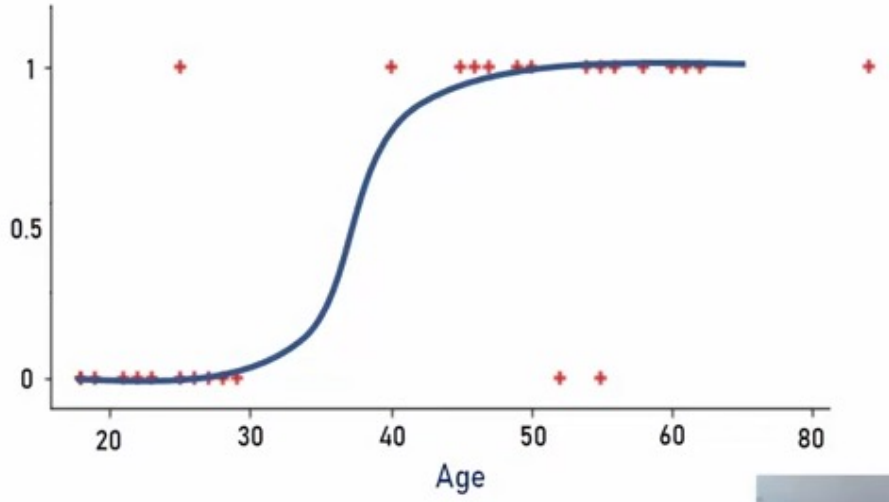
Age

Step 2

$$z = \frac{1}{1 + e^{-y}}$$

If person will buy insurance

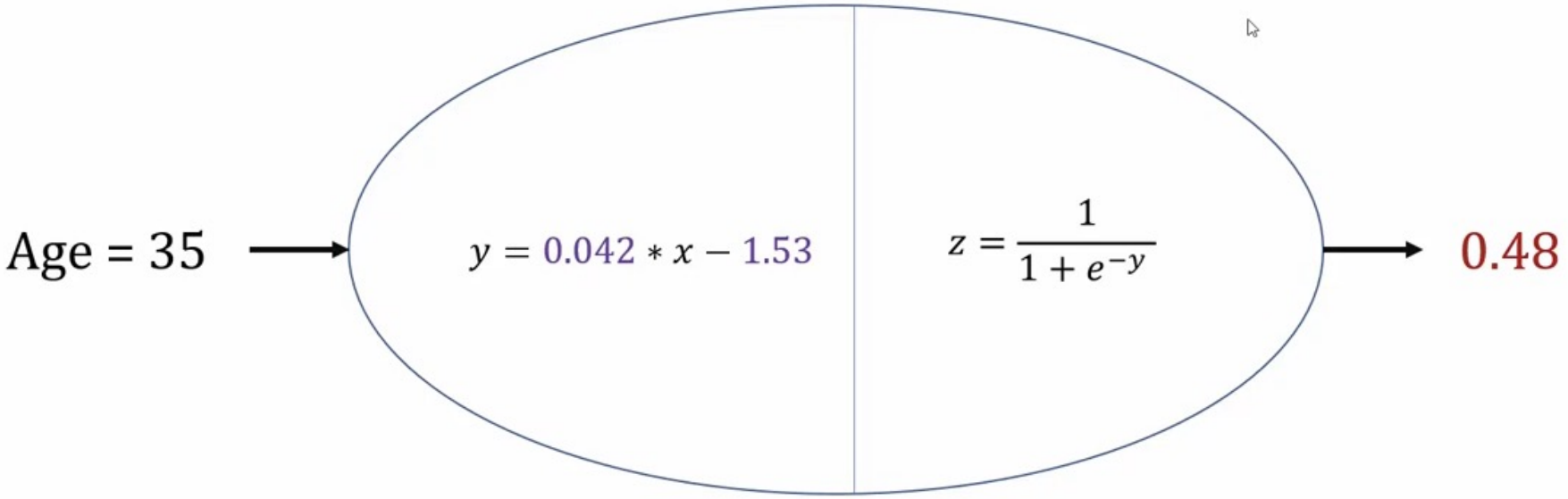




LINEAR TO LOGISTIC

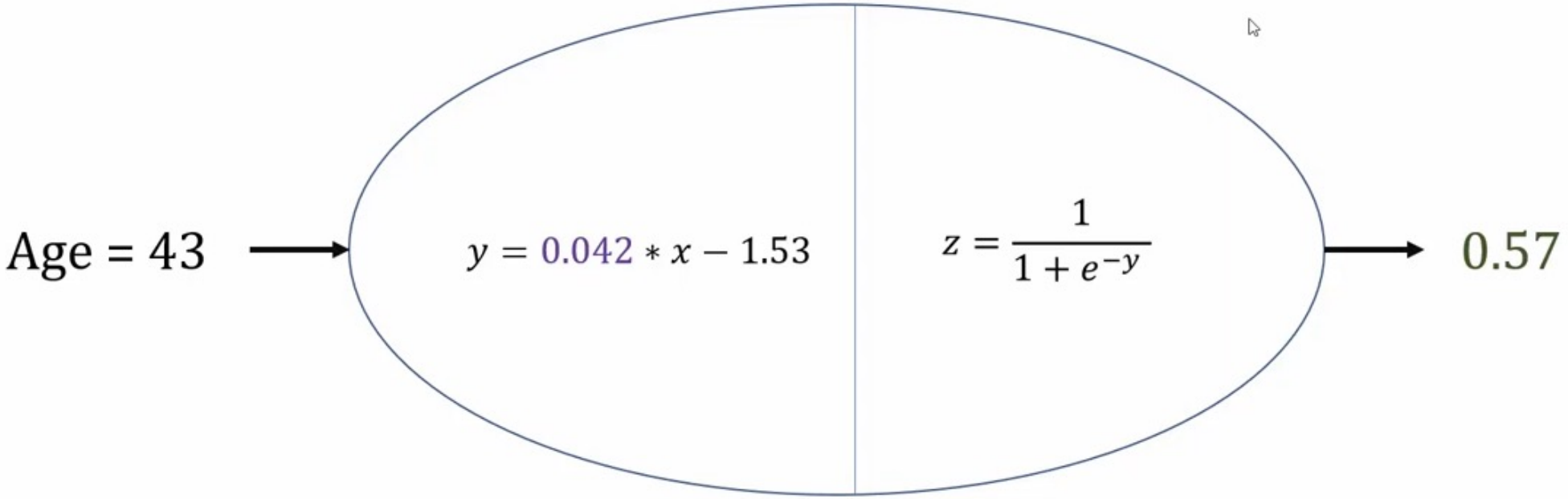
value < 0.5 = person will not buy insurance

value >= 0.5 = person **will** buy insurance



value < 0.5 = person will not buy insurance

value >= 0.5 = person **will** buy insurance



$$y = 0.042 * x - 1.53$$

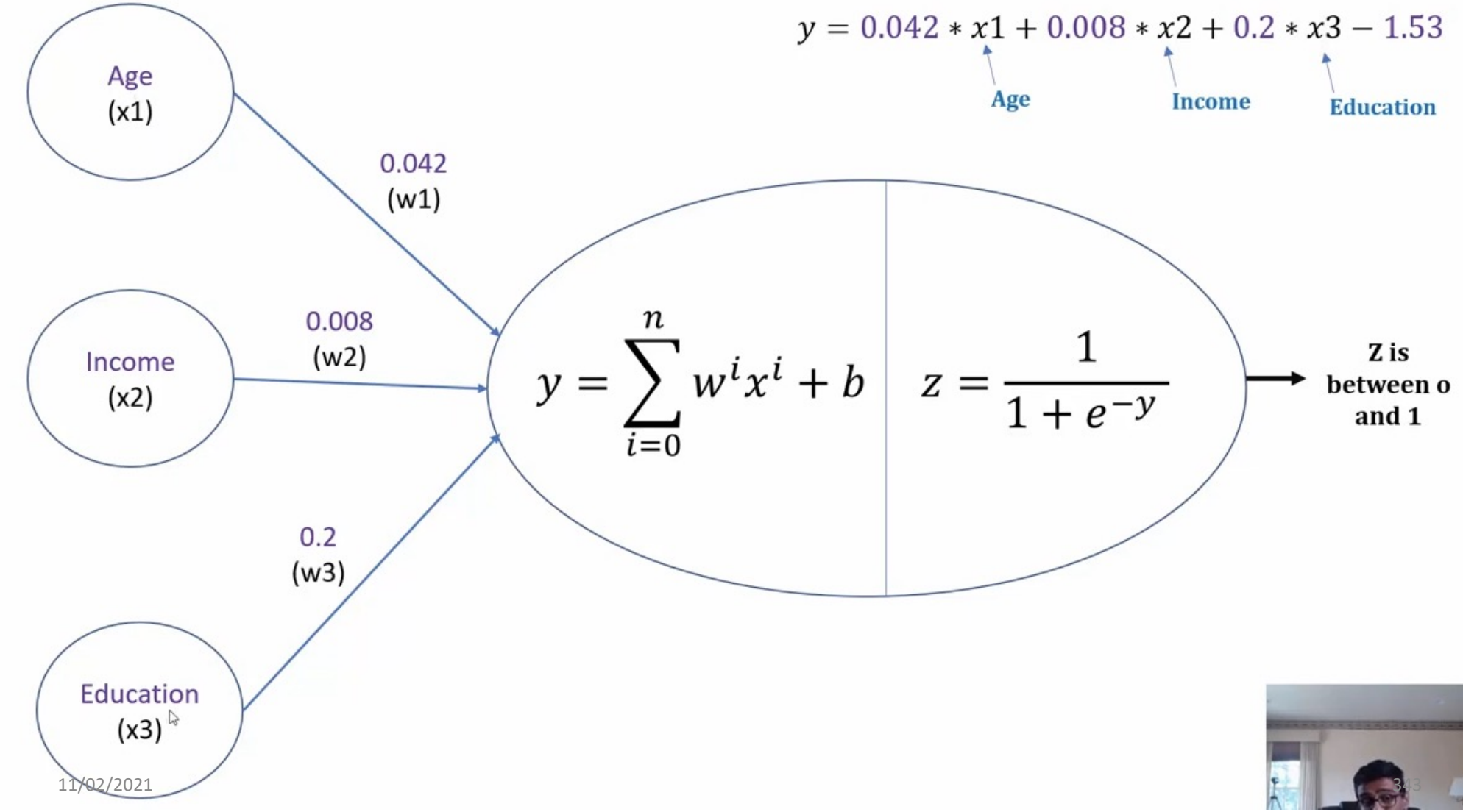
Age

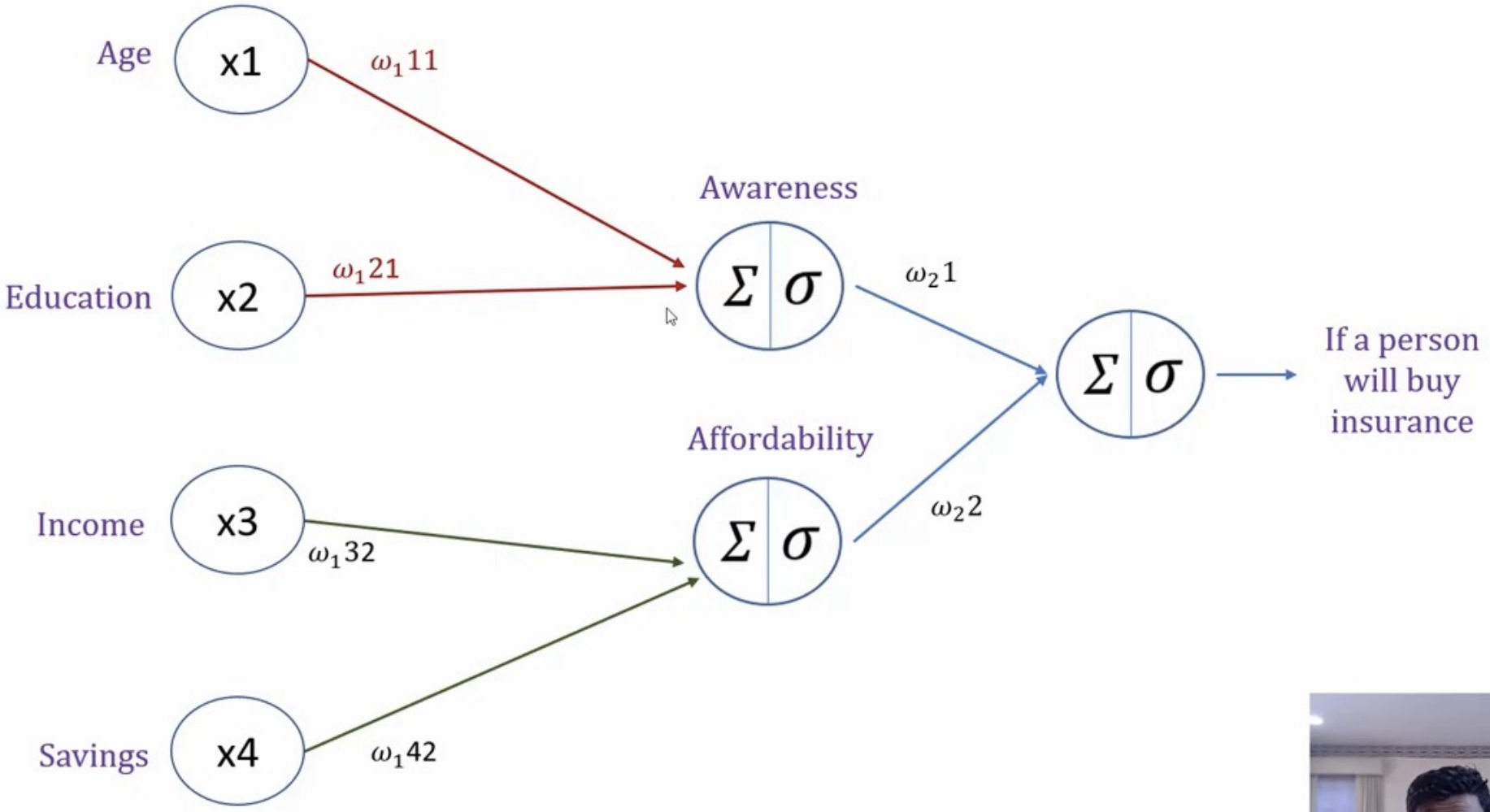
$$y = 0.042 * x1 + 0.008 * x2 + 0.2 * x3 - 1.53$$

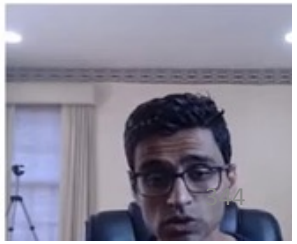
Age Income Education

$$y = w1 * x1 + w2 * x2 + w3 * x3 + \boldsymbol{b}$$

$$y = \sum_{i=0}^{n} w^i x^i + b$$

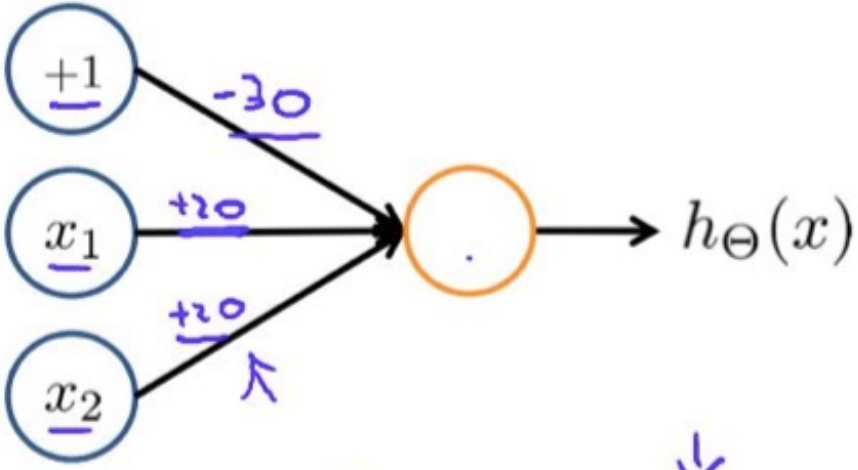

$y = 0.042 * x1 + 0.008 * x2 + 0.2 * x3 - 1.53$
Age
Income
Education
Age
(x1)
Income
(x2)
Education
(x3)
0.042
(w1)
0.008
(w2)
0.2
(w3)
$y = \sum_{i=0}^{n} w^i x^i + b$
$z = \frac{1}{1 + e^{-y}}$
Z is between o and 1

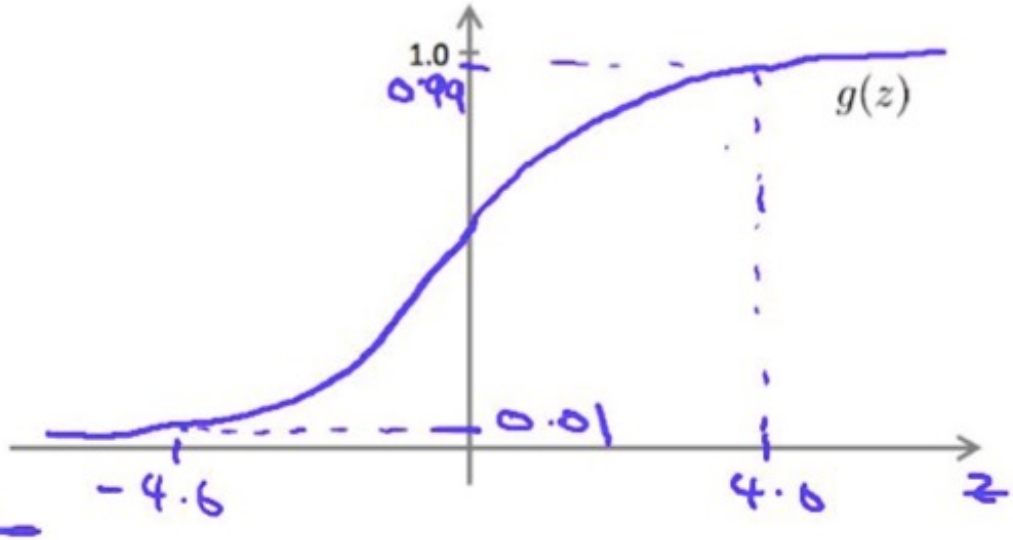

Age
x1
$\omega_1 11$
Awareness
Education
x2
$\omega_1 21$
Σ σ
$\omega_2 1$
Σ σ
If a person will buy insurance
Affordability
Income
x3
$\omega_1 32$
Σ σ
$\omega_2 2$
Savings
x4
$\omega_1 42$



Simple example: AND

- $x_1, x_2 \in \{0, 1\}$
- $y = x_1 \text{ AND } x_2$





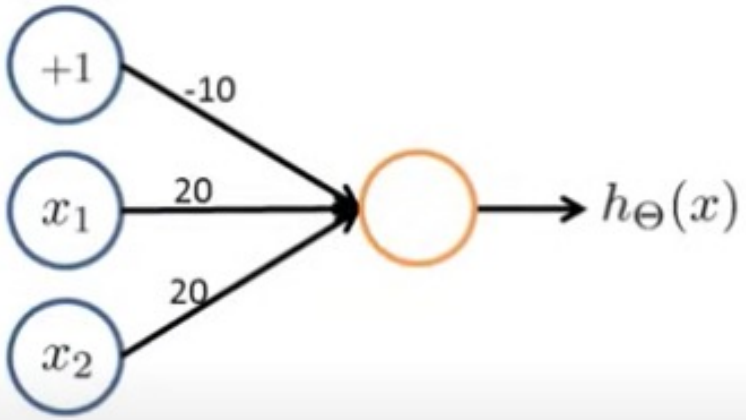
$h_\Theta(x) = g(-30 + 20x_1 + 20x_2)$

$\Theta_{10}^{(1)}$ $\Theta_{11}^{(1)}$ $\Theta_{12}^{(1)}$

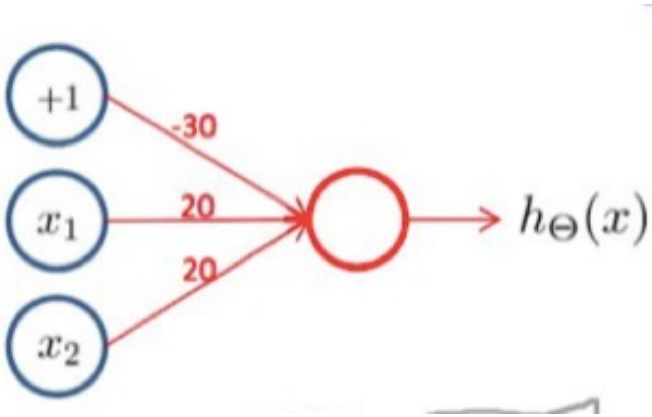
x_1	x_2	$h_\Theta(x)$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

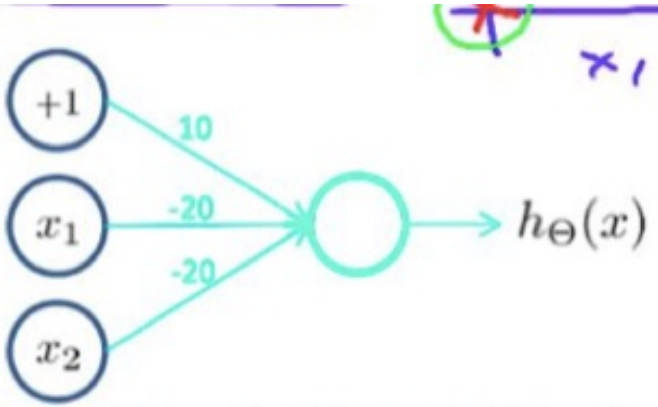
$h_\Theta(x) \approx x_1 \text{ AND } x_2$

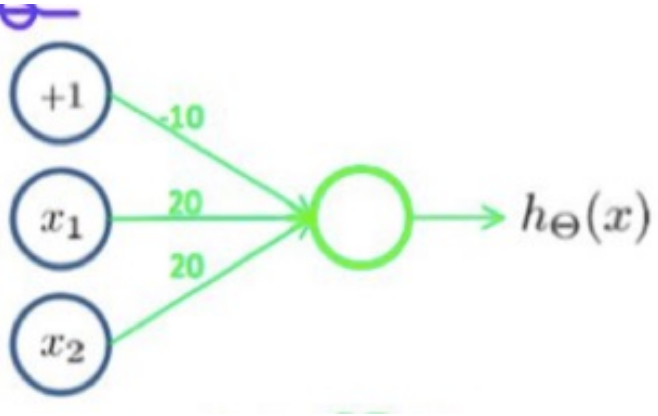
Or Function



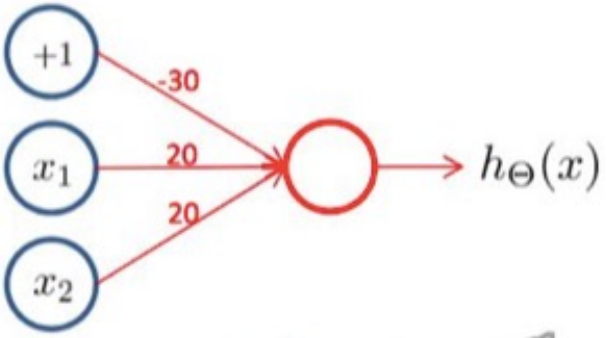
x_1	x_2	$h_\Theta(x)$
0	0	
0	1	
1	0	
1	1	

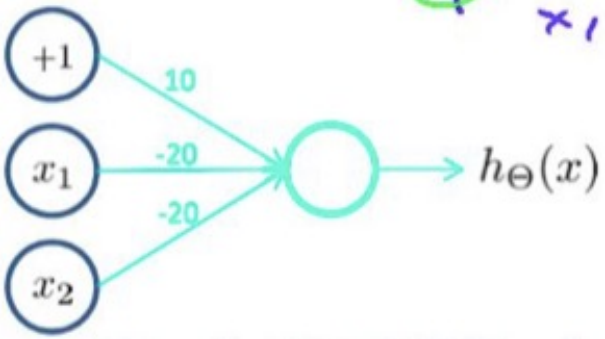

+1
x_1
x_2
-30
20
20
$h_\Theta(x)$

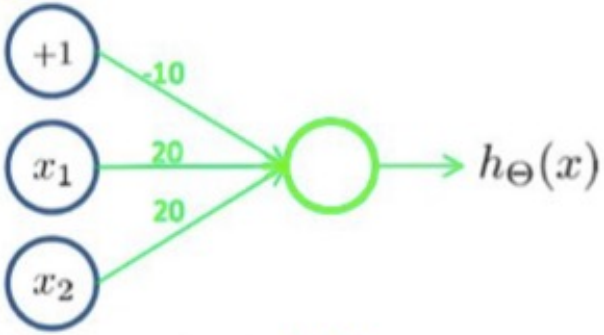

+1
x_1
x_2
10
-20
-20
$h_\Theta(x)$

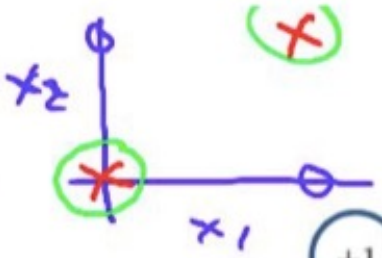

+1
x_1
x_2
-10
20
20
$h_\Theta(x)$

Putting it together: x_1 XNOR x_2





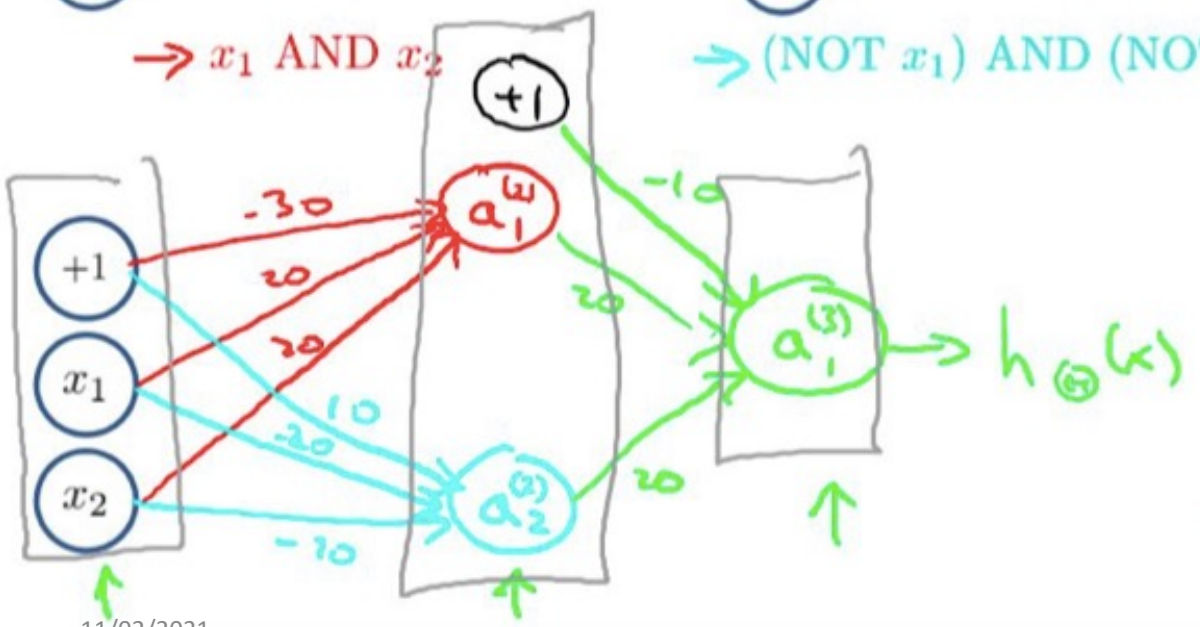




→ x_1 AND x_2

→ (NOT x_1) AND (NOT x_2)

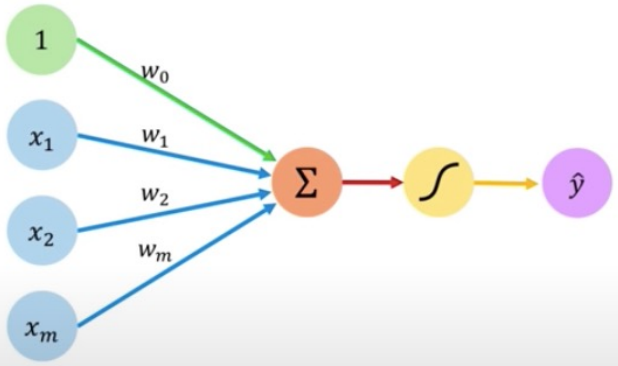
→ x_1 OR x_2

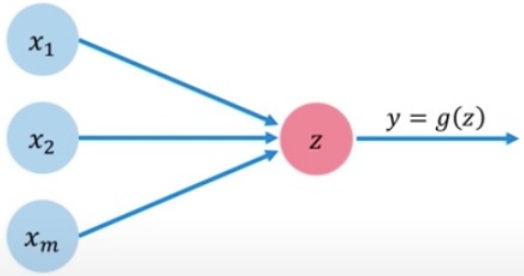


x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\Theta(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

The Perceptron: Simplified

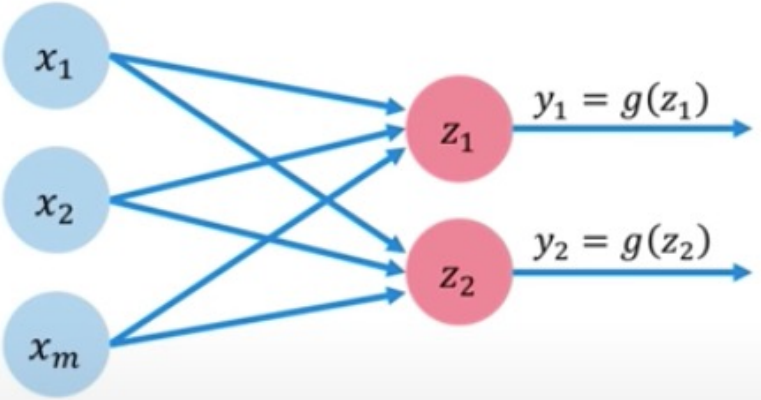
$$\hat{y} = g\left(w_0 + \boldsymbol{X}^T \boldsymbol{W} \right)$$





$$z = w_0 + \sum_{j=1}^{m} x_j \, w_j$$

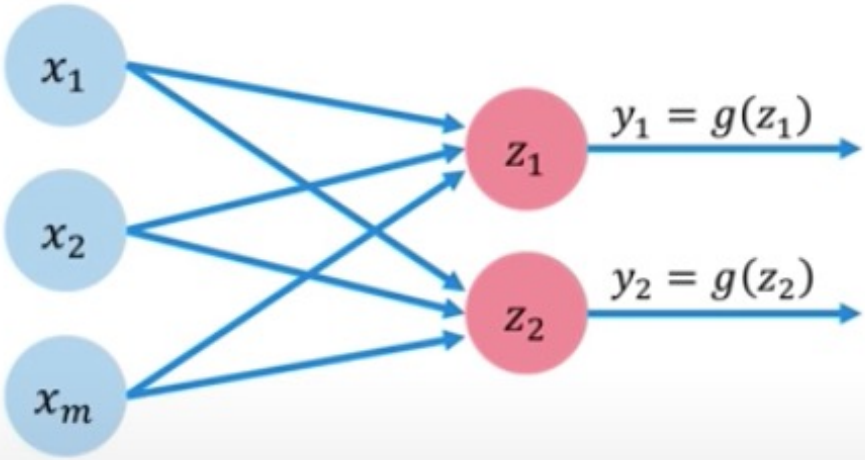
Multi Output Perceptron



$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

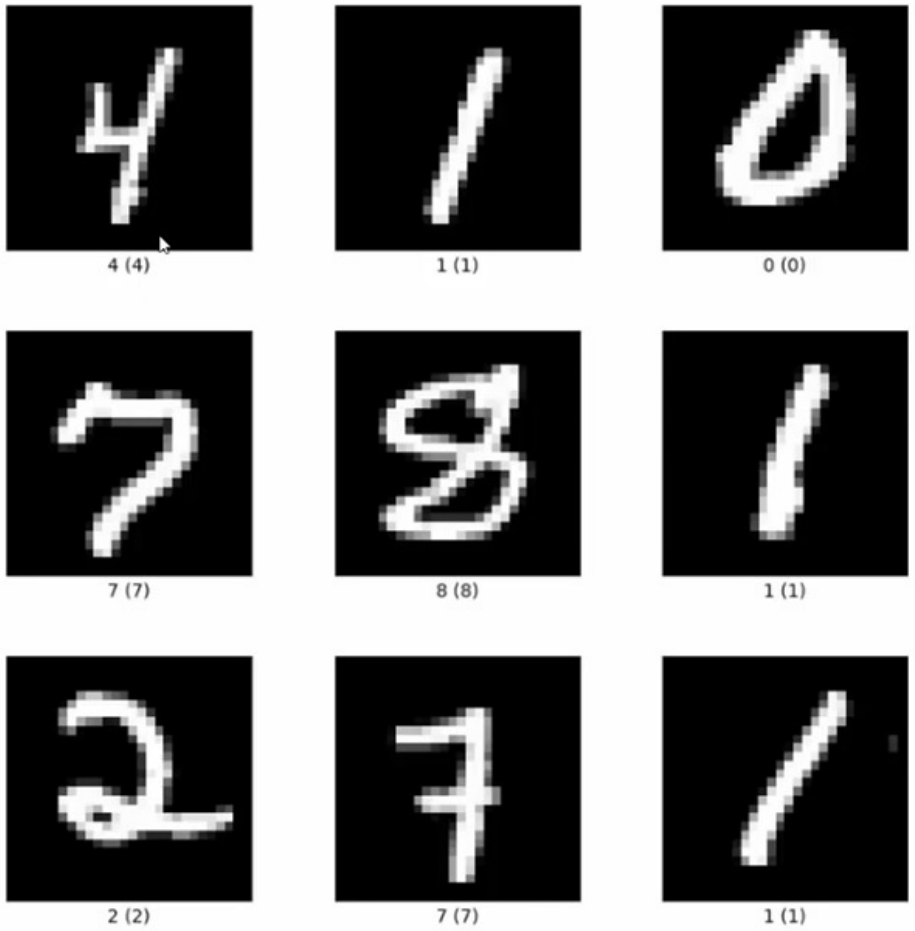
Multi Output Perceptron

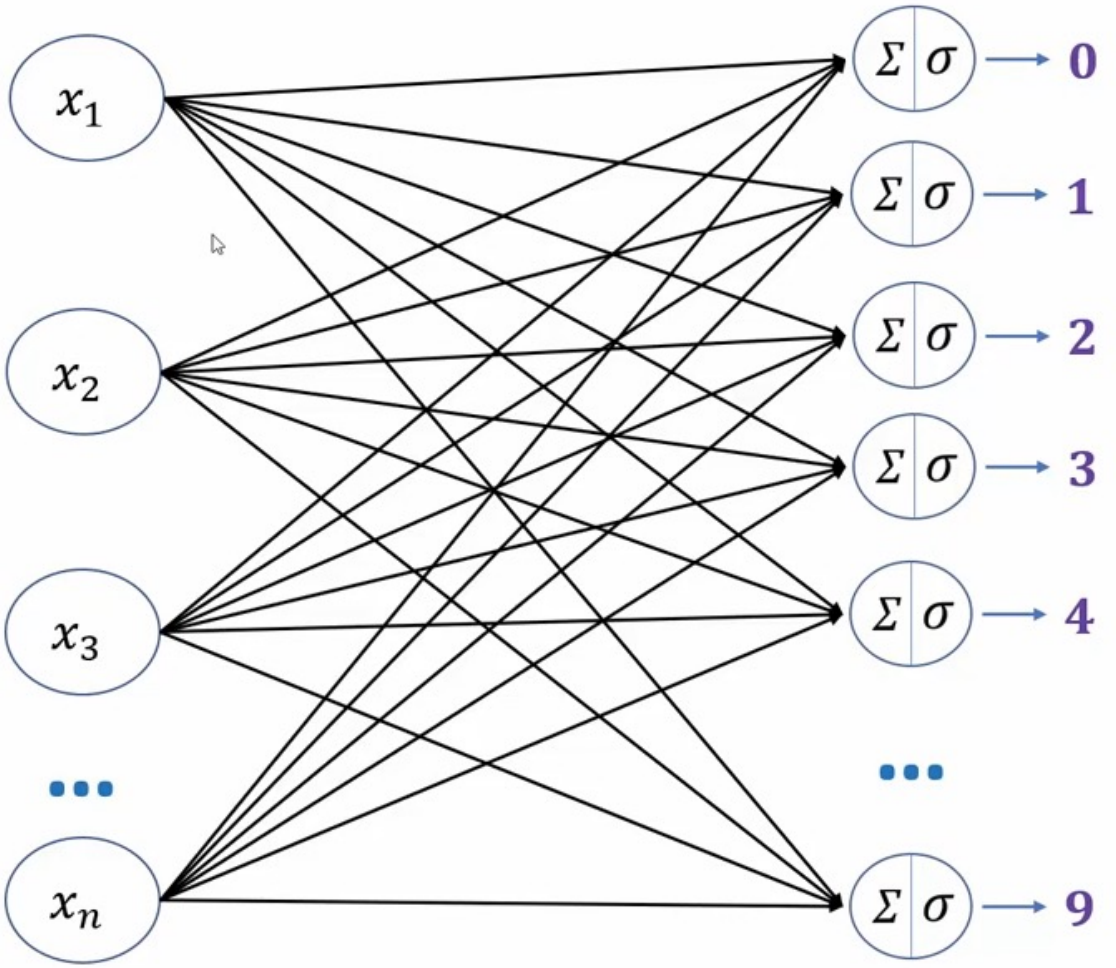
Because all inputs are densely connected to all outputs, these layers are called **Dense** layers

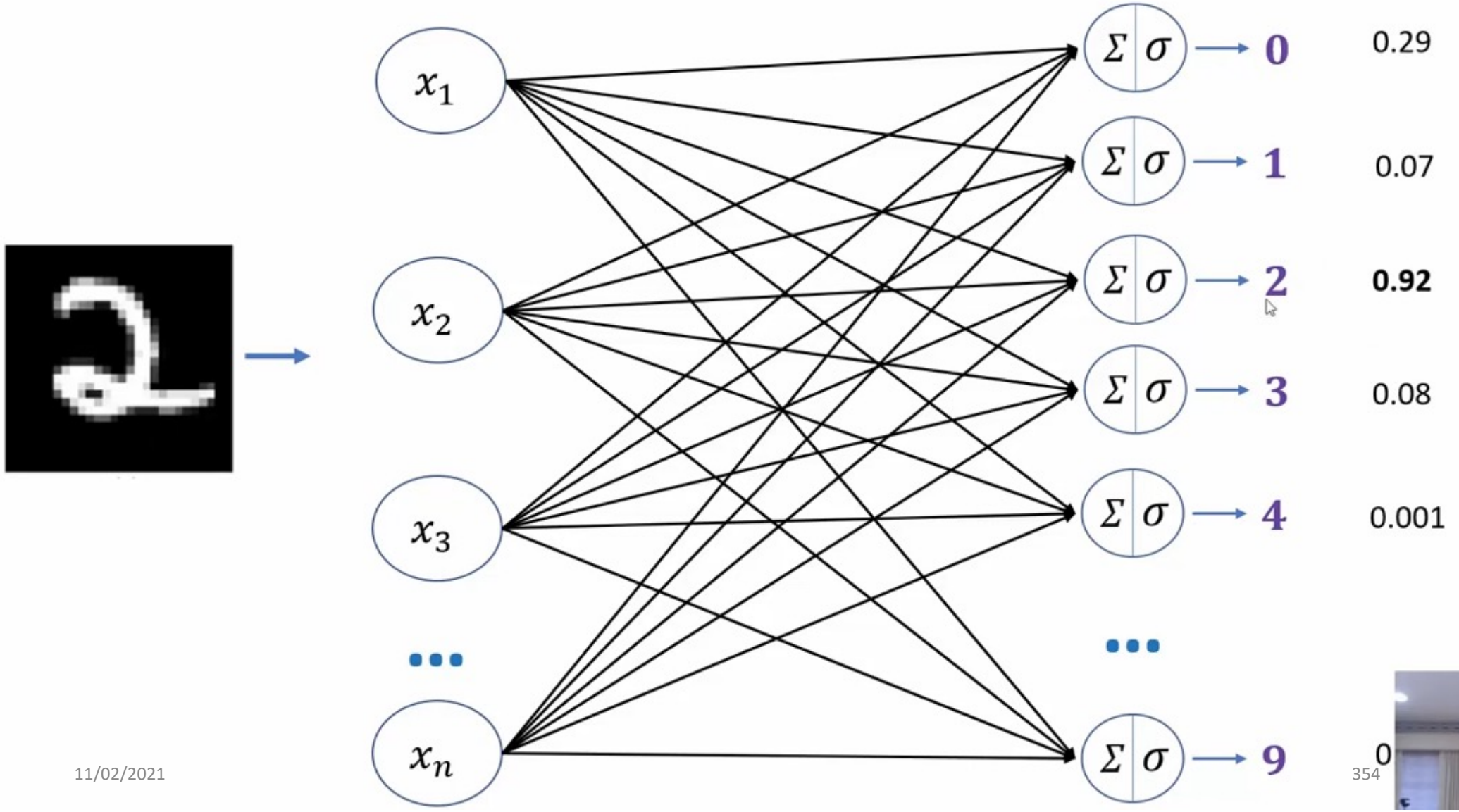


$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

EXAMPLE OF IMPLEMENTATION


4 (4)
1 (1)
0 (0)
7 (7)
8 (8)
1 (1)
2 (2)
7 (7)
1 (1)


x_1
x_2
x_3
x_n
Σ σ
0
1
2
3
4
9


x_1
x_2
x_3
x_n
Σ σ
0
1
2
3
4
9
0.29
0.07
0.92
0.08
0.001
0

0 – Black, 255 - White

0	0	0	0	0	0	0
0	87	240	210	24	0	0
0	13	0	101	195	0	0
0	35	167	99	210	0	0
0	145	230	240	201	189	140
0	0	102	67	17	13	0
0	0	0	0	0	0	0

0	0	0	0	0	0	0
0	87	240	210	24	0	0
0	13	0	101	195	0	0
0	35	167	99	210	0	0
0	145	230	240	201	189	140
0	0	102	67	17	13	0
0	0	0	0	0	0	0

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np
```

```
(X_train, y_train) , (X_test, y_test) = keras.datasets.mnist.load_data()
```

```
len(X_train)
```

```
len(X_test)
```

```
X_train[0].shape
```

```
X_train[0]
```

```
plt.matshow(X_train[0])
```

```
y_train[0]
```

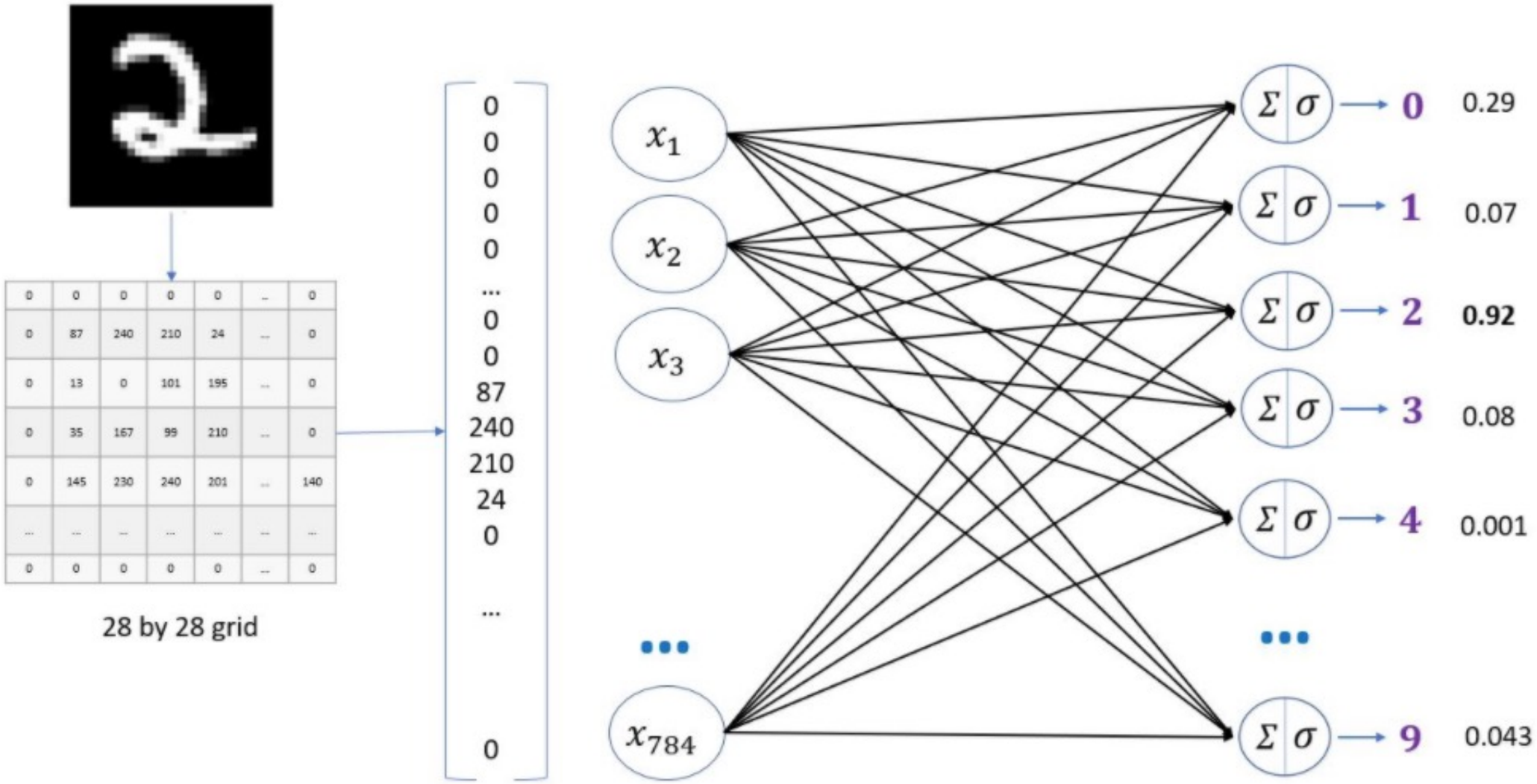
```
X_train = X_train / 255
X_test = X_test / 255
```

```
X_train[0]
```

```
X_train_flattened = X_train.reshape(len(X_train), 28*28)
X_test_flattened = X_test.reshape(len(X_test), 28*28)
```

```
X_train_flattened.shape
```

```
X_train_flattened[0]
```


0 0 0 0 0 .. 0
0 87 240 210 24 .. 0
0 13 0 101 195 .. 0
0 35 167 99 210 .. 0
0 145 230 240 201 .. 140
... ...
0 0 0 0 0 .. 0
28 by 28 grid
0
0
0
0
0
...
0
0
87
240
210
24
0
...
0
x_1
x_2
x_3
x_{784}
Σ σ
0 0.29
1 0.07
2 0.92
3 0.08
4 0.001
9 0.043

```
model = keras.Sequential([
    keras.layers.Dense(10, input_shape=(784,), activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train_flattened, y_train, epochs=5)
```

```
model.evaluate(X_test_flattened, y_test)
```

```
y_predicted = model.predict(X_test_flattened)
y_predicted[0]
```

```
plt.matshow(X_test[0])
```

np.argmax finds a maximum element from an array and returns the index of it

```
np.argmax(y_predicted[0])
```

```
y_predicted_labels = [np.argmax(i) for i in y_predicted]
```

```
y_predicted_labels[:5]
```

```
cm = tf.math.confusion_matrix(labels=y_test,predictions=y_predicted_labels)
cm
```

```
import seaborn as sn
plt.figure(figsize = (10,7))
sn.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

USING a 100 neurons HIDDEN LAYER

```
model = keras.Sequential([
    keras.layers.Dense(100, input_shape=(784,), activation='relu'),
    keras.layers.Dense(10, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train_flattened, y_train, epochs=5)
```

EVALUATING THE MODELS

```
model.evaluate(X_test_flattened,y_test)
```

```
y_predicted = model.predict(X_test_flattened)
y_predicted_labels = [np.argmax(i) for i in y_predicted]
cm = tf.math.confusion_matrix(labels=y_test,predictions=y_predicted_labels)

plt.figure(figsize = (10,7))
sn.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

Using Flatten layer so that we don't have to call .reshape on input dataset

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dense(10, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=10)
```

```
model.evaluate(X_test,y_test)
```

TensorFlow callback

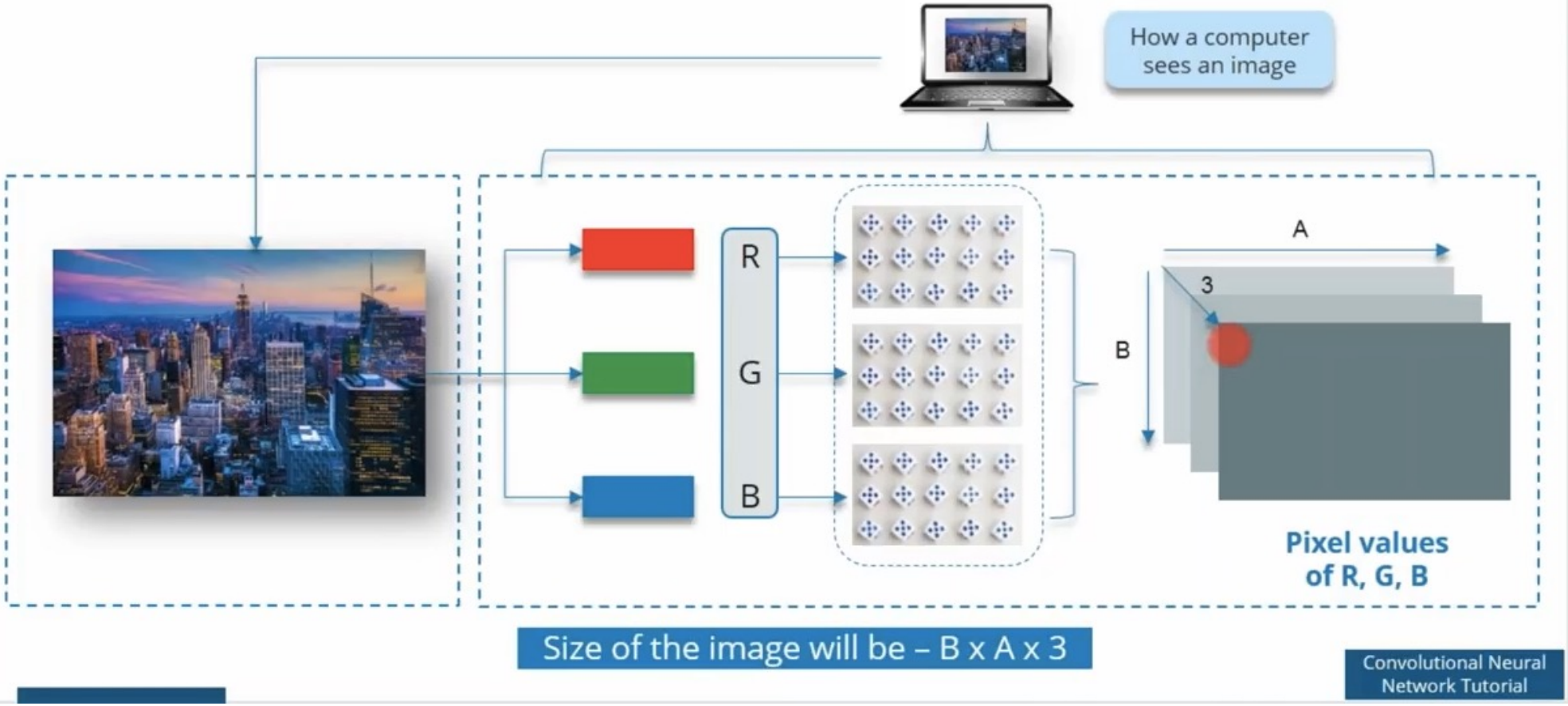
TensorFlow callbacks are functions or blocks of code which are executed during a specific instant while training a Deep Learning Model1. Available callbacks in TensorFlow 2.0 include2:

- EarlyStopping: This callback is used very often.
- ModelCheckpoint: This callback allows us to save the model regularly during training.
- TensorBoard: This is one of the best callbacks if you want to visualize the training summary for your model.
- LearningRateScheduler
- CSVLogger
- LambdaCallback
- ReduceLROnPlateau
- RemoteMonitor

CONVOLUTIONNAL NEURAL NETWORK
CNN
Ref: EDUREKA

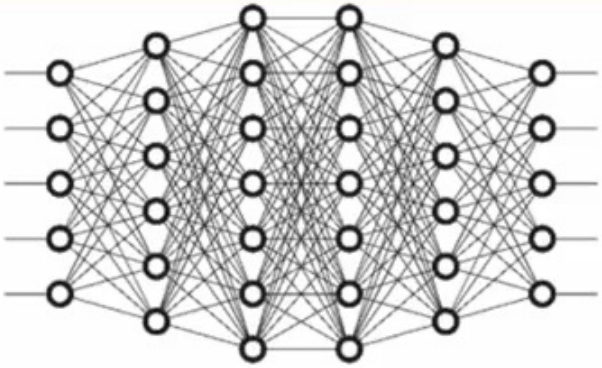
DEEP LEARNING WITH CNN

How a Computer Reads an Image



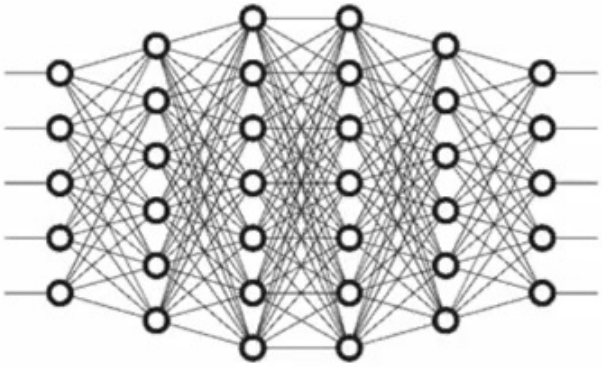
Why Not Fully Connected Networks

Image with 28 x 28 x 3 pixels



Number of weights in the first hidden layer will be 2352

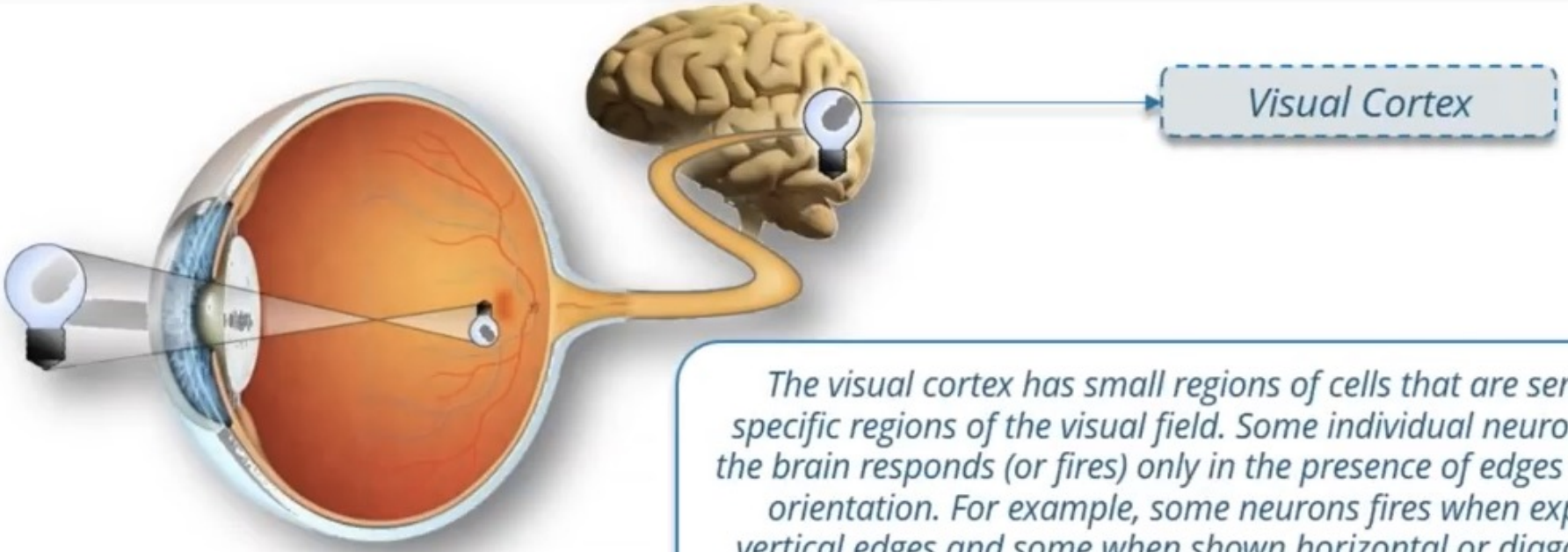
Image with 200 x 200 x 3 pixels



Number of weights in the first hidden layer will be 120,000

What is Convolutional Neural Network?

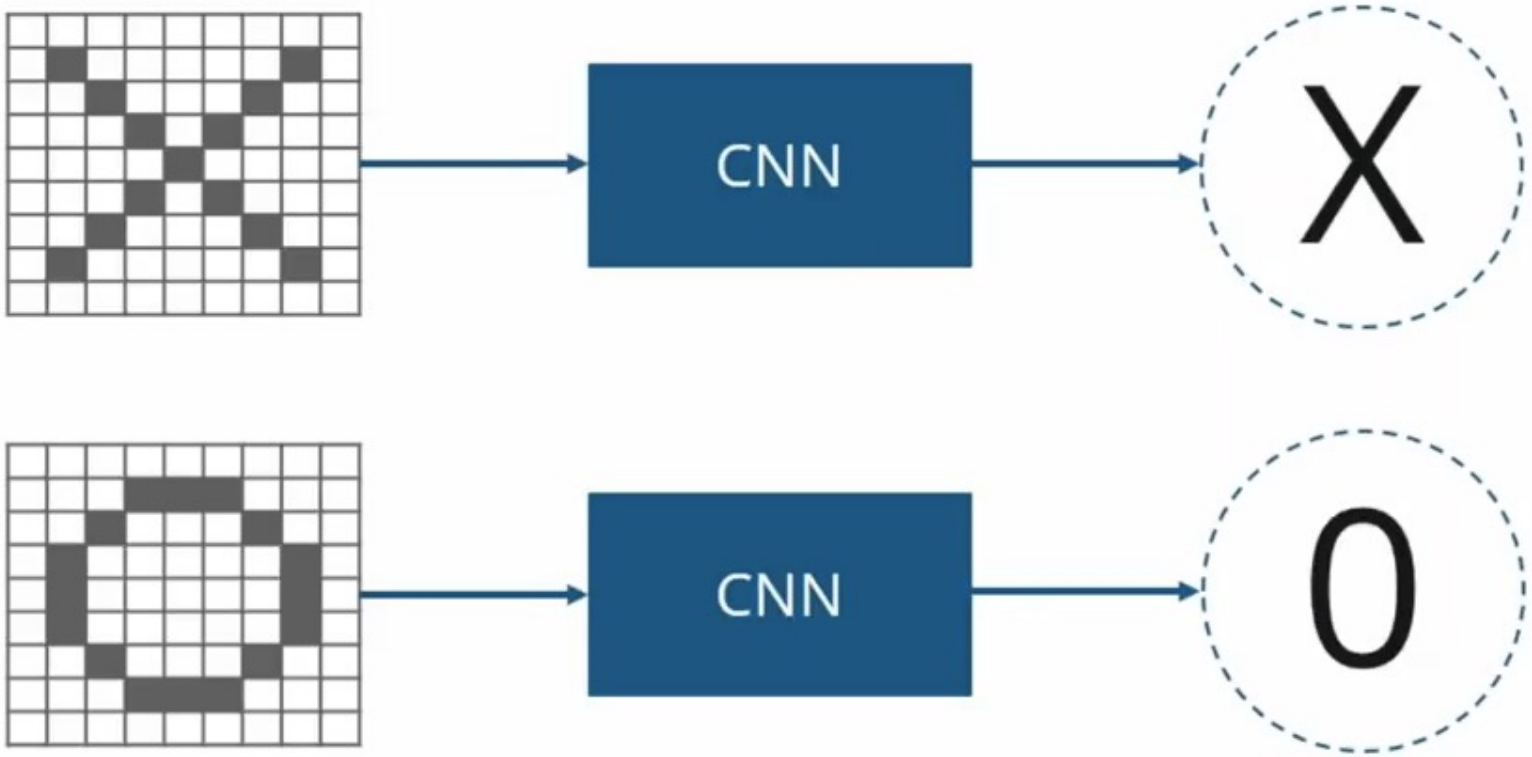
***Convolutional Neural Network** (**CNN**, or **ConvNet**)* is a type of feed-forward artificial neural network in which the connectivity pattern between its neurons is inspired by the organization of the animal visual cortex.



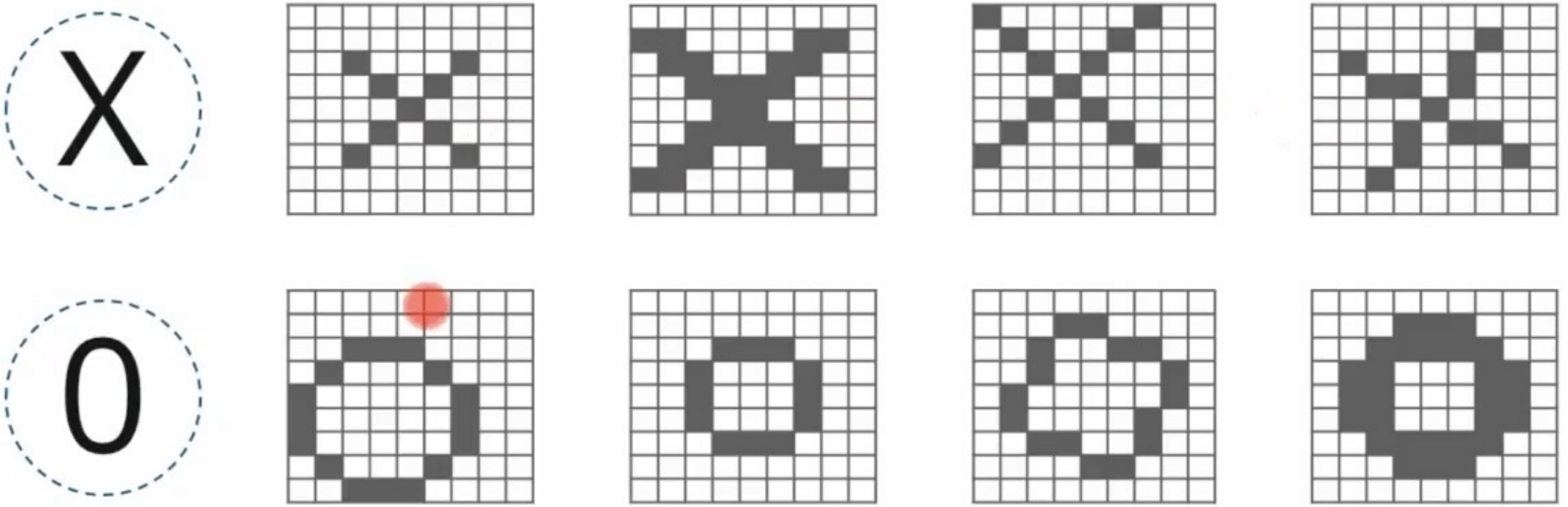
The visual cortex has small regions of cells that are sensitive to specific regions of the visual field. Some individual neuronal cells in the brain responds (or fires) only in the presence of edges of a certain orientation. For example, some neurons fires when exposed to vertical edges and some when shown horizontal or diagonal edges

Convolutional Neural Networks have following layers:

- ✓ Convolution
- ✓ ReLU Layer
- ✓ Pooling
- ✓ Fully Connected



Here, we will have some problems, because X and O images won't always have the same images.
There can be certain deformations. Consider the diagrams shown below:

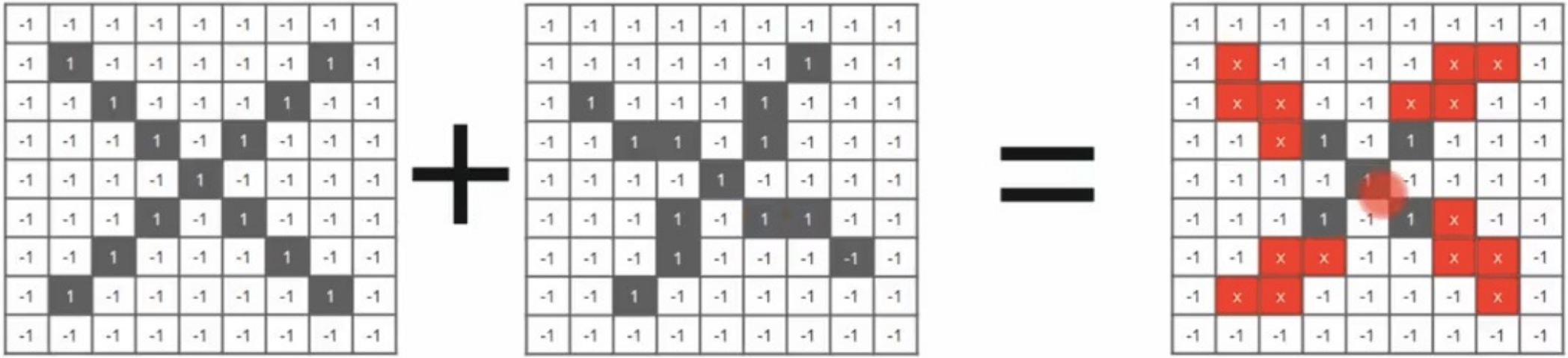


A computer understands an image using numbers at each pixels.
In our example, we have considered that a black pixel will have value 1 and a white pixel will have -1 value.

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

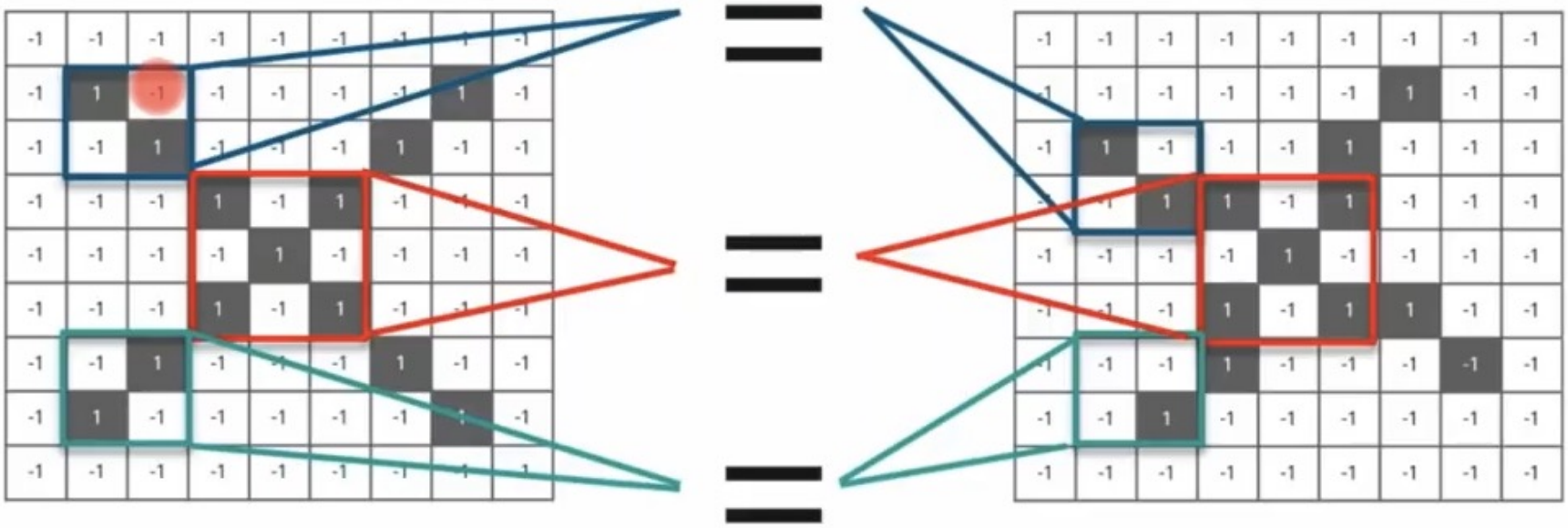
Convolutional Neural Network Tutorial

Using normal techniques, computers compare these images as:



How CNN Works?

CNN compares the images piece by piece. The pieces that it looks for are called features.
By finding rough feature matches, in roughly the same position in two images, CNN gets a lot better at seeing similarity than whole-image matching schemes.



How CNN Works?

1	-1	-1
-1	1	-1
-1	-1	1

1	-1	1
-1	1	-1
1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1

These are small pieces of the bigger image. We choose a feature and put it on the input image, if it matches then the image is classified correctly.

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

edureka!

Copyright © 2017, edureka and/or its affiliates. All rights reserved.

How CNN Works?

1	-1	-1
-1	1	-1
-1	-1	1

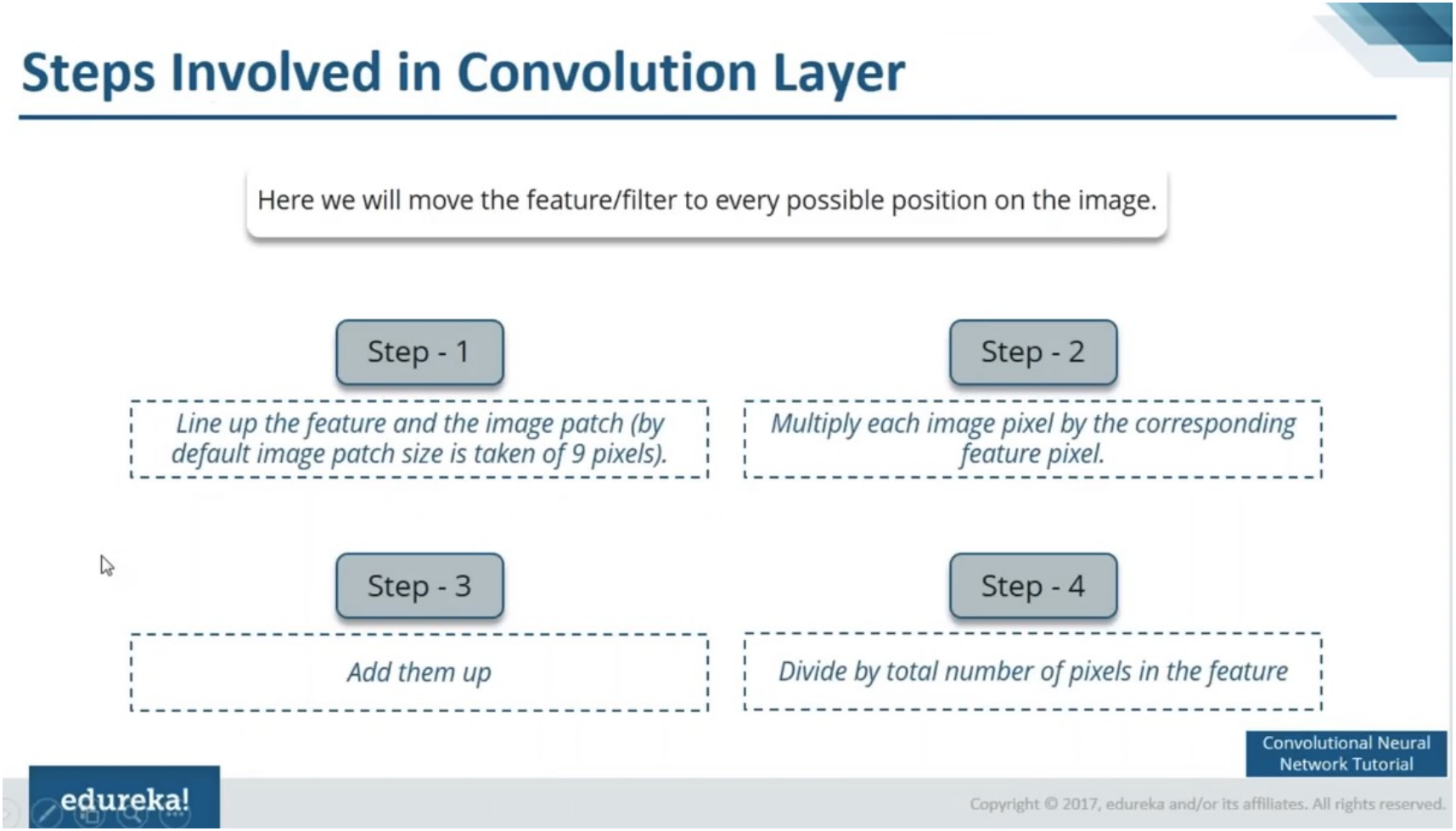
1	-1	1
-1	1	-1
1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1

These are small pieces of the bigger image. We choose a feature and put it on the input image, if it matches then the image is classified correctly.

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

Copyright © 2017, edureka and/or its affiliates. All rights reserved.

- 
Steps Involved in Convolution Layer
Here we will move the feature/filter to every possible position on the image.
Step - 1
Line up the feature and the image patch (by default image patch size is taken of 9 pixels).
Step - 2
Multiply each image pixel by the corresponding feature pixel.
Step - 3
Add them up
Step - 4
Divide by total number of pixels in the feature
Convolutional Neural Network Tutorial
edureka!
Copyright © 2017, edureka and/or its affiliates. All rights reserved.

Adding and Dividing by the Total Number of Pixels

1	-1	-1
-1	1	-1
-1	-1	1

$$\frac{1+1+1+1+1+1+1+1+1}{9} = 1$$

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

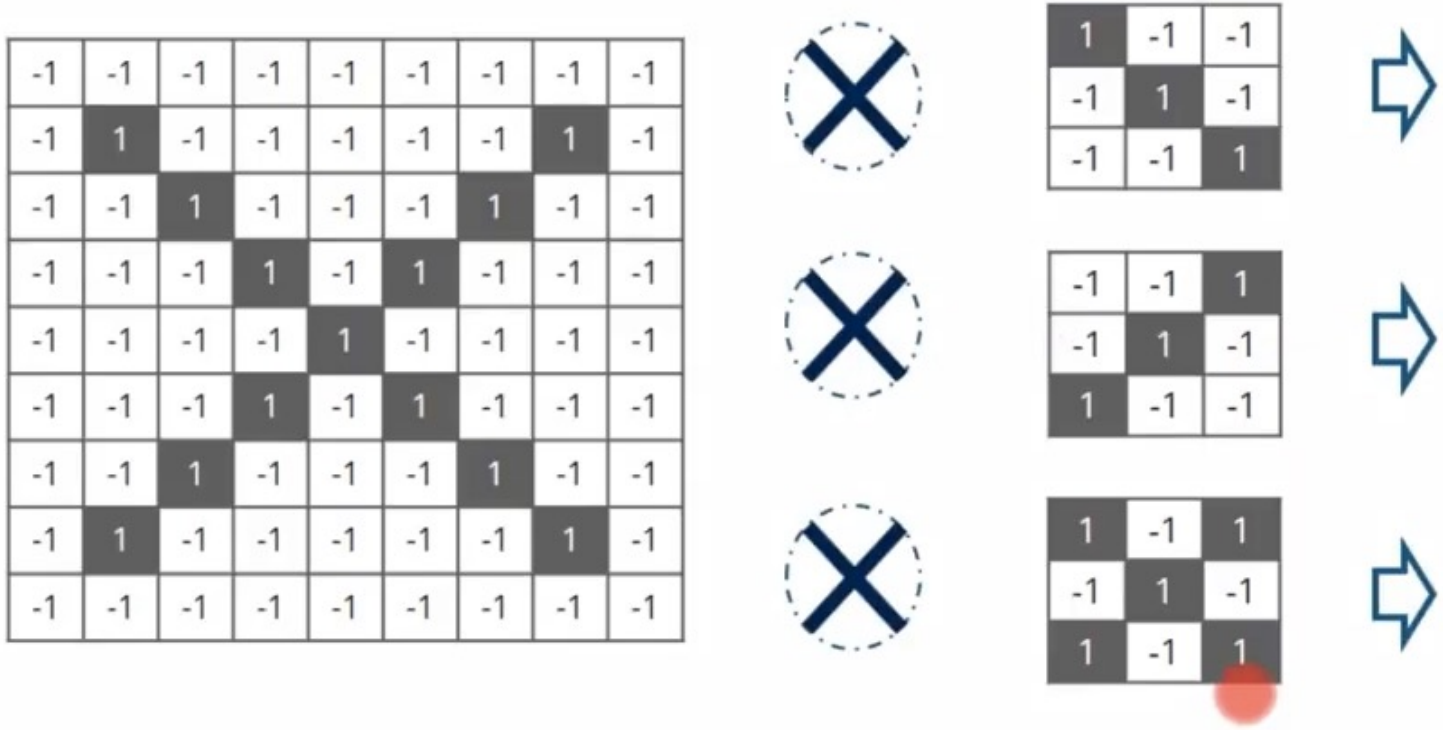
1	1	1
1	1	1
1	1	1

edureka!

Copyright © 2017, edureka and/or its affiliates. All rights reserved.

Convolution Layer Output

Similarly, we will perform the same convolution with every other filters



-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

1	-1	-1
-1	1	-1
-1	-1	1

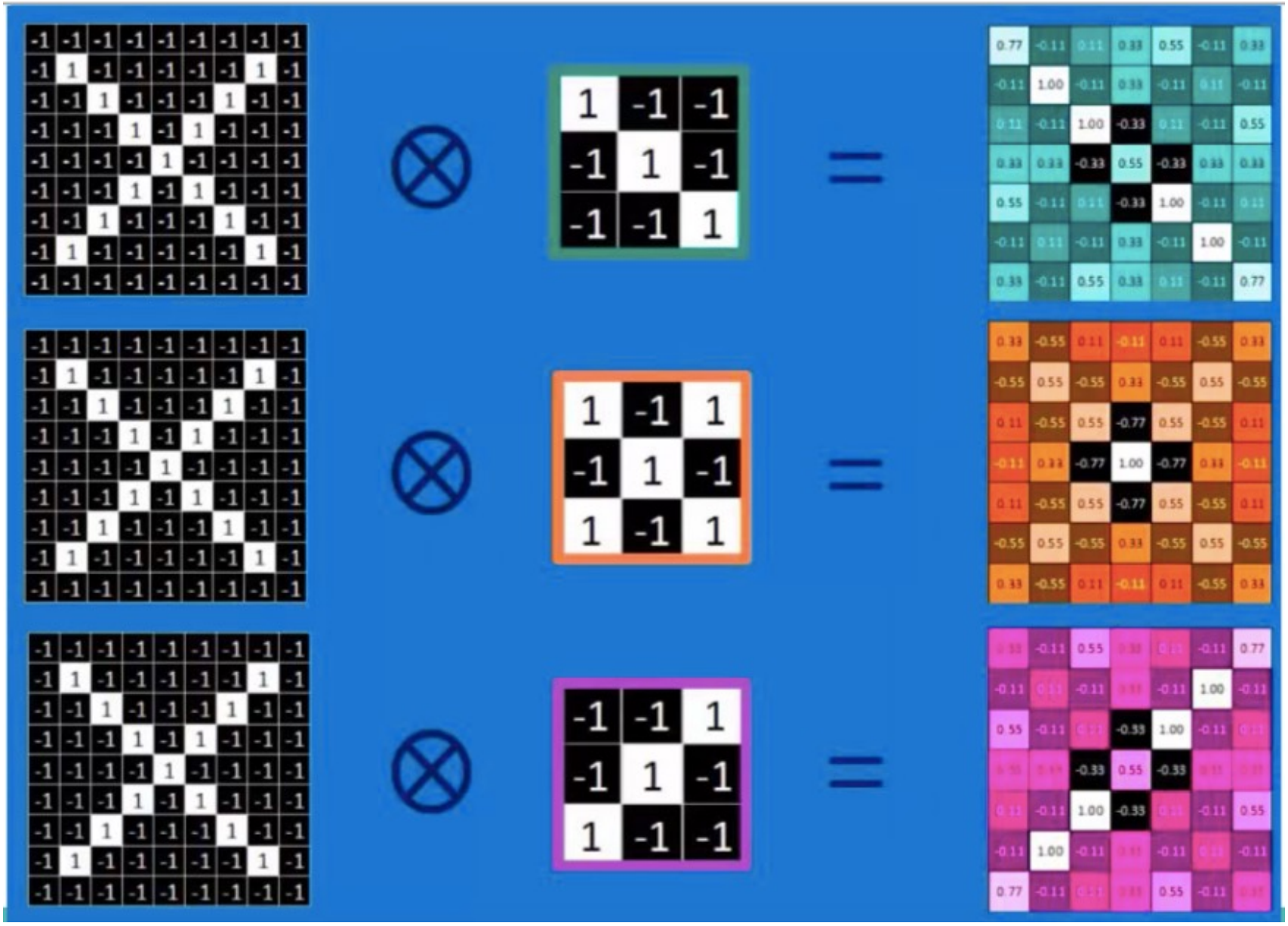
-1	-1	1
-1	1	-1
1	-1	-1

1	-1	1
-1	1	-1
1	-1	1

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.11	-0.55	0.55	-0.11	0.55	-0.55	0.11
-0.11	0.33	-0.77	1.00	-0.77	0.33	-0.11
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33

0.33	-0.11	0.55	0.33	0.11	-0.11	0.77
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.77	-0.11	0.11	0.33	0.55	-0.11	0.33



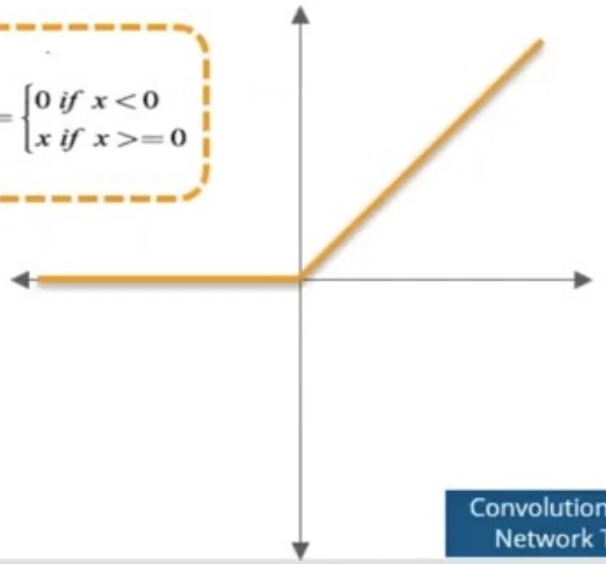
ReLU Layer

- ✓ In this layer we remove every negative values from the filtered images and replaces it with zero's
- ✓ This is done to avoid the values from summing up to zero

Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable

x	f(x)=x	F(x)
-3	f(-3) = 0	0
-5	f(-5) = 0	0
3	f(3) = 3	3
5	f(5) = 5	5

$$f(x) = \begin{cases} 0 \text{ if } x < 0 \\ x \text{ if } x >= 0 \end{cases}$$



Copyright © 2017, edureka and/or its affiliates. All rights reserved.

Output for One Feature

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

Output for All Features

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.11	-0.55	0.55	-0.11	0.55	-0.55	0.11
-0.11	0.33	-0.77	1.00	-0.77	0.33	-0.11
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33

0.33	0	0.11	0	0.11	0	0.33
0	0.55	0	0.33	0	0.55	0
0.11	0	0.55	0	0.55	0	0.11
0	0.33	0	1.00	0	0.33	0
0.11	0	0.55	0	0.55	0	0.11
0	0.55	0	0.33	0	0.55	0
0.33	0	0.11	0	0.11	0	0.33

0.33	-0.11	0.55	0.33	0.11	-0.11	0.77
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.77	-0.11	0.11	0.33	0.55	-0.11	0.33

0.33	0	0.55	0.33	0.11	0	0.77
0	0.11	0	0.33	0	1.00	0
0.55	0	0.11	0	1.00	0	0.11
0.33	0.33	0	0.55	0	0.33	0.33
0.11	0	1.00	0	0.11	0	0.55
0	1.00	0	0.33	0	0.11	0
0.77	0	0.11	0.33	0.55	0	0.33

POOLING LAYER

In this layer we shrink the image stack into a smaller size

Steps:

1. Pick a window size (usually 2 or 3).
2. Pick a stride (usually 2).
3. Walk your window across your filtered images.
4. From each window, take the maximum value.

Symbol:

Let's perform pooling with a window size 2 and a stride 2

Let's start with our first filtered image
In our first window the maximum or highest value is 1, so we track that and move the window two strides

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

1			

STRIDE & PADDING

Stride: is the number of pixels shifts over the input matrix. When the stride is 1 then we move the filters to 1 pixel at a time. When the stride is 2 then we move the filters to 2 pixels at a time and so on.

Padding Sometimes filter does not fit perfectly fit the input image. We have two options:

- Pad the picture with zeros (zero-padding) so that it fits
- Drop the part of the image where the filter did not fit. This is called valid padding which keeps only valid part of the image.

Moving the Window Across the Entire Image

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

Output After Passing Through Pooling Layer

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77

1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.33	0	0.11	0	0.11	0	0.33
0	0.55	0	0.33	0	0.55	0
0.11	0	0.55	0	0.55	0	0.11
0	0.33	0	1.00	0	0.33	0
0.11	0	0.55	0	0.55	0	0.11
0	0.55	0	0.33	0	0.55	0
0.33	0	0.11	0	0.11	0	0.33

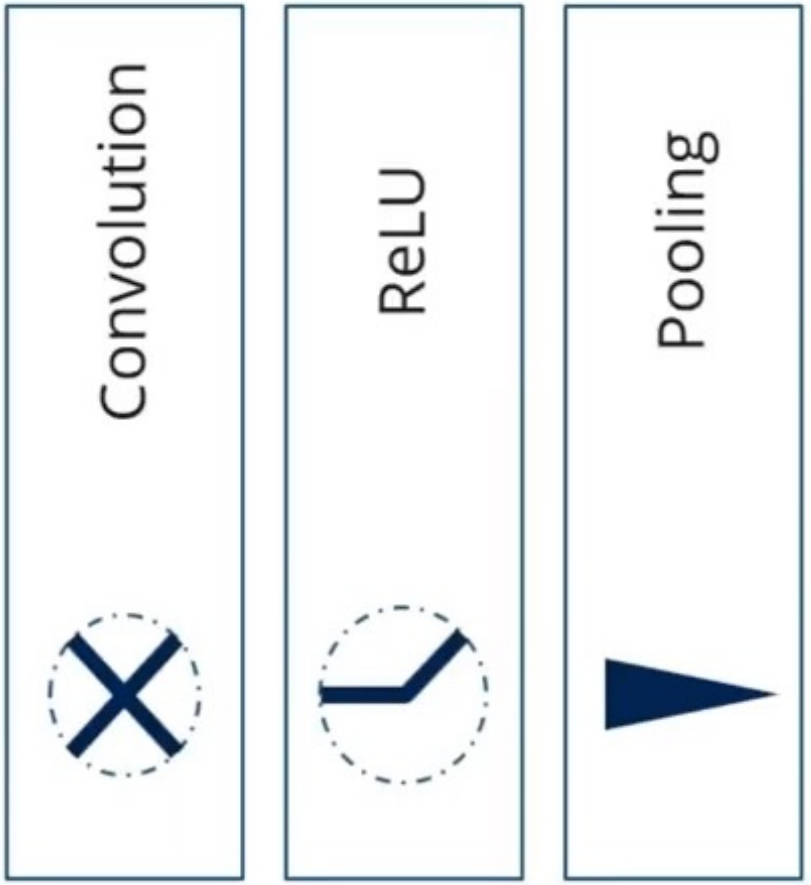
0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

0.33	0	0.55	0.33	0.11	0	0.77
0	0.11	0	0.33	0	1.00	0
0.55	0	0.11	0	1.00	0	0.11
0.33	0.33	0	0.55	0	0.33	0.33
0.11	0	1.00	0	0.11	0	0.55
0	1.00	0	0.33	0	0.11	0
0.77	0	0.11	0.33	0.55	0	0.33

0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

Stacking up the Layers

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

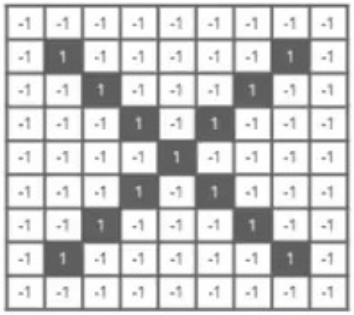


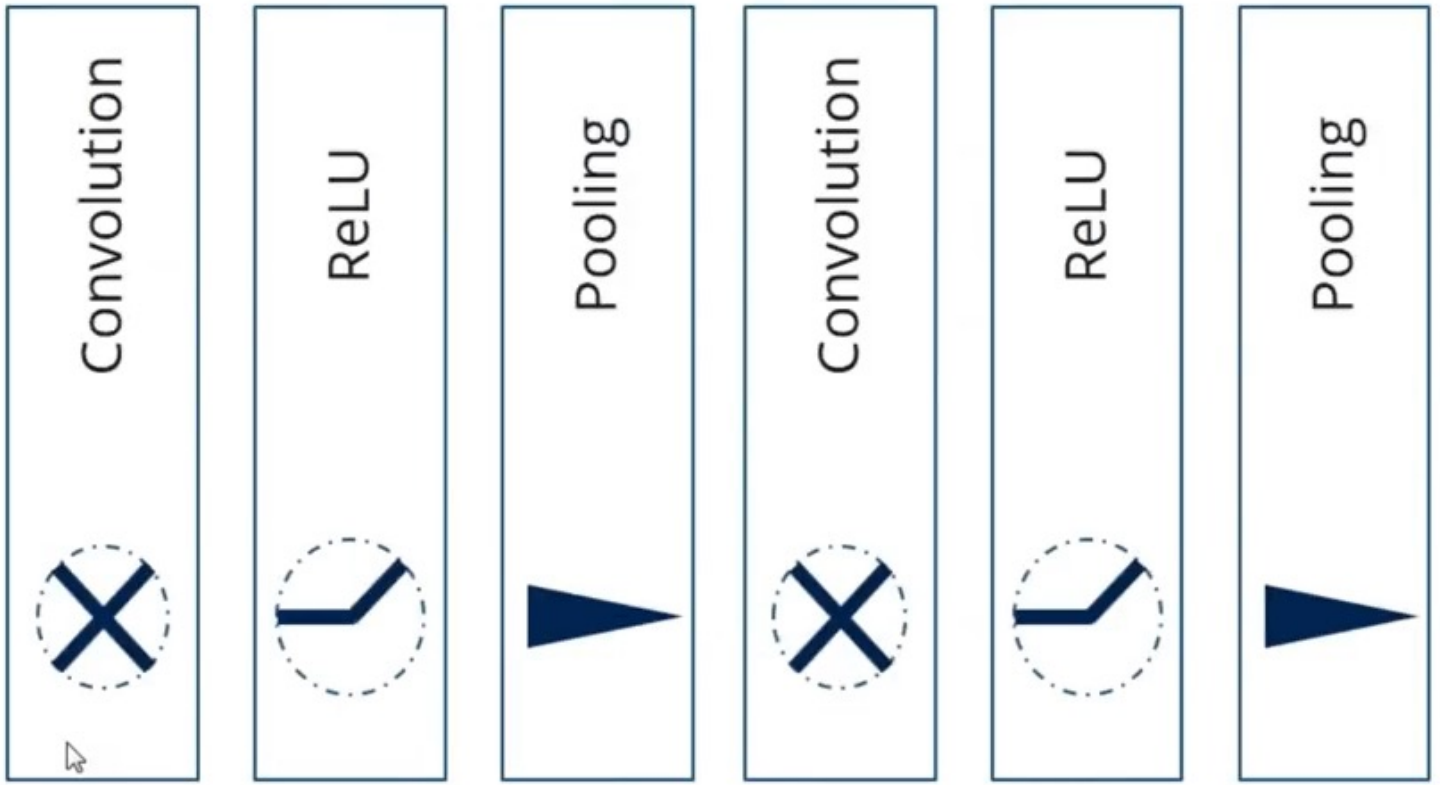
1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

Stacking up the Layers





1	0.55
0.55	1.00

1	0.55
0.55	0.55

0.55	1.00
1.00	0.55

Fully Connected Layer

This is the final layer where the actual classification happens

Here we take our filtered and shrinked images and put them into a single list

1	0.55
0.55	1.00

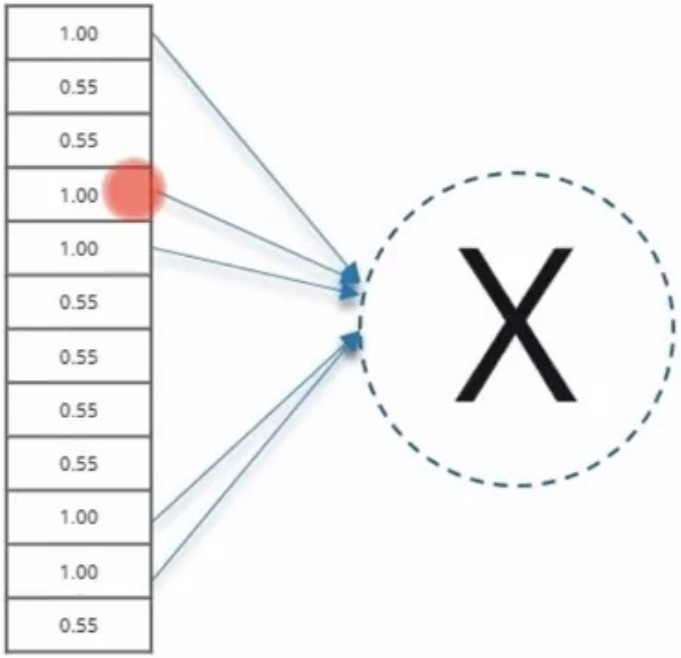
1	0.55
0.55	0.55

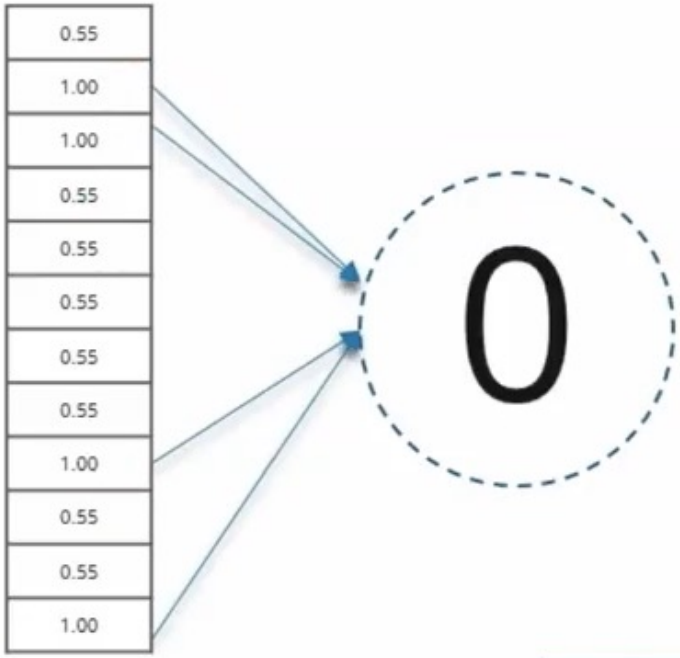
0.55	1.00
1.00	0.55

1.00
0.55
0.55
1.00
1.00
0.55
0.55
0.55
0.55
1.00
1.00
0.55

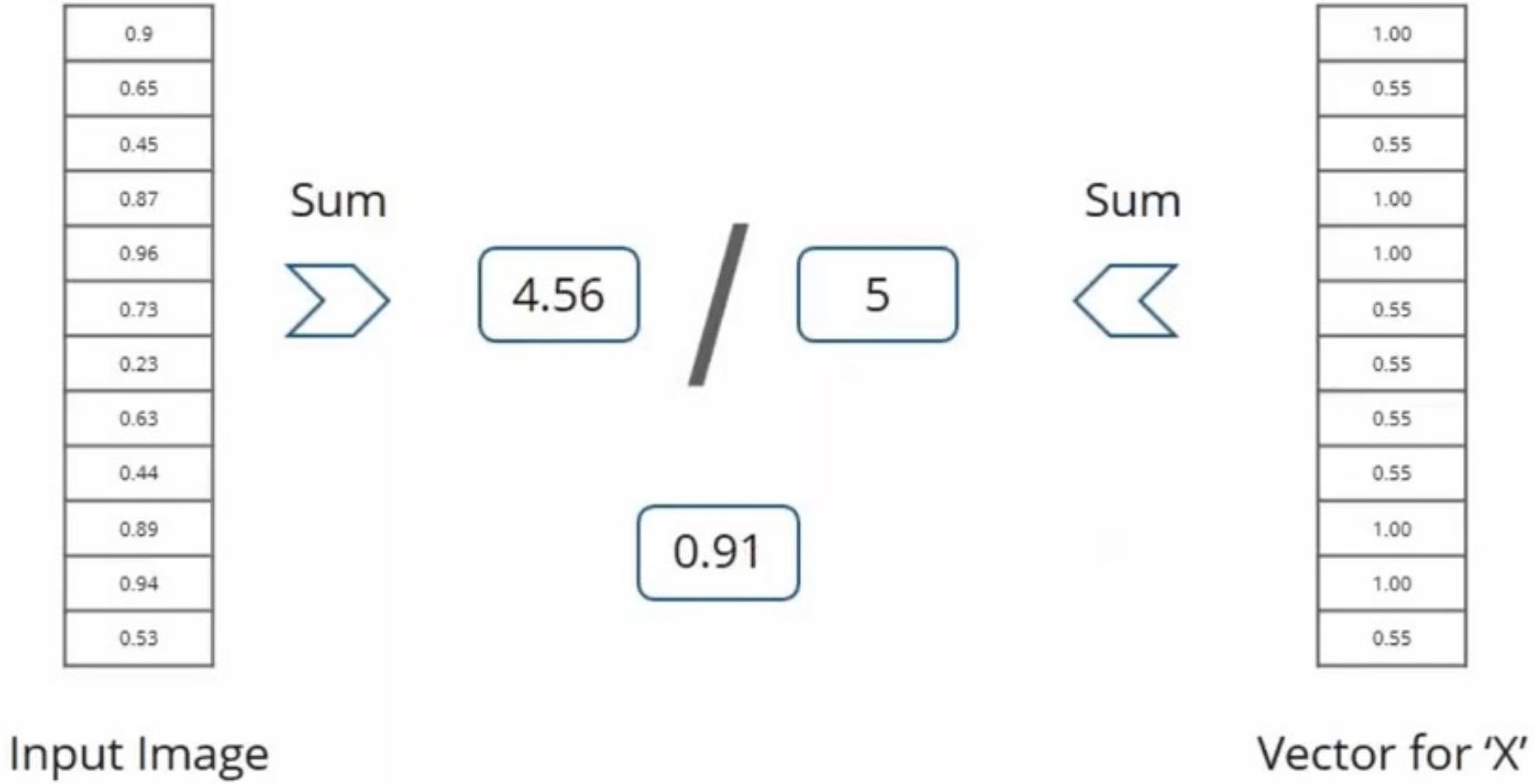
Output

When we feed in, 'X' and 'O'. Then there will be some element in the vector that will be high. Consider the image below, as you can see for 'X' there are different elements that are high and similarly, for 'O' we have different elements that are high.

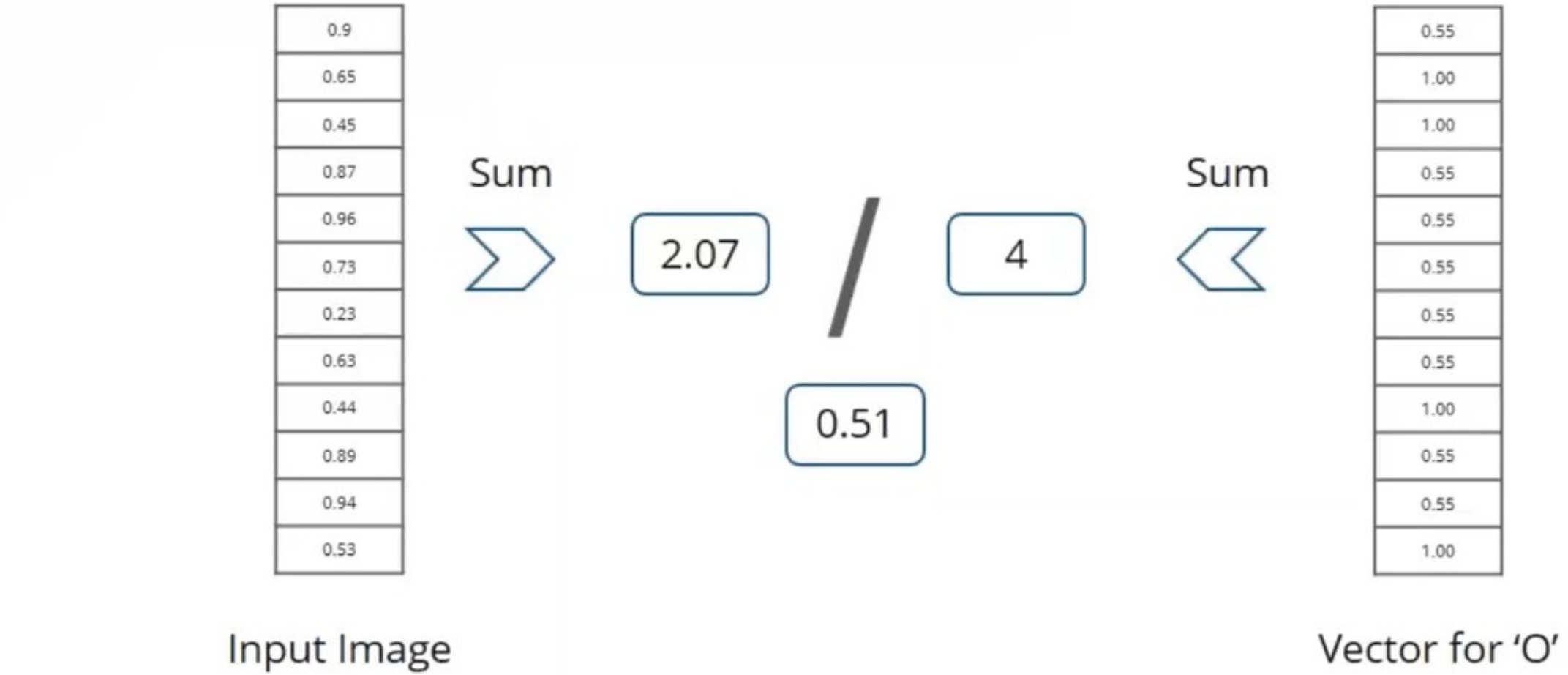




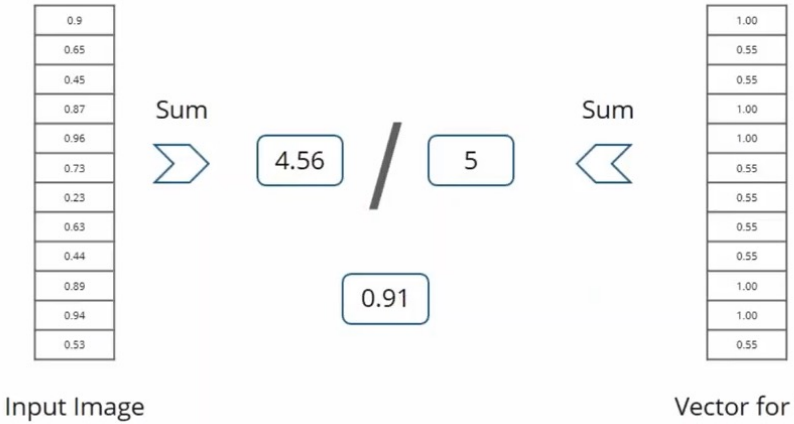
Comparing the Input Vector with X



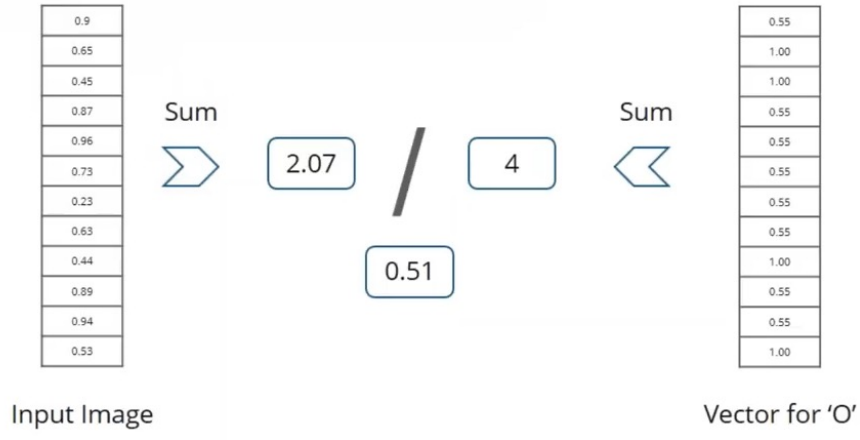
Comparing the Input Vector with O



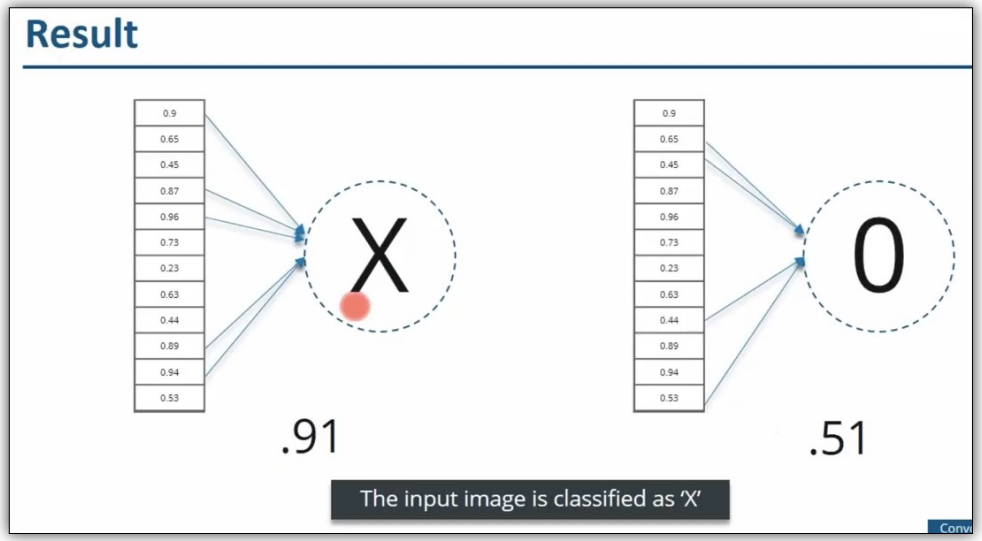
Comparing the Input Vector with X



Comparing the Input Vector with O



Result



USE CASE

Dog and Cat Identifier

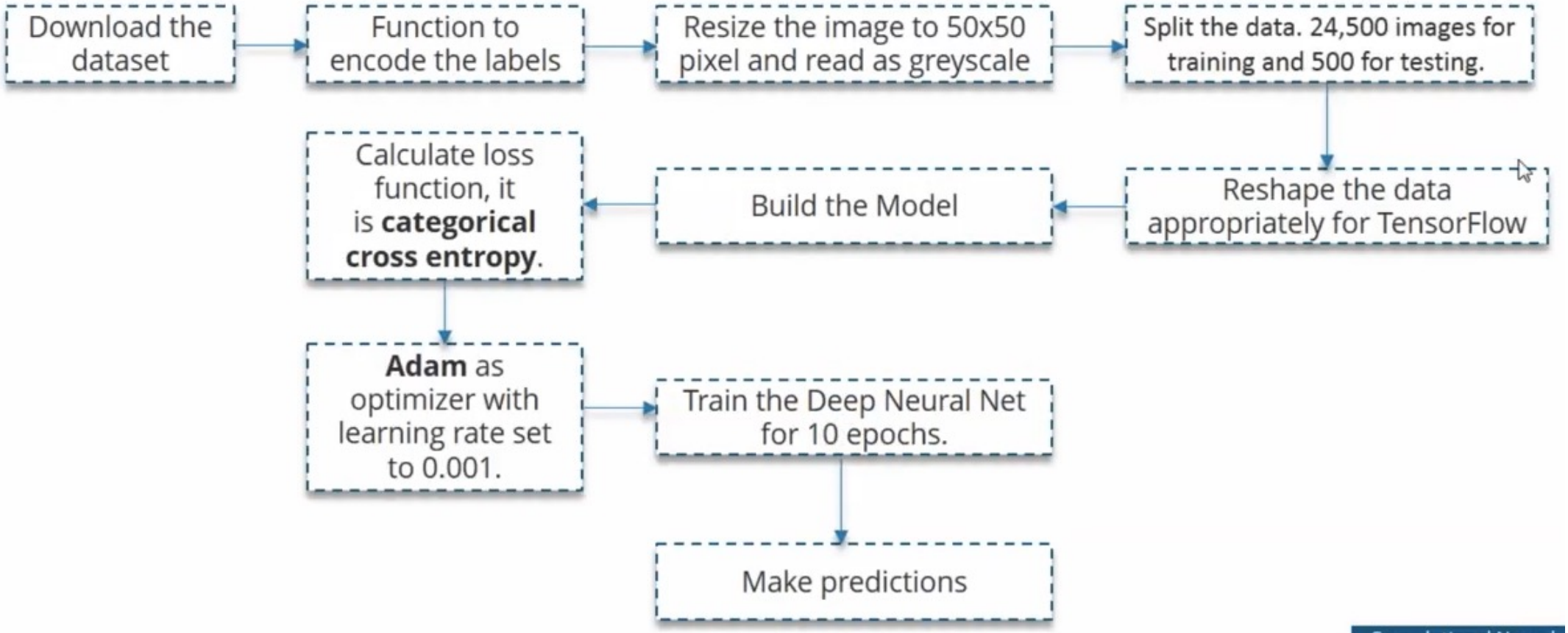
Training

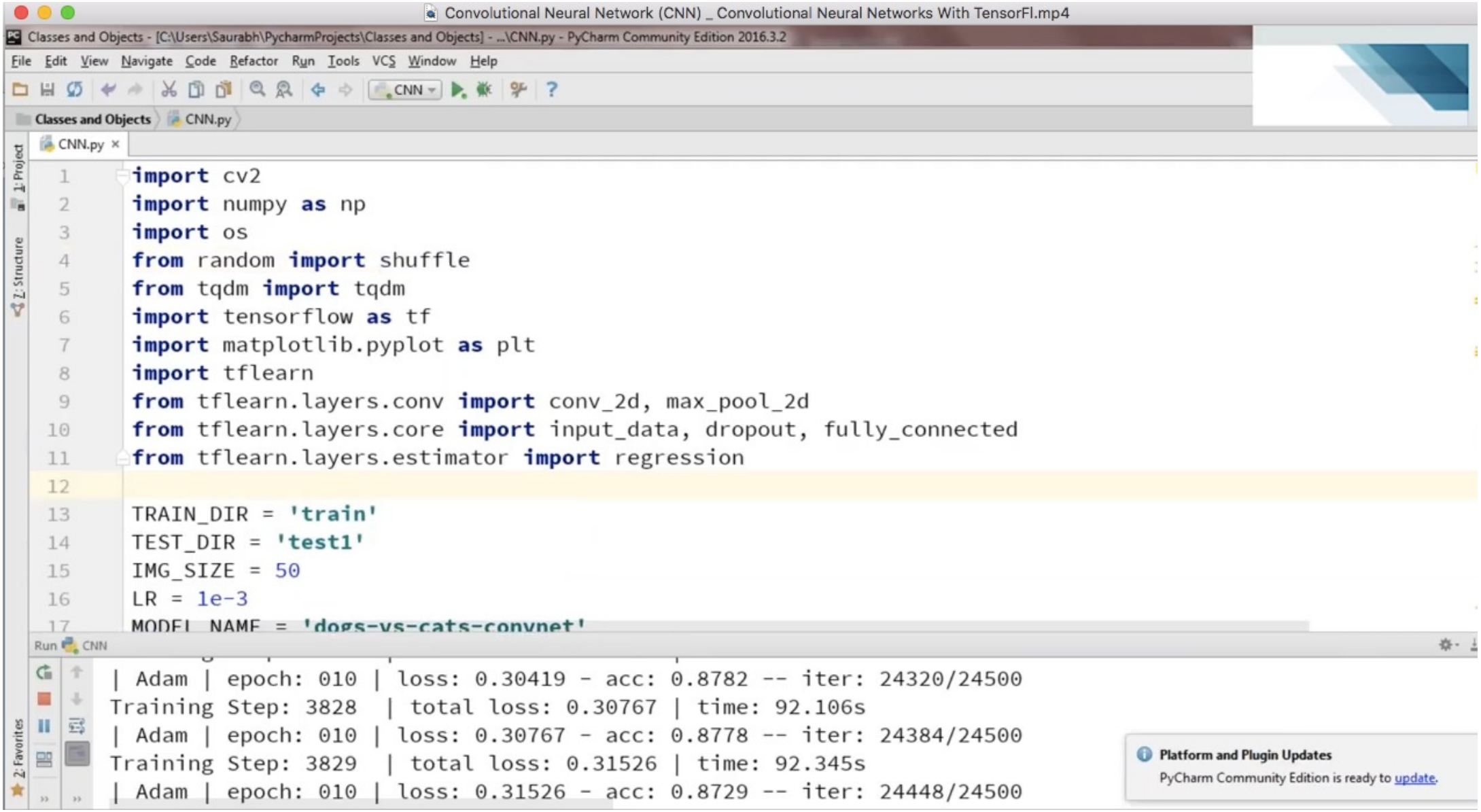
Input

Convolutional Neural Network

DOG

Implementing the Use-Case




Convolutional Neural Network (CNN) _ Convolutional Neural Networks With TensorFl.mp4
Classes and Objects - [C:\Users\Saurabh\PycharmProjects\Classes and Objects] - ...\CNN.py - PyCharm Community Edition 2016.3.2
File Edit View Navigate Code Refactor Run Tools VCS Window Help
CNN
Classes and Objects
CNN.py
CNN.py
1: Project
2: Favorites
Z: Structure
import cv2
import numpy as np
import os
from random import shuffle
from tqdm import tqdm
import tensorflow as tf
import matplotlib.pyplot as plt
import tflearn
from tflearn.layers.conv import conv_2d, max_pool_2d
from tflearn.layers.core import input_data, dropout, fully_connected
from tflearn.layers.estimator import regression
TRAIN_DIR = 'train'
TEST_DIR = 'test1'
IMG_SIZE = 50
LR = 1e-3
MODEL_NAME = 'dogs-vs-cats-convnet'
Run CNN
| Adam | epoch: 010 | loss: 0.30419 - acc: 0.8782 -- iter: 24320/24500
Training Step: 3828 | total loss: 0.30767 | time: 92.106s
| Adam | epoch: 010 | loss: 0.30767 - acc: 0.8778 -- iter: 24384/24500
Training Step: 3829 | total loss: 0.31526 | time: 92.345s
| Adam | epoch: 010 | loss: 0.31526 - acc: 0.8729 -- iter: 24448/24500
Platform and Plugin Updates
PyCharm Community Edition is ready to update.

Cat
Cat
Dog
Dog
Cat
Dog
Cat
Cat
Cat
Dog
Dog
Cat
Dog
Cat
Dog
Dog

CONVOLUTIONNAL NEURAL NETWORK

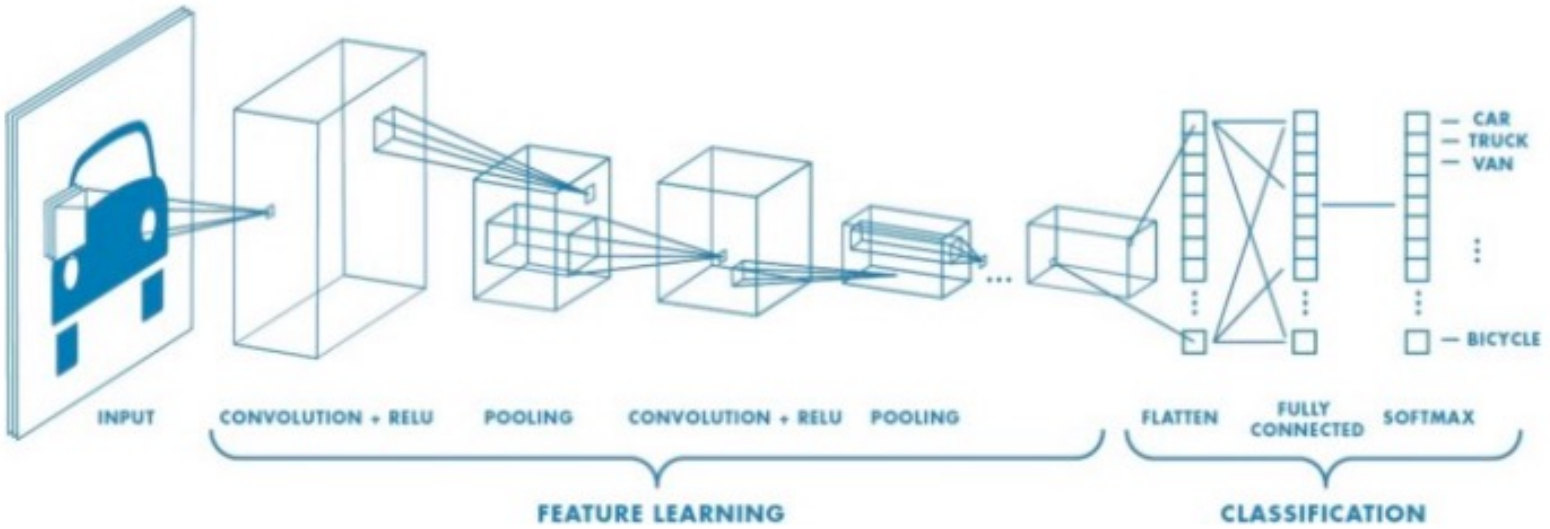


Figure 2 : Neural network with many convolutional layers

Hyperparameters (knobs)

- Convolution
  - Number of features
  - Size of features
- Pooling
  - Window size
  - Window stride
- Fully Connected
  - Number of neurons

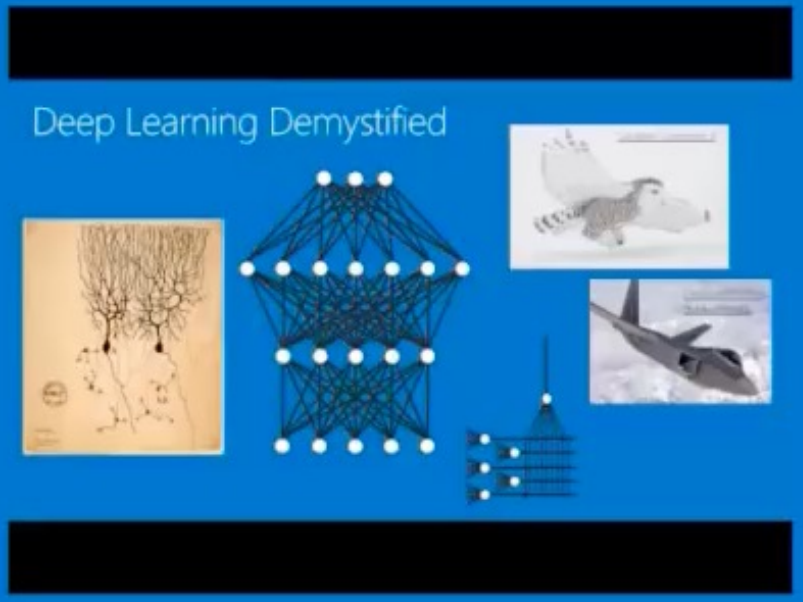
Limitations

ConvNets only capture local "spatial" patterns in data.

If the data can't be made to look like an image, ConvNets are less useful.

Also check out

Deep Learning Demystified



Notes from Stanford CS 231 course
(Justin Johnson and Andrej Karpathy)

The writings of Christopher Olah

Entrée [1]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np
```

```
/Users/christian/anaconda3/lib/python3.6/site-packages/tensorflow/python/framework/dtypes.py:516: FutureWarning: Pass
ing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will be understood as
(type, (1,)) / '(1,)type'.
  _np_qint8 = np.dtype([("qint8", np.int8, 1)])
/Users/christian/anaconda3/lib/python3.6/site-packages/tensorflow/python/framework/dtypes.py:517: FutureWarning: Pass
```

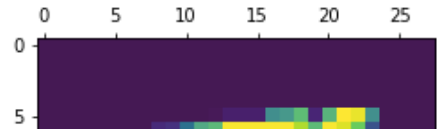
Entrée [2]:
```
(X_train, y_train) , (X_test, y_test) = keras.datasets.mnist.load_data()
```

Entrée [3]:
```
X_train.shape
```
Out[3]:
```
(60000, 28, 28)
```

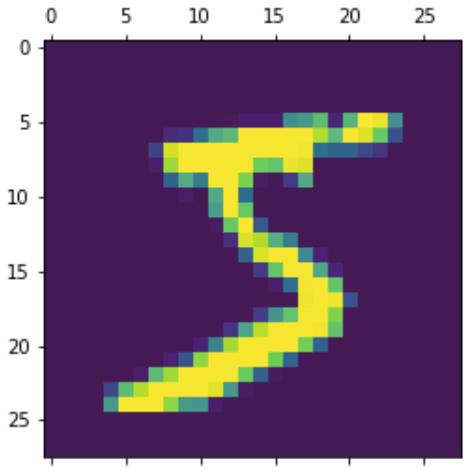
Entrée [4]:
```
X_test.shape
```
Out[4]:
```
(10000, 28, 28)
```

Entrée [5]:
```
X_train[0].shape
```
Out[5]:
```
(28, 28)
```

Entrée [6]:
```
plt.matshow(X_train[0])
```
Out[6]:
```
<matplotlib.image.AxesImage at 0x7fb1b6e481d0>
```



Entrée [6]:
```
plt.matshow(X_train[0])
```
Out[6]:
```
<matplotlib.image.AxesImage at 0x7fb1b6e481d0>
```



```
Entrée [7]: y_train[0]
```

```
Out[7]: 5
```

```
Entrée [8]: X_train = X_train / 255
            X_test = X_test / 255
```

Using ANN for classification

```
Entrée [10]: model = keras.Sequential([
                 keras.layers.Flatten(input_shape=(28, 28)),
                 keras.layers.Dense(100, activation='relu'),
                 keras.layers.Dense(10, activation='sigmoid')
             ])

             model.compile(optimizer='adam',
                           loss='sparse_categorical_crossentropy',
                           metrics=['accuracy'])

             model.fit(X_train, y_train, epochs=10)
```

```
Epoch 1/10
1875/1875 [==============================] - 2s 924us/step - loss: 0.2885 - accuracy: 0.9194
Epoch 2/10
1875/1875 [==============================] - 2s 920us/step - loss: 0.1363 - accuracy: 0.9603
Epoch 3/10
1875/1875 [==============================] - 2s 925us/step - loss: 0.0993 - accuracy: 0.9704
```

```
Entrée [11]: model.evaluate(X_test,y_test)

             313/313 [==============================] - 0s 988us/step - loss: 0.0831 - accuracy: 0.9780

  Out[11]: [0.0830635279417038, 0.9779999852180481]
```

```
Entrée [11]: X_train = X_train.reshape(-1,28,28,1)
             X_train.shape

  Out[11]: (60000, 28, 28, 1)
```

```
Entrée [12]: X_test = X_test.reshape(-1,28,28,1)
             X_test.shape

  Out[12]: (10000, 28, 28, 1)
```

Using CNN for classification

Entrée [13]:
```
model = keras.Sequential([

    layers.Conv2D(30, (3,3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(100, activation='relu'),
    keras.layers.Dense(10, activation='sigmoid')
])
```

Entrée [*]:
```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=5)
```

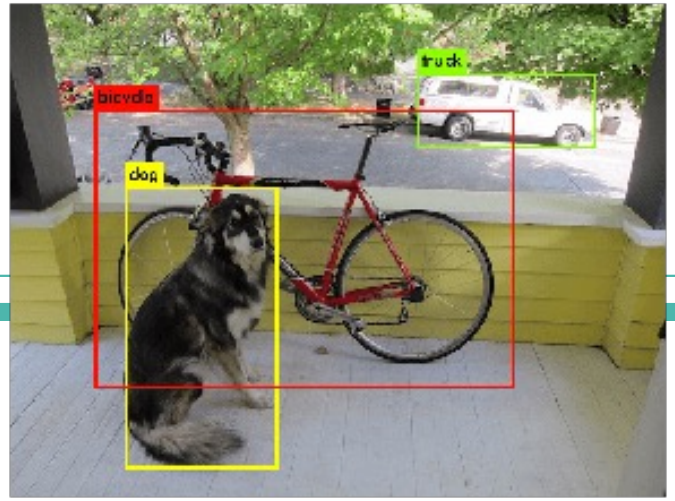
```
WARNING:tensorflow:From /Users/christian/anaconda3/lib/python3.6/site-packages/tensorflow/python/ops/math_grad.py:125
0: add_dispatch_support.<locals>.wrapper (from tensorflow.python.ops.array_ops) is deprecated and will be removed in
a future version.
Instructions for updating:
Use tf.where in 2.0, which has the same broadcast rule as np.where
Epoch 1/5
60000/60000 [==============================] - 157s 3ms/sample - loss: 0.1841 - acc: 0.9472
Epoch 2/5
60000/60000 [==============================] - 159s 3ms/sample - loss: 0.0642 - acc: 0.9806
Epoch 3/5
 6112/60000 [==>...........................] - ETA: 1:56 - loss: 0.0330 - acc: 0.9907
```

```
60000/60000 [==============================] - 159s 3ms/sample - loss: 0.0642 - acc: 0.9806
Epoch 3/5
28704/60000 [=============>................] - ETA: 1:14 - loss: 0.0418 - acc: 0.9878
```

```
Entrée [61]: y_train[:5]

  Out[61]: array([5, 0, 4, 1, 9], dtype=uint8)
```

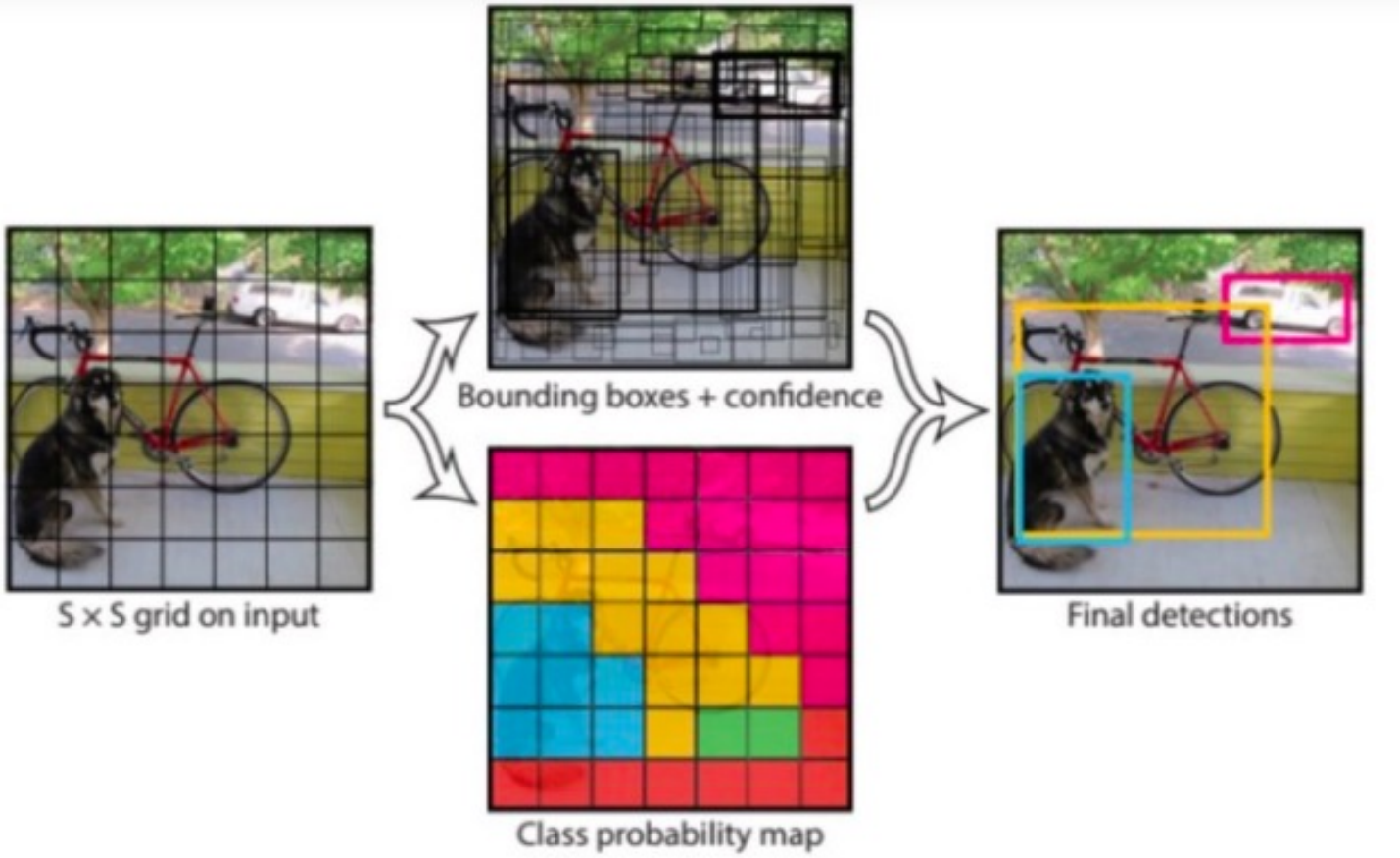
```
Entrée [56]: model.evaluate(X_test,y_test)

313/313 [==============================] - 0s 1ms/step - loss: 0.0541 - accuracy: 0.9843

  Out[56]: [0.05414153262972832, 0.9843000173568726]
```

YOLO – You only look once

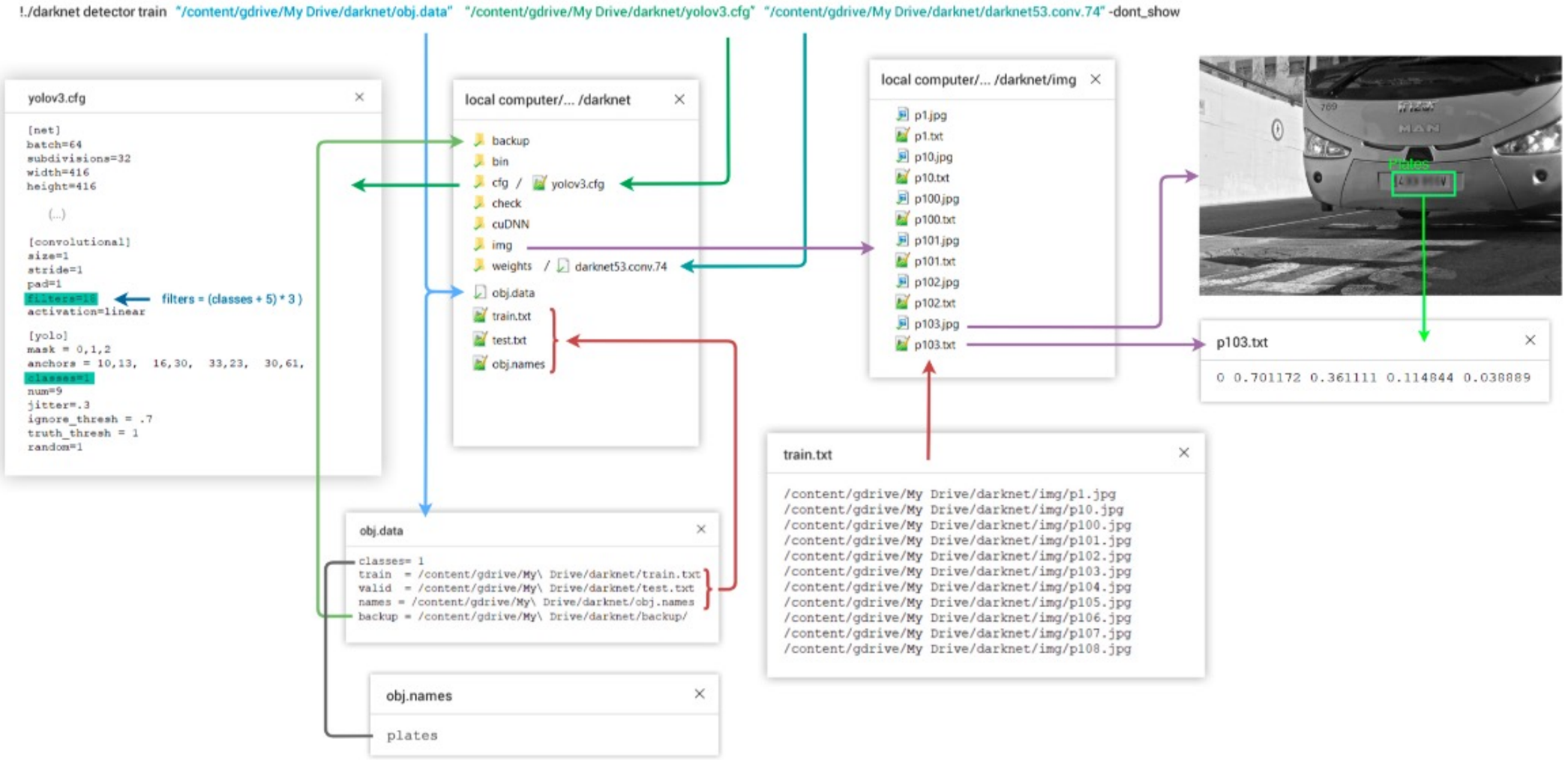


ASSETS

- Speed (45 frames per second — better than realtime)
- Network understands generalized object representation (This allowed them to train the network on real world images and predictions on artwork was still fairly accurate).
- faster version (with smaller architecture) — 155 frames per sec but is less accurate.
- open source: https://pjreddie.com/darknet/yolo/

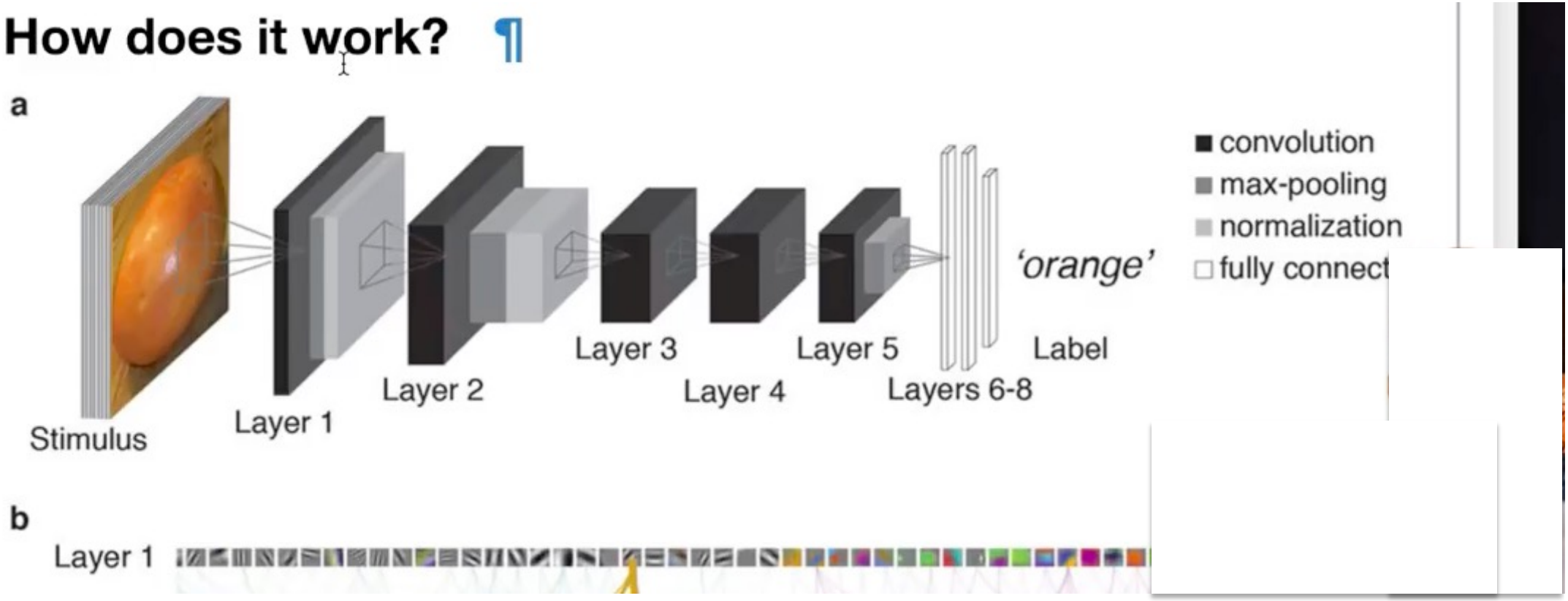

Bounding boxes + confidence
S x S grid on input
Class probability map
Final detections

Darknet. YOLOv3 Configuration files on colab notebook



Siraj Raval

https://dirtysalt.github.io/html/siraj-raval-youtube-channel-index.html



COMPUTER VISION

Sessions

- Linear Regression Single Variable
- Linear Regression Multiple Variables
- Gradient Descent and Cost Function
- Save Model Using Joblib And Pickle
- Dummy Variables & One Hot Encoding
- Data Preparation
- Logistic Regression
- Logistic Regression with Multiclass Classification
- Support Vector Machine
- Decision Tree
- Random Forest
- K-Fold Cross Validation
- K-Means
- Convolutionnal Neural Network
- Computer Vision
- Image processing (Object/Person - Recognition/Identification)
- Natural Language Processing

- Learn Data Science:
  - https://www.youtube.com/watch?v=9rDhY1P3YLA
- Machine Learning By Andrew Ng
  - https://www.youtube.com/watch?v=PPLop4L2eGk&list=PLLssT5z_DsK-h9vYZkQkYNWcItqhlRJLN
  - https://www.coursera.org/learn/machine-learning/home/welcome
- Codebasic
  - https://www.youtube.com/channel/UCh9nVJoWXmFb7sLApWGcLPQ
- Colab series
  - https://www.youtube.com/watch?v=inN8seMm7UI&list=PLQY2H8rRoyvyK5aEDAl3wUUqC_F0oEroL
- OpenCV
  - https://www.youtube.com/watch?v=YY9f-6u2Q_c